达梦数据库

DM 数据守护 V2.0

达梦技术丛书

前言

概述

本文档主要介绍 DM 数据守护的系统特性、基本概念、主要功能、各组成部件的详细介绍，以及如何搭建数据守护环境并使用等。

读者对象

本文档主要适用于 DM 数据库的：

- 开发工程师
- 测试工程师
- 技术支持工程师
- 数据库管理员

通用约定

在本文档中可能出现下列标志，它们所代表的含义如下：

表 0.1 标志含义

标志	说明
 警告：	表示可能导致系统损坏、数据丢失或不可预知的结果。
 注意：	表示可能导致性能降低、服务不可用。
 小窍门：	可以帮助您解决某个问题或节省您的时间。
 说明：	表示正文的附加信息，是对正文的强调和补充。

在本文档中可能出现下列格式，它们所代表的含义如下：

表 0.2 格式含义

格式	说明
宋体	表示正文。
Courier new	表示代码或者屏幕显示内容。
粗体	表示命令行中的关键字（命令中保持不变、必须照输的部分）或者正文中强调的内容。标题、警告、注意、小窍门、说明等内容均采用粗体。
<>	语法符号中，表示一个语法对象。
::=	语法符号中，表示定义符，用来定义一个语法对象。定义符左边为语法对象，右边为相应的语法描述。
|	语法符号中，表示或者符，限定的语法选项在实际语句中只能出现一个。
{ }	语法符号中，大括号内的语法选项在实际的语句中可以出现 0…N 次(N 为大于 0 的自然数)，但是大括号本身不能出现在语句中。
[ ]	语法符号中，中括号内的语法选项在实际的语句中可以出现 0…1 次，但是中括号本身不能出现在语句中。
关键字	关键字在 DM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字既可以大写也可以小写。

访问相关文档

如果您安装了 DM 数据库，可在安装目录的“\doc”子目录中找到 DM 数据库的各种手册与技术丛书。

您也可以通过访问我们的网站 www.dameng.com阅读或下载 DM 的各种相关文档。

联系我们

如果您有任何疑问或是想了解达梦数据库的最新动态消息，请联系我们：

网址：www.dameng.com

技术服务电话：400-991-6599

技术服务邮箱：dmtech@dameng.com

目录

1 引言 9

2 概述 11

2.1 系统特性 13

2.1.1 主要特性 13

2.1.2 主要改进 15

2.2 基本概念 16

2.2.1 数据库模式 16

2.2.3 数据库状态 17

2.2.3 LSN 介绍 19

2.2.4 Redo 日志 20

2.2.5 Redo 日志缓冲区 21

2.2.6 KEEP_BUF 介绍 22

2.2.7 联机 Redo 日志文件 23

2.2.8 归档介绍 24

2.2.9 MAL 系统 29

2.2.10 OGUID 29

2.2.11 守护进程组 29

2.2.12 组分裂 30

2.2.13 脑裂 30

2.3 术语定义 31

2.4 实时主备 32

2.4.1 主要功能 32

2.4.2 归档流程 33

2.5 MPP 主备 34

2.5.1 归档流程 35

2.5.2 功能扩展 35

2.5.3 Huge 表日志 36

2.5.4 dmmpp.ctl 维护 37

2.6 读写分离集群 39

2.6.1 归档流程 40

2.6.2 实现原理 41

2.6.3 事务一致性 43

2.6.4 性能调整 45

2.7 异步备库 46

3 守护进程 48

3.1 主要功能 48

3.1.1 监控数据库实例 48
3.1.2 发送状态信息 49
3.1.3 监控其他守护进程 49
3.1.4 接收监视器消息 49
3.1.5 主备库启动运行 50
3.1.6 备库故障处理 50
3.1.7 主库故障处理 51
3.1.8 故障恢复处理 51
3.2 守护类型 52
3.3 守护模式 53
3.4 守护状态 54
3.5 控制文件 56
3.5.1 概述 56
3.5.2 作用 56
3.5.3 内容 57
3.5.4 状态 58
3.6 相关命令 58
4 监视器 60
4.1 监视器类型 61
4.2 状态确认 61
4.3 自动接管 62
4.4 监视器命令 63
5 配置文件说明 79
5.1 dm.ini 79
5.2 dmmal.ini 81
5.3 dmarch.ini 82
5.4 dmwatcher.ini 83
5.5 dmwatcher.ctl 84
5.6 dmmonitor.ini 85
5.7 dmtimer.ini 87
5.8 端口配置关系说明 89
5.9 服务名配置 91
6 数据守护使用说明 93
6.1 正常运行状态 93

6.2 数据守护的启动 93
6.3 强制 Open 实例 94
6.4 关闭数据守护系统 95
6.5 主备库切换 96
6.6 主库故障、备库接管 98
6.7 备库强制接管 99
6.8 主库故障重启（备库接管前重启） 100
6.9 备库故障处理 100
6.10 公共网络故障 100
6.11 内部网络故障 101
6.12 故障库数据同步 102
6.13 备库重建 102
6.14 守护进程组分裂 104
6.15 dmmpp.ctl 不一致 105
6.16 确认监视器未启动 106
6.17 备库维护 106
6.18 滚动升级 107
6.19 备库文件损坏 109
6.20 实时/读写分离/MPP 备库数据同步情况分析 114
6.21 异步备库数据同步情况分析 115
6.22 注意事项 116
7 数据守护搭建 119
7.1 数据准备 119
7.1.1 数据文件拷贝 119
7.1.2 备份还原方式 119
7.1.3 检查数据一致性 120
7.1.4 守护进程控制文件 121
7.2 配置实时主备 121
7.2.1 环境说明 121
7.2.2 数据准备 123
7.2.3 配置主库 GRP1_RT_01 123
7.2.4 配置备库 GRP1_RT_02 126

7.2.5 配置监视器 129
7.2.6 启动守护进程 130
7.2.7 启动监视器 130
7.3 配置读写分离集群 130
7.3.1 环境说明 131
7.3.2 数据准备 132
7.3.3 配置主库 GRP1_RWW_01 133
7.3.4 配置备库 GRP1_RWW_02 136
7.3.5 配置备库 GRP1_RWW_03 140
7.3.6 配置监视器 143
7.3.7 启动守护进程 144
7.3.8 启动监视器 144
7.3.9 接口说明 145
7.4 配置 MPP 主备 147
7.4.1 环境说明 147
7.4.2 数据准备 149
7.4.3 配置主库 GRP1_MPP_EP01 149
7.4.4 配置主库 GRP2_MPP_EP02 153
7.4.5 配置备库 GRP1_MPP_EP11 155
7.4.6 配置备库 GRP2_MPP_EP22 157
7.4.7 配置 dmwatcher.ini 159
7.4.8 生成 dmwatcher.ctl 161
7.4.9 配置监视器 161
7.4.10 启动守护进程 162
7.4.11 启动监视器 162
7.5 配置异步备库 163
7.5.1 环境说明 163
7.5.2 数据准备 164
7.5.3 配置主库 GRP1_RT_01 164
7.5.4 配置备库 GRP1_RT_02 166
7.5.5 配置异步备库 GRP1_LOCAL_01 166
7.5.6 配置监视器 168
7.5.7 启动守护进程 169
7.5.8 启动监视器 169
7.6 注册服务 170
7.6.1 注册 dmserver 服务 170
7.6.2 注册 dmwatcher 服务 170
7.6.3 注册 dmmonitor 服务 171
7.7 动态增加读写分离集群节点 172
7.7.1 数据准备 172
7.7.2 配置新备库 173
7.7.3 动态添加 MAL 配置 176

7.7.4 动态添加归档配置 177
7.7.5 修改监视器 dmmonitor.ini 177
7.7.6 启动所有守护进程以及监视器 177
8 自动化脚本部署工具 178
8.1 配置文件 178
8.2 搭建步骤 182
8.3 集群维护 184
8.3.1 重启集群 184
8.3.2 批量更新 dm.ini 参数 185
8.3.3 配置 dm_svc.conf 186
8.3.4 达梦集群部署工具的维护 187
9 利用 DM 部署工具搭建数据守护 188
9.1 实时主备管理 188
9.1.1 部署 188
9.1.2 监控 197
10 附录 200
10.1 系统视图 200
10.2 监视器接口 206
10.2.1 DLL 依赖库 206
10.2.2 返回值说明 206
10.2.3 C 接口说明 207
10.2.4 JNI 接口说明 258
10.2.5 C 编程示例 292
10.2.6 Java 编程示例 296
10.2.7 错误码汇编 301

1 引言

DM 数据守护（Data Watch）是一种集成化的高可用、高性能数据库解决方案，是数据库异地容灾的首选方案。通过部署 DM 数据守护，可以在硬件故障（如磁盘损坏）、自然灾害（地震、火灾）等极端情况下，避免数据损坏、丢失，保障数据安全，并且可以快速恢复数据库服务，满足用户不间断提供数据库服务的要求。与常规的数据库备份（Backup）、还原（Restore）技术相比，数据守护可以更快地恢复数据库服务。随着数据规模不断增长，通过还原手段恢复数据，往往需要数个小时、甚至更长时间，而数据守护基本不受数据规模的影响，只需数秒时间就可以将备库切换为主库对外提供数据库服务。

DM数据守护提供多种解决方案，可以配置成实时主备、MPP主备或读写分离集群，满足用户关于系统可用性、数据安全性、性能等方面的综合需求，有效降低总体投入，获得超值的投资回报。

实时主备的主要目的是保障数据库的可用性。实时主备包含一个主库和一个实时备库，主库提供完整的数据库功能，备库提供只读服务，当主库出现故障时，备库可以切换为主库，继续对外提供服务。主库修改数据产生的Redo日志，在写入联机Redo日志文件之前发送到实时备库，实时备库收到Redo日志后马上响应主库，同时启动重演Redo日志流程，主库收到备库响应消息后再将Redo日志真正写入联机Redo日志文件。当主库出现故障时，备库只需要将收到的所有Redo日志重演结束，就可以切换为主库对外提供数据库服务了。

MPP主备的主要目的也是保障数据库的可用性。每个MPP节点（主库）可以配置1个或者多个MPP备库，MPP节点的主库和备库构成一个数据守护组（Group），这些MPP数据守护组一起构成了MPP主备系统。MPP主备Redo日志同步的流程与实时主备基本相同，先将Redo日志依次发送到备库，等接收到备库的响应后，再将主库Redo日志写入到联机Redo日志文件中。因此，当某个MPP主节点出现故障时，在其数据守护组内挑选一个备库切换为主库对外提供数据库服务，就可以确保整个MPP集群的正常使用。

读写分离集群的主要目标是在保障数据库可用性基础上，实现读与写分离，进一步提升数据库的业务支撑能力。读写分离集群包含一个主库，一个或者多个备库。读写分离集群主库先将Redo日志写入到联机Redo日志文件中，再依次发送Redo日志到备库，备库重演Redo日志后，再响应主库。因此，主库与备库的数据是严格保持一致的，在此基础上配合DM的各种接口（JDBC、DPI等），可以将只读操作自动分流到备库，有效降低主库的负载，

提升系统吞吐量。

2 概述

DM 数据守护（Data Watch）的实现原理非常简单：将主库（生产库）产生的 Redo 日志传输到备库，备库接收并重新应用 Redo 日志，从而实现备库与主库的数据同步。DM 数据守护的核心思想是监控数据库状态，获取主、备库数据同步情况，为 Redo 日志传输与重演过程中出现的各种异常情况提供一系列的解决方案。

DM 数据守护系统结构参考图 2.1。主要由主库、备库、Redo 日志、Redo 日志传输、Redo 日志重演、守护进程（dmwatcher）、监视器（dmmonitor）组成。

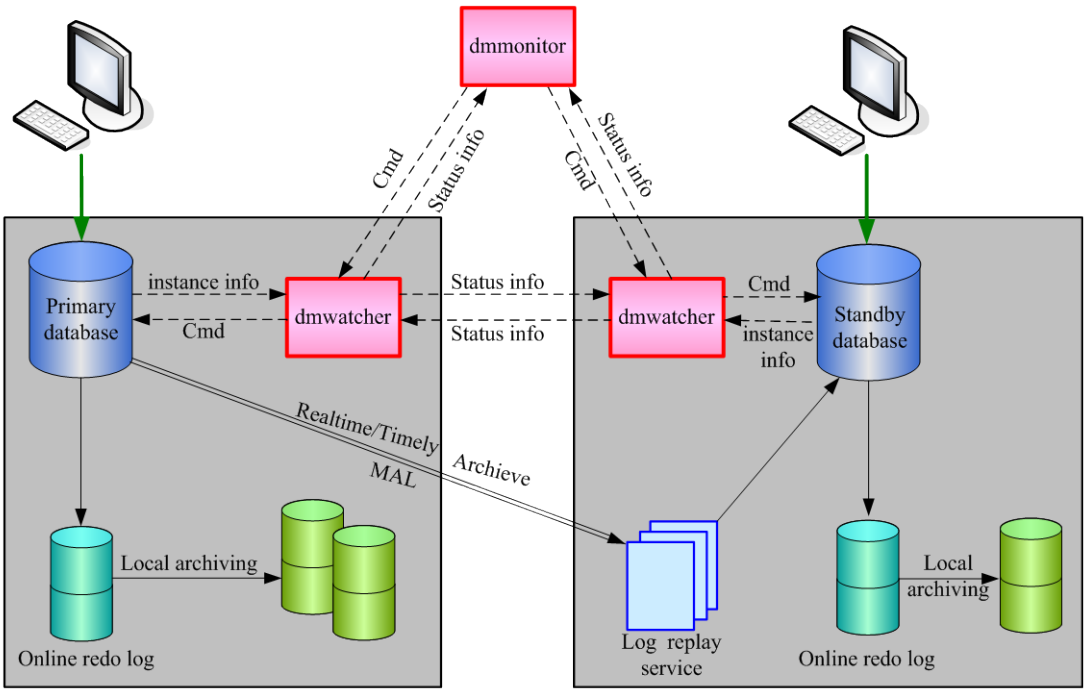


图 2.1 数据守护系统结构图

数据库与数据库实例

数据库（Database）是一个文件集合（包括数据文件、临时文件、重做日志文件和控制文件），保存在物理磁盘或文件系统中。

数据库实例（Instance）就是一组操作系统进程（或者是一个多线程的进程）以及一些内存。通过数据库实例，可以操作数据库，一般情况下，我们访问、修改数据库都是通过数据库实例来完成的。

本文档将不再严格区分数据库和数据库实例的概念，很多地方会笼统的以库来代替。考虑到数据守护系统中，数据库实例名是唯一的，为了更准确的进行描述，很多情况下我们会以实例 xxx 来标记某一个主库或者备库。

主库

Primary 模式，提供完整数据库服务的实例，一般来说主库是用来直接支撑应用系统的生产库。

备库

Standby 模式，提供只读数据库服务的实例，一般来说备库用来容灾，并可以承担数据库备份、查询等只读功能，不过备库临时表支持 Insert/Delete/Update 操作。

备库支持临时表修改主要基于两个因素：一是临时表数据的修改不会产生 Redo 日志，主备库临时表数据无法同步；二是提供更大灵活性，适应更多应用场景。

Redo 日志

Redo 日志记录物理数据页内容变动情况，是数据库十分重要的一个功能，在数据库系统故障（比如服务器掉电）重启时，利用 Redo 日志可以把数据恢复到故障前的状态。

Redo 日志也是数据守护的实现基础，数据库中 Insert、Delete、Update 等 DML 操作以及 Create TABLE 等 DDL 操作最终都会体现为对某一个或者多个物理数据页的修改，因此备库通过重做 Redo 日志可以与主库数据保持一致。

Redo 日志传输

主备库之间的 Redo 日志传输，以日志缓冲区 RLOG_BUF 为单位，主库通过 MAL 系统发送 Redo 日志到备库。各种不同数据守护类型的区别，就在于主库日志缓冲区 RLOG_BUF 的发送时机，以及备库收到 Redo 日志后的处理策略。

Redo 日志重演

Redo 日志重演的过程，就是备库收到主库发送的 Redo 日志后，在物理数据页上，重新修改数据的过程。Redo 日志重演由专门的 Redo 日志重演服务完成，重演服务严格按照收到的 Redo 日志的先后顺序，还原主库修改物理数据页的操作，并在重演过程中生成备库自身的 Redo 日志写入联机日志文件。

守护进程

守护进程（dmwatcher）是数据守护系统的核心工具，监控数据库实例的运行状态和主备库数据同步情况，在出现故障时启动各种处理预案。守护进程是各种消息的中转站，接收数据库实例、其他守护进程、以及监视器发送的各种消息；同时，守护进程也会将收到的数据库实例消息转发给其他守护进程和监视器。守护进程必须和被守护的数据库实例部署在同一台机器上。

监视器

监视器（dmmonitor）用来监控守护系统内守护进程、数据库实例信息，执行用户输入命令、监控实例故障、实现自动切换等。监视器一般配置在数据库实例和守护进程以外的机器上。

2.1 系统特性

2.1.1 主要特性

DM 数据守护的主要特性包括：

完整功能的主库

主库几乎没有任何功能上的限制，可以提供与 Normal 模式几乎完全一样的数据库功能。限制操作只有三项：不支持联机恢复表、不支持修改表空间文件名、不支持修改 arch_ini 参数。

活动的备库

基于独特的字典缓存技术和日志重演技术，备库在 Open 状态下执行数据同步，是真正意义上的热备库；在实现异地容灾的同时，用户可以只读的方式访问备库，执行报表生成、数据备份等功能，减轻主库系统的负载，提高资源利用率。

多重数据保护

每个备库都是一个完整的数据库备份，可以同时配置多个备库，为数据安全提供全方位的保护。

高可用性

当主库出现故障时，可以快速将备库切换为主库，继续提供数据库服务，确保数据库服务不中断。切换过程一般在数秒钟之内完成。

多种守护模式

提供自动切换和手动切换两种守护模式，满足用户不同需求。其中，配置自动切换的前提是已经部署确认监视器。在提供第三方机器部署确认监视器情况下，可以配置为故障自动切换模式，主库出现故障时，系统自动将备库切换为主库对外提供数据库服务。

多种守护类型

提供实时主备、MPP 主备、读写分离集群和本地守护等四种类型的数据守护，适应各种应用需求。

故障自动重连

配置、使用连接服务名访问数据库，在发生主备库切换后，接口会自动将连接迁移到新的主库上。

故障库自动重加入

主库故障，发生主备库切换。故障主库重启后，可以自动切换为 Standby 模式，作为备库重新加入数据守护系统。

历史数据自动同步

故障备库重启，同步历史数据的过程无需用户干预，主库自动发送历史归档 Redo 日志到备库，进行数据同步。

自动负载均衡

配置读写分离集群，可以将只读操作分流到备库上进行，来减轻主库访问压力，提高数据库系统的吞吐量。读写分离的过程由 JDBC 等接口配合服务器自动完成，无需用户干预。

滚动升级

可以在不间断数据库服务的情况下，滚动地对数据守护系统中的主备库进行数据库软件版本升级。

灵活的搭建方式

可以在不间断数据库服务的情况下，将单节点数据库升级为主备系统。DM 提供多种工具来完成数据守护搭建过程，如 SHELL 脚本或 DM 部署管理工具，均能方便地完成数据守护搭建。

完备的监控工具

通过命令行监控工具 dmmonitor、DM 部署管理工具可以实时更新、监控主备库的状态和数据同步情况。

完善的监控接口

提供完善的数据守护监控接口，可以定制监控项，并方便地集成到应用系统中。

丰富的守护命令

提供主备库切换、强制接管等功能，通过简单的命令，就可以实现主备库角色互换、故障接管等功能。


说明：**DM 的单节点和主库提供读未提交（Read Uncommitted）、读提交（Read Committed）和串行化（Seriablizable）三种事务隔离级，可重复读（Repeatable Read）升级为更严格的串行化事务隔离级。但是，备库只支持读提交（Read Committed）事务隔离级别：**

- **不能访问所有未提交事务的修改**
- **可以访问所有已提交事务的修改**

2.1.2 主要改进

在总结 DM 数据守护 V1.0 版本设计方案与用户使用反馈的基础上，V2.0 版本进行了一些重要改进，为用户提供更好的使用体验：

1.检测并处理组分裂

检测并处理组分裂场景，是 DM 数据守护 V2.0 的一个重大改进。在 V1.0 版本中，主备库之间的数据一致性完全由用户保证，备库强制接管后，没有办法检查重新恢复的主库是否可以作为备库重新加入主备系统。在 V2.0 版本中，我们引入控制文件 dmwatcher.ctl，将备库接管时的相关信息，记录在守护进程控制文件 dmwatcher.ctl 中，故障主库恢复后，根据 dmwatcher.ctl 控制文件中信息来判断是否满足故障重加入条件。

2.支持 TCP 协议，取消 UDP 协议

数据库实例与守护进程、守护进程与守护进程、以及守护进程与监视器之间的信息传递，都是基于 TCP 协议完成的。相比于采用 UDP 协议的 DM 数据守护系统 V1.0 版本，使用 TCP 协议的 V2.0 版本，具有以下优势：

1） 支持跨网段部署数据守护系统，降低部署的硬件要求；

2） 支持在一台物理机器上搭建数据守护系统，方便测试环境搭建；

3） 更加简化、统一的配置文件和配置参数。

3．实时主备/MPP 主备/读写分离集群均支持故障自动切换

数据守护系统配置为自动切换模式，主库故障时，备库可以自动切换为主库，这个过程叫作故障自动切换。在 V1.0 版本中，只有实时主备支持故障自动切换，在 V2.0 版本中，MPP 主备和读写分离集群也支持故障自动切换。

4．任意场景支持强制接管，避免繁琐的手工 SQL 干预

实时主备/MPP 主备/读写分离三种类型全部实现了强制接管命令。

在 V1.0 版本中，实时守护没有监视器命令，需要手工执行一系列 SQL 语句完成强制接管，比较繁琐；读写分离集群和 MPP 主备提供了强制接管命令，但使用场景受限，比如待接管备库处于 MOUNT 状态的情况下，无法通过强制接管命令将备库切换为主库。并且强制接管后，用户无法判断主备库数据是否一致，存在较大的数据安全隐患。

在 V2.0 版本中，实时主备/MPP 主备/读写分离集群都支持强制接管命令，并且根据守护进程控制文件信息，可以判断强制接管后，是否产生组分裂，主备库数据是否可以恢复到一致状态。

5．配置参数调整，风格统一、步骤简化。提供多种搭建数据守护的自动化工具

在 V1.0 版本中，实时主备、MPP 主备和读写分离集群，各自有一套配置参数。在 V2.0 版本中，将三类守护系统统一为一套配置参数，并且简化掉一部分配置，大大提高了数据守护系统的搭建效率。

在 V2.0 版本中，除了支持手动搭建数据守护系统， DM 还提供多种自动化工具来完成数据守护搭建过程，如 SHELL 脚本、DM 部署管理工具均能简便灵活的完成数据守护系统的搭建。

2.2 基本概念

在介绍数据守护主要功能之前，我们先熟悉几个基本概念，只有充分理解这些概念，才能更加深入地理解数据守护。

2.2.1 数据库模式

DM 支持 3 种数据库模式：Normal 模式、Primary 模式和 Standby 模式。

Normal 模式

提供正常的数据库服务，操作没有限制。正常生成本地归档，但不发送实时归档（Realtime）、MPP 归档（MArch）、即时归档（Timely）、和异步归档（Async）。

Primary 模式

提供正常的数据库服务，操作有极少限制。该模式下功能限制包括：不支持联机恢复表、不支持修改表空间文件名、不支持修改 arch_ini 参数。正常生成本地归档，支持实时归档（Realtime）、MPP 归档（MArch）、即时归档（Timely）和异步归档（Async）。Primary 模式下，所有的数据库对象修改操作都强制生成 Redo 日志。

Standby 模式

可以执行数据库备份、查询等只读数据库操作。正常生成本地归档，正常发送异步归档 Redo 日志；但实时归档（Realtime）、MPP 归档（MArch）、即时归档（Timely）均强制失效。

三种模式可以相互切换，模式切换必须在 Mount 状态下执行。切换的 SQL 语句如下：

将数据库切换为 Primary 模式：

```
alter database primary;
```

将数据库切换为 Standby 模式：

```
alter database standby;
```

将数据库切换为 Normal 模式：

```
alter database normal;
```

2.2.3 数据库状态

DM 的数据库状态包括：

Startup 状态

系统刚启动时设置为 Startup 状态。

After Redo 状态

系统启动过程中联机日志重做完成后，回滚活动事务前设置为 After Redo 状态。非 Standby 模式的实例在执行 alter database open 操作前，也会将系统设置为 After Redo 状态。

Open 状态

数据库处于正常提供服务的状态，但不能进行归档配置等操作。

Mount 状态

数据库在 Mount 状态下，限制 Redo 日志刷盘，不能修改数据，不能访问表、视图等数据库对象，但可以执行修改归档配置、控制文件和修改数据库模式等操作，也可以执行一些不修改数据库内容的操作，比如查询动态视图或者一些只读的系统过程。

系统从 Open 状态切换为 Mount 状态时，需要回滚所有活动事务，但不会断开用户连接，也不会强制 Buffer 中的脏页刷盘。

Suspend 状态

数据库在 Suspend 状态下，限制 Redo 日志刷盘，可以访问数据库对象，甚至可以修改数据，但是一旦执行 COMMIT 等操作触发 Redo 日志写盘时，当前操作将被挂起。

相比 Open 到 Mount 的状态切换，Open 到 Suspend 的状态切换更加简单、高效，不会回滚任何活动事务，在状态切换完成后，所有事务可以继续执行。

一般在修改归档状态之前将系统切换为 Suspend 状态，比如备库故障恢复后，在历史数据（归档日志）同步完成后，需要重新启用实时归档功能时：

1．将系统切换为 Suspend 状态，限制 Redo 日志写入联机 Redo 日志文件。

2．修改归档状态为 Valid。

3．重新将数据库切换为 Open 状态，恢复 Redo 日志写入功能。

4．备库与主库重新进入实时同步状态。

另外，实时归档、MPP 归档失败时（比如网络故障导致），Primary 实例将试图切换成 Suspend 状态，防止后续的日志写入。因为一旦写入，主备切换时，有可能备库没有收到最后那次的 RLOG_BUF，导致主库上多一段日志，很容易造成主备数据不一致。当实例成功切换为 SUSPEND 状态时，可直接退出，强制丢弃多余的日志，避免主备数据不一致。

Shutdown 状态

实例正常退出时设置为 Shutdown 状态。

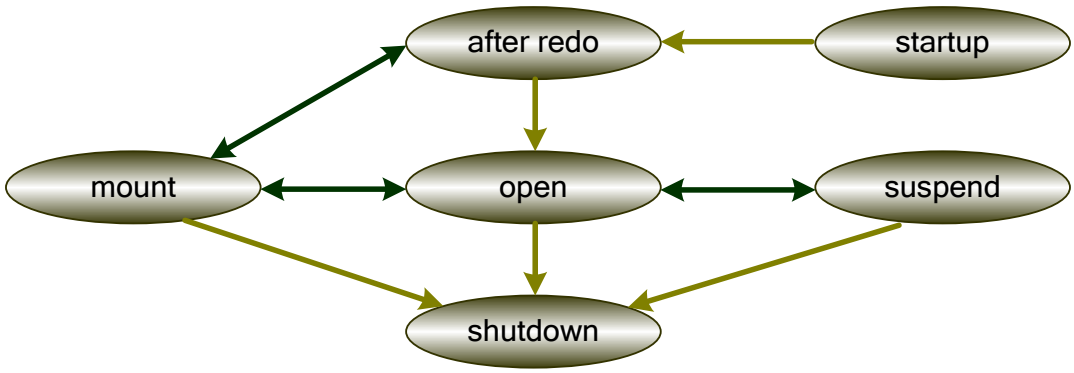


图 2.2 数据库状态转换

不同数据库状态之间的转换规则，如图 2.2 所示。用户可以通过 SQL 语句进行数据库状态切换：1. Open 状态与 Mount 状态可以相互切换；2. Open 状态与 Suspend 状态可以相互切换；3. Mount 和 Suspend 状态不能直接转换；4. 其他状态为系统内部状态，用户不能主动干预。

切换数据库状态的 SQL 如下：

1. 将数据库修改为 Open 状态。当系统处于 Primary/Standby 模式时，必须强制加上 force 子句。

```
alter database open [force];
```

2．将数据库修改为 Mount 状态。

```
alter database mount;
```

3．将数据库修改为 Suspend 状态。

```
alter database suspend;
```



小窍门：由于 dmwatcher 根据数据库模式、状态等信息作为故障处理、故障恢复的依据，建议在配置数据守护过程中，修改 dm.ini 参数 ALTER_MODE_STATUS 为 0，限制用户直接通过 SQL 语句修改数据库状态、模式，避免 dmwatcher 做出错误的决策。

2.2.3 LSN 介绍

LSN（Last Sequence Number）是由系统自动维护的 Bigint 类型数值，具有自动递增、全局唯一特性，每一个 LSN 值代表着 DM 系统内部产生的一个物理事务。物理事务（Physical Transaction，简称 ptx）是数据库内部一系列修改物理数据页操作的集合，与数据库管理系统中事务（Transaction）概念相对应，具有原子性、有序性、无法撤销等特性。

DM 数据库中与 LSN 相关的信息，可以通过查询 V$RLOG 表来获取。DM 主要包括以下几种类型的 LSN 值：

- **CUR_LSN** 是系统已经分配的最大 LSN 值。物理事务提交时，系统会为其分配一个唯一的 LSN 值，大小等于 CUR_LSN + 1，然后再修改 CUR_LSN=CUR_LSN+1。
- **FILE_LSN** 是已经写入联机 Redo 日志文件的最大 LSN 值。每次将 Redo 日志缓冲区 RLOG_BUF 写入联机 Redo 日志文件后，都要修改 FILE_LSN 值。
- **FLUSH_LSN** 是已经发起日志刷盘请求，但还没有真正写入联机 Redo 日志文件的最大 LSN 值。
- **CKPT_LSN** 是检查点 LSN，所有 LSN <= CKPT_LSN 的物理事务修改的数据页，都已经从 Buffer 缓冲区写入磁盘，CKPT_LSN 由检查点线程负责调整。

与 DM 数据守护相关的 LSN 信息有：

- ✓ **CLSN** 与 CUR_LSN 保持一致，数据库已经分配的最大 LSN 值。
- ✓ **FLSN** 与 FILE_LSN 保持一致，已写入联机日志文件的 LSN 值。

- ✓ **SLSN** 是 Standby LSN 的缩写，是备库收到的最后一个 RLOG_BUF 的最大 LSN 值，与主库的 FLUSH_LSN 保持一致。
- ✓ **SSLSN** 是 Second Standby LSN 的缩写，实时主备或 MPP 主备中备库收到的倒数第二个 RLOG_BUF 的最大 LSN 值。在读写分离集群中 SLSN == SSLSN。
- ✓ **TAKEOVER_LSN** 备库接管时的 SLSN 值，记录在 dmwatcher.ctl 文件中。同一守护进程组中故障库恢复时，根据是 TAKEOVER_LSN 和故障库的 FILE_LSN 值，来判断故障库数据是否可以恢复。

2.2.4 Redo 日志

Redo 日志包含了所有物理数据页的修改内容，Insert/delete/update 等 DML 操作、Create Table 等 DDL 操作，最终都会转化为对物理数据页的修改，这些修改都会反映到 Redo 日志中。一般说来一条 Insert 等 SQL 语句，在系统内部会转化为多个相互独立的物理事务来完成，物理事务提交时会将产生的 Redo 日志写入缓冲区 RLOG_BUF 中。

一个物理事务包含一个或者多个 Redo 记录（Redo Record），每条 Redo 记录（RREC）都对应一个修改物理数据页的动作。根据记录内容的不同，RREC 可以分为两类：物理 RREC 和逻辑 RREC。

物理 RREC 记录的是数据页的变化情况，内容包括：操作类型、修改数据页地址、页内偏移、数据页上的修改内容，如果是变长类型的 Redo 记录，在 RREC 记录头之后还会有一个两字节的长度信息。

逻辑 RREC 记录的是一些数据库逻辑操作步骤，主要包括：事务启动、事务提交、事务回滚、字典封锁、事务封锁、B 树封锁、字典淘汰等。

逻辑 RREC 记录是专门为数据守护增加的记录类型，用来解决备库重演 Redo 日志与用户访问备库之间的并发冲突，以及主库执行 DDL 后导致的主备数据库字典缓存不一致问题。备数据库解析到逻辑 RREC 记录时，根据记录内容，生成相应的事务，封锁对应的数据库对象，并从字典缓存中淘汰过期的字典对象。

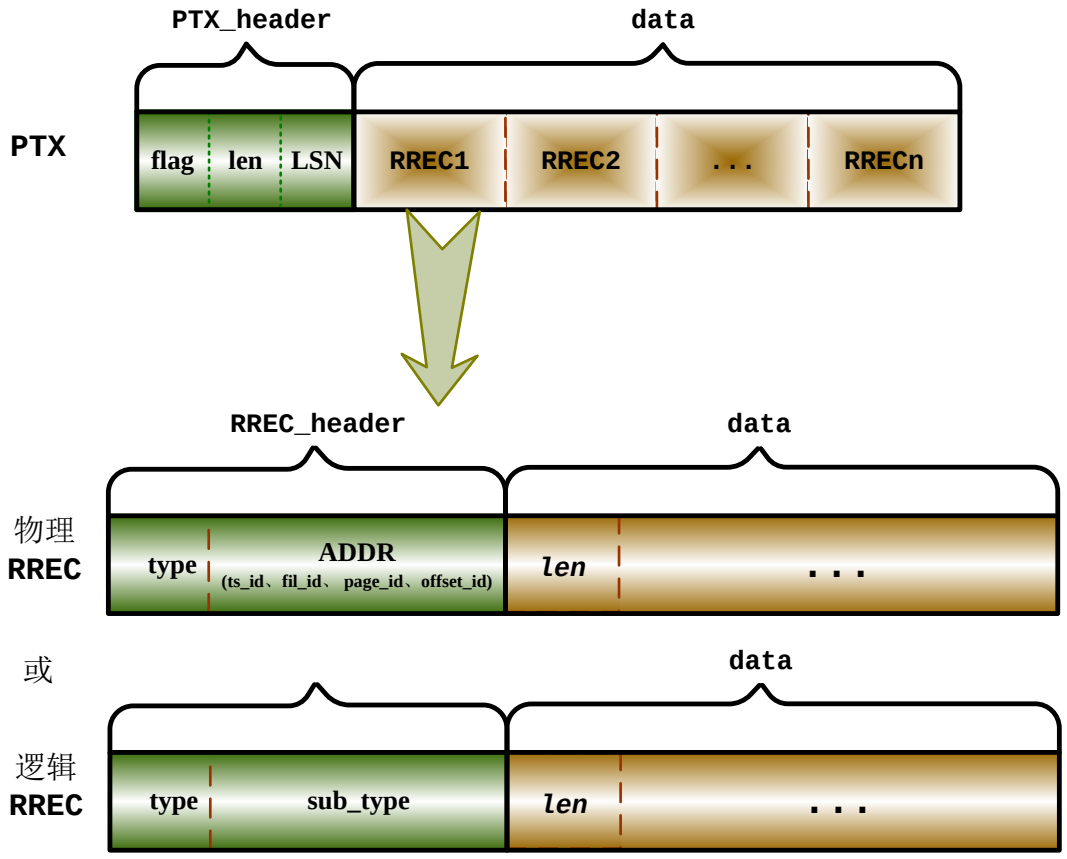


图 2.3 PTX/RREC 结构图

2.2.5 Redo 日志缓冲区

Redo 日志缓冲区 RLOG_BUF 是 DM 数据库内部的一个数据结构，用来优化、提升 Redo 日志刷盘效率。物理事务提交时将 Redo 日志写入到 RLOG_BUF 中，在数据库事务提交或 RLOG_BUF 缓冲区被写满时触发日志刷盘动作。日志刷盘线程负责将 RLOG_BUF 中的 Redo 日志写入联机日志文件，如果配置了 Redo 日志归档，日志刷盘线程还将负责触发归档动作。DM 数据守护系统中，主库以 RLOG_BUF 缓冲区为最小单位，发送 Redo 日志到备库。

RLOG_BUF 由一系列的 RLOG_PAGE 组成，RLOG_PAGE 是物理事务的保存载体，一个 RLOG_PAGE 可以保存一个或多个物理事务信息，一个物理事务、甚至一条 Redo 记录也可能需要存放到多个 RLOG_PAGE 中。

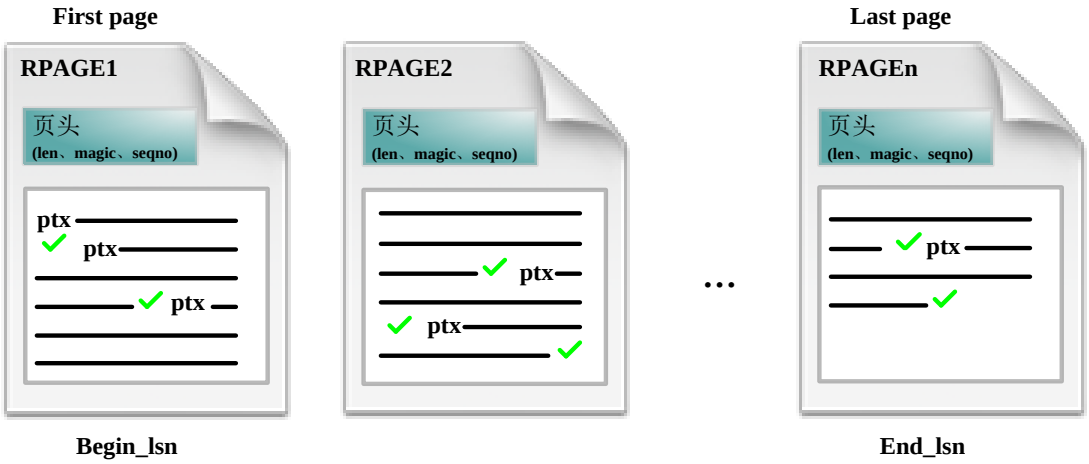


图 2.4 RLOG_BUF 结构图

2.2.6 KEEP_BUF 介绍

实时主备和 MPP 主备系统中，备库将最新收到的 RLOG_BUF 保存在内存中，不马上启动重演，这个 RLOG_BUF 我们称之为 KEEP_BUF。引入 KEEP_BUF 的主要目的是，避免下述场景中，主库故障重启后不必要的主备切换，减少用户干预。

场景说明（实时主备或 MPP 主备）：

1.用户登录主库 A 执行

```
create table tx(c1 int);

insert into tx values(1);

commit;
```

其中 commit 操作将触发实时（或 MPP）归档，发送 RLOG_BUF 到备库 B。

2.备库 B 收到 RLOG_BUF，响应主库 A，并启动日志重演。

3.主库 A 在 RLOG_BUF 写入联机日志文件之前故障。

4.主库 A 重新启动后，由于 RLOG_BUF 没有写入联机日志文件，之前插入 tx 表的数据丢失；但此时备库 B 已经重演日志成功，tx 表中已经插入一行数据。

上述场景中，主备库数据不再保持一致，必须将备库 B 切换为主库，并重新从 B 同步数据到 A。如果配置的是手动切换模式，则必须要有用户干预，进行备库接管后，才能恢复

数据库服务。

引入 KEEP_BUF 后，备库 B 收到主库 A 发送的 RLOG_BUF，并不会马上启动日志重演，主库 A 重启后，守护进程 A 检测到备库 B 存在 KEEP_BUF，通知备库 B 丢弃 KEEP_BUF 后，直接 Open 主库 A，就可以继续提供数据库服务。并且，这些操作是由守护进程自动完成，不需要用户干预。

如果备库自动接管、或者用户发起备库接管命令，那么备库的 KEEP_BUF 将会启动重演，不管主库是否已经将 KEEP_BUF 对应的 Redo 日志写入联机日志文件中，备库接管时的 TAKEOVER_LSN 一定是大于等于主库的 FILE_LSN。当故障主库重启后，仍然可以作为备库，自动重新加入数据守护系统。

备库 KEEP_BUF 日志重演的时机包括：

1. 收到主库的重演命令，并且备库的 SLSN 满足重演条件

主库会定时每 5 秒将 FILE_LSN 等信息发送到备库，当主库 FILE_LSN == 备库 SLSN 时，表明主库已经将 KEEP_BUF 对应的 Redo 日志写入联机日志文件中，此时备库会启动 KEEP_BUF 的日志重演。

2. 备库收到新的 RLOG_BUF

备库收到新的 RLOG_BUF 时，会将当前保存的 KEEP_BUF 日志重演，并将新收到的 RLOG_BUF 再次放入 KEEP_BUF 中。

3. 备库切换为新主库

在监视器执行 SWITCHOVER 或 TAKEOVER 命令，或者确认监视器通知备库自动接管时，备库会在切换为 PRIMARY 模式之前，启动 KEEP_BUF 的日志重演。



说明：读写分离集群先将 RLOG_BUF 写入主库联机 Redo 日志文件，再将 RLOG_BUF 发送到备库，因此它的即时备库没有 KEEP_BUF。

2.2.7 联机 Redo 日志文件

DM 数据库默认包含两个联机 Redo 日志文件（如 DAMENG01.log、DAMENG02.log，系统内部分别称为 0 号文件、1 号文件），用来保存 Redo 日志，RLOG_BUF 顺序写入联机 Redo 日志文件中，当一个日志文件写满后，自动切换到另外一个 Redo 日志文件。其中 0

号文件是 Redo 日志主文件，在日志主文件头里保存了诸如 CKPT_LSN，CKPT_FILE，CKPT_OFFSET，FILE_LSN 等信息。系统故障重启时，从[CKPT_FILE, CKPT_OFFSET]位置开始读取 Redo 日志，解析 RREC 记录，并重新修改对应数据页内容，确保将数据恢复到系统故障前状态。

随着检查点(Checkpoint)的推进，对应产生 Redo 日志的数据页从数据缓冲区(DATA Buffer）写入磁盘后，联机 Redo 日志文件可以覆盖重用、循环使用，确保 Redo 日志文件不会随着日志量的增加而增长。

注意：任何数据页从 Buffer 缓冲区写入磁盘之前，必须确保修改数据页产生的 Redo 日志已经写入到联机 Redo 日志文件中。

下图说明了联机日志文件、日志缓冲区 RLOG_BUF 以及相关各 LSN 之间的关系。

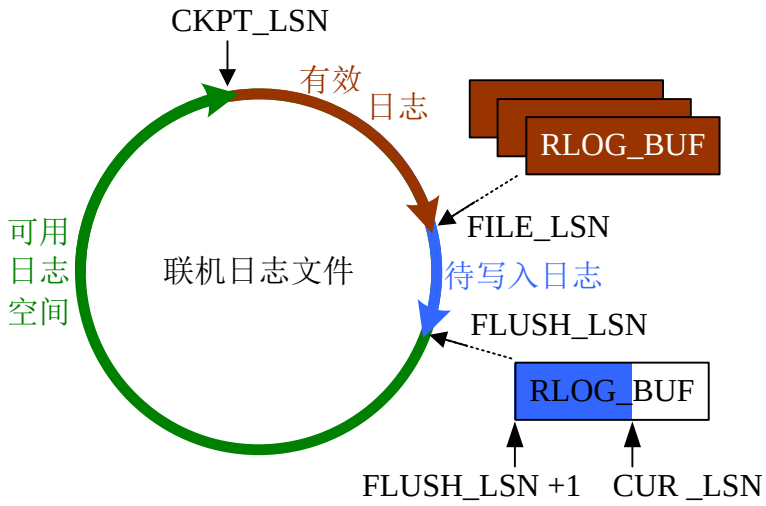


图 2.5 联机 Redo 日志文件与日志缓冲区

在联机日志文件中，可以覆盖写入 Redo 日志的文件长度为可用日志空间；不能被覆盖，系统故障重启需要重做部分，为有效 Redo 日志，有效 Redo 日志的 LSN 取值范围是(CKPT_LSN,FILE_LSN]；已经被发起日志刷盘请求，但还没有真正写入联机 Redo 日志文件的区间为(FILE_LSN,FLUSH_LSN]，称为待写入日志空间。

2.2.8 归档介绍

归档是实现数据守护系统的重要技术手段，根据功能与实现方式的不同，DM 数据库的归档可以分为 5 类：本地归档、实时归档、MPP 归档、即时归档和异步归档。

2.2.8.1 本地归档

Redo 日志本地归档（LOCAL），就是将 Redo 日志写入到本地归档日志文件的过程。配置本地归档情况下，Redo 日志刷盘线程将 Redo 日志写入联机 Redo 日志文件后，对应的 RLOG_BUF 由专门的归档线程负责写入本地归档日志文件中。

与联机 Redo 日志文件可以被覆盖重用不同，本地归档日志文件不能被覆盖，写入其中的 Redo 日志信息会一直保留，直到用户主动删除；如果配置了归档日志空间上限，系统会自动删除最早生成的归档 Redo 日志文件，腾出空间。

DM 提供了按指定的时间或指定的 LSN 删除归档日志的系统函数，但用户需要谨慎使用。例如，在守护系统中，如果备库故障了，主库继续服务，主库的日志在继续增长。此时如果删除尚未同步到备库的主库归档日志，那么等待备库重启之后，会由于备库收到的日志缺失导致主备库无法正常同步数据。相关的系统函数分别为 SF_ARCHIVELOG_DELETE_BEFORE_TIME 和 SF_ARCHIVELOG_DELETE_BEFORE_LSN。

本地归档文件在配置的归档目录下生成并保存，文件命名规则为：日志归档名_年月日时分秒毫秒.log，如：ARCHIVE_LOCAL1_20151014153933458.log。如果磁盘空间不足，且没有配置归档日志空间上限（或者配置的上限超过实际空间），系统将自动挂起，直到用户主动释放出足够的空间后继续运行。


注意：

为了最大限度的保护数据，当磁盘空间不足导致归档写入失败时，系统会挂起等待，直到用户释放出足够的磁盘空间。

当磁盘损坏导致归档日志写入失败时，系统会强制 HALT。

2.2.8.2 实时归档

与本地归档写入本地磁盘不同，实时归档（Realtime）需要将主库上产生的 Redo 日志，通过 MAL 系统发送到备库，实时归档是实时主备的实现基础。实时归档只有在主库配置为 Primary 模式时才能生效，一个主库只能配置一个实时备库。

实时归档的执行流程是，主库将 Redo 日志（RLOG_BUF）写入联机 Redo 日志文件前，将 Redo 日志发送到配置为 Standby 模式的备库，备库收到 Redo 日志（RLOG_BUF）后放入 KEEP_BUF，并将原有的 KEEP_BUF 中的内容加入日志重演任务系统后，马上响应主库，而不是等待 Redo 日志重演结束再响应主库，主库收到响应消息，确认备库已经收到

Redo 日志，再将 Redo 日志写入联机 Redo 日志文件中。

2.2.8.3 MPP 归档

MPP 归档（MArch）是实现 MPP 主备的基础，MPP 归档的原理、流程与实时归档基本一致，主要区别是：

1. 实时归档只能配置 1 个备库，MPP 归档可以配置 1~8 个备库。
2. 实时归档不支持 Huge 表数据传递，MPP 归档支持 Huge 表数据传递。

2.2.8.4 即时归档

即时归档（Timely）在主库将 Redo 日志写入联机 Redo 日志文件后，再通过 MAL 系统将 Redo 日志发送到备库。即时归档是读写分离集群的实现基础，与实时归档的主要区别是发送 Redo 日志的时机不同。一个主库可以配置 1~8 个即时备库。

根据备库重演 Redo 日志和响应主库时机的不同，即时归档分为两种模式：事务一致模式和高性能模式。即时归档模式根据配置文件 dmarch.ini 中的 ARCH_WAIT_APPLY 配置项来确定，配置为 1 就是事务一致模式，配置为 0 就是高性能模式，ARCH_WAIT_APPLY 缺省值是 1。

事务一致模式 主库事务 commit 触发 Redo 日志刷盘和即时归档，备库收到主库发送的 Redo 日志要在重演完成后再响应主库，主库才能响应用户 commit 成功。事务一致模式下，同一个事务的 SELEC 语句不管在是主库执行，还是在备库执行，查询结果都满足 Read Commit 隔离级要求。

高性能模式 与实时归档一样，备库收到主库发送的 Redo 日志后，马上响应主库再启动日志重演。高性能模式下，备库与主库的数据同步存在一定延时（一般情况下延迟时间非常短暂，用户几乎感觉不到），不能严格保证事务一致性。

事务一致模式，主备库之间严格维护事务一致性，但主库要等备库 Redo 日志重演完成后，再响应用户的 commit 请求，事务 commit 时间会变长，存在一定的性能损失。高性能模式则通过牺牲事务一致性，来获得更高的性能和提升系统的吞吐量。用户应该根据实际情况，选择合适的即时归档模式。

2.2.8.5 异步归档

异步归档（Async），由主、备库上配置的定时器触发，根据异步备库的 CUR_LSN 信息，扫描本地归档目录获取 Redo 日志，并通过 MAL 系统将 Redo 日志发送到异步备库。异步备库的 Redo 日志重演过程与实时归档等其他类型的归档完全一致。

每个 Primary 或 Standby 模式的数据库，最多可以配置 8 个异步备库，Normal 模式下配置的异步备库会被自动失效。

异步备库可以级联配置，异步备库本身也可以作为源库配置异步备库。理论上守护系统中可配置的异步备库的总数目只受 MAL 系统最大节点数（2048）的限制。

2.2.8.6 归档区别

表 2.1 归档类型比较

类型 / 比较	本地归档	实时归档	MPP 归档	即时归档	异步归档
备库数量	0	1	1~8	1~8	1~8
通过 MAL 传递数据	否	是	是	是	是
归档时机	写入联机日志后，再写入本地归档日志文件	写入联机日志前，发送到备库	写入联机日志前，发送到备库	写入联机日志后，发送到备库	定时启动
归档写入(发送)	归档线程	日志刷盘线程	日志刷盘线程	日志刷盘线程	异步归档线程
支持 Huge 表	否	否	是	否	否
数据来源	RLOG_BUF	RLOG_BUF	RLOG_BUF/ HUGE_BUFFER	RLOG_BUF	本地归档文件
失败处理	磁盘空间不足时，系统挂起等待用户释放	Suspend 数据库，保持归档状态不变，等待守	Suspend 数据库，保持归档状态不变，等待守	Suspend 数据库，并设置归档为无效状态	不做处理，等待下次触发继续发送

	出足够的磁盘空间。 磁盘损坏导致写入失败时，系统会强制 HALT	护进程干预	护进程干预		
备库响应时机	无	收到立即响应	收到立即响应	事务一致模式：重演完成后响应； 高性能模式：收到立即响应	收到立即响应
源库模式	Primary Standby Normal	Primary	Primary	Primary	Primary Standby
目标库模式	无	Standby	Standby	Standby	Standby



说明：任意一个备库的实时归档/MPP 归档/即时归档失败（即使其他备库归档成功了），主库都会切换为 Suspend 状态。

2.2.8.7 归档状态

本地归档、实时归档、MPP 归档和即时归档均包含两种状态：Valid 和 Invalid。异步归档只有一种归档状态：Valid。

- **Valid** 归档有效，正常执行各种数据库归档操作。
- **Invalid** 归档无效，主数据库不发送日志到备数据库。

在不同的归档类型中，归档状态转换时机不同。具体转换时机描述如下：

1. 主备库启动后，主库->备库的归档默认是 Valid 状态，守护进程 Open 主库前，会将主库->备库的归档状态修改为 Invalid 状态。

2. 备库故障恢复，同步主库数据完成后，守护进程先将主库修改为 Suspend 状态，

并将主库->备库的归档状态从 Invalid 修改为 Valid。当守护进程再次 Open 主库后，主备库数据重新恢复为一致状态。

3. 实时备库、MPP 备库故障，守护进程处理 Failover 过程中，将主库->备库的归档状态修改为 Invalid。

4. 读写分离集群中，主库发送归档失败后，直接将主库->备库的归档改为 Invalid 状态。

5. 异步归档始终保持 Valid 状态，一旦归档失败马上返回，等待下一次触发再继续发送。



说明：实时归档、MPP 归档、即时归档只对 Primary 模式的主库有效，备库上配置的实时归档、MPP 归档、即时归档状态没有实际意义，始终保持 Valid 状态。

2.2.9 MAL 系统

MAL 系统是基于 TCP 协议实现的一种内部通信机制，具有可靠、灵活、高效的特性。DM 通过 MAL 系统实现 Redo 日志传输，以及其他一些实例间的消息通讯。

2.2.10 OGUID

数据守护唯一标识码，配置数据守护时，需要由用户指定 OGUID 值。其中数据库的 OGUID 在 MOUNT 状态下由系统函数 SP_SET_OGUID 设置，守护进程和监视器的 OGUID 值在配置文件中设定。

同一守护进程组中的所有数据库、守护进程和监视器，都必须配置相同的 OGUID 值，取值范围为 0~2147483647。

2.2.11 守护进程组

配置了相同 OGUID 的两个或多个守护进程，构成一个守护进程组。为方便管理，对每个守护进程组进行命名，守护进程组中的所有守护进程和监视器，必须配置相同的组名。

2.2.12 组分裂

同一守护进程组中，不同数据库实例的数据出现不一致，并且无法通过重演 Redo 日志，重新同步数据的情况，我们称为组分裂。引发组分裂的主要原因包括：

1. 即时归档中，主库在将 Redo 日志写入本地联机 Redo 日志文件之后，发送 Redo 日志到备库之前出现故障，导致主备库数据不一致，为了继续提供服务，执行备库强制接管。此时，当故障主库重启后，就会引发组分裂。

2. 故障备库重新完成数据同步之前，主库硬件故障，并且长时间无法恢复；在用户接受丢失部分数据情况下，为了尽快恢复数据库服务，执行备库强制接管，将备库切换为主库。此时，如果故障主库重启，也会造成组分裂。

检测到组分裂后，守护进程会修改控制文件为分裂状态，被分裂出去的数据库需要通过备份还原等技术手段重新恢复，或者按照分裂库修复步骤重新将数据恢复到一致状态。

2.2.13 脑裂

脑裂是同一个守护进程组中，同时出现两个或者多个活动主库，并且这些主库都接收用户请求，提供完整数据库服务。一旦发生脑裂，将无法保证数据一致性，对数据安全造成严重后果。

DM 数据守护系统为预防脑裂做了大量工作，例如故障自动切换模式数据守护，必须配置确认监视器。确认监视器启动故障切换之前，会进行严格的条件检查，避免脑裂发生。守护进程一旦检测到脑裂发生，会马上强制退出主库，等待用户干预，避免数据差异进一步扩大。

造成脑裂的主要原因有两个：网络不稳定或错误的人工干预。为了避免出现脑裂，我们建议：

1. 设置 dm.ini 参数 ALTER_MODE_STATUS=0，限制用户进行直接通过 SQL 修改数据库模式和状态。

2. 提供稳定、可靠的网络环境。

3. 配置自动切换数据守护时，将确认监视器部署在独立的第三方机器上，不要与某一个数据库实例部署在一起，避免由于网络问题触发自动故障切换，导致脑裂发生。

4. 通过人工干预，将备库切换为主库之前，一定要确认主库已经发生故障，避免主库

活动情况下，备库强制接管，人为造成脑裂。

2.3 术语定义

数据守护系统包含主库、备库、守护进程、监视器等诸多部件，在主备库切换、故障处理等场景下，仅以主库或者备库这些称谓，已经无法准确描述对应部件。为了更加清晰的描述数据守护系统，特别定义以下术语。

表 2.2 数据守护术语

术语	含义
实时主备	配置实时归档的主备系统
MPP 主备	配置 MPP 归档的主备系统
读写分离集群	配置即时归档的主备系统
实时主库[实例名]	实时主备系统中 Primary 模式的库
实时备库[实例名]	实时主备系统中 Standby 模式的库
MPP 主库[实例名]	MPP 主备系统中 Primary 模式的库
MPP 备库[实例名]	MPP 主备系统中 Standby 模式的库
即时主库[实例名]	读写分离主备系统中 Primary 模式的库
即时备库[实例名]	读写分离主备系统中 Standby 模式的库
异步备库[实例名]	异步归档目标库，Standby 模式
异步源库[实例名]	异步归档源库，Primary/Standby 模式都可
故障主库[实例名]	发生故障的 Primary 模式实例
故障备库[实例名]	发生故障的 Standby 模式实例
数据一致备库[实例名]	主库到当前备库归档处于有效状态，备库与主库数据保持一致
可恢复备库[实例名]	主库到当前备库归档处于失效状态，备库与主库数据存在差异，但主库归档日志涵盖备库缺失的数据
分裂库[实例名]	与主库数据不一致，且无法通过重做归档日志将数据恢复到一致状态的库
守护进程组[组名]	配置了相同 OGUID 的两个或多个守护进程，构成一个守护进程组
监视器	基于监视器接口实现的命令行工具，用于监控、管理数据守护系统
确认监视器	运行在确认模式下的监视器



注意：配置数据守护时，不要将数据库实例名配置为 Primary/Standby，避免在描述对象时产生歧义。

2.4 实时主备

实时主备系统由主库、实时备库、守护进程和监视器组成。通过部署实时主备系统，可以及时检测并处理各种硬件故障、数据库实例异常，确保持续提供数据库服务。

2.4.1 主要功能

实时主备系统主要功能包括：

1．实时数据同步

主备库通过实时归档完成数据同步，实时归档要求主库将 RLOG_BUF 发送到备库后，再将 RLOG_BUF 写入本地联机 Redo 日志文件。但要注意的是，备库确认收到主库发送的 Redo 日志，并不保证备库已经完成重演这些 Redo 日志，因此主备库之间的数据同步存在一定的时间差。

2．主备库切换

主备库正常运行过程中，可以通过监视器的 Switchover 命令，一键完成主备库角色转换。主备库切换功能可以确保在软、硬件升级，或系统维护时，提供不间断的数据库服务。

3．自动故障处理

备库故障，不影响主库正常提供数据库服务，守护进程自动通知主库修改实时归档为 Invalid 状态，将实时备库失效。

4．自动数据同步

备库故障恢复后，守护进程自动通知主库发送归档 Redo 日志，重新进行主备库数据同步。并在历史数据同步后，修改主库的实时归档状态为 Valid，恢复实时备库功能。

备库接管后，原主库故障恢复，守护进程自动修改原主库的模式为 Standby，并重新作为备库加入主备系统。

5．备库接管

主库发生故障后，可以通过监视器的 Takeover 命令，将备库切换为主库，继续对外提供服务。如果配置为自动切换模式，确认监视器可以自动检测主库故障，并通知备库接管，

这个过程不需要人工干预。

6．备库强制接管

如果执行 Takeover 命令不成功，但主库可能由于硬件损坏等原因无法马上恢复，为了及时恢复数据库服务，DM 提供了 Takeover Force 命令，强制将备库切换为主库。但需要由用户确认主库故障前，主库与接管备库的数据是一致的（主库到备库的归档是 Valid 状态），避免引发守护进程组分裂。

说明： **执行 Takeover Force 有可能引发组分裂，而 Takeover 命令是在确保不会产生组分裂情况下才允许执行。**

2.4.2 归档流程

实时归档是实时主备数据同步的基础，其流程如下图所示：

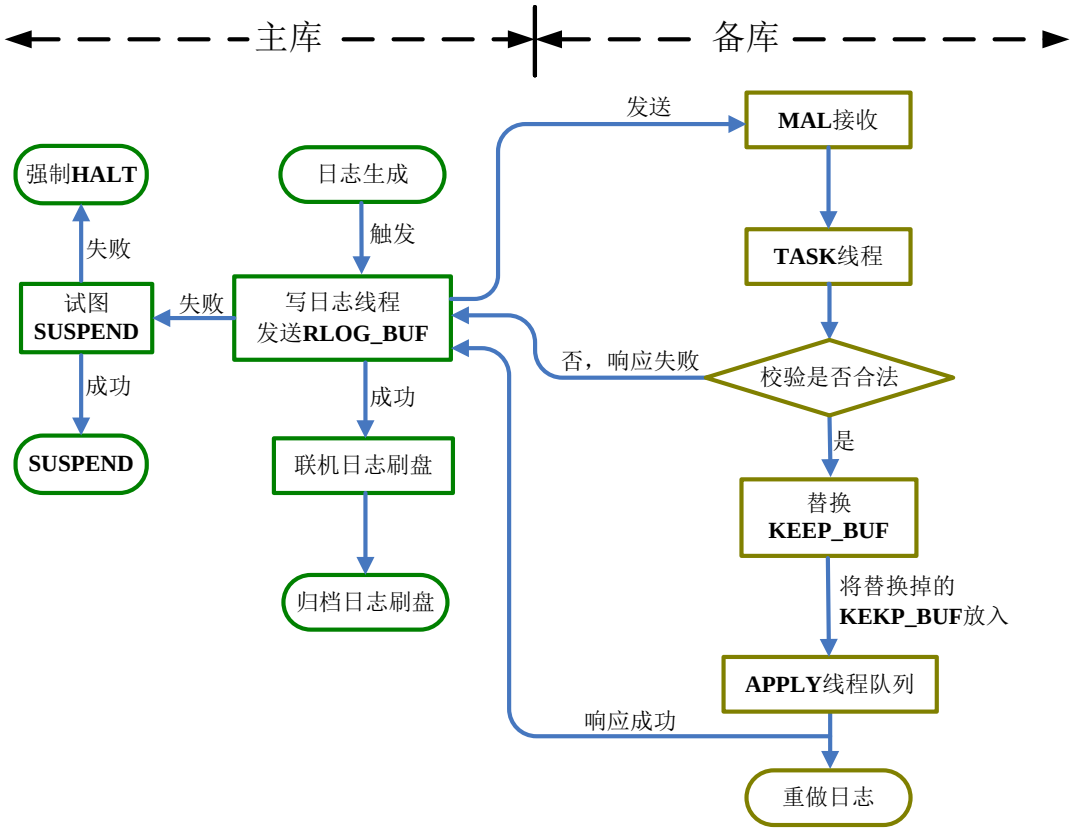


图 2.6　实时归档流程图

主库生成联机 Redo 日志，当触发日志写文件操作后，日志线程先将 RLOG_BUF 发送到备库，备库接收后进行合法性校验（包括日志是否连续、备库状态是否 Open 等），不合法则返回错误信息，合法则作为 KEEP_BUF 保留在内存中，原有 KEEP_BUF 的 Redo 日志

加入 Apply 任务队列进行 Redo 日志重演，并响应主库日志接收成功。

2.5 MPP 主备

MPP 主备是在 DM MPP 集群系统基础上，将每个节点配置为主库，并相应的配置 1 个或者多个备库，每个节点的主、备库构成一个守护进程组，这些守护进程组一起构成了一个 MPP 主备系统。因此，MPP 主备系统包含多个守护进程组，每个守护进程组都是一个单独的、增强版本的实时主备系统，这些守护进程组相互独立，具备实时主备的基本功能，可以进行主备切换、备库接管操作。

MPP 主备的主要目的是为 DM MPP 集群提供数据可靠性保障，备库只做数据容灾、备份，MPP 备库并不是 MPP 集群的一部分，只是某个 MPP 节点（主库）的镜像。MPP 备库不参与 MPP 操作，与其他 MPP 备库之间也没有任何关系，甚至 MPP 备库 dm.ini 配置中的 MPP_INI 参数也是设置为 0 的。因此，MPP 备库只能以单节点方式提供只读服务，但不提供全局的 MPP 只读服务。

MPP 主备系统中，一个守护进程 dmwatcher 可以监控、管理多个守护进程组的数据库实例。一般来说，一台物理机器上，可以部署 1 个 MPP 节点主库和多个 MPP 节点备库，充分利用硬件资源，节省投资。

下图以三个 MPP 节点，每个节点配备两个备库为例，说明 MPP 主备系统的结构。

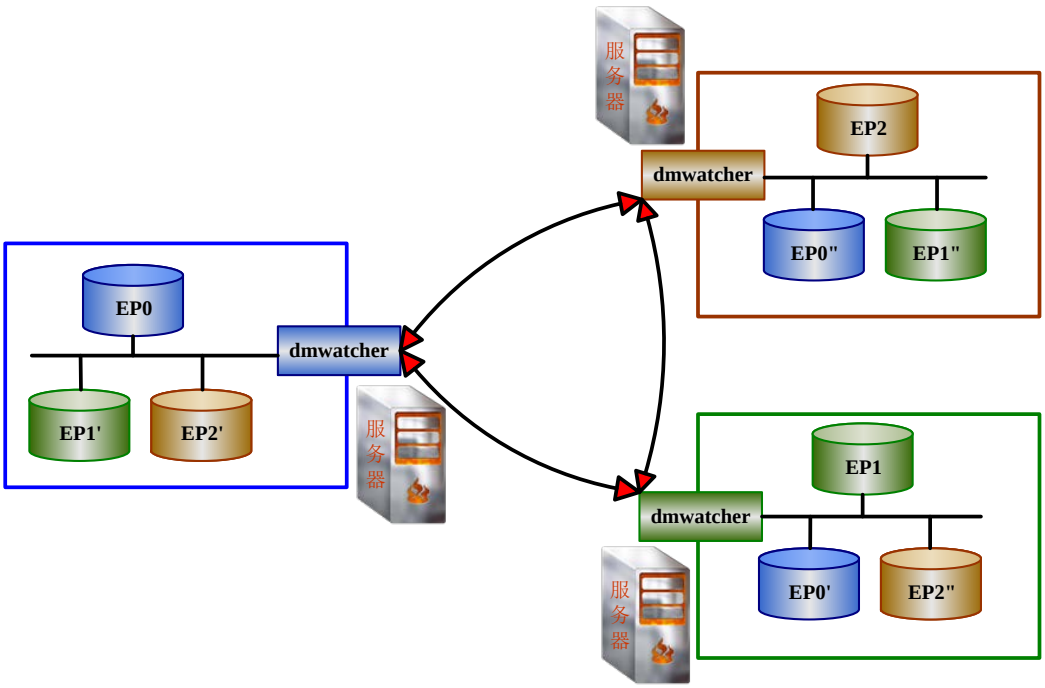


图 2.7 MPP 主备系统图

2.5.1 归档流程

MPP 主备的实现基础是 MPP 归档，MPP 归档流程与实时归档基本一致：主库先将 Redo 日志发送到备库，再写入本地联机 Redo 日志文件中。归档流程参考下图：

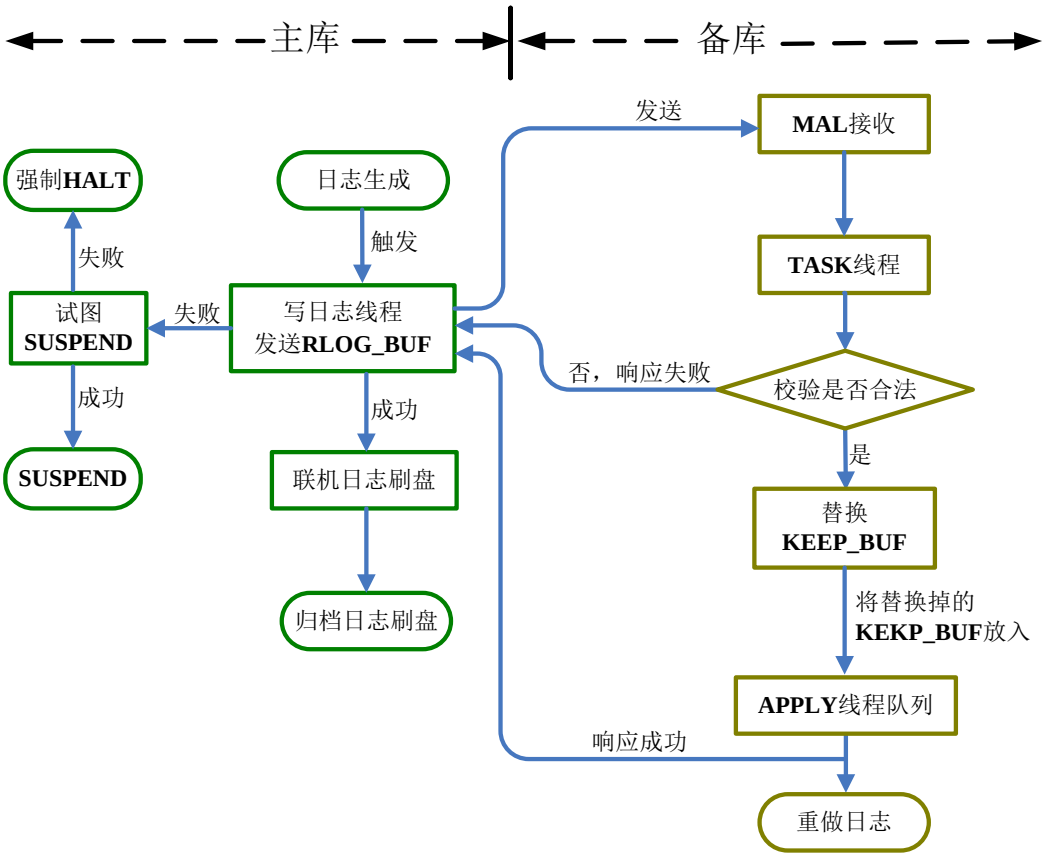


图 2.8 MPP 归档流程图

2.5.2 功能扩展

与实时主备系统相比，MPP 主备扩充了部分功能，包括：

- 支持多个备库　　每个 MPP 节点（主库）可以配置 1~8 个 MPP 备库。
- 支持 Huge 表数据同步　　主库上的 Huge 表修改，可以通过 MPP 归档同步到 MPP 备库。
- 维护 dmmpp.ctl 控制文件　dmmpp.ctl 文件记录 MPP 集群中的节点信息，在主备库切换、或者备库接管后，必须将新的主库信息更新到 dmmpp.ctl 文件中。

表 2.3 实时主备、MPP 主备比较

类型 / 比较	实时主备	MPP 主备
备库数量	1	1~8
归档类型	实时归档	MPP 归档
归档时机	写入联机日志前，发送到备库	写入联机日志前，发送到备库
Huge 表数据同步	不支持	支持
数据来源	RLOG_BUF	RLOG_BUF/HUGE_BUFFER
更新 dmmpp.ctl	否	是

2.5.3 Huge 表日志

Huge 表的存储使用独立的数据文件系统，分别存储在数据文件和辅助表中。数据文件存放数据；辅助表存放控制信息。数据文件不以数据页作为 IO 基本单位，所以数据修改无法产生常规的 Redo 日志；而其辅助表是一个普通的使用数据页存储的表，数据修改会产生 Redo 日志。

数据区（section）是 Huge 表操作的基本单位，Huge 表数据根据行号写入不同数据分区，同一行所有列的数据所属的数据区号必定相同。按照 Huge 表的存储方式，每个数据区都包含数据区数据（section data）和数据区辅助信息（section info），其中，数据区数据来自数据文件，数据区信息来自于辅助表。

Huge 表的刷盘请求，要求将所有列、相同数据区号的内容都写入磁盘。每一个数据区刷盘，包含两个动作：1）数据区写入数据文件；2）写入辅助表信息。所以，Huge 表的归档流程与一般归档流程不同，包括以下步骤：

1．数据区内容写入磁盘前，先通过 MAL 系统发送给备库，并等待备库响应。

2．备库收到数据区内容后，先缓存到内存中，再响应主库。

3．主库收到备库响应后，将数据区内容写入磁盘。

4． 主库发送所有列数据区内容到备库、并写入本地磁盘后，生成一个特殊的 Redo 日志，标记 Huge 表数据刷盘动作。

5．主库更新辅助表信息，由于辅助表是普通表，因此更新过程正常生成 Redo 日志。

6． 主库触发 Redo 日志刷盘，先将日志发送到备库，等待备库响应后再写入本地联机 Redo 日志文件中。

7．备库重做 Huge 表数据刷盘 Redo 日志时，将之前缓存的数据区内容写入磁盘。

8．随后重做更新辅助表的 Redo 日志。

同步 Huge 表历史数据的流程也存在一定差异：

1．备库初始化环境，装载缺失的数据信息，清理所有数据区数据缓存。

2．主库发送本地归档日志。

3． 备库重做 Huge 数据刷盘日志时，缓存对应 Huge 表 ID、数据区号等信息；辅助表相关的 Redo 日志正常重做。

4．主库同步历史归档完成后，守护进程增加了一个步骤，通知主库发送 Huge 表数据。

1） 主库从备库获取缺失的 Huge 表 ID、数据区号等信息。

2） 主库根据这些信息，读取对应的数据区，并发送给备库。

3） 对于每个数据区，备库接收到数据区后，将其写入到对应的数据文件中，写入完成后发送确认消息给主库；主库在收到确认消息后再发送下一个数据区。

4） 主库完成所有数据区的数据发送后，通知备库数据发送完成。

5） 备库接收到数据发送完成的消息后，清理缺失的数据信息。

6） 主库通知守护进程 HFS 数据文件的恢复完成。

5．数据同步完成。

在同步数据过程中，备库在重做 Huge 数据刷盘日志时缓存的 Huge 表 ID、数据区号等信息只有在恢复完成后才会清理。如果在同步数据过程中备库再次故障，这些缓存没有被清理，备库重启后会装载这些数据信息等待恢复。

2.5.4 dmmpp.ctl 维护

DM MPP 集群中，每个节点都是一个独立数据库实例，他们之间的联系信息记录在 dmmpp.ctl 控制文件中。每个节点都保存一个 dmmpp.ctl 文件，并要求文件内容在所有节点都是完全一致的。

dmmpp.ctl 里的信息主要包括：系统状态、MPP 总节点数、文件校验码、故障节点数、故障节点序号、配置项名、实例名、实例节点序号，以及根据节点数和实例序号生成的哈希映射数组等。

当某个节点的守护进程组执行主备切换后，需要修改 dmmpp.ctl 中对应实例的信息，

将新主库实例名替换原主库实例名。否则，其他 MPP 节点仍然会将信息发送到原主库；但主备切换后，原主库已经变成 Standby 模式，Insert/Delete/Update 等非只读操作将会报错。因此 MPP 主备系统中，任何 MPP 节点主库信息发生变化后，必须相应的修改全部 MPP 节点上的 dmmpp.ctl 文件。

对整个 MPP 集群来说，MPP 节点主备切换就是某个节点的主实例信息名发生变化。比如 0 号节点发生主备切换，只要在 dmmpp.ctl 中将 0 号节点对应的 mpp_inst_name 修改为新的主库实例名，并修改新 MPP 主节点 dm.ini 配置项 MPP_INI 为 1。

下面根据图 2.7 MPP 主备系统图，举例说明一个节点主备切换时相应的控制文件是如何变化的。由于 dmmpp.ctl 是二进制文件，为了便于识别，下面列出的是对应转换的文本格式内容。原始配置如下：

```
[service_name1]                 #配置项名
    mpp_seq_no      = 0         #节点序号
    mpp_inst_name   = EP0       #节点实例名
[service_name2]
    mpp_seq_no      = 1
    mpp_inst_name   = EP1
[service_name3]
    mpp_seq_no      = 2
    mpp_inst_name   = EP2
```

将 EP0 和 EP0'切换后，dmmpp.ctl 中对应的实例信息如下，可以看出配置项中只有节点实例名 EP0 变化为 EP0'了，节点序号以及其他配置项的内容都保持不变：

```
[service_name1]                 #配置项名
    mpp_seq_no      = 0         #节点序号
    mpp_inst_name   = EP0'      #节点实例名
[service_name2]
    mpp_seq_no      = 1
    mpp_inst_name   = EP1
[service_name3]
    mpp_seq_no      = 2
    mpp_inst_name   = EP2
```

MPP 集群中用户登录某个节点创建一个主会话连接（session)时，该节点通过 MAL 系统会主动发起到集群中其他所有节点的登录请求，在其他所有节点上建立对应的镜像会话(msession)，镜像会话和主会话会一直保持连接。当主会话断开连接，会通知所有镜像会话释放。或者当一个镜像会话释放（比如节点故障），主会话节点能够检测到并通知其他镜像会话全部释放，主会话再断开。

因此当 MPP 主备切换时，由于主节点发生变化，该节点上存在的所有主会话和镜像会话都要被释放，会将整个 MPP 集群的所有连接强制断开，切换过程中同时会阻止新的登录请求。主备切换完成后，恢复正常状态，接受登录请求，新连接会登录到新的主节点上。

2.6 读写分离集群

读写分离集群是基于即时归档实现的高性能数据库集群，不但提供数据保护、容灾等数据守护基本功能，还具有读写操作自动分离、负载均衡等特性。和 MPP 主备一样，读写分离集群最多可以配置 8 个备库，提供数据同步、备库故障自动处理、故障恢复自动数据同步等功能，也支持自动故障切换和手动故障切换两种守护模式。

一般情况下，应用系统中查询等只读操作的比例远大于 Insert/Delete/Update 等 DML 操作，修改对象定义等 DDL 操作的比例则更低。但是，这些操作往往混杂在一起，在高并发、高压力情况下，会导致数据库性能下降，响应时间变长。借助读写分离集群，将只读操作自动分发到备库执行，可以充分利用备库的硬件资源，降低主库的并发访问压力，进而提升数据库的吞吐量。

读写分离集群不依赖额外的中间件，而是通过数据库接口与数据库之间的密切配合，实现读、写操作自动分离特性。DM 的 JDBC、DPI、DCI、ODBC、Provider 等接口都可以用来部署读写分离集群。

根据是否满足读提交事务隔离级特性，读写分离集群可以配置为事务一致模式和高性能两种模式。简单的说，事务一致模式下，不论一个 Select 语句是在备库执行、还是在主库执行，其查询结果集都是一样的。高性能模式则不能保证查询是一致的，备库的数据与主库的数据同步存在一定的延迟，当 Select 语句发送到备库执行时，返回的有可能是主库上一个时间点的数据。

2.6.1 归档流程

读写分离集群的实现基础是即时归档。即时归档与实时归档、MPP 归档存在一定差异：

1. 主库先将日志写入本地联机 Redo 日志文件中，再发送 RLOG_BUF 到备库。

2. 备库日志重演时机有两种选择：

   - 事务一致模式　要求备库在重演 Redo 日志完成后再响应主库。
   - 高性能模式　与实时归档、MPP 归档一样，收到 Redo 日志后，马上响应主库。

3. 即时归档的同步机制，可以保证备库的 Redo 日志不会比主库的 Redo 日志多，因此即时备库不需要 KEEP_BUF，收到 RLOG_BUF 直接加入到 Apply 任务系统，启动 Redo 日志重演。

4. 备库故障、或者主备库之间网络故障，导致发送 RLOG_BUF 失败后，主库马上修改即时归档为 Invalid 状态，并切换数据库为 Suspend 状态。

5. 即时归档修改为 Invalid 状态后，会强制断开对应该备库上存在影子会话的用户会话，避免只读操作继续分发到该备库，导致查询数据不一致。

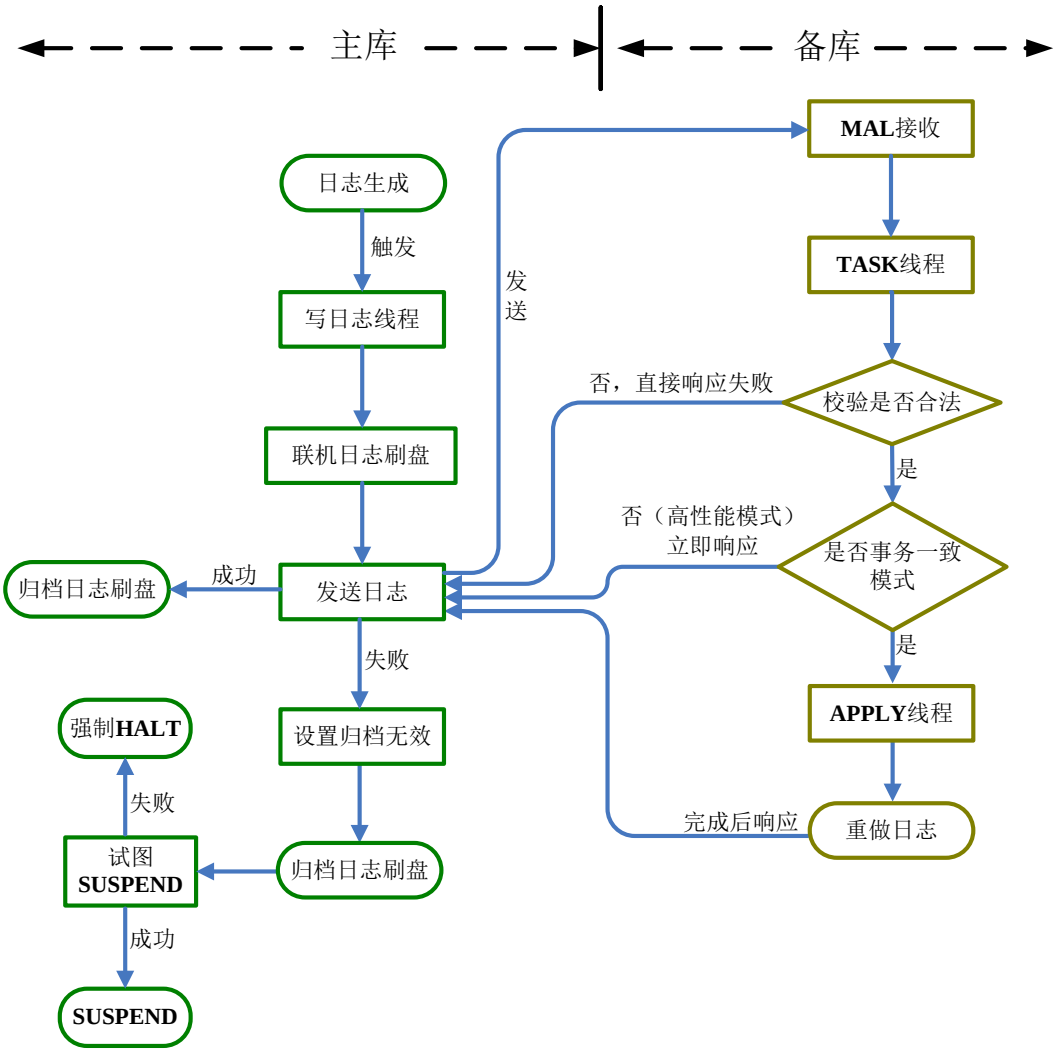


图 2.9 即时归档流程

2.6.2 实现原理

实现读写分离集群的基本思路是：利用备库提供只读服务、无法修改数据的特性，优先将所有操作发送到备库执行，一旦备库执行报错，则发送到主库重新执行。通过备库“试错”这么一个步骤，自然地将只读操作分流到备库执行。并且，备库“试错”由接口层自动完成，对应用透明。

读写分离集群数据库连接创建流程：

1. 用户发起数据库连接请求。
2. 接口（JDBC、DPI 等）根据服务名配置(dm_svc.conf)登录主库。
3. 主库挑选一个有效即时备库的 IP/Port 返回给接口。
4. 接口根据返回的备库 IP 和 Port 信息，向备库发起一个连接请求。

5． 备库返回连接成功信息。

6． 接口响应用户数据库连接创建成功。

接口在备库上创建的连接，是读写分离集群自动创建的；对用户而言，就是在主库上创建了一个数据库连接。下面以配置了两个备库的读写集群为例，说明读写分离集群的连接创建流程。

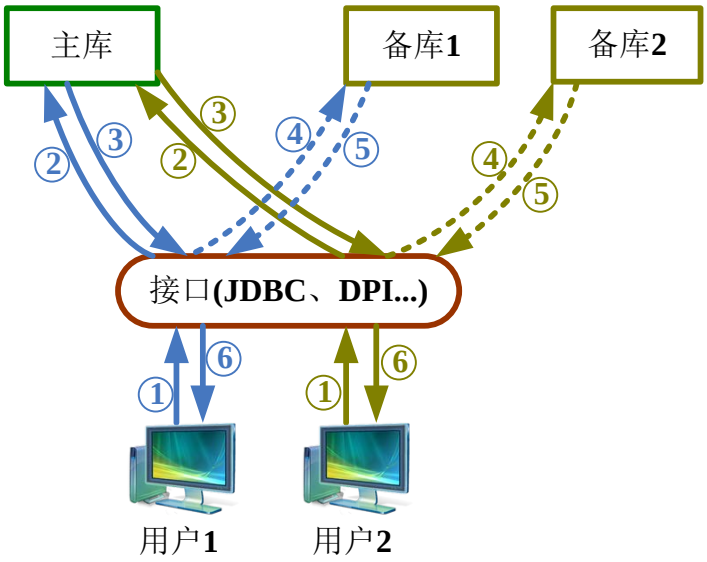


图 2.10 读写分离连接创建

读写分离集群语句分发流程：

1． 接口收到用户的请求。

2． 接口优先将 SQL 发送到备库执行。

3． 备库执行并返回执行结果。如果接口收到的是备库执行成功消息，则转到第 6 步，如果接口收到的是备库执行失败消息，则转到第 4 步。

4． 重新将执行失败的 SQL 发送到主库执行。只要第 3 步中的 SQL 在备库执行失败，则同一个事务后续的所有操作（包括只读操作）都会直接发送到主库执行。

5． 主库执行并返回执行结果给接口。一旦主库上执行的写事务提交，则下次继续从第 1 步开始执行。

6． 接口响应用户并将执行结果返回给用户。

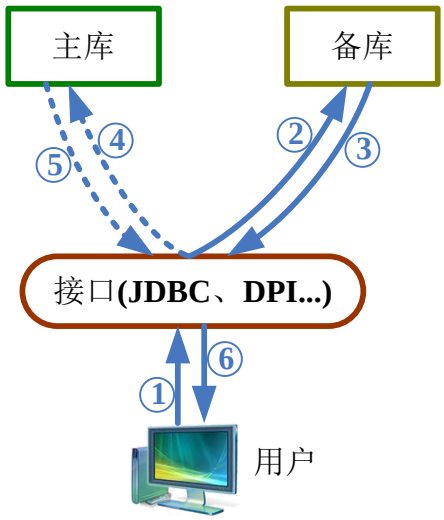


图 2.11 读写分离集群语句分发流程

执行举例说明：

```
--数据准备
create table t(c1 int);
--事务开始
select * from t;                    --首先在备库上执行
insert into t values(1);            --写操作转移到主库上执行
select * from t;                    --事务未提交，还在主库上执行
commit;                             --事务提交
select * from t;                    --事务已提交，重新转移到备库上执行
```

2.6.3 事务一致性

读写分离集群通过 JDBC、DPI 等接口自动分发语句，一个事务包含的多个语句可能分别在备库和主库上执行，但执行结果与单独在一个数据库实例上完全一致，满足读提交事务隔离级特性。

下面以两段伪代码 trx1 和 trx2 为例，说明读写分离的特性和如何实现事务隔离级。

```
trx1:
   update t set c1 = 2;
   commit;
```

```
trx2:
   rs = select c1 from t;
   fetch c1 from rs into var_x;
   insert into tx values(var_x);
```

```
commit;
```

根据读写分离特性，trx1 的 update 在主库执行；trx2 的 select 语句在备库执行，insert 语句转到主库执行，并且 trx2 的 insert 语句的插入值，是从之前执行的 select 结果集中获取。

下面根据即时归档流程，结合 trx2 执行 select 语句时机和以及 trx1 的不同状态进行讨论，详细地说明读写分离集群是如何实现提交事务隔离级别的。

第一种情况，trx2 执行 select 时，trx1 的 commit 还未执行。

trx2 的 select 语句，不管是在主库还是备库执行，查询结果都是 trx1 更新 t 表之前的值，var_x = 1。

第二种情况，trx2 执行 select 时，trx1 的 commit 已经执行完成。

trx2 的 select 语句，不管是在主库还是备库执行，查询结果都是 trx1 更新 t 表后的值，var_x = 2。

第三种情况，trx2 执行 select 时，trx1 正在执行 commit。

trx2 的 select 查询结果，与两个语句在系统内部的执行顺序有关，var_x 的值可能是 1 或者 2。但由于 trx1 的 commit 并没有明确响应用户，var_x 的最终值取决于数据库管理系统的实现策略，不管返回 1 还是 2，都符合读提交事务隔离级。

为了保证主备库上的一致性，目前读写分离集群有下列一些类型的语句不会在备库上执行，都在主库上执行：

1. 设置会话、事务为串行化隔离级语句。
2. 表对象上锁语句（lock table xx in exclusive mode）。
3. 查询上锁语句（select for update）。
4. 备份相关系统函数（由于备份文件放在主库上，备库查询不到）。
5. 自治事务操作（内部可能有提交操作）。
6. 包操作（比如有些操作将文件创建在备库上，主库上后续找不到）。
7. 动态视图查询（和上下文相关，主备实例上可能不一样）。
8. 设置自增列操作语句（SET IDENTITY_INSERT TABLE ON）。
9. 临时表查询（临时表不产生日志）。
10. 访问@@IDENTITY、@@ERROR 等全局变量。
11. SF_GET_PARA_STRING_VALUE、SF_GET_PARA_DOUBLE_VALUE 等函数。


说明：

读写分离集群中，当一个 SQL 从备库切换到主库执行时，主库会启动一个新的事务，主库事务与备库事务没有任何联系，事务 ID 也完全不同。备库事务 ID 与主库事务 ID 分配机制并不相同，主库的事务 ID 取值范围是[1 ~ 0x7FFFFFFFFFF]，备库事务 ID 取值范围是[0x800000000000 ~ 0xFFFFFFFFFFF]；备库事务 ID 是一个内存值，每次重启后都从 0x800000000000 开始重新分配；主库事务 ID 是一个物理值，一旦分配后，就不会再重复分配。

2.6.4 性能调整

根据读写分离语句分发流程可以发现，当一个应用系统中，只读事务占绝大多数情况下，可能出现备库高负载、高压力，主库反而比较空闲的情况。为了实现负载均衡，更好的利用主备库的硬件资源，JDBC 等数据库接口提供了配置项，允许将一定比例的只读事务分发到主库执行。因此，用户应该根据主备库的负载情况，灵活调接口的分发比例 rwPercent 配置项，以获得最佳的数据库性能。

备库数量是影响读写分离集群性能的一个重要因素，备库越多则每个备库需要承担的任务越少，有助于提升系统整体并发效率。但同时要考虑另外一个因素，事务一致模式下，主库要等所有备库重演 Redo 日志完成后，才能响应用户；虽然备库的 Redo 日志重演是并行进行的，但 Redo 日志是串行发送，因此随着备库的增加，即时归档时间会变长，最终降低非只读事务的响应速度。因此，部署多少备库，也需要综合考虑硬件资源、系统性能等各种因素。

配置为高性能模式，则是提升读写分离集群的另一个有效手段。如果应用系统对查询结果实时性要求并不太高，并且事务中修改数据的操作也不依赖同一个事务中的查询结果。那么，通过修改 dmarch.ini 中的 ARCH_WAIT_APPLY 配置项为 0，将读写分离集群配置为高性能模式，可以大幅提高系统整体性能。如果应用包含以下代码逻辑，则不适合使用高性能模式：

```
--tx1 事务开始
insert into t values(1);            --写操作在主库上执行
commit;                             --事务提交
```

```
--tx2 事务开始
select top 1 c1 into var1 from t;    --tx1 事务已提交，select 操作重新转移到备库上执行。高性能模式下备库可能还没有重做日志，查不出 tx1 中插入的结果
update t set c1 = var1 + 1 where c1 = var1;    --更新不到数据
```

此外，根据读写分离特性，合理的规划应用的事务逻辑，也可以获得更佳的性能：

1. 尽可能将事务规划为只读事务和纯修改事务，避免无效的备库试错。

2. 读操作尽量放在写操作之前，用备库可读的特点来分摊系统压力。

2.7 异步备库

异步备库一般用于历史数据统计、周期报表等对数据实时性要求不高的业务场合。异步归档时机可以选择在源库空闲的时候，可避免源库的业务高峰期同步数据对性能的影响。

每个 Primary 或者 Standby 模式的库，都可以配置最多 8 个异步备库。配置了异步备库的 Primary 或者 Standby 模式的库，统称为源库。可以在实时主备、MPP 主备和读写分离集群的主库和备库上配置异步备库，异步备库可级联配置，异步备库本身也可以作为源库配置异步备库。理论上守护系统中可配置的异步备库的数目只受 MAL 系统最大节点数（2048）的限制。

一个异步备库只能有一个源库，这个限制由用户在配置时保证。配置异步备库十分简单，确保源库和异步备库之间的 MAL 链路是正常的，在源库的 dm.ini 中打开定时器 TIMER_INI 开关，同时配置文件 dmtimer.ini，由定时器定时触发源库发送归档日志到异步备库即可。异步归档的最小触发间隔是 1 分钟。

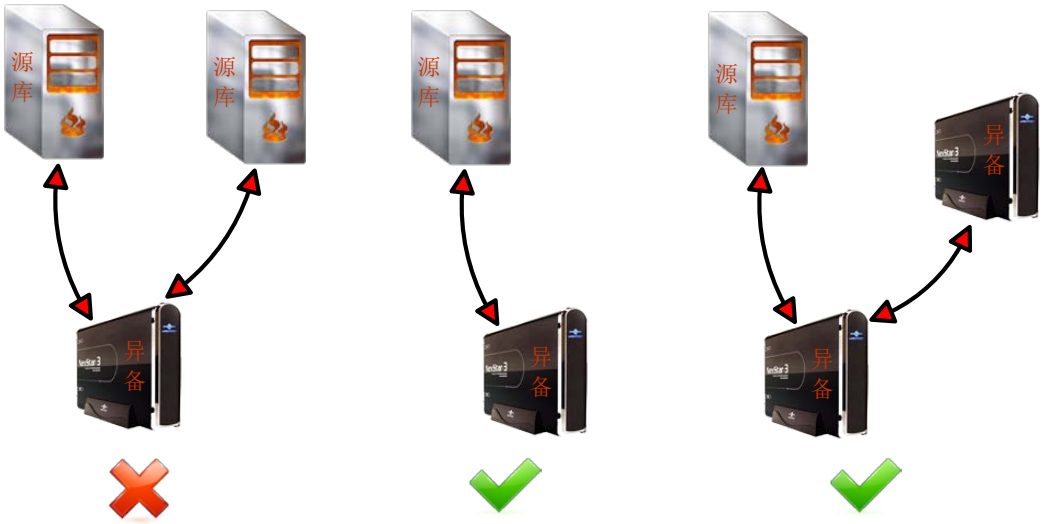


图 2.12 错误/正确/级联异步备库配置

3 守护进程

守护进程（dmwatcher）是 DM 数据守护系统不可或缺的核心部件，是数据库实例和监视器之间信息流转的桥梁。数据库实例只向本地守护进程发送信息，只接收本地守护进程的消息和命令；监视器（dmmonitor）接收守护进程的消息，并向守护进程发送命令；数据库实例与监视器之间没有直接的消息交互；守护进程解析并执行监视器发起的各种命令（Switchover/Takeover/Open force 等），并在必要时通知数据库实例执行相应的操作。

3.1 主要功能

守护进程是管理数据守护系统的核心部件，监视器（dmmonitor）负责发起命令，守护进程负责解析、处理命令。守护进程提供了数据库监控、故障检测、故障处理、故障恢复等各种功能。

3.1.1 监控数据库实例

守护进程负责监控数据库运行状态，必要时重启数据库服务。守护进程和实例链路建立成功后，数据库实例定时发送信息到守护进程，发送到守护进程的内容包括：实例进程 ID、实例名、数据库模式、数据库状态、FILE_LSN、CUR_LSN、MAL 链路状态、归档状态、公钥、MPP 控制文件等信息。

守护进程更新本地记录的实例信息后，同时记录该时间戳。当检测到实例进程 ID 已经不存在或者超过一段时间没有收到实例消息（INST_ERROR_TIME），则会认定实例故障。如果配置了自动重启，则会将实例重启。


注意：

守护进程采用超时机制判断实例是否故障，即当前时间和上次收到消息的时间差是否超过故障认定时间（INST_ERROR_TIME），因此不建议在数据守护系统运行过程中调整操作系统时间，避免导致这个差值很大，误判实例故障。

3.1.2 发送状态信息

守护进程将监控的数据库实例信息和守护进程自身的信息（包括守护类型、守护模式、守护状态、守护日志、监视器执行序列号、执行返回码等）捆绑在一起，定时发送给其他守护进程和所有监视器。

3.1.3 监控其他守护进程

接收并解析其他守护进程发送的消息，如果超过一段时间（DW_ERROR_TIME)没有收到远程守护进程消息，会将远程守护进程状态认定为 ERROR 状态。另外还会结合本地数据库信息和守护进程自身状态，切换数据库的运行模式和系统状态。



注意：守护进程采用超时机制判断远程守护进程是否故障，即当前时间和上次收到消息的时间差是否超过故障认定时间（DW_ERROR_TIME），因此不建议在数据守护系统运行过程中调整操作系统时间，避免导致这个差值很大，误判远程守护进程故障。

3.1.4 接收监视器消息

主备切换、实例接管等操作都是通过监视器命令进行，监视器将操作命令分解成多个步骤顺序执行。守护进程接收这些消息，通知实例进行相应操作，例如执行 SQL 语句修改实例模式、状态、INI 参数、设置归档状态等一系列动作，可实现主备库切换、备库接管等操作。

例如，主备切换操作，监视器首先通知待切换主备库的守护进程修改状态为 Switchover 状态，设置成功以后，其他监视器将不能再进行命令操作。守护进程收到监视器将实例 Mount 的命令，转发到本地实例执行，实例执行完成后返回执行结果。执行结果包含在实例向守护进程发送的消息中，守护进程根据消息中的执行码进行判断是否成功，再响应到监视器上。


注意：**监视器和守护进程之间也是采用超时机制对方是否故障，即当前时间和上次收到消息的时间差是否超过故障认定时间（守护进程配置的 DW_ERROR_TIME），因此不建议在数据守护系统运行过程中调整操作系统时间，避免导致这个差值很大，误判监视器故障。**

3.1.5 主备库启动运行

数据守护系统刚启动时，所有实例处于 Mount 状态，守护进程处于 Startup 状态，启动时需要将实例转换到 Open 状态，守护进程也切换到 Open 状态，对外提供服务。

启动条件和流程详见数据守护使用说明章节中的数据守护启动流程 6.2 数据守护的启动 。

3.1.6 备库故障处理

故障自动切换模式下，备库故障后，如果主备库之间的归档状态仍然有效，主库的守护进程会先切换为 Confirm 状态，等待确认监视器的确认消息，如果确认为符合故障处理条件，主库守护进程再切换至 Failover 状态，将备库的归档失效。

故障手动切换模式下，备库故障后，如果主备库之间的归档状态仍然有效，会直接切换到 Failover 状态，直接将备库的归档失效，不需要备库故障确认。

备库故障重启后，如果存在活动主库，主库守护进程根据备库实例的模式、状态、备库守护进程状态、守护进程控制文件信息、以及备库的恢复间隔信息判断，是否可以进行故障恢复，在满足故障恢复条件情况下，启动 Recovery 流程，重新恢复主备库到一致状态。

读写分离集群下，主库向即时备库发送归档失败后，会直接修改归档状态无效，并将数据库修改为 Suspend 状态。


说明：**如果主备库之间网络故障期间，主库没有修改操作触发归档日志发送，主库会一直保持 Open 状态。**

如果网络故障期间备库接管，网络恢复后，dmwatcher 会通知主库强制 Halt。

3.1.7 主库故障处理

故障自动切换模式下，主库故障后，确认监视器会捕获到故障信息，自动选出可接管的备库，并通知备库进行接管。备库接管由确认监视器自动触发，无需用户干预。

故障手动切换模式下，主库故障后，需要人工干预，通过监视器发起接管命令，将可接管备库切换为主库。

故障自动切换模式数据守护，可以实时处理故障，但对网络稳定性要求更高，需要确保主、备库之间，主、备库与守护进程之间的网络稳定可靠，否则可能会误判主库故障，备库自动接管后，出现多个 Open 状态的主库，引发脑裂。故障手动切换模式，备库不会自动接管，出现节点故障、或者网络故障时，由用户根据各种故障情况，进行人工干预。因此，我们认为故障手动切换模式，可以更好的保护数据一致性，建议尽量使用故障手动切换模式的数据守护。

主库故障重启后，守护进程根据控制文件中的接管记录，以及是否存在其他 Primary 实例来判定重启后的恢复策略。可能重新作为主库加入守护系统，也可能修改为 Standby 模式，以备库身份重新加入守护系统，如果出现组分裂，则需要用户干预才可以重加入守护系统。

3.1.8 故障恢复处理

故障恢复状态（Recovery）是守护进程自行判断，和监视器无关。如果符合以下条件，主库的守护进程可自动进入 Recovery 状态，进行数据恢复：

1. 本地主库 [Primary,Open]，守护进程 Open 状态
2. 远程备库[Standby,Open]，归档状态 Invalid，守护进程 Open 状态
3. 控制文件判断可加入，本地 LSN 更大等
4. 远程备库[Standby,Open]达到了设置的启动恢复的时间间隔

对于第 4 点，可通过配置主库守护进程 dmwatcher.ini 的 INST_RECOVER_TIME 来控制，取值(0s~ 86400 s)，为 0 时表示满足前 3 个条件就会立即启动恢复流程，该值对所有备库有效，可通过监视器的 show arch info 命令查看当前设置的到指定备库的恢复时间间隔。

可以使用监视器的 set recover time 命令来动态设置指定备库的恢复间隔，该命

令只会修改命令中指定的备库的恢复间隔，并且只保存在主库的守护进程内存中，不会写入到 dmwatcher.ini 文件。

在以下场景中，主库的守护进程会重置内存中的 INST_RECOVER_TIME 为 0，对满足前 3 个条件的备库会立即启动恢复流程：

1. 数据守护系统启动完成之后。
2. 守护系统运行过程中，主库的手动重启或者守护进程自动启动 Open 之后。
3. 监视器执行 Switchover 主备切换/Takeover 备库接管/强制 Open 主库的操作之后。

故障恢复流程如下：

1. 同步守护控制文件，发送 dmwatcher.ctl 到备库上。
2. 通知备库丢弃 KEEP_BUF。
3. 通知主库发送归档日志，同步历史数据。
4. 通知主库 SUSPEND。
5. 再次通知主库发送归档日志。
6. 主库发送 HUGE 表数据（MPP 主备）。
7. 通知主库设置备库归档有效 VALID。
8. 通知主库 OPEN


注意：

以下情况会导致 Recovery 过程中断：

1. Recovery 过程中收到监视器的命令

2. 存在多个备库时，Recovery 过程中发现其他备库故障，且符合 failover 条件，则中断 Recovery，先做 failover 处理。

3.2 守护类型

守护进程支持 4 种守护类型：

- 本地守护

提供最基本的守护进程功能，监控本地数据库服务，如果连续一段时间没有收到来自其监控数据库的消息，即认定数据库出现故障，根据配置确定是否使用配置的启动命令，重启数据库服务。异步备库也是采用这种方式配置。

- 实时主备

该类型只针对使用实时归档类型的主备库。在本地类型的基础上，通过和远程守护进程的交互，增加了 Open 主备数据库、切换主备模式、主备库故障检测、恢复等功能。

- 读写分离

该类型只针对使用读写分离即时归档类型的主备库。在本地类型的基础上，通过和远程守护进程的交互，增加了 Open 主备数据库、切换主备模式、主备库故障检测、恢复等功能。

- MPP 主备

该类型只针对使用 MPP 归档类型的主备库。在本地类型的基础上，通过和远程守护进程的交互，增加了 Open 主备数据库、切换主备模式、主备库故障检测、恢复等功能。

3.3 守护模式

守护进程支持 2 种故障切换模式：

- 故障自动切换

主库发生故障时，确认监视器自动选择一个备库，切换为主库对外提供服务。故障自动切换模式，要求必须配置，且只能配置一个确认监视器。

- 故障手动切换

主库发生故障时，由用户根据实际情况，通过监视器命令将备库切换为主库。在用户干预之前，备库可以继续提供只读服务和临时表的操作。

实时主备/MPP 主备/读写分离集群都可以配置为故障自动切换或故障手动切换模式，这两种模式下守护系统的启动流程、数据同步和故障处理机制存在一定的差异，主要差异参考表 3.1。

表 3.1 自动、手动切换比较

比较内容	故障自动切换	故障手动切换
硬件要求	>=3 台机器	>=2 台机器
主库故障需要人工干预	否	是
备库 KEEP_BUF	有	有
需要确认监视器	是	否
支持实时主备	是	是
支持 MPP 主备	是	是

支持读写分离集群	是	是
主库故障处理	备库自动接管	备库手动接管
备库故障处理	主库先进入 Confirm 状态，向确认监视器求证备库故障后，再进行 Failover 处理	主库直接 Failover 处理
主备库切换	支持	支持



一个数据守护集群，只能配置一个确认监视器。

说明：如果同时启动多个确认监视器，后启动的确认监视器将报错并自动退出。

3.4 守护状态

守护进程包括以下一些状态：

- **startup** 守护进程启动状态，需要根据远程守护进程发送的状态信息，结合本地数据库的初始模式、状态和数据同步情况，确定本地数据库的启动模式和状态后，进入 open 状态。
- **open** 守护进程正常工作，监控数据库，并定时发送数据库的状态信息，接收其他守护进程发送的信息，接收监视器发送的用户请求。
- **shutdown** 守护进程停止监控数据库状态，也不提供主备库切换功能。
- **switchover** 主备都正常的情况下，手动切换主备过程中设置为 switchover 状态。
- **failover** 远程备库故障后，本地主库执行故障处理时，守护进程设置为 failover 状态。
- **recovery** 故障恢复同步历史数据过程中设置为 recovery 状态。
- **confirm** 通过监视器确认远程主（备）库是否活动的过程中，守护进程设置为 confirm 状态。
- **takeover** 主库确认故障后，备库手工接管或监视器通知自动接管过程中，守护进程设置为 takeover 状态。
- **open force** 主库没有收到远程所有实例消息，或组中没有活动主库，需要借助监视器命令强制 open 主库或备库实例时，守护进程设置为 open force 状态。
- **error** 超过一段时间（DW_ERROR_TIME）没有接收到远程守护进 程消息，本地

守护进程或监视器认定远程守护进程故障，修改远程守护进程为 error 状态。

- **login check** 监视器执行命令登录校验时，守护进程所处的状态。
- **mppctl update** 修改主库 MPP 控制文件（dmmpp.ctl）时，守护进程所处的状态，只在 MPP 主备系统出现。
- **change arch** set arch invalid 命令执行时守护进程所处的状态。

守护进程所有状态变换和它监控的数据库的状态变换都会生成相应的 LOG 信息，写入到../log 目录中以'dm_实例名_当前年月.log'和'dmwatcher_实例名_当前年月.log'方式命名的日志文件中。用户可以通过查看日志文件，分析实例和守护进程的运行状态、监控故障处理过程。

守护进程的状态转换入下图所示：

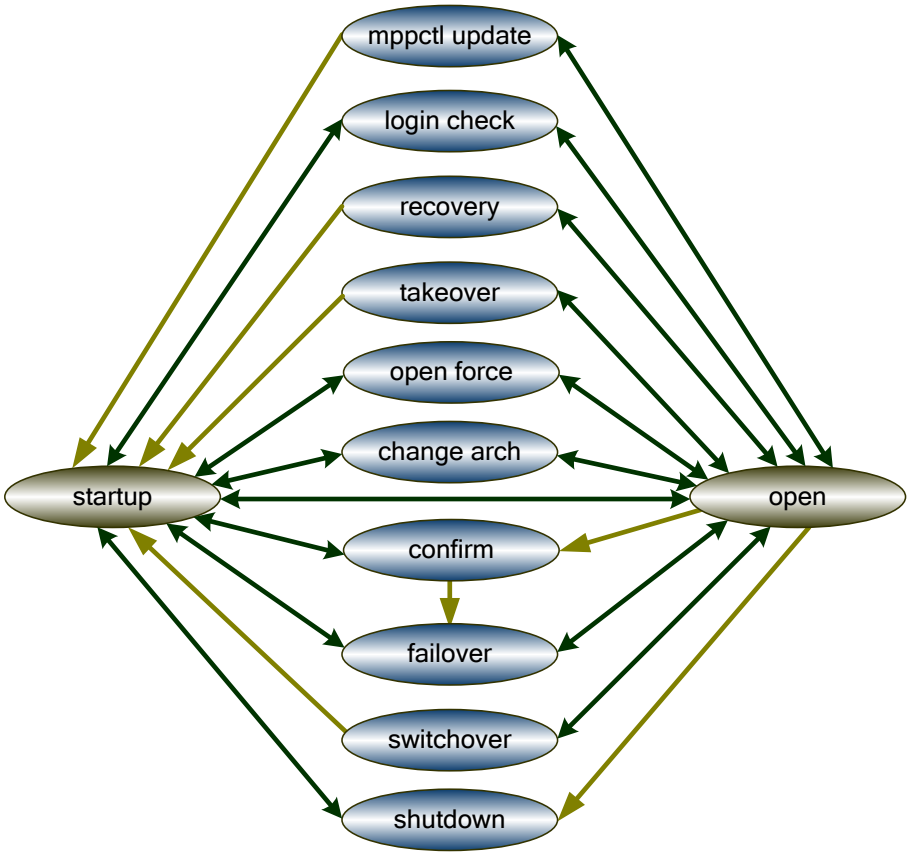


图 3.1 守护进程状态转换图

从图中可以看出，守护系统主要工作在 startup 和 open 状态，几乎任何状态都可能转到这两种状态，并且这两种状态之间也可以相互转换。

另外，当远程守护故障，任何状态都可转到 error 状态。

3.5 控制文件

3.5.1 概述

守护进程控制文件（dmwatcher.ctl）记录数据守护系统运行过程中守护进程组内主库变迁历史信息，是根据守护 V2.0 版本引入的一个新特性。同一个守护进程组内，所有 dmwatcher.ctl 文件的内容保持一致，不同守护进程组的 dmwatcher.ctl 没有任何关联关系。Local 类型的守护进程不需要配置 dmwatcher.ctl，MPP/Realtime/Timely 类型的守护进程都必须配置 dmwatcher.ctl 文件。

在引入控制文件之前，为了保证主备库数据一致性，备库接管的条件十分严格，类似 2.3.1 中备库接管章节。这些限制，导致很多主库故障场景，监视器的 takeover 命令报错，只能通过一系列 SQL 进行备库强制接管。并且，用户还要记录备库接管时，主备库的 LSN 信息，待故障主库恢复后，用户要根据这些信息，判断故障主库是否可以直接作为备库、重新加入数据守护系统。

引入控制文件后，守护进程根据 dmwatcher.ctl 记录的信息，判断故障主库是否可以直接作为备库、重新加入数据守护系统，如果故障主库的 FILE_LSN > 接管备库的 TAKEOVER_LSN，守护进程会将故障主库的 dmwatcher.ctl 修改为 split 状态，避免将无法保证数据一致性的库加入到数据守护系统中。

3.5.2 作用

控制文件作用如下：

1. 记录数据库模式变化过程
2. 判断实例是否能够加入当前主备系统
3. 判断系统是否出现分裂情况
4. 降低用户使用数据守护复杂度
5. 提供一些更加实用、更加强大的功能特性：
   - 在无法满足自动 Open 情况下，可以通过 Open Force 命令强制 Open 主库。
   - 在主库故障，但备库不满足接管条件时，可以使用 Takeover Force 命令，及时恢复数据库服务。

3.5.3 内容

守护进程控制文件 dmwatcher.ctl 存放在实例的数据文件目录下。dmwatcher.ctl 是由 dmwatcher.ini 文件转换生成的，使用 dmctlcvt 工具。举例如下：

```
./dmctlcvt t2dwctl /dm7/data/DAMENG/dmwatcher.ini /dm7/data/DAMENG
```

控制文件的内容可以通过 dmctlcvt 工具转换成文本来查看，也可以通过 dmmonitor 工具的 show dmwatcher ctl 命令在控制台上查看。举例如下：

```
dmctlcvt dwctl2t /dm7/data/DAMENG/dmwatcher.ctl /dm7/data/DAMENG/dmwatcher.txt
```

进程控制文件 dmwatcher.ctl 维护一个全局唯一值（TGUID），每次更新控制文件，都会生成一个新的 TGUID，再加上当前旧的 TGUID 以及一些其他的描述信息，构成一个控制记录项，添加到控制文件中。文件中所有的控制记录项根据 TGUID 链接起来，可以重现守护进程组内主备库的模式、状态变化历史。

控制文件内容包括控制文件头和控制记录项。

控制文件头：

内容主要包括（记录项数目, 当前 TGUID, 控制文件状态)。

控制记录项：

控制记录项的内容包含如下信息：(TGUID 旧值，TGUID 新值，执行操作实例的 SLSN，记录生成时间，监视器 ID，操作描述信息，监视器 IP 地址)。

操作描述信息包括：Open Force、Switchover、Takeover、Takeover Force、Auto Takeover 这五种类型。

每次执行相关操作时，会在控制文件中产生一条控制记录项。相关操作如下：

1. 监视器执行 Open Force 命令，强制 Open 主库实例
2. 监视器执行 Switchover 命令，切换主备库模式
3. 监视器执行 Takeover 命令，使用备库接管故障主库
4. 监视器执行 Takeover Force 命令，使用备库强制接管故障主库
5. 自动切换模式下，主库故障，确认监视器通知备库自动接管为新主库

3.5.4 状态

守护进程控制文件包含以下状态：

- 有效（VALID）　　正常运行时状态。
- 无效（INVALID）　　存在多个主实例时设置。
- 分裂（SPLIT）　　数据和有效主实例的数据不一致时设置。

在某些情况下，守护进程会将控制文件状态设置为分裂或无效，设置为这两种状态后，对应的实例将不能加入守护组，需要人工干预。

无效状态产生时机：

1．同时出现两个或两个以上的 Open 状态的主实例。

2．同时出现两个或两个以上的 SUSPEND 状态的主实例。

3．同时出现 Open 状态以及 SUSPEND 状态的主实例。

4．出现分裂的情况，重启之后，不能加入 Open 主实例。

以上有些情况只会在人工非法干预下才会出现，置无效状态是为了保护数据的有效性。

分裂状态产生时机：

1．本地实例[Primary/MOUNT]，不能加入远程实例[Primary/OPEN]。

2．本地实例[Standby/MOUNT]，不能加入远程 Primary 实例；

本地实例[Standby/OPEN]，不能加入远程 Primary 实例。

3.6 相关命令

守护进程（dmwatcher）既能以控制台方式启动，也可以配置为服务方式启动。可以在守护进程的控制台上输入命令，关闭守护进程，显示守护进程组的状态信息等，具体命令参考下表：

表 3.2 守护进程命令

命令	含义
HELP	帮助
EXIT	退出守护进程
STATUS	显示守护进程运行状态
SHOW	显示所有组的守护进程及其监控数据库的状态信息

SHOW GROUP GROUP_NAME	显示指定组的守护进程和监控数据库的状态信息

4 监视器

监视器（dmmonitor）是基于监视器接口（监视器接口详见 9.2 监视器接口 ）实现的一个命令行工具，是 DM 数据守护系统的重要组成部分。

通过监视器，可以监控数据守护系统的运行情况，获取主备库状态、守护进程状态、以及主备库数据同步情况等信息。同时，监视器（dmmonitor）还提供了一系列命令来管理数据守护系统。

监视器的基本作用如下：

- 监控数据守护系统

接收守护进程发送的消息，显示主、备数据库状态变化，以及故障切换过程中，数据库模式、状态变化的完整过程。

- 管理数据守护系统

用户可以在监视器上输入命令，启动、停止守护进程的监控功能，执行主备库切换、备库故障接管等操作。

- 确认状态信息

用于故障自动切换的数据守护系统中，主、备库进行故障处理之前，需要通过监视器进行信息确认，确保对应的备库或者主库是真的产生异常了，避免主备库之间网络故障引发脑裂。

- 发起故障自动接管命令

用于故障自动切换的数据守护系统中，主库发生故障时，挑选符合接管条件的备库，并通知备库执行接管操作。


注意：

对于实时守护类型和读写分离守护类型，监视器只允许配置一个守护进程组；MPP 守护类型允许配置多组，并且要求这些组的主库必须是同一套 MPP 系统；对于本地守护类型，允许和实时/读写分离/MPP 守护类型配置到同一组作为异步备库，如果不作为某个实例的异步备库，只是普通的单机配置为本地守护类型，则需要单独成组，并且监视器也不允许配置多组。

4.1 监视器类型

监视器支持两种运行模式：监控模式和确认模式。监视器运行模式由配置文件（dmmonitor.ini）的 MON_DW_CONFIRM 参数来确定。MON_DW_CONFIRM 参数的默认值是 0，表示监控模式；MON_DW_CONFIRM 参数值为 1 时，表示监视器运行在确认模式下。为了区分监视器的两种模式，我们将运行在确认模式下的监视器称为确认监视器。

1．监控模式

一个数据守护系统中，最多允许同时启动 10 个监视器，所有监视器都可以接收守护进程消息，获取守护系统状态。所有监视器都可以发起 Switchover 等命令，但守护进程一次只能接收一个监视器的命令，在一个监视器命令执行完成之前，守护进程收到其他监视器发起的请求，会直接报错返回。

2．确认模式

和监控模式一样，确认监视器接收守护进程消息，获取数据守护系统状态，也可以执行各种监控命令。区别在于，除了具备监控模式监视器所有功能之外，确认监视器还具有状态确认和自动接管两个功能。此外，一个数据守护系统中，只能配置 1 个确认监视器。故障自动切换模式的数据守护系统，必须部署一个确认监视器，否则在出现实例故障时，会导致数据库服务中断。

4.2 状态确认

故障自动切换模式数据守护系统中，主库守护进程监测到备库故障时，需要向确认监视器求证，确认备库是真的故障了，再启动故障处理流程将归档失效，避免引发脑裂。比如，主库网络故障，主库直接将归档失效继续以 Primary 模式提供服务；同时，确认监视器认为主库故障，将备库切换为 Primary 模式，守护进程组中同时出现多个主库，引发脑裂。

状态确认只对故障自动切换数据守护系统有效，手动切换模式下，不需要状态确认。主库守护进程，在以下两种故障场景中，会切换到 Confirm 状态，向确认监视器进行求证：

1.主库处于 Open 状态，主备库之间的 MAL 链路断开，但归档仍然处于有效状态。

2.主库处于 Suspend 状态，主库到备库的归档仍然处于有效状态。

主库守护进程切换到 Confirm 之后，根据不同的场景决定是否切换为 Failover 状态、启动故障处理流程：

1.主库网络故障，主库到备库、主库到确认监视器的连接异常。这种情况下主库守护进程，一直保持 Confirm 状态，不会启动故障处理流程。

2.备库故障或者备库网络故障，主库守护进程会切换为 Failover 状态，启动故障处理流程。

3.主备库之间网络故障，但与确认监视器之间的网络正常，主库守护进程会切换为 Failover 状态，启动故障处理流程。

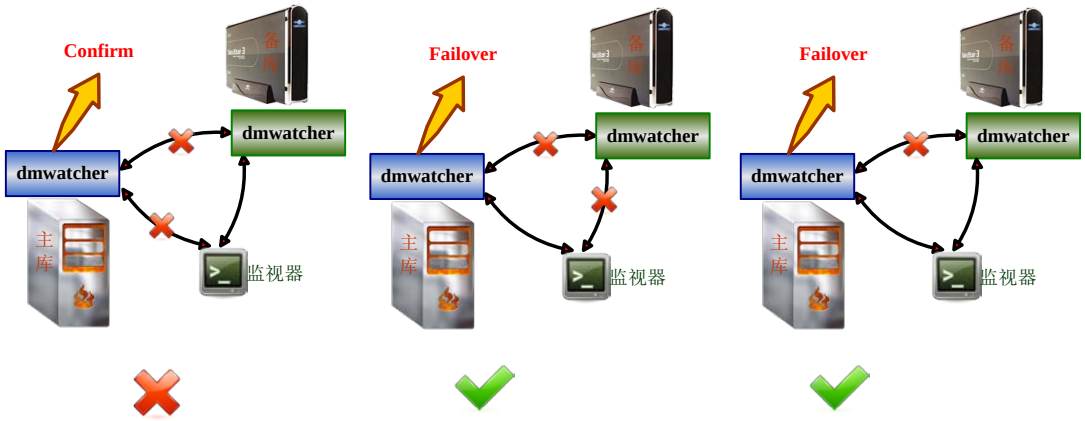


图 4.1 状态确认

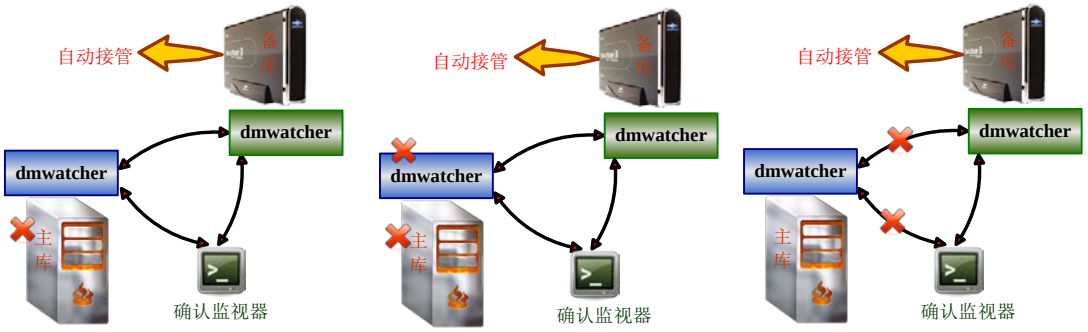
4.3 自动接管

故障自动切换模式下，确认监视器检测到主库故障后，根据收到的主备库 LSN、归档状态、MAL 链路状态等信息，确定一个接管备库，并将其切换为主库。确认监视器启动自动接管流程的主要场景有三种，任何一种都会导致备库自动接管。场景如下：

1.主库数据库实例异常终止，主库守护进程正常。

2.主库硬件故障、或者数据库实例和守护进程同时故障。

3.主库网络故障，主备库之间、主库与监视器之间连接异常。



左　　中　　右

图 4.2 自动接管

若想实现备库自动接管，主库，归档状态，备库都必须符合一定条件才行。条件如下：

■ 主库

1. 主库是 Primary 模式、Open 状态时，产生故障。（如图 4.2 左，中）
2. 主库守护进程故障，故障前是 Open/Recovery 状态。（如图 4.2 中）
3. 故障主库与接管备库和确认监视器之间的 MAL 链路断开。（如图 4.2 右）

■ 归档状态

故障主库到接管备库的归档状态为 Valid。

■ 备库

接管备库是 Standby 模式、Open 状态；接管备库的 dmwatcher.ctl 控制文件状态为 Valid；故障主库和接管备库的 dmwatcher.ctl 控制文件相同。



如果主库故障前，正在执行 Switchover/Takeover 等命令，则备库不会自动接管，需要人工干预。

确认监视器要求一开始就启动，保证出现故障情况时，确认监视器已经收到了故障主库或备库的历史消息，否则会因为条件不足无法自动处理，需要通过命令方式人工干预。

确认监视器不要和主库部署在一台机器上，避免主库网络故障时，备库无法自动切换为主库。

确认监视器不要和备库部署在一台机器上，避免主备库之间网络异常时，确认监视器误认为主库故障，将备库切换为主库，引发双主问题。

4.4 监视器命令

监视器提供以下命令来管理守护系统：

表 4.1 监视器命令

命令	含义
help	显示帮助信息
show [group_name]	显示指定组的实例信息
show [all]	显示所有组的实例信息

show i[nterval] n	每隔n秒自动显示所有组的实例信息
q	取消自动显示
list [group_name.]inst_name	列出指定组的实例对应的守护进程配置信息
list [all]	列出所有实例对应的守护进程配置信息
show monitor [group_name[.]] [inst_name]	列出连接到指定守护进程的所有监视器信息
tip	查看系统当前运行状态
exit	退出监视器
login	登录监视器
logout	退出登录
startup dmwatcher [group_name]	启动指定组的守护进程监控功能
stop dmwatcher [group_name]	关闭指定组的守护进程监控功能
startup instance [group_name]	启动指定组的实例
stop instance [group_name]	关闭指定组的实例
open instance [group_name.] inst_name	强制open指定组的指定实例
choose switchover [group_name]	选择可切换为Primary实例的实例列表
switchover [group_name[.]] [inst_name]	切换指定组的指定实例为Primary实例
choose takeover [group_name]	选择可接管故障Primary实例的实例列表
takeover [group_name[.]] [inst_name]	使用指定组的指定实例接管故障Primary实例
choose takeover force [group_name]	选择可强制接管故障Primary实例的实例列表
takeover force [group_name[.]] [inst_name]	使用指定组的指定实例强制接管故障Primary实例
show dmwatcher ctl [group_name.] inst_name	显示指定守护进程的控制文件内容
clear dmwatcher ctl [group_name]	清理指定组的守护进程控制文件
set [group_name.]para_name value	修改指定组的守护进程的指定配置参数,支持修改以下参数: DW_ERROR_TIME/INST_RECOVER_TIME/ INST_ERROR_TIME/INST_AUTO_RESTART
set recover time [group_name.]inst_name	修改指定组的指定实例的恢复间隔(只修改指定实

value	例对应主库的守护进程的内存值）
set arch invalid [group_name.]inst_name	修改指定组的指定实例的归档为无效状态
show arch info [group_name.]inst_name	查看源实例到指定组的指定实例的归档同步信息
remove instance [group_name.]inst_name	将指定备库移出守护进程组
以下命令仅支持MPP守护类型的实例	
show mpp	显示MPP节点信息
startup dmwatcher all	启动所有组的守护进程监控功能
stop dmwatcher all	关闭所有组的守护进程监控功能
startup instance all	启动所有组的实例
stop instance all	关闭所有组的实例
clear dmwatcher ctl all	清理所有组的守护进程控制文件
check mppctl	检查MPP控制文件是否处于一致状态
recover mppctl	恢复MPP控制文件到一致状态

下面对监视器命令进行详细的介绍：


注意：

命令中参数 GROUP_NAME 和 INST_NAME 的使用说明：

对于读写分离和实时守护类型，因为监视器只允许配置一组，所以下面命令中的组名[group_name]都可以不指定；对于 MPP 守护类型，因为有多个组，所以需要指定组名。

[inst_name]在有多个备实例时需要指定实例名。

组名和实例名均不唯一的情况下，需要全部指定。组名和实例名用符号“.”分隔。

1. help

显示帮助信息。

2. show [group_name]

如果指定有 group_name，则显示指定组的所有实例状态信息，如果没有指定，则显示所有组的实例状态信息。

show 命令显示结果中各字段含义说明如下：

1) 全局字段信息

GROUP：守护进程组名。

OGUID：守护进程组配置的唯一 OGUID 值。

MON_CONFIRM：监视器是否配置为确认模式，值为 TRUE 或 FALSE。

FAILOVER_TYPE：当前配置的切换模式，AUTO FAILOVER 表示故障自动切换模式，MANUAL FAILOVER 表示故障手动切换模式。

2) 实例各字段信息

IP:实例所在的 IP 地址，也是实例对应的守护进程的 IP 地址。

PORT：监视器连接实例对应的守护进程的 TCP 端口。

WTIME：实例对应的守护进程本地的当前时间。

WTYPE：守护进程配置的守护类型，包括 LOCAL/REALTIME/TIMELY/MARCH 四种类型。

WCTLSTAT：守护进程控制文件状态，包括 Valid/SPLIT/Invalid 三种状态，对于配置为本地守护类型的实例，由于不需要有守护进程控制文件，该字段值为 NULL。

WSTATUS：守护进程状态。

INST_OK：守护进程认定的实例状态，OK 或 Error。

INAME：实例名称。

ISTATUS：数据库实例状态，包括 Startup/AFTER Redo/Mount/Open/SUSPEND/Shutdown 这几种状态。

IMODE：数据库实例模式，包括 NORMAL/Primary/Standby 这三种模式。

RSTAT：此字段对备实例有效，表示主实例到备实例的归档状态，可能为 Valid/Invalid/UNKNOWN，对于本地守护类型的备实例，此字段为 NULL，对于主实例本身，此字段值为 Valid。

N_Open：表示实例的 Open 次数，主实例每 Open 一次，该值对应加 1，并产生日志同步到备实例重做，主备实例数据一致的情况下，N_Open 数值也应该一致。

FLSN：文件 LSN，指已经写入联机日志文件的最大 LSN 值。

CLSN：系统当前 LSN，指当前数据库最新产生的 LSN 值。

SSLSN：备实例收到的主实例发送过来日志的次大 LSN，主实例的 SSLSN 和 CLSN 相等。

SLSN：备实例收到的主实例发送过来日志的最大 LSN，主实例的 SLSN 和 CLSN 相等。

3. show all

显示所有组的实例状态信息，相关字段含义可参考 show [group_name] 命令说明。

4. show i[nterval] n

在 dmmonitor 工具界面上定时每隔 n 秒自动显示所有组的所有实例状态信息，相当于每隔 n 秒自动执行一次 show 命令，n 取值范围为 5~3600(s)，相关字段含义可参考 show [group_name] 命令说明。

如果在工具界面上手动执行其他命令，则会自动取消定时显示功能。

5. q

取消 show i[nterval] n 命令设置的自动显示功能。

6. list [group_name.]inst_name

此命令列出指定组中指定实例的守护进程配置信息，只有一组时允许不指定 group_name。

list 命令显示的字段和 dmwatcher.ini 的配置内容完全一致，可参考 5.4 dmwatcher.ini配置表格的各个字段说明。

7. list all

列出所有实例的守护进程配置信息。

list 命令显示的字段和 dmwatcher.ini 的配置内容完全一致，可参考 5.4 dmwatcher.ini配置表格的各个字段说明。

8. show monitor [group_name[.]] [inst_name]

列出连接到指定实例守护进程的所有监视器信息。

对于 group_name，在配置有多组时需要指定，否则可以不指定。对于 inst_name，如果指定，则显示指定实例守护进程上的连接信息，如果没有指定，则根据 dmmonitor.ini 对应组中的配置顺序，显示第一个活动的守护进程上的监视器连接信息。如果 group_name 和 inst_name 同时指定，则需要用“.”符合分隔。

同一个守护系统中允许最多 10 个监视器同时连接，可通过此命令获取指定守护进程上的监视器连接信息，包括连接时间、确认监视器配置、监视器 ID 和 IP 信息。其中显示的连接信息中，第一行结果为本地监视器的连接信息。命令显示结果的字段说明如下：

- DW_CONN_TIME：守护进程和监视器建立连接的时间。

- MON_Confirm：监视器是否配置为确认模式。
- MID：唯一标识监视器的 ID 值。
- MON_IP：监视器所在机器的 IP 地址。

9. tip

打印当前系统的运行状态。

检查实例状态、实例的守护进程状态、实例的分裂信息、实例的可加入信息、是否存在多主实例以及 MPP 环境下 dmmpp.ctl 是否处于一致状态等系统信息，并打印检查结果。

10. exit

退出监视器。

11. login

登录监视器，登录口令和服务器的用户登录口令一致，且必须有 SYSDBA 权限，默认使用 SYSDBA/SYSDBA。

监视器优先选择主实例校验登录口令，如果没有活动主实例，则选择一个活动备实例校验登录口令，部分监视器命令需要先登录才允许执行。

12. logout

退出登录。

13. startup dmwatcher [group_name]

启动指定组的所有守护进程监控功能，只有一组的情况下，可以不指定 group_name。

此命令是指在守护进程处于 shutdown 状态时，将其切换至 startup 状态，恢复守护进程的监控功能，而不是启动守护程序。如果守护进程不是处于 shutdown 状态，则跳过不再执行。

另外切换至 startup 状态后，守护进程可能会根据本地或远程实例状态信息自动做一些状态切换处理，不一定会一直保持在 Startup 状态。

14. stop dmwatcher [group_name]

关闭指定组的所有守护进程监控功能，只有一组的情况下，可以不指定 group_name。


注意：**执行此命令需要先登录监视器。**

此命令只是将守护进程切换为 shutdown 状态，并不会退出守护进程，守护进程切换 shutdown 状态后，不会再自动处理故障，也不会自动启动实例，但仍然能够正常接收、发送消息，监视器仍然能够监控到最新的实例状态。

15. startup instance [group_name]

启动指定组的所有实例，只有一组的情况下，可以不指定 group_name。

如果守护进程的监控功能被关闭，处于 shutdown 状态，在通知实例启动之前，会先打开守护进程的监控功能，再通知守护进程启动实例。



注意：如果守护进程 dmwatcher.ini 中 inst_auto_restart 配置为 1，则对应实例不允许通过命令方式启动，需要等待守护进程自动启动实例。

16. stop instance [group_name]

退出指定组的所有实例，只有一组的情况下，可以不指定 group_name。执行此命令需要先登录监视器。

为了避免守护进程将实例再次启动，退出时会先通知关闭所有守护进程的监控，重启等功能，切换守护进程为 shutdown 状态。如果已经是 Shutdown 状态，则跳过不再通知。

退出时，如果组中存在 Open 状态的主实例，则先通知主实例退出，再依次通知其他备实例退出，否则就按照 dmmonitor.ini 中的配置顺序，依次通知每个处于活动状态的实例退出。

17. open instance [group_name.] inst_name

强制 Open 指定组的指定实例，只有一组的情况下，可以不指定 group_name，但实例名 inst_name 必须指定。执行此命令需要先登录监视器。

如果指定有组名 group_name，则组名和实例名需要用“.”分隔。

正常情况下，守护进程会自动 Open 实例，不需要手动执行此命令，如果发现某个实例一直无法自动 Open，可尝试使用此命令。

命令执行时，首先会检查守护进程是否可以执行自动 Open，如果符合执行条件，则命令会报错返回，需要等待守护进程自动处理，否则允许命令执行，将指定实例强制 Open。

18. choose switchover [group_name]

选出指定组中允许执行 switchover 的备实例列表，只有一组的情况下，可以不指定

group_name。

19. switchover [group_name[.]] [inst_name]

将指定组的指定实例切换为新的主实例，将当前的主实例切换为新的备实例，此命令要求执行切换的主备实例都是正常 open 状态。执行此命令需要先登录监视器。

只有一组的情况下，可以不指定 group_name，组中只有一个备实例的情况下，可以不指定 inst_name，如果 group_name 和 inst_name 同时指定，则需要用“.”符合分隔。

20. choose takeover [group_name]

指定组中主实例出现故障时，可以通过此命令选出组中可以正常接管的备实例列表，只有一组的情况下，可以不指定 group_name。

如果指定组中存在有活动主实例，则不再选择可接管的备实例列表。

21. takeover [group_name[.]] [inst_name]

使用指定组的指定实例接管组中的故障主实例。执行此命令需要先登录监视器。

只有一组的情况下，可以不指定 group_name，组中只有一个备实例的情况下，可以不指定 inst_name，如果 group_name 和 inst_name 同时指定，则需要用“.”符合分隔。

该命令要求监视器收到过故障主实例的历史信息，并且主实例的历史信息必须满足被接管条件，可以先通过 choose takeover [group_name]选出符合正常接管条件的备实例列表，选择其中一个备实例执行接管操作即可。

22. choose takeover force [group_name]

指定组中主实例出现故障时，可以通过此命令选出组中可以强制接管的备实例列表，只有一组的情况下，可以不指定 group_name。

如果指定组中存在有活动主实例，则不再选择可强制接管的备实例列表。



注意：出现主实例故障时，优先使用 choose takeover [group_name] 选出可接管主库，并使用 takeover [group_name[.]] [inst_name]来接管。如果不存在可以正常接管的备实例，但又需要主实例对外提供服务，可再考虑使用 choose takeover force [group_name]选出可强制接管的备实例，并使用 takeover force [group_name[.]] [inst_name]进行强制接管。但是强制接管可能会导致出现分裂情况。

23. takeover force [group_name[.]] [inst_name]

使用指定组的指定实例强制接管组中的故障主实例。执行此命令需要先登录监视器。

只有一组的情况下，可以不指定 group_name，组中只有一个备实例的情况下，可以不指定 inst_name，如果 group_name 和 inst_name 同时指定，则需要用“.”符合分隔。

24. show dmwatcher ctl [group_name.] inst_name

除了配置为本地守护类型的守护进程外，其他守护类型都需要有守护进程控制文件，通过此命令可查看守护进程控制文件的内容。

只有一组的情况下，可以不指定 group_name，但实例名 inst_name 必须指定，如果 group_name 和 inst_name 同时指定，则需要用“.”符合分隔。

1) 全局字段信息

INST_NAME：守护实例名称。

N_ITEM：控制文件中的记录项个数，对于新初始化的或者执行过 clear dmwatcher ctl 命令的 dmwatcher.ctl，记录项个数为 0。

CTL_STATUS：控制文件状态，包括 Valid/SPLIT/Invalid 三种状态。

CUR_TGUID：控制文件当前的 TGUID 值，每次初始化或增加一项记录都会生成新的 TGUID 值，新生成的 TGUID 以当时执行操作的实例名作为后缀。

2) 每条记录项的字段信息

如果控制文件中有记录项，每行记录相关字段含义如下：

TGUID_ORG：增加此记录时，控制文件的原始 TGUID 值。

TGUID_NEW：增加此条记录时，新生成的 TGUID 值，最后一条记录的 TGUID_NEW 和全局的 CUR_TGUID 值一致。

SLSN：增加此条记录时，执行操作的实例的 SLSN 值，实例名附加在此条记录的 TGUID_NEW 末尾，操作描述信息可参考 DESC 字段。

DATETIME：增加此记录时守护进程的本地时间。

MID：执行操作的监视器唯一 ID 标识。

MON_IP：执行操作的监视器所在机器 IP 地址。

DESC：监视器执行的操作描述信息。

DESC 字段的描述信息包括下面几种类型：

- Open Force 表示监视器执行 open instance 增加的 ctl 记录

- Switchover 表示监视器执行 switchover 增加的 ctl 记录
- Takeover 表示监视器执行正常 takeover 增加的 ctl 记录
- Takeover Force 表示监视器执行强制 takeover 增加的 ctl 记录
- AUTO Takeover 表示确认监视器自动接管故障主实例时增加的 ctl 记录
- 其他描述信息 如果 CTL_STATUS 不是 Valid 状态，则 DESC 是一段描述信息，描述修改 CTL_STATUS 为 SPLIT 或 Invalid 状态的原因。

25. clear dmwatcher ctl [group_name]

清理指定组的守护进程 dmwatcher.ctl 文件，此命令要求系统所有主备实例运行正常，不存在分裂情况。执行此命令需要先登录监视器。

只有一组时，可以不指定 group_name。

监视器每执行一次相关操作，dmwatcher.ctl 就会增加一条记录。相关操作如下：

- 确认监视器的自动接管
- Open Instance
- Switchover
- Takeover
- Takeover Force
- 处理分裂情况时，会增加一条修改控制文件无效或分裂状态的记录

当 dmwatcher.ctl 文件很大的时候，可以选择使用此命令执行清理操作。

dmwatcher.ctl 清理流程：先清理主库控制文件再清理备库控制文件。如果主库控制文件清理失败则命令执行失败；如果备库控制文件清理失败，继续往下执行，清理失败的备库由用户自己干预来保证控制文件的一致性。用户可以统一拷贝同一组中执行过清理的 dmwatcher.ctl，也可以拷贝同一组中尚未执行清理的 dmwatcher.ctl，保证同一组的所有主备实例都要使用同一份 dmwatcher.ctl 文件。

对重新拷贝过 dmwatcher.ctl 的实例，需要重启对应的守护进程，重新加载控制文件内容。

26. set [group_name.]para_name value

修改指定组的所有守护进程的指定参数值，并同步修改配置文件 dmwatcher.ini 中的参数值。执行此命令需要先登录监视器。

如果组中某个守护进程故障，则跳过不修改，如果要重启故障的守护进程，需要在重启

前修改守护进程 dmwatcher.ini 中的 DW_ERROR_TIME 和远程一致，否则 dmmonitor 执行命令时会报错处理。

group_name 指定组名，只有一组时可以不指定，para_name 指定参数名称，value 指定对应的参数值，目前可修改的参数包括：DW_ERROR_TIME、INST_RECOVER_TIME、INST_ERROR_TIME、INST_AUTO_RESTART。

27. set recover time [group_name.]inst_name value

设置主库到指定备库的 Recovery 时间间隔，取值范围(0~ 86400)s。执行此命令需要先登录监视器。

只有一组时可以不指定 group_name，inst_name 必须是备库实例名。命令执行时会查找 inst_name 对应的主库，并通知主库的守护进程修改到备库的 recovery 间隔时间为指定的 value 值，注意此命令只修改主库守护进程的内存值(INST_RECOVER_TIME)，并且只对指定备库起作用，不会写入到 dmwatcher.ini 文件。

命令执行时要求主库和主库的守护进程必须是活动的，否则无法查找 inst_name 对应的主库信息，导致命令执行失败。

设置成功后，在指定备库发生故障重启时（不包括正常退出重启的情况），除了要满足 Recovery 的基本条件外，还需要满足距离指定备库上次的恢复时间超过指定的时间间隔，主库的守护进程才会再次启动 recovery 流程。

28. set arch invalid [group_name.]inst_name

设置主库到指定备库的归档状态无效。执行此命令需要先登录监视器。

只有一组时可以不指定 group_name，inst_name 必须是备库实例名。

命令执行时会查找 inst_name 对应的主库，并通知主库修改到备库的归档状态无效。

命令执行时要求主库和主库的守护进程必须是活动的，否则无法查找 inst_name 对应的主库信息，导致命令执行失败。

在主备库正常运行时，用户如果想人为退出备库（例如，需要修改备库配置），并且不想触发主库的 Failover 流程（避免影响应用执行），可先使用 set recover time 命令，将备库的 Recovery 间隔修改到一个较大的值，然后使用此命令通知主库修改备库的归档状态无效，即可达到目的。

之所以先使用 set recover time 命令设置恢复间隔，是为了防止先设置归档无效后，

在备库退出前，主库的守护进程立马启动 Recovery 流程，又重新将备库归档恢复到有效状态，避免出现备库退出后还会触发主库执行 Failover 的情况。

29. show arch info [group_name.]inst_name

显示源实例到指定备库的归档发送信息，以及源实例的守护进程本地到指定备库的 Recovery 间隔信息。也支持异步备库的查询，其中 Recovery 间隔信息对异步备库无效。

查询结果中的字段说明如下：

■ 主库的守护进程字段信息

下面这些信息只对非 local 守护类型的备库有效。

LAST RECOVER TIME：主库守护进程上次启动对指定备库的 Recovery 时间，NONE 表示未执行过恢复操作。

LAST_RECOVER CODE：主库守护进程上次对指定备库执行 Recovery 的结果，Code 小于 0 表示执行失败，等于 0 表示执行成功，等于 100 表示实际未执行。

INST_RECOVER_TIME(IN DMWATCHER.INI)：主库的 dmwatcher.ini 中配置的 INST_RECOVER_TIME 值，对所有备库有效。

INST_RECOVER_TIME(IN DMWATCHER MEMORY)：主库的守护进程内存中的 INST_RECOVER_TIME 值，只对指定备库有效，可通过 SET RECOVER TIME 命令动态修改。

■ 主库实例字段信息

ARCH TYPE：主库到指定备库的归档类型，包括 REALTIME/TIMELY/MPP/ASYNC 四种类型。

MAL STATUS：主库到指定备库的 MAL 链路状态，包括 CONNECTED/ DISCONNECTED 两种状态。

ARCH STATUS：主库到指定备库的归档状态，包括 VALID/INVALID 两种状态。

SEND TYPE：主库到指定备库的归档发送类型。NORMAL SEND 表示主备库正常运行时的归档发送，FOR RECOVER SEND 表示主库的守护进程发起的 Recovery 流程中的归档发送，FOR ASYNC SEND 表示定时器定时触发的异步归档发送。

SEND START TIME：主库到指定备库的归档发送起始时间，NONE 表示未执行过发送操作。

SEND END TIME：主库到指定备库的归档发送结束时间，该字段为 NONE 时，如果 SEND

START TIME 也是 NONE，则表示未执行过发送操作，如果 SEND START TIME 是具体的时间，则表示发送尚未结束。

SEND START LSN：主库到指定备库的归档发送起始 LSN。

SEND END LSN：主库到指定备库的归档发送结束 LSN。

SEND CODE：主库到指定备库的归档发送结果，Code 小于 0 表示发送失败，等于 0 表示发送成功，等于 100 表示实际未发送。

SEND DESC INFO：主库到指定备库的归档发送结果描述信息。

30. remove instance [group_name.]inst_name

将指定备库移出当前的守护进程组，并且不触发主库的 Failover 故障处理流程。执行此命令需要先登录监视器。该操作不会修改相关的配置信息，只是为了需要主动退出维护备库时使用。

该命令包括下面几个操作步骤：

1) 通知主库设置指定备库的恢复间隔为 86400s。

2) 通知主库修改指定备库的归档状态无效。

3) 通知切换备库的守护进程为 SHUTDOWN 状态。

4) 通知备库正常退出。

从上面步骤可以看出，此命令是前面几个命令的结合，步骤 1 相当于执行 set recover time 命令，步骤 2 相当于执行 set arch invalid 命令，步骤 3、4 相当于正常退出实例命令，使用此命令可简化手工操作步骤，一次完成将指定备库移出守护进程组的所有操作。

另外此命令也支持异步备库的执行，对于异步备库，会跳过步骤 1、2，直接执行步骤 3、4，执行退出操作。


注意：

此命令执行成功后，在备库维护工作完成重启之后，主库内存中对指定备库的恢复间隔时间仍然是 86400 s，如果想尽快启动恢复流程，需要再通过手工执行 set recover time 调整指定备库的恢复间隔，此限制只针对非 local 守护类型的备库。

执行此命令需要先登录监视器。

31. show mpp

显示 MPP 站点信息，该命令仅支持 MPP 守护类型下使用。

显示的信息和 dmmpp.ctl 中记录的节点信息一致。

相关字段含义如下：

INST_NAME：配置的 MPP 节点实例名。

EP_SEQNO：配置的 MPP 节点号。

32. startup dmwatcher all

启动所有组的守护进程监控功能，该命令仅支持 MPP 守护类型下使用。

此命令是指在守护进程处于 Shutdown 状态时，将其切换至 Startup 状态，恢复守护进程的监控功能，并不是启动守护程序，如果守护进程不是处于 Shutdown 状态，则跳过不再处理。

另外，切换至 Startup 状态后，守护进程可能会根据本地或远程实例状态信息自动做一些状态切换处理，不一定会一直保持在 Startup 状态。

33. stop dmwatcher all

关闭所有组的守护进程监控功能，该命令仅支持 MPP 守护类型下使用。执行此命令需要先登录监视器。

此命令只是将守护进程切换为 Shutdown 状态，并不会退出守护进程，守护进程切换 Shutdown 状态后，不会再自动处理故障，也不会自动启动实例，但仍然能够正常接收、发送消息，监视器仍然能够监控到最新的实例状态。

34. startup instance all

启动所有组的所有实例，该命令仅支持 MPP 守护类型下使用。

如果守护进程的监控功能被关闭，处于 Shutdown 状态，在通知实例启动之前，会先打开守护进程的监控功能，再通知守护进程启动实例。



注意：如果守护进程 dmwatcher.ini 中 INST_AUTO_RESTART 配置为 1，则对应实例不允许通过命令方式启动，需要等待守护进程自动启动实例。

35. stop instance all

退出所有组的所有实例，该命令仅支持 MPP 守护类型下使用。执行此命令需要先登录监视器。

退出时会先通知关闭所有守护进程的监控功能，切换守护进程为 Shutdown 状态，避免守护进程将实例再次启动，如果已经是 Shutdown 状态，则跳过不再通知。

退出时，如果组中存在 Open 状态的主实例，则先通知主实例退出，再依次通知其他备实例退出，否则就按照 dmmonitor.ini 中的配置顺序，依次通知每个处于活动状态的实例退出。

36. clear dmwatcher ctl all

清理所有组的所有守护进程的 dmwatcher.ctl 文件，仅支持 MPP 守护类型下使用，此命令要求系统所有主备实例运行正常，不存在分裂情况。

监视器每执行一次相关操作，dmwatcher.ctl 就会增加一条记录。相关操作如下：

- 确认监视器的自动接管
- open instance
- switchover
- takeover
- takeover force
- 处理分裂情况时，会增加一条修改控制文件无效或分裂状态的记录

当 dmwatcher.ctl 文件很大的时候，可以选择使用此命令执行清理操作。

dmwatcher.ctl 清理流程：先清理主库控制文件再清理备库控制文件。如果主库控制文件清理失败则命令执行失败；如果备库控制文件清理失败，继续往下执行，清理失败的备库由用户自己干预来保证控制文件的一致性。用户可以统一拷贝同一组中执行过清理的 dmwatcher.ctl，也可以拷贝同一组中尚未执行清理的 dmwatcher.ctl，保证同一组的所有主备实例都要用同一份 dmwatcher.ctl 文件。

对重新拷贝过 dmwatcher.ctl 的实例，需要重启对应的守护进程，重新加载控制文件内容。

37. check mppctl

检查当前所有活动主实例的 dmmpp.ctl 文件内容是否一致，仅支持 MPP 守护类型下使用，如果某个组的主实例出现故障，则跳过不检查。

38. recover mppctl

恢复当前所有活动主实例的 dmmpp.ctl 到一致状态，仅支持 MPP 守护类型下使用。执行此命令需要先登录监视器。

如果某个组的主实例故障，则跳过不恢复，待主实例恢复正常后，可再通过 check mppctl 命令校验，如果不一致，可再尝试使用 recover mppctl 恢复到一致状态。

例如，MPP 系统中，主备库切换，备库接管等过程中，主备信息发生变更，新主库的信息将更新到 dmmpp.ctl 中。如果此时，另外一个主库发生故障，因为还未来得及同步最新 dmmpp.ctl 的情况下，所以重启后，可能造成主实例的 dmmpp.ctl 不一致的情况，导致 MPP 系统不能正常工作。这种情况下，可通过 dmmonitor 的检查（check mppctl）和修复（recover mppctl）命令进行同步该控制文件。

5 配置文件说明

与 DM 数据守护相关的配置文件包括：

- 数据库配置文件 dm.ini
- 数据库控制文件 dm.ctl
- MAL 配置文件 dmmal.ini
- Redo 日志归档配置文件 dmarch.ini
- 守护进程配置文件 dmwatcher.ini
- 守护进程控制文件 dmwatcher.ctl
- 监视器配置文件 dmmonitor.ini
- 定时器配置文件 dmtimer.ini
- MPP 控制文件 dmmpp.ctl 等等

其中，dmwatcher.ctl 是 dmwatcher.ini 经由 dmctlcvt 工具转换生成；dmmpp.ctl 在 2.5.4 dmmpp.ctl 维护 中介绍；dm.ctl 不需要用户修改，只要放在指定的目录即可。

各配置文件的存放路径：

1. dm.ini 存放目录没有限制，一般直接放在数据库目录中。
2. dm.ctl 存放目录由 dm.ini 的 CTL_PATH 配置项指定。
3. dmwatcher.ini 存放目录没有限制，一般和 dm.ini 存放同一个目录。
4. dmmonitor.ini 一般单独放在一台机器上，存放目录没有限制。
5. dmmal.ini、dmarch.ini、dmtimer.ini 、dmmpp.ctl 、dmwatcher.ctl 存放目录由在 dm.ini 的 SYSTEM_PATH 配置项指定。

下面分别介绍各配置文件中相关配置项的含义。

5.1 dm.ini

dm.ini 是数据库配置文件。

表 5.1 dm.ini 相关配置项

配置项	配置含义
INSTANCE_NAME	数据库实例名（长度不超过 16 个字符），与 dmmal.ini 中的 MAL_INST_NAME 对应。配置数据守护系统时，应该保持 INSTANCE_NAME 是全局唯一的。

PORT_NUM	数据库实例监听端口（范围 1024~65534），与 dmmal.ini 中的 MAL_INST_NAME 对应。
DW_PORT	守护进程监听端口（范围 1024~65534）
DW_ERROR_TIME	守护进程启动后，用户执行修改数据库模式、状态等操作将会报错，不允许执行 alter database mount/alter database standby 等 SQL。 默认守护进程退出 60s 后，才允许执行这些操作，取值范围为 0~1800s。
ALTER_MODE_STATUS	是否允许手工修改数据库的模式和状态，1 表示允许，0 表示不允许，此参数可动态修改，默认为 1，数据守护环境下建议配置为 0，避免用户手工干预。
ENABLE_OFFLINE_TS	是否允许 offline 表空间，1 表示允许，0 表示不允许，2 表示禁止备库，其他放开。守护环境下建议配置为 2。
SESS_FREE_IN_SUSPEND	远程归档失败会导致系统挂起，为了防止应用的连接一直挂住不切换到新主库，设置该参数，表示归档失败挂起后隔一段时间自动断开所有连接。默认值 60s，取值范围 0-1800s，0 表示不断开。
MAL_INI	MAL 系统配置开关，0 表示不启用 MAL 系统，1 表示启用 MAL 系统。
ARCH_INI	Redo 日志归档配置开关，0 表示不启动 Redo 日志归档，1 表示启用 Redo 日志归档
TIMER_INI	是否启用定时器，0：不启用；1：启用
MPP_INI	是否启用 MPP 系统，0：不启用；1：启用
DW_MAX_SVR_WAIT_TIME	数据库等待守护进程启动的最大时间（范围 0~65534s）。如果设定时间内，守护进程没有启动，数据库实例强制退出。0 代表不检测，缺省为 0
HA_INST_CHECK_FLAG	检测多个实例同时启动标记。 配置为 1 时，服务器每隔 1s 解析一次 dminst.sys 文件，检查是否有多个实例进程同时启动，如果没有则递增修改一次

	dminst.sys 文件中记录的序列值。 配置为 0 则不做检查，默认为 1。 配置为 1 时，守护进程会解析 dminst.sys 文件，根据文件中记录的序列值判断实例进程是否处于活动状态。

5.2 dmmal.ini

dmmal.ini 是 MAL 配置文件。需要用到 MAL 环境的实例，所有站点 dmmal.ini 需要保证严格一致。

表 5.2 dmmal.ini 配置项

配置项	配置含义
MAL_CHECK_INTERVAL	MAL 链路检测时间间隔，取值范围（0s-1800s），默认 30s，配置为 0 表示不进行 MAL 链路检测，数据守护环境，不建议配置为 0，防止网络故障导致服务长时间阻塞。
MAL_CONN_FAIL_INTERVAL	判定 MAL 链路断开的时间，取值范围（2s-1800s），默认 10s
MAL_LOGIN_TIMEOUT	MPP/DBLINK 等实例间登录时的超时检测间隔（3-1800），以秒为单位，默认 15s
MAL_BUF_SIZE	单个 MAL 缓存大小限制，以兆为单位。当此 MAL 的缓存邮件超过此大小，则会将邮件存储到文件中。有效值范围（0~500000），默认为 100，如果配置为 0，则表示不限制单个 MAL 缓存大小
MAL_SYS_BUF_SIZE	MAL 系统总内存大小限制，单位：M。有效值范围（0~500000），默认为 0，表示 MAL 系统无总内存限制
MAL_VPOOL_SIZE	MAL 系统使用的内存初始化大小，以兆为单位。有效值范围（1~500000），默认为 128，此值一般要设置的比 MAL_BUF_SIZE 大一些
MAL_COMPRESS_LEVEL	MAL 消息压缩等级，取值范围（0-10）。默认为 0，不进行压缩；1–9 表示采用 lz 算法，从 1 到 9 表示压缩速度依次递减，压缩率依次递增；10 表示采用 quicklz 算法，压缩速度高于 lz 算法，压缩率相对低
[MAL_NAME]	MAL 名称，同一个配置文件中 MAL 名称需保持唯一性

MAL_INST_NAME	数据库实例名，与 dm.ini 的 INSTANCE_NAME 配置项保持一致，MAL 系统中数据库实例名要保持唯一
MAL_HOST	MAL IP 地址，使用 MAL_HOST + MAL_PORT 创建 MAL 链路
MAL_PORT	MAL 监听端口
MAL_INST_HOST	MAL_INST_NAME 实例对外服务 IP 地址
MAL_INST_PORT	MAL_INST_NAME 实例对外服务端口，和 dm.ini 中的 PORT_NUM 保持一致
MAL_DW_PORT	MAL_INST_NAME 实例守护进程监听端口，其他守护进程或监视器使用 MAL_HOST + MAL_DW_PORT 创建 TCP 连接。

5.3 dmarch.ini

dmarch.ini 是 Redo 日志归档配置文件。

表 5.3 dmarch.ini 配置项

配置项	配置含义
ARCH_WAIT_APPLY	备库收到 Redo 日志后，是否需要重演完成后，再响应主库。0 表示收到马上响应，1 表示重演完成后响应。配置即时归档时，默认值为 1；其他归档忽略这个配置项，强制设置为 0。
[ARCH_NAME]	Redo 日志归档名
ARCH_TYPE	Redo 日志归档类型， LOCAL/ REALTIME/ TIMELY/ MARCH/ ASYNC 分别表示本地归档/实时归档/即时归档/MPP 归档/异步归档
ARCH_DEST	归档目标，本地归档为归档文件存放路径，其他归档方式设置为目标数据库实例名
ARCH_FILE_SIZE	单个 Redo 日志归档文件大小，取值范围（64M~2048M），仅对本地归档有效，缺省 128M
ARCH_SPACE_LIMIT	Redo 日志归档空间限制，当所有本地归档文件达到限制值时，系统自动删除最早生成的归档日志文件。0 表示无空间限制，取值范围（1024M~4294967294M），仅对本地归档有效，缺省为 0

ARCH_TIMER_NAME	定时器名称，仅对异步归档有效

5.4 dmwatcher.ini

dmwatcher.ini 是守护进程配置文件。

表 5.4 dmwatcher.ini 配置项

配置项	配置含义
[GROUP_NAME]	守护进程组名（长度不能超过 16）
DW_TYPE	守护类型，默认为 LOCAL LOCAL：本地守护 REALTIME：实时主备 TIMELY：读写分离 MARCH：MPP 主备
DW_MODE	切换模式，默认为 MANUAL MANUAL：故障手动切换模式 AUTO：故障自动切换模式
DW_ERROR_TIME	守护进程故障认定时间，取值范围为（3s~32767s），缺省 15 秒没有收到远程守护进程消息，即认定远程守护进程故障，对本地模式无效。 另外此参数也是监视器认定守护进程的故障时间，超过设置的时间间隔仍没有收到守护进程消息，监视器认为守护进程出现故障。
INST_ERROR_TIME	数据库故障认定时间，取值范围为（3s~32767s），缺省 15 秒没有收到数据库发送的状态信息，即认定其监控的数据库出现故障
INST_OGUID	数据守护唯一标识码，同一守护进程组中的所有数据库、守护进程和监视器，都必须配置相同的 OGUID 值，取值范围为 0-2147483647
INST_INI	监控数据库 dm.ini 路径。dmwatcher 从 dm.ini 配置文件获取 DW_PORT 信息，并进一步从 dmmal.ini 中获取

	MAL_HOST/MAL_DW_PORT 等信息。
INST_AUTO_RESTART	是否自动重启数据库实例，0：不自动重启 1：自动重启
INST_STARTUP_CMD	数据库启动命令。 1. linux 命令行方式启动（不能出现带有空格的路径）： INST_STARTUP_CMD = /opt/dm/bin/dmserver 2. linux 服务方式启动： INST_STARTUP_CMD = service dmserverd restart 3. Windows 命令行启动： INST_STARTUP_CMD = c:\dm\bin\dmserver 4. Windows 服务方式启动： INST_STARTUP_CMD = net start 注册服务名（注册服务名，可通过 DM 服务查看器获取）
INST_RECOVER_TIME	备库故障恢复检测时间间隔，取值范围 0~86400s，缺省每 60 秒检查一下备库状态，满足故障恢复条件时，启动历史数据同步流程。数据守护系统启动完成后、Switchover 主备切换后、Takeover 备库接管后以及强制 Open 主库后，主库守护进程 INST_RECOVER_TIME 内存值会强制设置成 0，确保尽快启动数据同步。另外，还可以通过监视器命令 set recover time 修改 INST_RECOVER_TIME 内存值。

5.5 dmwatcher.ctl

除了配置为 Local 守护类型的实例外，其他实例的库目录下都需要有守护进程控制文件 dmwatcher.ctl，且初始配置时同一个守护进程组的控制文件必须相同。

该控制文件由 dmctlcvt 工具根据 dmwatcher.ini 转化生成。控制文件中有一个随机的 tguid 值，每次调用 dmctlcvt 生成的 tguid 都不同，因此初始搭建环境时，同一个组只能通过 dmctlcvt 工具生成一次，并分别拷贝到各数据库目录下，不能采用分别生成的方式。

dmwatcher.ctl 按照 dmwatcher.ini 中配置的组名分别生成到各自的目录下，同一个守护进程组需要使用同一份控制文件。同一个组使用了不同的 dmwatcher.ctl 控制

文件将造成组分裂。

dmctlcvt 工具用法格式如下：

```
dmctlcvt t2dwctl [dmwatcher.ini path] [ctl file path]
dmctlcvt dwctl2t [dmwatcher.ctl path] [txt file path]
```

其中 t2c 命令中的 ctl file path 只需要指定到目录，不需要指定文件名。

1．生成 ctl 文件

需要指定 dmwatcher.ini 路径和文件输出路径，生成的 dmwatcher.ctl 按组名分开存放，同一组的所有实例使用同一个 ctl 文件。

参考命令如下：

```
./dmctlcvt t2dwctl /dm7/data/DAMENG/dmwatcher.ini /dm7/data
```

2．解析 ctl 文件

可将 dmwatcher.ctl 内容读出到 txt 文件中，方便查看内容。

参考命令如下：

```
./dmctlcvt dwctl2t /dm7/data/DAMENG/dmwatcher.ctl
/dm7/data/DAMENG/dmwatcher.txt
```

5.6 dmmonitor.ini

dmmonitor.ini 是监视器配置文件。

表 5.5 dmmonitor.ini 配置项

配置项	配置含义
MON_DW_CONFIRM	是否配置为确认模式，缺省为 0。0：监控模式 1：确认模式
MON_LOG_PATH	日志文件路径，日志文件命名方式为 “dmmonitor_年月日时分秒.log”，例如“dmmonitor_20150418230523.log”。 其他请参考表后附加说明。
MON_LOG_INTERVAL	自动记录系统状态信息到日志文件的时间间隔，取值(0，1 或者(5~3600))，单位秒，0 表示不记录任何日志，1 表示只记录监视器正常接收到的消息，5~3600 表示除了记录监视器正常接收消息外，每隔指定的间隔另外记录系统信息到日志文件中，默认值为 1。

MON_LOG_FILE_SIZE	单个日志文件大小，范围 16~2048，单位为 M，默认值为 64，达到最大值后，会自动生成并切换到新的日志文件中，如果达到设定的总空间限制，会自动删除创建时间最早的日志文件。
MON_LOG_SPACE_LIMIT	日志总空间大小，取值 0 或者 256~4096，单位为 M，默认值为 0，表示没有空间限制。
[GROUP_NAME]	守护进程组名，与 dmwatcher.ini 中的守护进程组名保持一致
MON_INST_OGUID	数据守护唯一标识码，与 dmwatcher.ini 中的 INST_OGUID 保持一致
MON_DW_IP	守护进程 IP 地址和监听端口。 配置格式为：“守护进程 IP 地址:守护进程监听端口”。 其中 IP 地址和 dmmal.ini 中的 MAL_HOST 保持一致，端口和 dmmal.ini 中的 MAL_DW_PORT 保持一致。

MON_LOG_PATH 配置说明：

1. 对于 dmmonitor.exe 命令行工具

如果 dmmonitor.ini 中配置有 MON_LOG_PATH 路径，则将 MON_LOG_PATH 作为日志文件路径，如果没有配置，则将 dmmonitor.ini 配置文件所在的路径作为日志文件路径。

2. 对于 dwmon_init 接口

接口说明可参考附录部分。

对于通过调用接口方式监控守护系统的情况，如果调用 dwmon_init 接口时指定有 log_path 参数，则将指定的 log_path 作为日志文件路径，如果没有指定 log_path 参数，则将 dmmonitor.ini 中配置的 MON_LOG_PATH 作为日志文件路径，如果 dmmonitor.ini 中没有配置 MON_LOG_PATH，则将 dmmonitor.ini 配置文件所在的路径作为日志文件路径。


注意：只有在 MON_LOG_INTERVAL 配置大于 0 的情况下才会产生日志信息，并写入到日志文件中，日志文件路径请参考上述说明。

在有日志写入操作时，如果日志路径下没有日志文件，会自动创建一个新的日志文件，如果已经有日志文件，则根据设定的单个日志文件大小（MON_LOG_FILE_SIZE）决定继续写入已有的日志文件或者创建新的日志文件写入。

创建新的日志文件时，根据设定的日志总空间大小（MON_LOG_SPACE_LIMIT）决定是否删除创建时间最早的日志文件。

5.7 dmtimer.ini

dmtimer.ini 用于配置定时器，可记录异步备库的定时器信息，具体使用请见异步备库配置章节。dmtimer.ini 的配置项见下表。

表 5.6 dmtimer.ini 的配置项

项目	项目意义	字段	字段意义
[TIMER_NAME1]	定时器信息	TIMER_NAME	定时器名称
		TYPE	定时器调度类型： 1：执行一次 2：按日执行 3：按周执行 4：按月执行的第几天 5：按月执行的第一周 6：按月执行的第二周 7：按月执行的第三周 8：按月执行的第四周 9：按月执行的最后一周
		FREQ_MONTH_WEEK_INTERVAL	间隔月或周数
		FREQ_SUB_INTERVAL	间隔天数
		FREQ_MINUTE_INTERVAL	间隔分钟数
		START_TIME	开始时间
		END_TIME	结束时间

		DURING_START_DATE	开始时间点
		DURING_END_DATE	结束时间点
		NO_END_DATE_FLAG	是否结束标记
		DESCRIBE	定时器描述
		IS_VALID	有效标记

例 1，配置 TYPE = 1，开始时间为 2016-02-22 17:30:00，只执行一次。

```
[TIMER_01]
  TYPE                          = 1
  FREQ_MONTH_WEEK_INTERVAL      = 0
  FREQ_SUB_INTERVAL         = 0
  FREQ_MINUTE_INTERVAL      = 0
  START_TIME                = 00:00:00
  END_TIME                      = 00:00:00
  DURING_START_DATE             = 2016-02-22 17:30:00
  DURING_END_DATE               = 2016-02-22 17:30:30
  NO_END_DATE_FLAG              = 0
  DESCRIBE                      = RT TIMER
  IS_VALID                      = 1
```

例 2：TYPE = 2，每天 01:00:00 触发。

```
[TIMER_01]
  TYPE                          = 2
  FREQ_MONTH_WEEK_INTERVAL      = 1
  FREQ_SUB_INTERVAL         = 0
  FREQ_MINUTE_INTERVAL      = 0
  START_TIME                = 01:00:00
  END_TIME                      = 01:00:10
  DURING_START_DATE             = 2016-02-11 17:36:09
  DURING_END_DATE               = 9999-12-31 23:59:59
```

```
NO_END_DATE_FLAG              = 1

DESCRIBE                      = RT TIMER

IS_VALID                      = 1
```

5.8 端口配置关系说明

对上述 ini 文件中的各个端口之间的对应关系说明如下。

1. dm.ini

dm.ini 中端口相关的有两个配置项：PORT_NUM、DW_PORT。

- PORT_NUM

是数据库实例的监听端口，监听用户的连接请求，dmmal.ini 中的[MAL_INST_NAME: MAL_INST_PORT]要与 dm.ini 中的[INSTANCE_NAME: PORT_NUM]保持一致。

- DW_PORT

是守护系统中，数据库监听守护进程连接请求的端口，由守护进程主动建立到数据库的 TCP 连接，这个端口只用于数据库和守护进程之间的消息交互使用。

例如，以主库 GRP1_RT_A，备库 GRP1_RT_B 为例，来说明如何使用 PORT_NUM、DW_PORT。

主库 GRP1_RT_A 的 dm.ini：

```
#这里只列举端口相关的参数，其他参数配置请参考后文的配置举例

INSTANCE_NAME           = GRP1_RT_A

PORT_NUM                = 5236  #数据库实例监听端口

DW_PORT                 = 5237  #守护进程监听端口
```

备库 GRP1_RT_B 的 dm.ini：

```
#这里只列举端口相关的参数，其他参数配置请参考后文的配置举例

INSTANCE_NAME           = GRP1_RT_B

PORT_NUM                = 6236  #数据库实例监听端口

DW_PORT                 = 6237  #守护进程监听端口
```

2. dmmal.ini

dmmal.ini 中每个实例都有一个单独的 mal 配置项，每个实例中有三个端口相关的配置：MAL_PORT、MAL_INST_PORT、MAL_DW_PORT。

- MAL_PORT

MAL 监听端口，用于创建 MAL 链路，同一个实例的 MAL 配置项中，MAL_PORT 不能和实例 dm.ini 中的两个端口相同，避免端口绑定冲突。

- MAL_INST_PORT

MAL 配置项中，MAL_INST_NAME 实例的监听端口，和实例 dm.ini 的 PORT_NUM 值相同。

- MAL_DW_PORT

守护进程监听端口,其他守护进程或监视器使用 MAL_HOST + MAL_DW_PORT 创建 TCP 连接。监视器配置文件 dmmonitor.ini 中，MON_DW_IP 就是一组 MAL_HOST:MAL_DW_PORT。

例如，这里以主库 GRP1_RT_A，备库 GRP1_RT_B 为例进行说明。

```
MAL_CHECK_INTERVAL      = 5
MAL_CONN_FAIL_INTERVAL  = 5
[MAL_INST1]
    MAL_INST_NAME           = GRP1_RT_A
    MAL_HOST                = 192.168.0.141
    MAL_PORT                = 5238
    MAL_INST_HOST           = 192.168.1.131
    MAL_INST_PORT           = 5236    #数据库实例监听端口，对应 dm.ini 中的 PORT_NUM
    MAL_DW_PORT         = 5239    #守护进程监听端口
[MAL_INST2]
    MAL_INST_NAME           = GRP1_RT_B
    MAL_HOST                = 192.168.0.142
    MAL_PORT                = 6238
    MAL_INST_HOST           = 192.168.1.132
    MAL_INST_PORT           = 6236   #数据库实例监听端口，对应 dm.ini 中的 PORT_NUM
    MAL_DW_PORT         = 6239   #守护进程监听端口
```

3. dmwatcher.ini

守护进程启动时，需要获取远程实例对应的配置项信息，创建到远程守护进程的 TCP 连接，这些信息通过 INST_INI 配置项来获取。

守护进程从 INST_INI 路径找到并读取 dm.ini、dmmal.ini、dmarch.ini，综合各个配置项得到远程守护进程的 IP、端口等信息并建立连接。

4. dmmonitor.ini

监视器配置项中以守护进程组为单位，通过 MON_DW_IP 配置到每组的所有守护进程的 IP 和端口信息。

配置格式为：“守护进程 IP 地址:守护进程 TCP 端口”。

其中守护进程 IP 地址和 dmmal.ini 中的 MAL_HOST 保持一致，守护进程 TCP 端口和 dmmal.ini 中的 MAL_DW_PORT 保持一致。

dmmal.ini 中配置有多少个 IP 和 PORT，dmmonitor.ini 就需要配置多少个对应的 “IP:PORT”信息，否则会无法接收到消息。

例如，这里以监控主库 GRP1_RT_A，备库 GRP1_RT_B 说明：

```
#这里只对 IP/端口信息举例说明，其他配置参数请参考后面的配置举例
[GRP1]
    MON_INST_OGUID          = 82379


    #IP 和 PORT 信息和 dmmal.ini 中的 MAL_HOST 和 MAL_DW_PORT 配置项一致
    MON_DW_IP               = 192.168.0.141:5239
    MON_DW_IP               = 192.168.0.142:6239
```

5.9 服务名配置

配置 DM 数据守护，一般要求配置连接服务名，以实现故障自动重连。连接服务名可以在 DM 提供的 JDBC、DPI 等接口中使用，连接数据库时指定连接服务名，接口会随机选择一个 IP 进行连接，如果连接不成功或者服务器状态不正确，则顺序获取下一个 IP 进行连接，直至连接成功或者遍历了所有 IP。

可以通过编辑 dm_svc.conf 文件，配置连接服务名。dm_svc.conf 配置文件在 DM 安装时生成，Windows 平台下位于%SystemRoot%\system32 目录，Linux 平台下位于 /etc 目录。

连接服务名格式：

```
SERVERNAME=(IP[:PORT],IP[:PORT],......)
```

dm_svc.conf 文件，常用配置项目说明：

- SERVERNAME

连接服务名，用户通过连接服务名访问数据库。

- IP

数据库所在的 IP 地址，如果是 IPv6 地址，为了区分端口，需要用[]封闭 IP 地址。

- PORT

数据库使用的 TCP 连接端口，可选配置，不配置则使用连接上指定的端口。

- LOGIN_Primary

服务名方式登录，始终会优先登录主库；LOGIN_Primary 表示是否仅登录主库，可以配置为 1 或者 0；1 表示不登录 Standby 模式的库，如果系统中只有 Standby 模式的库，登录失败、并报错；0 表示优先登录主库，没有可用主库情况下登录备库。默认值为 1。

- SWITCH_TIME

检测到数据库实例故障时，接口在服务器之间切换的次数；超过设置次数没有连接到有效数据库时，断开连接并报错。有效值范围 1~9223372036854775807，默认值为 3。

- SWITCH_INTERVAL

表示在服务器之间切换的时间间隔，单位为毫秒，有效值范围 1~9223372036854775807，默认值为 200。

例如，这里我们配置一个名为 dw_svc 的连接服务名，使用 dw_svc 连接数据守护中的数据库，即可实现故障自动重连。

```
dw_svc=(192.168.1.131:5236,192.168.1.132:5236)
LOGIN_Primary=(1)
SWITCH_TIME=(3)
SWITCH_INTERVAL=(1000)
```

关于 dm_svc.conf 的详细设置，请参考《DM7 系统管理员手册》2.1.1.4 dm_svc.conf 章节。

6 数据守护使用说明

本章以 1 个主库、2 个备库的读写分离集群（主库 A、备库 B、和备库 C）为例，对一些常见的数据守护场景进行说明，并相应的介绍各种监视器命令的执行流程。

6.1 正常运行状态

守护系统正常运行时，同一个守护进程组中，只有一个主库，其他的都是备库。

主库处于 Open 状态，主库守护进程也处于 Open 状态，守护进程控制文件是 Valid 有效状态。

所有备库也处于 Open 状态，所有备库守护进程处于 Open 状态，守护进程控制文件是 Valid 有效状态。

主库到所有备库的归档也都处于 Valid 有效状态。

MPP 主备系统中，所有主库的 dmmpp.ctl 都处于一致状态。

6.2 数据守护的启动

Normal 模式的库默认以 Open 状态启动，也可以通过增加启动参数 Mount，将数据库启动到 Mount 状态。而 Primary/Standby 模式的库启动后，自动进入 Mount 状态，因此，数据守护系统启动时，所有数据库实例处于 Mount 状态。所有守护进程处于 Startup 状态。

Local 类型的守护进程，直接 Open 数据库实例，并修改守护进程状态为 Open。其他类型的守护进程，需要相互协调信息，自动将数据库实例切换到 Open 状态，并将守护进程状态也切换为 Open。

非 Local 类型守护进程将 Standby 模式数据库实例切换为 Open 状态的条件是：

1.守护进程控制文件有效，并且没有标记为分裂状态。

2.检测到活动主库（Primary/Open），并且主备库数据保持一致（根据 LSN 判断）、或者是活动主库的可恢复备库。

3.守护进程 Open 数据库实例后，再将自己切换为 Open 状态。

因此，在检测到活动主库之前，备库一直保持 Mount 状态，守护进程保持 startup 状态，分裂备库也会一直保持 mount 状态。

非 Local 类型守护进程将 Primary 模式数据库实例切换为 Open 状态的条件是：

1.守护进程控制文件有效，并且没有标记为分裂状态。

2.守护进程组内所有数据库已经启动（不一定处于 Open 状态）。

3.根据数据库模式、LSN 信息判断，监控的数据库不是其他数据库的可恢复备库。

4.MPP 主备的 Primary 模式实例 Open 前，检查是否收到 dmmpp.ctl 信息，如果没有则报错（可能是配置错误，比如 dm.ini 中的 MPP_INI 配置项没有设置）。


注意：

数据守护系统启动流程，会设置主库 MPP 归档/实时归档/即时归档为 Invalid 状态。Primary 模式数据库实例切换为 Open 状态时，需要回滚活动事务、Purge 已提交事务，并重构回滚段，会引发数据变化，LSN 增长，因此数据守护启动完成后，主备库数据肯定是处于不一致状态。

主库守护进程 Open 主库后，会修改 INST_RECOVER_TIME 内存值为 0（默认 60 秒），确保备库 Open 后，马上启动故障恢复流程，同步主库数据完成后，重新将归档设置为 Valid 状态。

如果在故障恢复流程完成之前，主库故障，将无法执行备库接管；备库强制接管会引发守护进程组分裂。

读写分离集群，在 Timely 归档变为 Valid 之前，不会在备库上创建数据库连接，只读操作也无法分流到对应的备库。

6.3 强制 Open 实例

正常情况下，守护进程 dmwatcher 可以自动 Open 数据库实例，但某些情况下（比如备库硬件故障无法启动），数据守护系统不满足 6.2 介绍的启动条件，我们可以通过监视器执行 Open instance 命令，强制 Open 数据库实例。主备库都可以强制 Open，其执行流程如下：

假设需要强制 Open 数据库 A，只需要启动一个监视器，并在控制台上输入 Open instance A 即可完成强制启动。

如果实例 A 是 Standby 模式，强制 Open 的执行流程如下：

1. 通知 A 的守护进程切换为 Open Force 状态

2. 通知实例执行 Open 操作

3. 通知守护进程切换 Open 状态

如果实例 A 是 Primary 模式，强制 Open 的执行流程如下：

1. 通知 A 的守护进程切换为 Open Force 状态

2. 修改 A 的 MPP 归档/实时归档/即时归档状态为无效

3. 修改守护进程控制文件，增加实例 A 强制 Open 记录项

4. 通知实例执行 Open 操作

5. 通知守护进程切换 Open 状态

Primary 模式数据库实例切换为 Open 状态时，需要回滚活动事务、Purge 已提交事务，并重构回滚段，会引发数据变化，LSN 增长。因此，强制 Open 主库后，需要在守护进程控制文件中增加强制 Open 项。并且，这个操作可能会引发守护进程组分裂，比如：

1. 主库 A 故障

2. 备库 B 接管，成为主库

3. B 故障

4. A 重启，并强制 Open

5. A 和 B 数据不一致，并且无法恢复到一致状态。此时，B 起来，就产生守护进程组分裂



强制 Open 主库前，会设置主库 MPP 归档/实时归档/即时归档为 Invalid 状态。

强制 Open 主库命令，会修改主库守护进程 INST_RECOVER_TIME 内存值为 0（默认 60 秒），确保备库 Open 后，马上启动故障恢复流程，同步主库数据完成后，重新将归档设置为 Valid 状态。

如果在故障恢复流程完成之前，主库故障，将无法执行备库接管；备库强制接管会引发守护进程组分裂。

6.4 关闭数据守护系统

退出守护系统时，必须按照一定的顺序来关闭守护进程和实例。特别是自动切换模式，如果退出守护进程或主备库的顺序不正确，可能会引起主备切换，甚至造成守护进程组分裂。

通过监视器执行 Stop Instance 命令关闭数据守护系统，是最简单、安全的方式。命令执行成功后，数据库实例正常关闭。但守护进程并没有真正退出，而是将状态切换为 Shutdown 状态。

Stop Instance 命令内部流程如下：

1) 通知守护进程切换为 Shutdown 状态

2) 通知主库退出

3) 通知其他备库退出

如果使用手动方式关闭数据守护系统，请严格按照以下顺序：

1) 如果启动了确认监视器，先关闭确认监视器（防止自动接管）

2) 关闭备库守护进程（防止重启实例）

3) 关闭主库守护进程（防止重启实例）

4) Shutdown 主库

5) Shutdown 备库



注意：先关闭主库再关闭备库，顺序一定不能错。

因为主库 Shutdown 过程中，需要 Purge 所有已提交事务，会修改数据，并产生 Redo 日志。如果先 Shutdown 备库，会导致主库发送归档日志失败，并且由于主库已经处于 Shutdown 状态，会导致主库异常关闭。

6.5 主备库切换

主库维护，滚动升级等场景，可以执行 Switchover 命令，实现主备库切换。如果存在多个备库，需要先执行 Choose Switchover 命令，选出守护进程组中可以切换的备库。

Choose Switchover 命令选择可切换备库的条件如下：

1. 主库守护进程是 Open 状态

2. 备库守护进程是 Open 状态

3. 主、备库守护进程控制文件 dmwatcher.ctl 相同，且为 Valid 状态

4. 主库正常运行

5. 备库正常运行

6. 主库处于 Open 状态

7. 备库处于 Open 状态

8. 主库到备库的归档是 Valid 状态

假定选出的可切换备库是 B，Switchover 切换流程如下：

1. 通知主备库守护进程，切换为 Switchover 状态
2. 通知主库(A) Mount
3. 实时守护或 MPP 守护类型下，通知备库(B) APPLY KEEP_BUF
4. 通知备库(B) Mount
5. 通知(A) 切换为 Standby 模式
6. MPP 守护类型下，通知(A) 修改 MPP_INI 为 0
7. 通知(B) 切换为 Primary 模式
8. 通知(B) 修改归档状态为无效
9. 通知守护进程(B)修改控制文件，增加一条切换记录
10. MPP 守护类型下，通知先更新执行 Switchover 命令的守护进程组，更新主库的 dmmpp.ctl 文件，再通知更新其他组主库的 dmmpp.ctl 文件
11. 通知新的备库(A) Open
12. 通知新的主库(B) Open
13. 通知守护进程切换为 Open 状态
14. 清理所有守护进程上记录的监视器信息

主备库切换在实现逻辑上等同于主备库正常状态下，用户主动发起的 Takeover 操作。Swithover 完成后，主备库之间数据是不完全同步的；要由新主库 B 的守护进程通过 Recover 流程，重新同步数据和 dmwatcher.ctl 到新备库 A。



Switchover 命令，会修改切换后主库守护进程 INST_RECOVER_TIME 内存值为 0（默认 60 秒），确保尽快启动故障恢复流程，同步主库数据完成后，重新将归档设置为 Valid 状态。在故障恢复流程完成之前，再次执行 Switchover 命令会报错，如果主库故障，备库接管将会报错；备库强制接管会引发守护进程组分裂。

6.6 主库故障、备库接管

当出现硬件故障（掉电、存储损坏等）原因，主库无法启动，并且主库短期不能恢复情况下，可使用备库接管功能，将备库切换为主库继续对外服务。

故障自动切换模式下，确认监视器会自动选择符合条件的备库进行接管。故障手动切换模式下，可以先在监视器上执行 Choose Takeover 命令，选出守护进程组中可以接管的备库。

为了避免备库接管后，造成守护进程组分裂，执行 Takeover 必须满足下列条件：

1. 主库是 Primary 模式、Open 状态时，产生故障
2. 主库守护进程故障，故障前是 Open/Recovery 状态；或者主库守护进程正常
3. 故障主库到接管备库的归档状态为 Valid
4. 故障主库与接管备库之间的 MAL 链路断开
5. 接管备库是 Standby 模式、Open 状态
6. 接管备库的 dmwatcher.ctl 控制文件状态为 Valid
7. 故障主库和接管备库的 dmwatcher.ctl 控制文件相同

假设主库 A 故障时，选出接管备库 B，在监视器上输入 Takeover 命令，就可以执行备库接管。备库接管的执行流程包括：

1. 监视器通知守护进程切换为 Takeover 状态
2. 实时/MPP 守护类型下，通知备库(B) APPLY KEEP_BUF
3. 通知备库(B) Mount
4. 通知(B) 切换为 Primary 模式
5. 通知(B) 修改 MPP 归档/实时归档/即时归档状态为 Invalid
6. 通知守护进程(B)修改控制文件， 增加一条接管记录
7. MPP 守护类型下，通知接管守护进程组更新主库的 dmmpp.ctl 文件，再通知更新其他守护进程组主库的 dmmpp.ctl 文件
8. 通知新的主库(B) Open
9. 通知守护进程切换为 Open 状态


注意：

例如，主库 C 故障，备库 B 接管。执行 Takeover 命令后，会修改守护进程(B)的 INST_RECOVER_TIME 内存值为 0（默认 60 秒），确保尽快启动 B->C 的故障恢复流程，同步主库数据完成后，重新将 B->C 的归档设置为 Valid 状态。在故障恢复流程完成之前，主库（B）故障，备库（C）无法接管；强制备库（C）接管会引发守护进程组分裂。

6.7 备库强制接管

有些情况下，备库接管会失败，但主库不能启动或者及时恢复对外服务的情况下，可以使用 Takeover Force 命令，进行备库强制接管。强制接管具有一定的风险，可能导致备库和故障主库数据不一致，而造成部分数据的丢失，出现数据库分裂的情况，所以应该慎重使用。

比如主库和守护进程故障时，监视器未启动，用户启动监视器后，由于监视器并未收到故障主库任何信息，因此不满足 Takeover 条件，执行 Takeover 会报错。如果用户可确认主库故障时主备数据是一致的（比如故障时主库未执行操作，主备库归档有效的，并且两者的 SLSN 一致）、或者丢失小部分数据的影响可忽略、或者丢失小部分数据的影响小于主库持续宕机造成的影响，则可以考虑执行 takeover force 强制接管。

强制接管，先通过 Choose Takeover Force 选出符合接管条件的备库，再执行 Takeover Force 命令。备库强制接管时，如果接管备库是处于 Mount 状态/Standby 模式的库，则会自动 Open 备库，其他执行流程与备库接管一致。强制接管的条件包括：

1. 不存在活动主库
2. 备库守护进程处于 Open 或 Startup 状态
3. 备库实例运行正常
4. 备库是 Standby 模式
5. 备库处于 Open 或 Mount 状态
6. 备库 SLSN 必须是所有备库中最大的
7. 备库守护进程控制文件必须有效



注意：与 Takeover 命令一样，Takeover Force 命令会修改主库守护进程的 INST_RECOVER_TIME 内存值为 0（默认 60 秒），确保尽快启动故障恢复流程。强制接管具有一定的风险，可能导致备库和故障主库数据不一致，而造成部分数据的丢失、数据库分裂，所以应该综合考虑当时情况，慎重使用。

6.8 主库故障重启（备库接管前重启）

主库故障后立即重启，此时主库的守护进程变成 Startup 状态，重新进入守护进程的启动流程，将所有备库归档设置成无效状态，并重新 Open 主数据库。Open 成功后继续作为主库，当检测到备库正常时会启动 Recovery 处理流程，重新同步主备库数据。

6.9 备库故障处理

备库产生故障时，主库的处理流程对手动切换、自动切换模式处理上有些差异。对于手动切换模式，检测到备库故障，满足 Failover 条件时，立即切换到 Failover 状态。自动切换模式备库数据库异常，如果备库守护进程是活动的，主库守护进程直接切换到 Failover 状态；如果备库守护进程一起故障，则切换为 Confirm 确认状态，等待确认监视器的确认消息，详情参考 4.2 状态确认 。

主库 Failover 条件：

1. 备库实例故障
2. 主库->故障备库的归档状态是 Valid
3. 主库和故障备库之间的 MAL 链路断开、或者主库处于 Suspend 状态

主库 Failover 处理流程：

1. Suspend 主数据库
2. 设置主库->故障备库归档无效
3. 重新 Open 主库

6.10 公共网络故障

主库公共网络故障，主备库正常、内部网络正常情况下，用户无法连接主库执行正常的

数据库操作。这种情况下，用户可以通过 Switchover 命令，将备库切换为主库，确保数据库服务不受影响。

读写分离集群，备库公共网络故障，主备库正常、内部网络正常情况下，用户的所有请求会自动分发到主库执行。

6.11 内部网络故障

DM 数据守护对内部网络的可靠性提出了很高的要求，但是在实际应用中（比如异地容灾），存在很多不可控因素，内部网络无法保证绝对可靠。守护进程和守护进程之间、守护进程和监视器之间会超时机制来检测是否出现故障，当内部网络出现故障时，超过设置时间未收到远程消息，会认定远程故障。

下面的表格说明内部网络故障场景以及守护进程的处理策略：

表 6.1 故障场景和处理策略

故障类型	手动切换处理策略	自动切换处理策略
主库内部网卡故障	外部网卡正常时，归档失败，主库挂起，主库 FAILOVER，归档设置无效，异步工作。	主库守护进程保持 confirm 状态，确认监视器自动寻找备库自动接管。详见 4.2、4.3 节
备库内部网卡故障	外部网卡正常时，归档失败，主库挂起，主库 FAILOVER，归档设置无效，异步工作。	主库守护进程 FAILOVER，归档设置无效，异步工作。详见 4.2、4.3 节
主备库交换机故障	外部网卡正常时，归档失败，主库挂起，主库 FAILOVER，归档设置无效，异步工作。	主库守护进程 FAILOVER，归档设置无效，异步工作。详见 4.2、4.3 节
主备网卡同时故障	外部网卡正常时，归档失败，主库挂起，主库 FAILOVER，归档设置无效，异步工作。	主库守护进程保持 confirm 状态，备库不会自动切换

自动切换模式下，当发生主库内部网络故障，其他网络均正常，归档失败，主库 SUSPEND，导致连接主库的会话被挂起，一直无法返回，备库自动接管后，这些会话连接无法自动切换到新主库上。为此，达梦采用超时机制断开由于归档失败导致挂起时的所有会

话，通过配置 dm.ini 中的 SESS_FREE_IN_SUSPEND 实现，默认 60s。

网络故障恢复后，对于手动切换模式，没有强制接管的情况下，主库的守护进程会自动判断备库是否能加入，能加入则启动 RECOVERY 流程。对于自动切换模式已经被接管的情况，守护进程会将故障主库强制退出（shutdown），故障主库重启后，还需要守护进程再去判断是否可作为备库重加入守护系统，如果可以，则启动恢复流程（切换为 standy 模式，重新将实例 open），如果不能，才修改为分裂状态。

当内部网络是由于拔掉网线、或者禁用网卡等原因导致的故障，MAL 系统检测到链路断开的时间由 dmmal.ini 中配置的 MAL_CHECK_INTERVAL 以及 MAL_CONN_FAIL_INTERVAL 决定。会话的中断时间会比杀掉实例、掉电等花费的时间长。下面举例说明：

dmmal.ini 配置

```
MAL_CHECK_INTERVAL      = 10
MAL_CONN_FAIL_INTERVAL = 10
```

dmwatcher.ini 配置

```
DW_ERROR_TIME               = 10
```

拔掉备库网线后，主库上 mal 检测线程根据 MAL_CHECK_INTERVAL 间隔检查，时间<=10s，检查认定网络故障需要 10s，再加上守护进程认定远程网络故障时间 DW_ERROR_TIME 为 10s，在最坏的情况下，应用连接等待主库响应最长的时间可能会是三个时间的累加，也许会达到 30s。

6.12 故障库数据同步

当实例由于各种原因故障，重启或者网络恢复后，守护进程会自动判断能否加入当前主库系统，判断条件和数据恢复处理流程详见细 3.1.8 故障恢复处理 章节。恢复的中间过程可通过附录中的动态视图进行查看。

6.13 备库重建

当主备库所在硬件出现故障、或者数据库文件损坏导致数据损毁时，可按照数据守护搭建一章中介绍的几种方式来重新准备数据。

重建库要选定一个当前有效库或者一个历史备份。

下面以备库 B 数据损坏，重新配置数据为例，说明备库重建流程：

1．重新初始化数据库

```
./dminit path=/dm7/data
```

2．启动 DIsql 登录主库 A，执行联机备份

```
SQL> BACKUP DATABASE FULL BACKUPSET '/dm7/data/BACKUP_FILE_01';
```

3．将备份文件拷贝至 B 所在的机器上，执行脱机还原与恢复

```
./dmrman CTLSTMT="RESTORE DATABASE '/dm7/data/DAMENG/dm.ini' FROM BACKUPSET
'/dm7/data/BACKUP_FILE_01'"
./dmrman CTLSTMT="Recover DATABASE '/dm7/data/DAMENG/dm.ini' FROM BACKUPSET
'/dm7/data/BACKUP_FILE_01'"
```

4． 重新配置 B 的 dm.ini、dmmal.ini、dmarch.ini 和 dmwatcher.ini 配置文件，并拷贝主库的 dmwatcher.ctl 文件

5．以 mount 方式启动 B

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```

6．DIsql 登录 B，设置 OGUID

```
SQL>sp_set_oguid(82379);
```

7．修改数据库模式为 Standby

```
SQL>alter database standby;
```

8．启动 B 的守护进程

```
./dmwatcher /dm7/data/DAMENG/dmwatcher.ini
```

执行以上步骤后，恢复 B 的准备过程已经完成。接下来，数据守护系统会将 B 作为备库重加入数据守护系统，A 的守护进程会自动通知同步数据到 B，最终恢复主备库数据到一致状态。

如果数据规模比较大、联机备份耗时较长、应用压力比较大的情况下，主库联机备份、备库脱机还原过程中，主库可能又新产生了大量归档日志。使用上述步骤重建备库后，主库向备库同步历史数据的时间会比较久，主备库数据在比较长的一段时间内处于不一致状态。用户还可以利用归档备份、还原和归档恢复功能，将备库数据恢复到更加接近主库最新状态，有效减少备库加入主备系统后，历史数据同步时间。执行步骤如下：

9．重新初始化数据库

```
./dminit path=/dm7/data
```

10. 启动 DIsql 登录主库 A，执行联机备份

```
SQL> BACKUP DATABASE FULL BACKUPSET '/dm7/data/BACKUP_FILE_02' WITHOUT LOG;
```

11. 将备份文件拷贝至 B 所在的机器上，执行脱机还原

```
./dmrman CTLSTMT="RESTORE DATABASE '/dm7/data/DAMENG/dm.ini' FROM BACKUPSET
'/dm7/data/BACKUP_FILE_02'"
```

12. 启动 DIsql 登录主库 A，查看备份集的 BEGIN_LSN，并从 BEGIN_LSN 开始备份主库的归档日志。（这里假设查询到的备份集 BEGIN_LSN 是 38491）

```
SQL> SELECT BACKUP_NAME,BEGIN_LSN FROM V$BACKUPSET;
SQL> BACKUP ARCHIVE LOG FROM LSN 38491 BACKUPSET '/dm7/data/ARCH_BAK_02';
```

13. 将归档备份集拷贝至 B 所在的机器，还原归档后、利用这些归档日志进行数据库恢复

```
./dmrman CTLSTMT="RESTORE ARCHIVE LOG FROM BACKUPSET '/dm7/data/ARCH_BAK_02' TO
ARCHIVEDIR '/dm7/data/arch'"
./dmrman CTLSTMT="Recover DATABASE '/dm7/data/DAMENG/dm.ini' WITH ARCHIVEDIR
'/dm7/data/arch'"
```

14. 重新配置 B 的 dm.ini、dmmal.ini、dmarch.ini 和 dmwatcher.ini 配置文件，并拷贝主库的 dmwatcher.ctl 文件

15. 以 mount 方式启动 B

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```

16. DIsql 登录 B，设置 OGUID

```
SQL>sp_set_oguid(82379);
```

17. 修改数据库模式为 Standby

```
SQL>alter database standby;
```

18. 启动 B 的守护进程

```
./dmwatcher /dm7/data/DAMENG/dmwatcher.ini
```

6.14 守护进程组分裂

组分裂的概念和导致分裂的场景参考 2.2.12 一节，对于分裂的数据库需要进行人工

干预，重新准备数据或者对分裂数据进行修复后，才允许重新加入守护系统。

借助监视器的 show 命令、或者 tip 命令可以查看守护系统中是否发生实例的分裂，对于已发生的分裂，可以借助以下方法找出分裂产生的原因：

1. 查看分裂实例的服务器 log 日志，查找带有“[!!!”和“!!!]”标签的 log 信息，log 信息格式形如 “[!!! LOG_INFO !!!]”，记录有实例分裂的详细原因。

2. 如果分裂实例的守护进程控制文件状态不是 Valid，可借助监视器的 show dmwatcher ctl 命令，根据修改状态的那条记录信息对应的 DESC 字段找出原因。

3. 根据服务器和监视器的 log 日志，找出历史操作信息，分析产生分裂的原因。

发生分裂后，用户需要选择适当的主库作为最新主库。当库的数据量不太大、重新准备数据时间不需太长的情况下，可简单采用备库重建的方法加入系统。当数据量非常大，备库重建短时间内无法完成，可考虑使用分裂库修复的方式修复好分裂库后，再重新加入系统。

6.15 dmmpp.ctl 不一致

MPP 主备环境，在某些故障场景中，可能会出现 dmmpp.ctl 控制文件不一致的情况，可以借助监视器的 check mppctl 和 recover mppctl 命令来检测和恢复。

正常运行时，所有 MPP 主节点的 dmmpp.ctl 文件内容是完全一致的。如果某个组中的主备库实例发生改变（如执行 switchover、takeover 等），导致 dmmpp.ctl 变化，需要及时同步到所有组的主库节点上。

在执行数据同步过程中可能发生故障，导致 dmmpp.ctl 未同步，就可能造成控制文件的不一致。例如，两节点 MPP，分成两个守护进程组（GRP1 和 GRP2），组 GRP1 包含两个实例（主库 GRP1_MPP_EP01 和备库 GRP1_MPP_EP11），组 GRP2 包含两个实例（主库 GRP2_MPP_EP02 和备库 GRP2_MPP_EP22）。在监视器上执行 switchover 命令，将 GRP1 的两个实例进行切换。

当执行到 switchover 流程中更新 dmmpp.ctl 文件时，可能会出现下列故障情况，导致 dmmpp.ctl 处于不一致状态。

1. 更新 GRP1 的 dmmpp.ctl 文件时，监视器故障或者链路断开等原因导致更新失败，GRP1 和 GRP2 的 dmmpp.ctl 文件都没有更新，但 GRP1 的主库已经切换为 GRP1_MPP_EP11。

2. 更新 GRP1 的 dmmpp.ctl 文件成功，但更新 GRP2 的 dmmpp.ctl 失败，GRP1

和 GRP2 的 dmmpp.ctl 文件不一致。

这两种不一致状态可以通过 recover mppctl 命令尝试重新恢复到一致状态。

监视器执行 recover mppctl 的步骤如下：

1. dmmonitor 根据当前守护系统中的主库信息，重新生成有效的 dmmpp.ctl 文件。
2. dmmonitor 将有效的 dmmpp.ctl 文件内容发送给主库。



注意：如果存在主库故障，还没有处理时，recover mppctl 会跳过故障组，只恢复存在有效主库组的 dmmpp.ctl 文件。故障组主库恢复正常后，需要重新调用 recover mppctl 进行恢复。dmmpp.ctl 文件不一致时，MPP 无法正常提供数据库功能。

6.16 确认监视器未启动

故障自动切换模式下，确认监视器必须一直处于启动状态，如果故障前未启动确认监视器，主库故障后备库并不会自动切换为主库。

故障自动切换模式下，确认监视器故障或者未启动，如果主备库之间出现网络故障，主库守护进程会处于确认状态，实例处于挂起 SUSPEND 状态。当主库守护进程检测到网络恢复正常后，即使在没有确认监视器的情况下，也会变成 FAILOVER 状态，后续进行正常的备机恢复处理，而不会一直处于 CONFIRM 确认状态。

启动一个监视器，通过该监视器的 show monitor 命令可以查看连接到指定守护进程的所有监视器信息，以此可以检查确认监视器是否启动了。

6.17 备库维护

守护进程正常运行时，如果需要主动维护备库，并且不启动主库的 failover 故障处理流程（可参考 6.9 备库故障处理 ），可以通过监视器命令调整主库守护进程的恢复时间间隔，并主动将备库归档失效来达到目的。

监视器提供 set recover time 命令和 set arch invalid 命令支持备库的主动维护操作。

其中 remove instance 命令包含了设置恢复间隔、通知归档失效以及通知备库退出的操作，如果用户想简化手工操作步骤，可直接使用 remove instance 来完成，如果暂

时不需要退出备库，可结合 set recover time 命令和 set arch invalid 命令来完成，具体命令使用说明如下：

1. set recover time 和 set arch invalid 命令

set recover time 命令可动态设置主库守护进程内存中对指定备库的恢复间隔，在将备库归档失效之前，需要先使用此命令对需要主动维护的备库设置一个较大的间隔值，然后再使用 set arch invalid 命令，修改主库到备库的归档为无效状态，执行过这两个操作后，即可正常退出备库，执行维护操作。

执行 set arch invalid 将备库归档失效，是为了防止备库退出后，主库启动故障处理流程，执行 suspend 等操作影响正常的应用执行。而在此之前先执行 set recover time 是为了防止在备库归档失效之后，实际退出之前，主库的守护进程立马发起 recover 恢复流程，又重新将备库归档置为有效状态，在备库退出后，还会执行故障处理的情况发生。

2. remove instance 命令

remove instance 命令包含有以下 4 个操作步骤，此命令主要是为了简化备库维护的手工操作步骤，一次完成将指定备库移出守护进程组的所有操作。

1) 通知主库设置指定备库的恢复间隔为 86400 s（相当于执行 set recover time 命令）

2) 通知主库修改指定备库的归档状态无效（相当于执行 set arch invalid 命令）

3) 通知切换备库的守护进程为 SHUTDOWN 状态

4) 通知备库正常退出

另外此命令也支持异步备库的执行，对于异步备库，会跳过步骤 1)、2)，直接执行步骤 3)、4)，执行退出操作。

此命令执行成功后，在备库维护工作完成重启之后，主库内存中对指定备库的恢复间隔时间仍然是 86400 s，如果想尽快启动恢复流程，需要再通过手工执行 set recover time 命令调整指定备库的恢复间隔，此限制只针对非 local 守护类型的备库。

6.18 滚动升级

通过 DM 数据守护，可以确保数据库服务不中断情况下，实现滚动升级。

这里以一主一备的配置方式举例说明，如图图 6.1 所示。

假设初始状态为主库 DM1，部署在 DW_P 机器上，备库 DM2，部署在 DW_S 机器上，监

视机 DW_M 滚动升级操作步骤如下：

1． 关闭备库 DM2 的守护进程，正常关闭备库实例 DM2；这个过程中，主库 DM1 的守护进程会检测到备库故障，并进行故障处理，修改归档状态无效。

2． 升级 DW_S，重新安装新版本服务器，或者直接将新的执行程序和动态库替换旧版本

3． 启动 DM2 实例，启动 DM2 的守护进程

4． 等待历史数据同步完成，DM2 重新加入数据守护系统（可以通过监视器 DW_M 查看数据守护系统是否恢复正常）

5． 关闭主库 DM1 的守护进程，正常关闭实例 DM1；使用监视器的接管命令将 DM2 接管为新的主库。

6． 升级 DW_P，重新安装新版本服务器，或者直接将新的执行程序和动态库替换旧版本

7． 启动 DM1 实例，启动 DM1 的守护进程

8． 等待历史数据同步完成，DM1 作为备库重新加入数据守护系统（可以通过监视器 DW_M 查看数据守护系统是否恢复正常）

9． 在 DW_M 监视器控制台上输入 Switchover 命令，重新将 DM1 切换为主库，至此滚动升级完成。

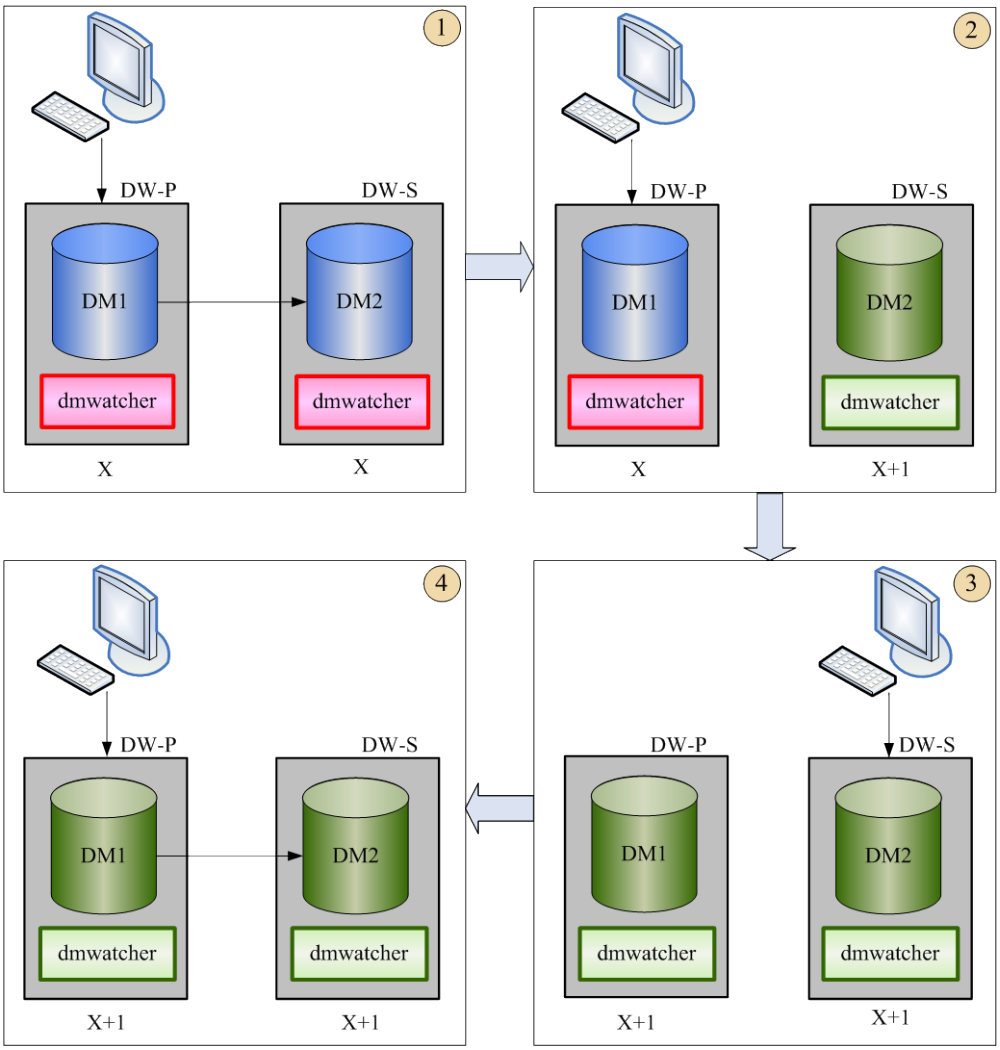


图 6.1 滚动升级

多个备库的情况，可同样参考上面的步骤 1 到步骤 4，先统一升级所有备库实例，步骤 5 中，可任意选择一个升级后的备库接管，最后升级主库完成后，再借助 Switchover 命令切换主备库。

警告：**由于涉及回滚段改造，DB_VERSION 版本号为 0x00070005 的数据库无法滚动升级到 0x00070006 版本。由于日志压缩格式重构，0x00070007 版本数据库无法滚动升级到 0x00070008 版本。**

6.19 备库文件损坏

重演 Redo 日志过程中，如果访问数据文件失败（用户通过操作系统命令删除、或者文件所在存储故障、或者表空间处于 Offline 状态），备库会强制 Halt，相关错误信息会写

入日志文件中。因此，建议用户在确保主库不会修改对应表空间文件情况下，执行 Offline 备库表空间操作；或者先通过 Remove Instance 命令将备库移出守护进程组后，再进行 Offline 表空间操作。

如果是由于备库表空间处于 Offline 状态，导致备库故障，但数据文件本身并没有损坏情况下，我们可以直接在备库执行 Alter tablespace *ts1* online;避免备库重演 Redo 日志异常。

如果备库数据文件损坏，则可以通过表空间还原、或者还原指定数据文件来修复。在数据修复前，要先通过监视器执行 remove instance 命令将对应备库移出守护进出组，数据修复完成后，再重新将备库加入守护进程组。

修复的前提条件是有备库的完全备份集或者该表空间的备份集，可以任选其中一个备份集，来执行修复操作。

参考步骤：

以下示例中备库所在的守护进程组名为 GRP1，备库实例名为 GRP1_RT_02，备库数据文件存放路径为 F:\data，备库上发生损坏的表空间名称为 TS1，当前已经有一个完全备份集“DBBAK”和一个 TS1 的表空间备份集“TS1BAK”。

完全备份示例：

```
BACKUP DATABASE FULL BACKUPSET 'DBBAK';
```

表空间 TS1 备份示例：

```
BACKUP TABLESPACE TS1 BACKUPSET 'TS1BAK';
```

对表空间 TS1 进行数据修复：

修复方法 1：

1．使用监视器命令将备库移出守护环境

命令格式：

```
remove instance [group_name.]inst_name
```

只有一组时可以省略组名 group_name，如果没有登录监视器，则需要先在监视器上执行 login 登录命令。

```
remove instance GRP1.GRP1_RT_02
```



注意：此命令会通知主库将备库归档失效，强制设置主库到备库的恢复间隔为 24 小时，并关闭备库的守护进程，将备库实例退出。

在命令执行时，如果备库由于数据文件损坏已经被强制 halt，则备库的退出操作会执行失败，如果备库被强制 halt 后监视器又重启过，没有收到过备库的历史消息，命令会执行失败，可参考“修复方法 2”的步骤进行修复。

2.MOUNT 方式重新启动备库

实例启动时，如果存在有损坏的数据文件，对应的表空间会被置为 offline 状态，可以通过还原表空间或者还原指定的数据文件进行修复，使用 disql 登录备库，选择其中一种还原方式执行即可。

- 还原表空间

```
RESTORE TABLESPACE TS1 FROM BACKUPSET 'TS1BAK';
```

- 还原指定的数据文件

1) 查找数据文件路径

```
--通过动态视图查找需要还原的数据文件路径，必要时，可以导出 dm.ctl 文件定位表空间
select id,path from v$datafile;
行号      ID          PATH
---------- ----------- -----------------------------------
1         0           f:\data\DAMENG\SYSTEM.DBF
2         0           f:\data\DAMENG\ROLL.DBF
3         0           f:\data\DAMENG\TEMP.DBF
4         0           f:\data\DAMENG\MAIN.DBF
5         NULL        文件[f:\data\DAMENG\ts1.dbf]不存在
```

2) 使用查到的数据文件路径执行还原

```
RESTORE TABLESPACE TS1 DATAFILE 'f:\data\DAMENG\ts1.dbf' FROM BACKUPSET
'TS1BAK';
```

3．打开备库的守护进程监控

步骤 1 remove instance 命令执行完成后，会将关闭备库的守护进程监控功能，备库的守护进程处于 SHUTDOWN 状态，需要在监视器上执行下面命令，打开守护进程监控，：

```
startup dmwatcher GRP1
```

正常情况下，监控打开后，主库正常的情况下，守护进程会将备库自动 OPEN（这里不

考虑备库发生分裂的情况），主库的守护进程在 24 小时之后才会启动 recovery 流程恢复备库，此时主备库之间并不会有数据同步操作（remove instance 命令将备库的 recover 恢复间隔设置为 24 小时）。

如果此时主库故障，备库的守护进程无法执行自动 OPEN 操作，可以借助监视器的强制 OPEN 命令将备库手动 OPEN，如果没有登录监视器，则需要先在监视器上执行 login 登录命令。

```
open instance GRP1.GRP1_RT_02
```

4．Online 表空间

使用 disql 登录备库，执行下面语句 Online TS1 表空间：

```
ALTER TABLESPACE TS1 ONLINE;
```

5．使用监视器命令设置备库的恢复间隔

如步骤 3 中描述，remove instance 命令会强制设置主库的守护进程在 24 小时之后再启动到备库的恢复间隔，如果需要立即启动主备库之间的数据同步，将备库恢复正常，可以使用监视器的 set recover time 命令将恢复间隔设置为 0，如果没有登录监视器，则需要先在监视器上执行 login 登录命令。

```
set recover time GRP1.GRP1_RT_02 0
```

命令执行完成后，主库的守护进程会启动 recovery 流程，待主备数据同步到一致状态后，备库即恢复到正常状态，重加入守护系统成功。

修复方法 2：

此修复过程使用 disql 执行语句来完成，备库示例环境同修复方法 1，在修复期间不允许启动守护进程，避免 disql 执行语句报错，另外需要根据情况修改 dm.ini 的配置参数。执行命令前，备库已经被 halt，或者手动退出，守护进程也正常 exit 退出。

1．根据情况修改 dm.ini 的配置参数

查看 dm.ini 中的 ALTER_MODE_STATUS 参数，如果配置为 0（不允许手动修改实例的模式/状态），则需要将此参数暂时修改为 1，后续需要执行 OPEN FORCE 操作将实例强制 OPEN。

```
ALTER_MODE_STATUS       = 1    #允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 1    #允许 offline 表空间
```

2．MOUNT 方式重新启动备库，不启动备库的守护进程

实例启动时，如果存在有损坏的数据文件，对应的表空间会被置为 offline 状态。

使用 disql 登录备库：

1) 将备库强制 OPEN

```
ALTER DATABASE OPEN FORCE;
```

2) 还原表空间或还原指定的数据文件，选择其中一种还原方式即可。

a. 还原表空间：

```
RESTORE TABLESPACE TS1 FROM BACKUPSET 'TS1BAK';
```

b. 还原指定的数据文件：

```
1) 查找数据文件路径
    --通过动态视图查找需要还原的数据文件路径，必要时，可以导出 dm.ctl 文件定位表空间
    select id,path from v$datafile;
    行号      ID          PATH
    ---------- ----------- -----------------------------------
    1         0           f:\data\DAMENG\SYSTEM.DBF
    2         0           f:\data\DAMENG\ROLL.DBF
    3         0           f:\data\DAMENG\TEMP.DBF
    4         0           f:\data\DAMENG\MAIN.DBF
    5         NULL        文件[f:\data\DAMENG\ts1.dbf]不存在


2) 使用查到的数据文件路径执行还原
RESTORE TABLESPACE TS1 DATAFILE 'f:\data\DAMENG\ts1.dbf' FROM BACKUPSET
'TS1BAK';
```

3. Online 还原过的表空间

使用 disql 登录备库，执行以下语句：

```
ALTER TABLESPACE TS1 ONLINE;
```

4. 改回 dm.ini 的配置参数

如果在步骤 1 中修改过 ALTER_MODE_STATUS 参数，需要再执行此步骤，重新将其修改为 0，否则可跳过此步骤。

```
ALTER_MODE_STATUS       = 0    #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2    #不允许备库 offline 表空间
```

5.MOUNT 方式重新启动备库，启动备库的守护进程

守护进程组中有正常主库的情况下，备库的守护进程会通知备库自动 Open，可以参考修复方法 1 中的步骤 5，在监视器上执行 set recover time 命令，通知主库的守护进程立即启动备库的恢复流程，将备库恢复到正常状态，如果没有登录监视器，执行此命令前需要先在监视器上执行 login 登录命令。

```
set recover time GRP1.GRP1_RT_02 0
```

命令执行完成后，主库的守护进程会启动 recovery 流程，待主备数据同步到一致状态后，备库即恢复到正常状态，重加入守护系统成功。

引发备库表空处于 Offline 状态的主要原因包括：

1.直接在备库上执行 Offline 表空间操作

2.在主库上执行 Offline 表空间操作后，执行 Switchover 命令进行主备库切换

3.备库数据文件损坏，启动时自动将对应表空间变成 Offline 状态



注意：主库表空间 Offline/online 操作，不会生成 Redo 日志，也不会同步到备库，也就是说主库的表空间 Offline/online 操作不会影响备库。但用户可以直接登录备库执行 Offline/Online 表空间操作，同样，备库的表空间 Offline/online 操作也不会影响主库。

6.20 实时/读写分离/MPP 备库数据同步情况分析

正常情况下实时/MPP 备库数据与主库数据基本一致，可以通过监视器的 show 命令观察主备库的 LSN 差距来大致了解数据同步情况，如果主备库的 LSN 值差距很小，说明主备库数据基本一致。当备库的 LSN 值远小于主库 LSN 值时，就需要引起重视，分析原因。

主备库 LSN 值相差很大的情况下，可以借助监视器的 show 命令查看状态来进行分析，下面的分析以守护系统中存在正常主库为前提，如果主库发生故障，是无法再向备库同步数据的，需要等主库恢复正常后，再观察主备库的 LSN 同步情况。

1．备库的守护进程状态是否正常（WSTATUS 字段）

如果 WSTATUS 是 ERROR 状态，说明监视器接收备库的守护进程消息超时，需要查看守护进程是否故障，如果发生故障，则需要根据守护进程日志分析故障原因，待守护进程恢复正常后再观察备库的 LSN 增长情况。

如果 WSTATUS 是 SHUTDOWN 状态，说明备库的守护进程监控被关闭，需要借助监视器的 startup dmwatcher 命令打开监控，打开之后再观察备库的 LSN 增长情况。

2．备库实例状态是否正常（INST_OK 字段）

如果 INST_OK 字段是 ERROR 状态，说明备库的守护进程接收备库实例消息超时，需要确认备库实例是否故障，如果发生故障，则需要根据服务器日志分析故障原因，待备库恢复正常后，再观察 LSN 增长情况。

3．备库归档状态是否有效（RSTAT 字段）

1）备库的 RSTAT 字段是 INVALID 状态

需要等主库的守护进程发起 recover 流程，同步主备库数据。

使用监视器的 show 命令，查看主库的守护进程状态，如果已经处于 recovery 状态，可以使用 disql 登录主库查询 V$RECOVER_STATUS 视图，查看当前的恢复信息，如果正在恢复的是其他备库，则需要等待其他备库恢复完成。

如果主库的守护进程不是 recovery 状态，则需要根据 3.1.8 故障恢复处理小节描述的 recovey 条件来判断备库是否满足恢复条件，如果不满足条件，则需要根据不满足的情况来做不同的处理。

2）备库的 RSTAT 字段是 VALID 状态

如果备库归档处于有效状态，并且备库实例是 OPEN 状态，主库也是正常 OPEN 状态，则说明主备库在正常同步数据，可以通过监视器的 show 命令查看备库的 lsn 是否在持续增长，以及备库的 slsn/sslsn 与 cur_lsn/file_lsn 的差值，如果差值很大，说明备库上有比较多的日志堆积。

可以登录备库查询 v$rapply_sys/v$rapply_log_task 视图来查看日志的堆积情况，如果日志堆积数量超过 3000 并且新收到的日志重演任务数和未处理的任务数差值超过 1000，则备库会执行强制 halt 处理。

6.21 异步备库数据同步情况分析

源库通过配置的定时器来定时同步数据到异步备库，可以根据定时器的时间间隔，使用监视器的 show 命令来观察异步备库的 LSN 是否在定期增长，或者使用监视器的 show arch info 命令来查看源库到异步备库的数据同步情况。

如果这两种方式都没有观察到源库到异步备库的数据同步，则需要从以下几个方面来分析原因：

1．源库是否配置正确

1) 源库的 dmarch.ini 中是否有异步归档配置项，是否有定时器名称配置 ARCH_TIMER_NAME，并且需要和 dmtimer.ini 中的定时器名称一致。

2) 源库的数据文件目录下是否有定时器配置文件 dmtimer.ini。

3) 源库的 dm.ini 中的 TIMER_INI 是否配置为 1。

4) 源库的 dmmal.ini 中是否有异步备库的配置项，并且守护系统中所有实例的 dmmal.ini 的内容完全一致。

5) 源库是否唯一，同一个异步备库只能配置在一个源库上。

可使用 disql 登录源库，查询 v$dm_arch_ini 和 v$dm_timer_ini 视图来确认配置信息是否正确。

2．源库和异步备库状态是否正常

通过监视器的 show 命令查看源库和异步备库状态，如果源库或异步备库发生故障，则不会再同步数据，需要等两者都恢复正常后，再观察异步备库的数据同步情况。

3．源库到异步备库的 mal 链路是否正常

通过监视器的 show arch info 观察源库到异步备库的 mal 链路状态，如果处于 DISCONNECTED 状态，也无法正常同步数据，需要等链路重建成功后，再观察异步备库的数据同步情况。

4．定时器是否满足触发条件

查看源库的 dmtimer.ini 中的触发条件，或者 disql 登录源库，通过 v$dm_timer_ini 视图查看定时器的触发条件，包括起始时间、触发频率等，再根据操作系统当前时间来判断是否满足触发条件，可能会有人为修改操作系统时间导致定时器无法触发的情况，如果不满足触发条件，也无法同步数据到异步备库。

6.22 注意事项

1． 守护进程、监视器启动后，如果收不到其他守护进程消息，检查端口、OGUID 等配置，并确认是否消息被防火墙屏蔽。

2． 备库故障重启，主库同步归档日志失败，可以查看监视器的 Recovery 信息，或者查看数据库日志信息、dmwatcher 日志信息，检查失败原因。如果是主库归档日志不完整、或者主备库日志不连续等原因造成同步失败，则需要尽快重新重建备库。

3． 建议使用故障手动切换模式数据守护系统。如果配置故障自动切换方式模式，则必

须配置确认监视器。

4. 同时配置数据守护和高可靠集群（HIGH-AVAILABILITY，简称 HA）软件情况下，需要将 dmwatcher.ini 的 INST_AUTO_RESTART 设置为 0，避免 dmwatcher 与 HA 软件同时启动 dmserver 实例。同时，配置 dm.ini 中的 HA_INST_CHECK_IP/HA_INST_CHECK_PORT 参数，避免由于 HA 软件故障，引发多个 dmserver 实例同时操作一份数据，导致数据损坏。

5. 用户应该使用“正常接管”方式将备库切换为主库，在无法正常接管的情况下，再考虑使用“强制接管”方式，但“强制接管”前一定要确认主备库数据是一致的，避免引发组分裂。

6. 根据磁盘空间情况，合理设置 dmarch.ini 的 ARCH_SPACE_LIMIT 值，防止磁盘空间被归档日志文件占满。

7. 备库重做新建数据库文件操作时，缺省按照主库的目录进行创建，如果创建失败则会将文件创建到 dm.ini 中指定的 SYSTEM_PATH 目录下。

8. DM 根据 dm.ini 参数 TS_MAX_ID/TS_FIL_MAX_ID 分配表空间和文件对象内存存放空间，用户需要确保主备库参数一致。如果配置不一致，比如：主库 TS_MAX_ID > 备库 TS_MAX_ID、并且新建表空间 ID > 备库 TS_MAX_ID 情况下，主库创建表空间成功，备库重演创建表空间操作时检测到 ID 越界会强制退出系统。

9. 守护进程通过当前系统时间与最后一次接收消息时间的差值，来检测目标对象是否异常，因此修改操作系统时间，可能会导致误判系统出现故障，不建议在数据守护系统运行过程中调整操作系统时间。

10. 主库修改表空间的 cache 方式的 DDL 操作，不会同步到备库。备库不允许修改表空间的 cache 方式。

11. 默认配置下，主备库都允许执行 Offline 表空间操作，并且主库的 Offline 表空间操作不会同步到备库。如果用户直接登录备库 Offline 表空间，重做 Redo 日志访问此表空间，将导致备库强制退出。因此，针对备库的表空间 Offline 操作要十分谨慎。用户可以修改 dm.ini 配置参数 ENABLE_OFFLINE_TS = 2，限制备库 Offline 表空间操作，降低误操作概率。

12. 主库上新建表空间或增加文件等操作，当备库磁盘空间不足时，备库上重做日志会导致宕机。

13. 两节点主备 MPP 环境，交叉配置环境中，创建表空间必须使用相对路径。例如，CREATE TABLESPACE TS4 DATAFILE 'TS4.DBF' SIZE 128;

7 数据守护搭建

7.1 数据准备

配置数据守护之前，必须先同步实例数据，确保两者保持完全一致；主库可以是新初始化的数据，也可以是正在生产、使用中的数据。DM 提供了两种方式初始化同步实例数据，数据文件拷贝以及备份还原方式。

不能使用分别初始化库的方法，原因如下：

1. 每个库都有一个永久魔数（permenant_magic），主库传送日志时会判断这个值是否一样，确保来自同一个库，不同的库传送不了日志

2. 由于 dminit 初始化数据库时，会生成随机密钥用于加密，每次生成的密钥都不相同，备库无法解析采用主库密钥加密的数据



对于新初始化的库，首次启动不允许使用 mount 方式，需要先正常启动并正常退出，然后才允许 mount 方式启动。

准备数据时，如果主库是新初始化的库，先正常启动并正常退出，然后再使用数据文件拷贝或者备份还原方式准备备库数据。

7.1.1 数据文件拷贝

如果搭建数据守护之前，数据库系统已经上线运行了，那么我们可以通过拷贝数据文件的方式实现主库和备库的数据同步。具体步骤包括：

1. 正常关闭数据库

2. 严格按照数据文件在主库上的分布，拷贝数据文件到备库的对应目录

3. 如果数据文件统一存放在一个目录下，则直接拷贝整个目录即可

7.1.2 备份还原方式

用户也可以通过脱机备份、脱机还原的方式同步主库和备库数据，更详细的说明可以参考备份恢复相关文档。具体步骤包括：

1. 正常关闭数据库

2. 进行脱机备份

```
./dmrman CTLSTMT="BACKUP DATABASE '/dm7/data/DAMENG/dm.ini' FULL TO BACKUP_FILE1 BACKUPSET '/dm7/data/BACKUP_FILE_01'"
```

3．拷贝备份文件到备库所在机器

4．执行脱机数据库还原与恢复

```
./dmrman CTLSTMT="RESTORE DATABASE '/dm7/data/DAMENG/dm.ini' FROM BACKUPSET '/dm7/data/BACKUP_FILE_01'"
./dmrman CTLSTMT="Recover DATABASE '/dm7/data/DAMENG/dm.ini' FROM BACKUPSET '/dm7/data/BACKUP_FILE_01'"
```



如果单节点系统已经上线，数据库服务不允许中断情况下，可以按照第 6.13 节“备库重建 ”的步骤完成备库数据准备。

7.1.3 检查数据一致性

依次到每个实例上，切换到执行码目录/dm7/bin，以 mount 方式启动数据库。

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```

启动命令行工具 DIsql,分别登录各个实例查询 LSN 值,如果主库和备库的 FILE_LSN 和 CUR_LSN 值相同，可以确保数据完全一致。

```
SQL>select file_LSN, cur_LSN from v$rlog;
--结果如下;
行号      FILE_LSN           CUR_LSN
-------- ------------------ --------------------
1         10250              10250
```

检查实例的永久魔数是否一致：

```
SQL>select permanent_magic;
--结果如下;
行号      PERMANENT_MAGIC
---------- ---------------
1385101984
```

确保 LSN 和永久魔数都一致的情况下，正常关闭各个实例，继续配置数据守护。

7.1.4 守护进程控制文件

对于故障场景中，某个非本地守护类型的实例所在机器故障，数据无法继续使用的情况下，从当前的有效主库直接拷贝数据文件或者备份还原方式重新准备数据后，需要同时拷贝主库的守护进程控制文件 dmwatcher.ctl 到实例的数据文件目录下。

7.2 配置实时主备

配置实时守护类型的数据守护系统，有以下几种配置方案，可以根据实际情况部署：

1. 只配置主库和实时备库。
2. 只配置主库和最多 8 个异步备库。
3. 配置主库、实时备库和最多 8 个异步备库。

在实际应用中，如果数据库规模很大，只有一个实时备库的情况下性能可能会受到影响，无法分担更多的任务，如果对数据的实时性要求不是很严格，则可以配置多个异步备库用于分担统计报表等任务。

异步备库的配置可以参考 7.5 配置异步备库 ，由于实时主备、读写分离集群、MPP 主备都支持配置异步备库，因此单独在 7.5 配置异步备库 进行配置说明。

为了帮助大家更加直观的理解 DM 数据守护方案，下面举例说明如何配置一个完整的实时守护类型的数据守护的过程，配置方案为一个主库和一个实时备库。



注意：实际配置时，相关的端口配置和 OGUID 值建议不要和手册示例使用完全相同的值，避免多个用户在同一个环境下搭建不同的数据守护系统，出现消息乱掉或者端口冲突等问题。

7.2.1 环境说明

下列机器事先都安装了 DM，安装路径为'/dm7'，执行程序保存在'/dm7/bin'目录中，数据存放路径为'/dm7/data'。

各主备库的实例名建议采用“组名_守护类型_序号”的方式命名，方便按组区分不同实例，注意总长度不能超过 16。本示例中组名为“GRP1”，采用实时守护类型，主库命名为“GRP1_RT_01”，备库命名为“GRP1_RT_02”。

表 7.1 配置环境说明

机器名	IP 地址	初始状态	操作系统	备注
DW_P	192.168.1.131 192.168.0.141	主库 GRP1_RT_01	Linux rh6-141.test 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.1.131 外部服务 IP； 192.168.0.141 内部通信 IP
DW_S1	192.168.1.132 192.168.0.142	备库 GRP1_RT_02	Linux rh6-142.localdomain 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.1.132 外部服务 IP； 192.168.0.142 内部通信 IP
DW_M	192.168.0.73	确认监视器	Linux rh6-73.localdomain 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	

表 7.2 端口规划

实例名	PORT_NUM	DW_PORT	MAL_HOST	MAL_PORT	MAL_DW_PORT
GRP1_RT_01	32141	33141	192.168.0.141	61141	52141
GRP1_RT_02	32142	33142	192.168.0.142	61142	52142

7.2.2 数据准备

DW_P 机器上初始化库至目录/dm7/data：

```
/dminit path=/dm7/data
```

然后按照 7.1 数据准备 中的方法准备备数据库数据。

7.2.3 配置主库 GRP1_RT_01

7.2.3.1 配置 dm.ini

在 DW_P 机器上配置主库的实例名为 GRP1_RT_01，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP1_RT_01
PORT_NUM                = 32141 #数据库实例监听端口
DW_PORT                 = 33141 #守护环境下，监听守护进程连接端口
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.2.3.2 配置 dmmal.ini

配置 MAL 系统，各主备库的 dmmal.ini 配置必须完全一致，MAL_HOST 使用内部网络 IP，MAL_PORT 与 dm.ini 中 PORT_NUM 使用不同的端口值，MAL_DW_PORT 是各实例对应的守护进程之间，以及守护进程和监视器之间的通信端口，配置如下：

```
MAL_CHECK_INTERVAL      = 5     #MAL 链路检测时间间隔
MAL_CONN_FAIL_INTERVAL  = 5     #判定 MAL 链路断开的时间
[MAL_INST1]
  MAL_INST_NAME         = GRP1_RT_01 #实例名，和 dm.ini 中的 INSTANCE_NAME 一致
```

```
 MAL_HOST                = 192.168.0.141  #MAL 系统监听 TCP 连接的 IP 地址
 MAL_PORT                = 61141          #MAL 系统监听 TCP 连接的端口
 MAL_INST_HOST           = 192.168.1.131  #实例的对外服务 IP 地址
 MAL_INST_PORT           = 32141 #实例的对外服务端口，和 dm.ini 中的 PORT_NUM 一致
 MAL_DW_PORT             = 52141 #实例对应的守护进程监听 TCP 连接的端口
[MAL_INST2]
 MAL_INST_NAME           = GRP1_RT_02
 MAL_HOST                = 192.168.0.142
 MAL_PORT                = 61142
 MAL_INST_HOST           = 192.168.1.132
 MAL_INST_PORT           = 32142
 MAL_DW_PORT             = 52142
```

7.2.3.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和实时归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP1_RT_01 是主库，需要向 GRP1_RT_02（实时守护类型的备库）同步数据，因此实时归档的 ARCH_DEST 配置为 GRP1_RT_02。

```
[ARCHIVE_REALTIME]
ARCH_TYPE               = REALTIME      #实时归档类型
ARCH_DEST               = GRP1_RT_02    #实时归档目标实例名
[ARCHIVE_LOCAL1]
ARCH_TYPE               = LOCAL #本地归档类型
ARCH_DEST               = /dm7/data/DAMENG/arch     #本地归档文件存放路径
ARCH_FILE_SIZE          = 128   #单位 Mb，本地单个归档文件最大值
ARCH_SPACE_LIMIT        = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.2.3.4 配置 dmwatcher.ini

修改 dmwatcher.ini 配置守护进程，配置为实时守护类型，使用自动切换模式。

```
[GRP1]
DW_TYPE             = REALTIME  #实时守护类型
DW_MODE             = AUTO      #自动切换模式
DW_ERROR_TIME       = 10        #远程守护进程故障认定时间
INST_RECOVER_TIME  = 60        #主库守护进程启动恢复的间隔时间
INST_ERROR_TIME     = 10        #本地实例故障认定时间
INST_OGUID          = 453331    #守护系统唯一 OGUID 值
INST_INI            = /dm7/data/DAMENG/dm.ini   #dm.ini 配置文件路径
INST_AUTO_RESTART  = 1          #打开实例的自动启动功能
INST_STARTUP_CMD   = /dm7/bin/dmserver  #命令行方式启动
```

7.2.3.5 配置 dmwatcher.ctl

同一个守护进程组，必须使用同一份 dmwatcher.ctl。因此，只需要使用 dmctlcvt 工具生成一份 dmwatcher.ctl 文件，然后分别拷贝到各个数据库目录下即可。

在配置完成 dmwatcher.ini 后，使用 dmctlcvt 工具生成 dmwatcher.ctl：

```
./dmctlcvt t2dwctl /dm7/data/DAMENG/dmwatcher.ini /dm7/data
```

拷贝生成的 dmwatcher.ctl 文件到数据文件目录/dm7/data/DAMENG。

7.2.3.6 启动主库

以 mount 方式启动主库

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```


注意：

一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。

7.2.3.7 设置 OGUID

启动命令行工具 DIsql，登录主库设置 OGUID 值。

```
SQL>sp_set_oguid(453331);
```



注意：系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。

7.2.3.8 修改数据库模式

启动命令行工具 DIsql，登录主库修改数据库为 primary 模式

```
SQL>alter database primary;
```

7.2.4 配置备库 GRP1_RT_02

7.2.4.1 配置 dm.ini

在 DW_S1 机器上配置备库的实例名为 GRP1_RT_02，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP1_RT_02
PORT_NUM                = 32142 #数据库实例监听端口
DW_PORT                 = 33142 #守护环境下，监听守护进程连接端口
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.2.4.2 配置 dmmal.ini

配置 MAL 系统，各主备库的 dmmal.ini 配置必须完全一致，MAL_HOST 使用内部网络 IP，MAL_PORT 与 dm.ini 中 PORT_NUM 使用不同的端口值，MAL_DW_PORT 是各实例对应的守护进程之间，以及守护进程和监视器之间的通信端口，配置如下：

```
MAL_CHECK_INTERVAL      = 5      #MAL 链路检测时间间隔
MAL_CONN_FAIL_INTERVAL  = 5      #判定 MAL 链路断开的时间
[MAL_INST1]
    MAL_INST_NAME           = GRP1_RT_01 #实例名，和 dm.ini 中的 INSTANCE_NAME 一致
    MAL_HOST                = 192.168.0.141  #MAL 系统监听 TCP 连接的 IP 地址
    MAL_PORT                = 61141          #MAL 系统监听 TCP 连接的端口
    MAL_INST_HOST           = 192.168.1.131  #实例的对外服务 IP 地址
    MAL_INST_PORT           = 32141 #实例的对外服务端口，和 dm.ini 中的 PORT_NUM 一致
    MAL_DW_PORT             = 52141 #实例对应的守护进程监听 TCP 连接的端口
[MAL_INST2]
    MAL_INST_NAME           = GRP1_RT_02
    MAL_HOST                = 192.168.0.142
    MAL_PORT                = 61142
    MAL_INST_HOST           = 192.168.1.132
    MAL_INST_PORT           = 32142
    MAL_DW_PORT             = 52142
```

7.2.4.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和实时归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP1_RT_02 是备库，守护系统配置完成后，可能在各种故障处理中，GRP1_RT_02 切换为新的主库，正常情况下，GRP1_RT_01 会切换为新的备库，需要向 GRP1_RT_01 同步数据，因此实时归档的 ARCH_DEST 配置为 GRP1_RT_01。

```
[ARCHIVE_REALTIME]
    ARCH_TYPE               = REALTIME      #实时归档类型
    ARCH_DEST               = GRP1_RT_01    #实时归档目标实例名
[ARCHIVE_LOCAL1]
    ARCH_TYPE               = LOCAL         #本地归档类型
    ARCH_DEST               = /dm7/data/DAMENG/arch     #本地归档文件路径
    ARCH_FILE_SIZE          = 128   #单位 Mb，本地单个归档文件最大值
    ARCH_SPACE_LIMIT        = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.2.4.4 配置 dmwatcher.ini

修改 dmwatcher.ini 配置守护进程，配置为实时守护类型，使用自动切换模式。

```
[GRP1]
    DW_TYPE             = REALTIME  #实时守护类型
    DW_MODE             = AUTO      #自动切换模式
    DW_ERROR_TIME       = 10        #远程守护进程故障认定时间
    INST_RECOVER_TIME  = 60         #主库守护进程启动恢复的间隔时间
    INST_ERROR_TIME     = 10        #本地实例故障认定时间
    INST_OGUID          = 453331    #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/DAMENG/dm.ini   #dm.ini 配置文件路径
    INST_AUTO_RESTART  = 1          #打开实例的自动启动功能
    INST_STARTUP_CMD   = /dm7/bin/dmserver  #命令行方式启动
```

7.2.4.5 配置 dmwatcher.ctl

同一个守护进程组，必须使用同一份 dmwatcher.ctl 文件，这里直接拷贝配置主库时已经生成的 dmwatcher.ctl 到本地数据文件目录/dm7/data/DAMENG。

7.2.4.6 启动备库

以 mount 方式启动备库

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```


注意：**一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。**

7.2.4.7 设置 OGUID

启动命令行工具 DIsql，登录备库设置 OGUID 值为 453331

```
SQL>sp_set_oguid(453331);
```


注意：**系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。**

7.2.4.8 修改数据库模式

启动命令行工具 DIsql，登录备库修改数据库为 Standby 模式

```
SQL>alter database standby;
```

7.2.5 配置监视器

由于主库和实时备库的守护进程配置为自动切换模式，因此这里选择配置确认监视器。和普通监视器相比，确认监视器除了相同的命令支持外，在主库发生故障时，能够自动通知实时备库接管为新的主库，具有自动故障处理的功能。


注意：**确认监视器最多只能配置一个。**

修改 dmmonitor.ini 配置确认监视器，其中 MON_DW_IP 中的 IP 和 PORT 和 dmmal.ini 中的 MAL_HOST 和 MAL_DW_PORT 配置项保持一致。

```
MON_DW_Confirm          = 1     #确认监视器模式
MON_LOG_PATH            = /dm7/data/log     #监视器日志文件存放路径
MON_LOG_INTERVAL        = 60    #每隔 60s 定时记录系统信息到日志文件
```

```
MON_LOG_FILE_SIZE       = 32     #每个日志文件最大 32M
MON_LOG_SPACE_LIMIT     = 0      #不限定日志文件总占用空间
[GRP1]
    MON_INST_OGUID          = 453331 #组 GRP1 的唯一 OGUID 值
#以下配置为监视器到组 GRP1 的守护进程的连接信息，以“IP:PORT”的形式配置
#IP 对应 dmmal.ini 中的 MAL_HOST，PORT 对应 dmmal.ini 中的 MAL_DW_PORT
    MON_DW_IP               = 192.168.0.141:52141
    MON_DW_IP               = 192.168.0.142:52142
```

7.2.6 启动守护进程

启动各个主备库上的守护进程：

```
./dmwatcher /dm7/data/DAMENG/dmwatcher.ini
```

守护进程启动后，进入 Startup 状态，此时实例都处于 Mount 状态。守护进程开始广播自身和其监控实例的状态信息，结合自身信息和远程守护进程的广播信息，守护进程将本地实例 Open，并切换为 Open 状态。

7.2.7 启动监视器

启动监视器：

```
./dmmonitor /dm7/data/dmmonitor.ini
```

监视器提供一系列命令，支持当前守护系统状态查看以及故障处理，可输入 help 命令，查看各种命令说明使用，结合实际情况选择使用。

至此实时守护类型的数据守护系统搭建完毕，在搭建步骤和各项配置都正确的情况下，在监视器上执行 show 命令，可以监控到所有实例都处于 Open 状态，所有守护进程也都处于 Open 状态，即为正常运行状态。

7.3 配置读写分离集群

配置读写分离守护类型的数据守护系统，有以下几种配置方案，可以根据实际情况部署：

1. 只配置主库和最多 8 个即时备库。

2． 只配置主库和最多 8 个异步备库。

3． 配置主库、最多 8 个即时备库和最多 8 个异步备库。

在实际应用中，如果数据库规模很大，并且对数据的实时性要求不是很严格，则可以配置多个异步备库用于分担统计报表等任务。

异步备库的配置可以参考 7.5 配置异步备库，由于实时主备、读写分离集群、MPP 主备都支持配置异步备库，因此单独在 7.5 配置异步备库 进行配置说明。

为了帮助大家更加直观的理解 DM 数据守护方案，下面举例说明如何配置一个完整的读写分离守护类型的数据守护的过程，配置方案为一个主库和两个即时备库。


注意：**实际配置时，相关的端口配置和 OGUID 值建议不要和手册示例使用完全相同的值，避免多个用户在同一个环境下搭建不同的数据守护系统，出现消息乱掉或者端口冲突等问题。**

7.3.1 环境说明

下列机器事先都安装了 DM，安装路径为’/dm7’，执行程序保存在’/dm7/bin’目录中，数据存放路径为’/dm7/data’。

各主备库的实例名建议采用“组名_守护类型_序号”的方式命名，方便按组区分不同实例，注意总长度不能超过 16。本示例中组名为“GRP1”，采用读写分离守护类型，主库命名为“GRP1_RWW_01”，备库分别命名为“GRP1_RWW_02”和“GRP1_RWW_03”。

表 7.3 配置环境说明

机器名	IP 地址	初始状态	操作系统	备注
DW_P	192.168.1.131 192.168.0.141	主库 GRP1_RWW_01	Linux rh6-141.test 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.1.131 外部服务 IP； 192.168.0.141 内部通信 IP
DW_S1	192.168.1.132 192.168.0.14	备库 GRP1_RWW_02	Linux rh6-142.localdomain 2.6.32-220.el6.x86_	192.168.1.132 外部服务 IP； 192.168.0.14

	2		64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	2 内部通信 IP
DW_S2	192.168.1.133 192.168.0.143	备库 GRP1_RWW_03	Linux rh6-143.localdomain 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.1.133 外部服务 IP； 192.168.0.143 内部通信 IP
DW_M	192.168.0.73	确认监视器	Linux rh6-73.localdomain 2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	

表 7.4 端口规划

实例名	PORT_NUM	DW_PORT	MAL_HOST	MAL_PORT	MAL_DW_PORT
GRP1_RWW_01	32141	33141	192.168.0.141	61141	52141
GRP1_RWW_02	32142	33142	192.168.0.142	61142	52142
GRP1_RWW_03	32143	33143	192.168.0.143	61143	52143

7.3.2 数据准备

DW_P 机器上初始化库至目录/dm7/data：

```
/dminit path=/dm7/data
```

然后按照 7.1 数据准备 中的方法分别准备备数据库数据。

7.3.3 配置主库 GRP1_RWW_01

7.3.3.1 配置 dm.ini

在 DW_P 机器上配置主库的实例名为 GRP1_RWW_01，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP1_RWW_01
PORT_NUM                = 32141 #数据库实例监听端口
DW_PORT                 = 33141 #守护环境下，监听守护进程连接端口
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.3.3.2 配置 dmmal.ini

配置 MAL 系统，各主备库的 dmmal.ini 配置必须完全一致，MAL_HOST 使用内部网络 IP，MAL_PORT 与 dm.ini 中 PORT_NUM 使用不同的端口值，MAL_DW_PORT 是各实例对应的守护进程之间，以及守护进程和监视器之间的通信端口，配置如下：

```
MAL_CHECK_INTERVAL      = 5     #MAL 链路检测时间间隔
MAL_CONN_FAIL_INTERVAL  = 5     #判定 MAL 链路断开的时间
[MAL_INST1]
    MAL_INST_NAME       = GRP1_RWW_01 #实例名，和 dm.ini 中的 INSTANCE_NAME 一致
    MAL_HOST            = 192.168.0.141 #MAL 系统监听 TCP 连接的 IP 地址
    MAL_PORT            = 61141         #MAL 系统监听 TCP 连接的端口
    MAL_INST_HOST       = 192.168.1.131  #实例的对外服务 IP 地址
    MAL_INST_PORT       = 32141 #实例的对外服务端口，和 dm.ini 中的 PORT_NUM 一致
    MAL_DW_PORT         = 52141 #实例对应的守护进程监听 TCP 连接的端口
```

```
[MAL_INST2]
    MAL_INST_NAME          = GRP1_RWW_02
    MAL_HOST               = 192.168.0.142
    MAL_PORT               = 61142
    MAL_INST_HOST          = 192.168.1.132
    MAL_INST_PORT          = 32142
    MAL_DW_PORT            = 52142
[MAL_INST3]
    MAL_INST_NAME          = GRP1_RWW_03
    MAL_HOST               = 192.168.0.143
    MAL_PORT               = 61143
    MAL_INST_HOST          = 192.168.1.133
    MAL_INST_PORT          = 32143
    MAL_DW_PORT            = 52143
```

7.3.3.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和即时归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP1_RWW_01 是主库，需要向 GRP1_RWW_02/ GRP1_RWW_03（读写分离守护类型的备库）同步数据，因此即时归档的 ARCH_DEST 分别配置为 GRP1_RWW_02 和 GRP1_RWW_03。

```
[ARCHIVE_TIMELY1]
     ARCH_TYPE             = TIMELY        #即时归档类型
     ARCH_DEST             = GRP1_RWW_02   #即时归档目标实例名
[ARCHIVE_TIMELY2]
     ARCH_TYPE             = TIMELY        #即时归档类型
     ARCH_DEST             = GRP1_RWW_03   #即时归档目标实例名
[ARCHIVE_LOCAL1]
```

```
    ARCH_TYPE               = LOCAL          #本地归档类型
    ARCH_DEST               = /dm7/data/DAMENG/arch     #本地归档文件存放路径
    ARCH_FILE_SIZE          = 128    #单位 Mb，本地单个归档文件最大值
    ARCH_SPACE_LIMIT        = 0      #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.3.3.4 配置 dmwatcher.ini

修改 dmwatcher.ini 配置守护进程，配置为读写分离守护类型，使用自动切换模式。

```
[GRP1]
    DW_TYPE             = TIMELY    #读写分离守护类型
    DW_MODE             = AUTO      #自动切换模式
    DW_ERROR_TIME       = 10        #远程守护进程故障认定时间
    INST_RECOVER_TIME   = 60        #主库守护进程启动恢复的间隔时间
    INST_ERROR_TIME     = 10        #本地实例故障认定时间
    INST_OGUID          = 453332    #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/DAMENG/dm.ini   #dm.ini 配置文件路径
    INST_AUTO_RESTART   = 1         #打开实例的自动启动功能
    INST_STARTUP_CMD    = /dm7/bin/dmserver #命令行方式启动
```

7.3.3.5 配置 dmwatcher.ctl

同一个守护进程组，必须使用同一份 dmwatcher.ctl。因此，只需要使用 dmctlcvt 工具生成一份 dmwatcher.ctl 文件，然后分别拷贝到各个数据库目录下即可。

在配置完成 dmwatcher.ini 后，使用 dmctlcvt 工具生成 dmwatcher.ctl：

```
./dmctlcvt t2dwctl /dm7/data/DAMENG/dmwatcher.ini /dm7/data
```

拷贝生成的 dmwatcher.ctl 文件到数据文件目录/dm7/data/DAMENG。

7.3.3.6 启动主库

以 mount 方式启动主库

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```


注意：**一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。**

7.3.3.7 设置 OGUID

启动命令行工具 DIsql，登录主库设置 OGUID 值。

```
SQL>sp_set_oguid(453332);
```


注意：**系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。**

7.3.3.8 修改数据库模式

启动命令行工具 DIsql，登录主库修改数据库为 Primary 模式

```
SQL>alter database primary;
```

7.3.4 配置备库 GRP1_RWW_02

7.3.4.1 配置 dm.ini

在 DW_S1 机器上配置备库的实例名为 GRP1_RWW_02，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP1_RWW_02
PORT_NUM                = 32142 #数据库实例监听端口
DW_PORT                 = 33142 #守护环境下，监听守护进程连接端口
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
```

```
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.3.4.2 配置 dmmal.ini

配置 MAL 系统，各主备库的 dmmal.ini 配置必须完全一致，MAL_HOST 使用内部网络 IP，MAL_PORT 与 dm.ini 中 PORT_NUM 使用不同的端口值，MAL_DW_PORT 是各实例对应的守护进程之间，以及守护进程和监视器之间的通信端口，配置如下：

```
MAL_CHECK_INTERVAL          = 5     #MAL 链路检测时间间隔
MAL_CONN_FAIL_INTERVAL      = 5     #判定 MAL 链路断开的时间
[MAL_INST1]
      MAL_INST_NAME        = GRP1_RWW_01 #实例名，和 dm.ini 中的 INSTANCE_NAME 一致
      MAL_HOST              = 192.168.0.141 #MAL 系统监听 TCP 连接的 IP 地址
      MAL_PORT              = 61141          #MAL 系统监听 TCP 连接的端口
      MAL_INST_HOST         = 192.168.1.131  #实例的对外服务 IP 地址
      MAL_INST_PORT         = 32141 #实例的对外服务端口，和 dm.ini 中的 PORT_NUM 一致
      MAL_DW_PORT           = 52141 #实例对应的守护进程监听 TCP 连接的端口
[MAL_INST2]
      MAL_INST_NAME         = GRP1_RWW_02
      MAL_HOST              = 192.168.0.142
      MAL_PORT              = 61142
      MAL_INST_HOST         = 192.168.1.132
      MAL_INST_PORT         = 32142
      MAL_DW_PORT           = 52142
[MAL_INST3]
      MAL_INST_NAME         = GRP1_RWW_03
      MAL_HOST              = 192.168.0.143
      MAL_PORT              = 61143
      MAL_INST_HOST         = 192.168.1.133
      MAL_INST_PORT         = 32143
```

```
    MAL_DW_PORT           = 52143
```

7.3.4.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和即时归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP1_RWW_02 是备库，守护系统配置完成后，可能在各种故障处理中，GRP1_RWW_02 切换为新的主库，正常情况下，GRP1_RWW_01 会切换为新的备库，需要向 GRP1_RWW_01 和 GRP1_RWW_03 同步数据，因此即时归档的 ARCH_DEST 分别配置为 GRP1_RWW_01 和 GRP1_RWW_03。

```
[ARCHIVE_TIMELY1]
    ARCH_TYPE             = TIMELY        #即时归档类型
    ARCH_DEST             = GRP1_RWW_01   #即时归档目标实例名
[ARCHIVE_TIMELY2]
    ARCH_TYPE             = TIMELY        #即时归档类型
    ARCH_DEST             = GRP1_RWW_03   #即时归档目标实例名
[ARCHIVE_LOCAL1]
    ARCH_TYPE             = LOCAL         #本地归档类型
    ARCH_DEST             = /dm7/data/DAMENG/arch     #本地归档文件存放路径
    ARCH_FILE_SIZE        = 128   #单位 Mb，本地单个归档文件最大值
    ARCH_SPACE_LIMIT      = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.3.4.4 配置 dmwatcher.ini

修改 dmwatcher.ini 配置守护进程，配置为读写分离守护类型，使用自动切换模式。

```
[GRP1]
    DW_TYPE           = TIMELY    #读写分离守护类型
    DW_MODE           = AUTO      #自动切换模式
    DW_ERROR_TIME     = 10        #远程守护进程故障认定时间
```

```
INST_RECOVER_TIME  = 60         #主库守护进程启动恢复的间隔时间
INST_ERROR_TIME    = 10         #本地实例故障认定时间
INST_OGUID         = 453332     #守护系统唯一 OGUID 值
INST_INI           = /dm7/data/DAMENG/dm.ini   #dm.ini 配置文件路径
INST_AUTO_RESTART  = 1          #打开实例的自动启动功能
INST_STARTUP_CMD   = /dm7/bin/dmserver #命令行方式启动
```

7.3.4.5 配置 dmwatcher.ctl

同一个守护进程组，必须使用同一份 dmwatcher.ctl 文件，这里直接拷贝配置主库时已经生成的 dmwatcher.ctl 到本地数据文件目录/dm7/data/DAMENG。

7.3.4.6 启动备库

以 mount 方式启动备库。

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```


注意：**一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。**

7.3.4.7 设置 OGUID

启动命令行工具 DIsql，登录备库设置 OGUID 值。

```
SQL>sp_set_oguid(453332);
```


注意：**系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。**

7.3.4.8 修改数据库模式

启动命令行工具 DIsql，登录备库修改数据库为 Standby 模式。

```
SQL>alter database standby;
```

7.3.5 配置备库 GRP1_RWW_03

7.3.5.1 配置 dm.ini

在 DW_S2 机器上配置备库的实例名为 GRP1_RWW_03，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP1_RWW_03


PORT_NUM                = 32143 #数据库实例监听端口
DW_PORT                 = 33143 #守护环境下，监听守护进程连接端口
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.3.5.2 配置 dmmal.ini

配置 MAL 系统，各主备库的 dmmal.ini 配置必须完全一致，MAL_HOST 使用内部网络 IP，MAL_PORT 与 dm.ini 中 PORT_NUM 使用不同的端口值，MAL_DW_PORT 是各实例对应的守护进程之间，以及守护进程和监视器之间的通信端口，配置如下：

```
MAL_CHECK_INTERVAL          = 5     #MAL 链路检测时间间隔
MAL_CONN_FAIL_INTERVAL      = 5     #判定 MAL 链路断开的时间
[MAL_INST1]
      MAL_INST_NAME        = GRP1_RWW_01 #实例名，和 dm.ini 中的 INSTANCE_NAME 一致
```

```
      MAL_HOST               = 192.168.0.141 #MAL 系统监听 TCP 连接的 IP 地址
      MAL_PORT               = 61141          #MAL 系统监听 TCP 连接的端口
      MAL_INST_HOST          = 192.168.1.131  #实例的对外服务 IP 地址
      MAL_INST_PORT          = 32141 #实例的对外服务端口，和 dm.ini 中的 PORT_NUM 一致
      MAL_DW_PORT            = 52141 #实例对应的守护进程监听 TCP 连接的端口
[MAL_INST2]
      MAL_INST_NAME          = GRP1_RWW_02
      MAL_HOST               = 192.168.0.142
      MAL_PORT               = 61142
      MAL_INST_HOST          = 192.168.1.132
      MAL_INST_PORT          = 32142
      MAL_DW_PORT            = 52142
[MAL_INST3]
      MAL_INST_NAME          = GRP1_RWW_03
      MAL_HOST               = 192.168.0.143
      MAL_PORT               = 61143
      MAL_INST_HOST          = 192.168.1.133
      MAL_INST_PORT          = 32143
      MAL_DW_PORT            = 52143
```

7.3.5.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和即时归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP1_RWW_03 是备库，守护系统配置完成后，可能在各种故障处理中，GRP1_RWW_03 切换为新的主库，正常情况下，GRP1_RWW_01 会切换为新的备库，需要向 GRP1_RWW_01 和 GRP1_RWW_02 同步数据，因此即时归档的 ARCH_DEST 分别配置为 GRP1_RWW_01 和 GRP1_RWW_02。

```
[ARCHIVE_TIMELY1]
```

```
    ARCH_TYPE               = TIMELY        #即时归档类型
    ARCH_DEST               = GRP1_RWW_01   #即时归档目标实例名
[ARCHIVE_TIMELY2]
    ARCH_TYPE               = TIMELY        #即时归档类型
    ARCH_DEST               = GRP1_RWW_02   #即时归档目标实例名
 [ARCHIVE_LOCAL1]
    ARCH_TYPE               = LOCAL         #本地归档类型
    ARCH_DEST               = /dm7/data/DAMENG/arch     #本地归档文件存放路径
    ARCH_FILE_SIZE          = 128   #单位 Mb，本地单个归档文件最大值
    ARCH_SPACE_LIMIT        = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.3.5.4 配置 dmwatcher.ini

修改 dmwatcher.ini 配置守护进程，配置为读写分离守护类型，使用自动切换模式。

```
[GRP1]
    DW_TYPE             = TIMELY    #读写分离守护类型
    DW_MODE             = AUTO      #自动切换模式
    DW_ERROR_TIME       = 10        #远程守护进程故障认定时间
    INST_RECOVER_TIME  = 60        #主库守护进程启动恢复的间隔时间
    INST_ERROR_TIME     = 10        #本地实例故障认定时间
    INST_OGUID          = 453332    #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/DAMENG/dm.ini   #dm.ini 配置文件路径
    INST_AUTO_RESTART  = 1          #打开实例的自动启动功能
    INST_STARTUP_CMD   = /dm7/bin/dmserver  #命令行方式启动
```

7.3.5.5 配置 dmwatcher.ctl

同一个守护进程组，必须使用同一份 dmwatcher.ctl 文件，这里直接拷贝配置主库时已经生成的 dmwatcher.ctl 到本地数据文件目录/dm7/data/DAMENG。

7.3.5.6 启动备库

以 mount 方式启动备库

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```



注意：一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。

7.3.5.7 设置 OGUID

启动命令行工具 DIsql，登录备库设置 OGUID 值。

```
SQL>sp_set_oguid(453332);
```



注意：系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。

7.3.5.8 修改数据库模式

启动命令行工具 DIsql，登录备库修改数据库为 Standby 模式

```
SQL>alter database standby;
```

7.3.6 配置监视器

由于主库和即时备库的守护进程配置为自动切换模式，因此这里选择配置确认监视器。和普通监视器相比，确认监视器除了相同的命令支持外，在主库发生故障时，能够自动通知即时备库接管为新的主库，具有自动故障处理的功能。

确认监视器最多只能配置一个。

修改 dmmonitor.ini 配置确认监视器，其中 MON_DW_IP 中的 IP 和 PORT 和 dmmal.ini 中的 MAL_HOST 和 MAL_DW_PORT 配置项保持一致。

```
MON_DW_Confirm          = 1     #确认监视器模式
```

```
MON_LOG_PATH            = /dm7/data/log #监视器日志文件存放路径
MON_LOG_INTERVAL        = 60    #每隔 60s 定时记录系统信息到日志文件
MON_LOG_FILE_SIZE       = 32    #每个日志文件最大 32M
MON_LOG_SPACE_LIMIT     = 0     #不限定日志文件总占用空间
[GRP1]
    MON_INST_OGUID          = 453332 #组 GRP1 的唯一 OGUID 值
    #以下配置为监视器到组 GRP1 的守护进程的连接信息，以“IP:PORT”的形式配置
    #IP 对应 dmmal.ini 中的 MAL_HOST，PORT 对应 dmmal.ini 中的 MAL_DW_PORT
    MON_DW_IP               = 192.168.0.141:52141
    MON_DW_IP               = 192.168.0.142:52142
    MON_DW_IP               = 192.168.0.143:52143
```

7.3.7 启动守护进程

启动各个主备库上的守护进程：

```
./dmwatcher /dm7/data/DAMENG/dmwatcher.ini
```

守护进程启动后，进入 Startup 状态，此时实例都处于 Mount 状态。守护进程开始广播自身和其监控实例的状态信息，结合自身信息和远程守护进程的广播信息，守护进程将本地实例 Open，并切换为 Open 状态。

7.3.8 启动监视器

启动监视器：

```
./dmmonitor /dm7/data/dmmonitor.ini
```

监视器提供一系列命令，支持当前守护系统状态查看以及故障处理，可输入 help 命令，查看各种命令说明使用，结合实际情况选择使用。

至此读写分离守护类型的数据守护系统搭建完毕，在搭建步骤和各项配置都正确的情况下，在监视器上执行 show 命令，可以监控到所有实例都处于 Open 状态，所有守护进程也都处于 Open 状态，即为正常运行状态。

7.3.9 接口说明

DM 多种客户端接口都支持读写分离集群连接设置，以下说明客户端连接服务器时如何设置读写分离属性，详细可参考《DM7 程序员手册》。

7.3.9.1 JDBC 接口

在 JDBC 连接串中增加了两个连接属性：

- rwSeparate 是否使用读写分离系统，默认 0；取值（0 不使用，1 使用）。
- rwPercent 分发到主库的事务占主备库总事务的百分比，有效值 0~100，默认值 25。

```
<DRIVER>dm.jdbc.driver.DmDriver</DRIVER>
<URL>jdbc:dm://192.168.0.206:5236?rwSeparate=1&rwPercent=10</URL>
```

7.3.9.2 DPI 接口

DPI 接口的连接句柄上的读写分离属性：

DSQL_ATTR_RWSEPARATE： 读写分离（可读写）

DSQL_ATTR_RWSEPARATE_PERCENT：读写分离比例（可读写）

属性设置举例：

```
dhenv           env;
dhcon           con;
dpi_alloc_env(&env);
dpi_alloc_con(env, &con);
dpi_set_con_attr(con, DSQL_ATTR_RWSEPARATE, (dpointer) DSQL_RWSEPARATE_ON, 0);
dpi_set_con_attr(con, DSQL_ATTR_RWSEPARATE_PERCENT, (dpointer)25, 0);
```

Disql 工具可以直接设置读写分离的属性：

```
>disql /nolog
SQL> login
```

```
服务名:

用户名:

密码:

端口号:

SSL 路径:

SSL 密码:

UKEY 名称:

UKEY PIN 码:

MPP 类型:

是否读写分离(y/n):y

读写分离百分比(0-100):25
```

7.3.9.3 ODBC 接口

ODBC 接口的连接关键字：

RW_SEPARATE：　　是否配置读写分离：TRUE，FALSE

RW_SEPARATE_PERCENT：　　读写分离的比例：0~100

连接串举例说明：

```
"DSN=DM7;DRIVER=DM ODBC

DRIVER;UID=SYSDBA;PWD=SYSDBA;TCP_PORT=5236;RW_SEPARATE=TRUE;RW_SEPARATE_PERC

ENT=25";
```

7.3.9.4 Provider 接口

DM .NET Provider 接口主要实现了 DmConnection，DmConnection 对象表示一个 DM 数据库打开的连接。其支持的读写分离属性包括：

RwSeparate：　是否读写分离，有效值为 true 或者 false；

RwPercent：　表示分发到主库的事务占主备库总事务的百分比，有效值范围：0~100，默认值为 25。

连接串举例：

```
static DmConnection cnn = new DmConnection();
```

```
cnn.ConnectionString = "Server=localhost; User Id=SYSDBA; PWD=SYSDBA;
RwSeparate=true; RwPercent=25";
```

7.3.9.5 DCI 接口

DCI 接口支持会话上的读写分离属性设置：

- OCI_ATTR_RW_SEPARATE 是否读写分离，有效值为 1 或者 0，默认 0；
- OCI_ATTR_RW_SEPARATE_PERCENT 表示分发到主库的事务占主备库总事务的百分比，有效值范围：0~100，默认值为 25。

例如，利用接口编程进行举例说明。

```
OCIEnv*         envhp;
OCISession*     authp;
OCIError*       errhp;
OCIInitialize(OCI_DEFAULT, NULL, NULL, NULL, NULL);
OCIEnvInit(&envhp, OCI_DEFAULT,0, 0);
OCIHandleAlloc(envhp, (dvoid**)&authp, OCI_HTYPE_SESSION, 0, 0);
OCIAttrSet(authp,OCI_HTYPE_SESSION,(void
*)OCI_RW_SEPARATE_ON,(ub4)sizeof(ub4),OCI_ATTR_RW_SEPARATE,errhp);
OCIAttrSet(authp,OCI_HTYPE_SESSION,(void
*)25,(ub4)sizeof(ub4),OCI_ATTR_RW_SEPARATE_PERCENT,errhp);
```

7.4 配置 MPP 主备

为了提高 MPP 系统可靠性，克服由于单节点故障导致整个系统不能继续正常工作，DM 在普通的 MPP 系统基础上，引入主备守护机制，将 MPP 节点作为主库节点，增加备库作为备份节点，必要时可切换为主库代替故障节点工作，提高系统的可靠性和可用性。

7.4.1 环境说明

本例配置 2 个 MPP 节点数，每个节点作为主库，与其备库组成一个守护组，因此需要配置两个守护组，取名分别为 GRP1、GRP2，主库名为 GRP1_MPP_EP01/ GRP2_MPP_EP02，

对应的备实例实例名分别为 GRP1_MPP_EP11/ GRP1_MPP_EP22。

准备 3 台机器 A、B、C，A 和 B 用来交叉部署实例，C 用来部署监视器。其中 A 和 B 配置两块网卡，一块接入内部网络交换模块，一块接入到外部交换机，C 接入内部网络。

机器事先都安装了 DM，安装路径为'/dm7'，执行程序保存在'/dm7/bin'目录中，数据存放路径为'/dm7/data/EP01', '/dm7/data/EP02'。


注意：**实际配置时，相关的端口配置和 OGUID 值建议不要和手册示例使用完全相同的值，避免多个用户在同一个环境下搭建不同的数据守护系统，出现消息乱掉或者端口冲突等问题。**

表 7.5 配置环境说明

机器名	IP 地址	初始状态	操作系统
A	192.168.1.131 192.168.0.141	主库 GRP1_MPP_EP01 备库 GRP2_MPP_EP22	Linux rh6-2.6.32-220.el6.x 86_64
B	192.168.1.132 192.168.0.142	主库 GRP2_MPP_EP02 备库 GRP1_MPP_EP11	Linux rh6-2.6.32-220.el6.x 86_64
C	192.168.0.144	监视器	Linux rh6-2.6.32-220.el6.x 86_64

表 7.6 端口规划—主库

实例名	PORT_NUM	DW_PORT	MAL_HOST	MAL_PORT	MAL_DW_PORT	MPP 实例序号
GRP1_MPP_EP01	5236	5243	192.168.0.141	5337	5253	0
GRP2_MPP_EP02	5236	5243	192.168.0.142	5337	5253	1

表 7.7 端口规划—备库

实例名	PORT_NUM	DW_PORT	MAL_HOST	MAL_PORT	MAL_DW_PORT	对应主实例
GRP1_MPP_EP11	5237	5244	192.168.0.142	5338	5254	GRP1_MPP_EP01

GRP2_MPP_EP22	5237	5244	192.168.0.141	5338	5254	GRP2_MPP_EP02

表 7.8 守护进程规划

组名	实例名	所在机器
GRP1	GRP1_MPP_EP01	192.168.0.141
	GRP1_MPP_EP11	192.168.0.142
GRP2	GRP2_MPP_EP02	192.168.0.142
	GRP2_MPP_EP22	192.168.0.141

7.4.2 数据准备

A 机器上初始化库至目录/dm7/data/EP01：

```
./dminit path=/dm7/data/EP01
```

B 机器上初始化库至目录/dm7/data/EP02：

```
./dminit path=/dm7/data/EP02
```

即完成两个主数据库的初始化，然后按照 7.1 数据准备 中的方法分别同步两个备数据库。

本例中采取机器交叉的方式配置两个备数据库，分别对应存放的目录为：

```
B 机器的/dm7/data/EP01

A 机器的/dm7/data/EP02
```

7.4.3 配置主库 GRP1_MPP_EP01

7.4.3.1 配置 dm.ini

在 A 机器上配置主库的实例名为 GRP1_MPP_EP01，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16

INSTANCE_NAME          = GRP1_MPP_EP01

PORT_NUM               = 5236  #数据库实例监听端口

DW_PORT                = 5243  #守护环境下，监听守护进程连接端口

DW_ERROR_TIME          = 60    #接收守护进程消息超时时间
```

```
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
MPP_INI                 = 1     #启用 MPP 配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.4.3.2 配置 dmmal.ini

配置 MAL 系统，各主备库的 dmmal.ini 配置必须完全一致，MAL_HOST 使用内部网络 IP，MAL_PORT 与 dm.ini 中 PORT_NUM 使用不同的端口值，MAL_DW_PORT 是各实例对应的守护进程之间，以及守护进程和监视器之间的通信端口，配置如下：

```
MAL_CHECK_INTERVAL      = 5     #MAL 链路检测时间间隔
MAL_CONN_FAIL_INTERVAL  = 5     #判定 MAL 链路断开的时间
[MAL_INST1]
      MAL_INST_NAME      = GRP1_MPP_EP01 #实例名，和 dm.ini 中的 INSTANCE_NAME 一致
      MAL_HOST              = 192.168.0.141 #MAL 系统监听 TCP 连接的 IP 地址
      MAL_PORT              = 5337        #MAL 系统监听 TCP 连接的端口
      MAL_INST_HOST         = 192.168.1.131 #实例的对外服务 IP 地址
      MAL_INST_PORT         = 5236  #实例的对外服务端口，和 dm.ini 中的 PORT_NUM 一致
      MAL_DW_PORT           = 5253  #实例对应的守护进程监听 TCP 连接的端口
[MAL_INST2]
      MAL_INST_NAME         = GRP2_MPP_EP02
      MAL_HOST              = 192.168.0.142
      MAL_PORT              = 5337
      MAL_INST_HOST         = 192.168.1.132
      MAL_INST_PORT         = 5236
      MAL_DW_PORT           = 5253
[MAL_INST3]
      MAL_INST_NAME         = GRP1_MPP_EP11
```

```
      MAL_HOST              = 192.168.0.142
      MAL_PORT              = 5338
      MAL_INST_HOST         = 192.168.1.132
      MAL_INST_PORT         = 5237
      MAL_DW_PORT           = 5254
[MAL_INST4]
      MAL_INST_NAME         = GRP2_MPP_EP22
      MAL_HOST              = 192.168.0.141
      MAL_PORT              = 5338
      MAL_INST_HOST         = 192.168.1.131
      MAL_INST_PORT         = 5237
      MAL_DW_PORT           = 5254
```

7.4.3.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和 MPP 归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP1_MPP_EP01 是主库，需要向 GRP1_MPP_EP11（MPP 守护类型的备库）同步数据，因此 MPP 归档的 ARCH_DEST 配置为 GRP1_MPP_EP11。

```
[ARCHIVE_MARCH1]
     ARCH_TYPE              = MARCH          #MPP 归档类型
     ARCH_DEST              = GRP1_MPP_EP11 #MPP 归档目标实例名
[ARCHIVE_LOCAL1]
     ARCH_TYPE              = LOCAL #本地归档类型
     ARCH_DEST              = /dm7/data/EP01/DAMENG/arch #本地归档文件存放路径
     ARCH_FILE_SIZE         = 128   #单位 Mb，本地单个归档文件最大值
     ARCH_SPACE_LIMIT       = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.4.3.4 配置 dmmpp.ctl

dmmpp.ctl 是二进制文件，由 dmmpp.ini 文本通过 dmctlcvt 工具转换而来,dmmpp.ini 配置项如下表：

表 7.9 dmmpp.ini 配置项

配置项	配置含义
[SERVICE_NAME]	标识每个实例的选项名
MPP_SEQ_NO	实例在 MPP 系统内的序号
MPP_INST_NAME	节点实例名

本例中两节点的 dmmpp.ini 配置如下：

```
[service_name1]
    mpp_seq_no      = 0
    mpp_inst_name   = GRP1_MPP_EP01
[service_name2]
    mpp_seq_no      = 1
    mpp_inst_name   = GRP2_MPP_EP02
```

转换命令如下：

```
./dmctlcvt t2c /dm7/data/EP01/DAMENG/dmmpp.ini /dm7/data/EP01/DAMENG/dmmpp.ctl
```

7.4.3.5 启动主库

以 mount 方式启动主库。

```
./dmserver /dm7/data/EP01/DAMENG/dm.ini mount
```



一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。

7.4.3.6 设置 OGUID

启动命令行工具 DIsql，使用 MPP 类型为 LOCAL 方式，登录主库设置 OGUID 值。

```
SQL>sp_set_oguid(45330);
```



注意：系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。

7.4.3.7 修改数据库模式

启动命令行工具 DIsql，使用 MPP 类型为 LOCAL 方式，登录主库修改数据库为 Primary 模式。

```
SQL>alter database primary;
```

7.4.4 配置主库 GRP2_MPP_EP02

7.4.4.1 配置 dm.ini

在 B 机器上配置主库的实例名为 GRP2_MPP_EP02，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP2_MPP_EP02
PORT_NUM                = 5236  #数据库实例监听端口
DW_PORT                 = 5243  #守护环境下，监听守护进程连接端口
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
MPP_INI                 = 1     #启用 MPP 配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.4.4.2 配置 dmmal.ini

直接将 A 机器上实例 GRP1_MPP_EP01 配置的 dmmal.ini 拷贝到/dm7/data/EP02/DAMENG 目录中。

7.4.4.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和 MPP 归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP2_MPP_EP02 是主库，需要向 GRP2_MPP_EP22（MPP 守护类型的备库）同步数据，因此 MPP 归档的 ARCH_DEST 配置为 GRP2_MPP_EP22。

```
[ARCHIVE_MARCH1]
     ARCH_TYPE               = MARCH          #MPP 归档类型
     ARCH_DEST               = GRP2_MPP_EP22 #MPP 归档目标实例名
[ARCHIVE_LOCAL1]
     ARCH_TYPE               = LOCAL #本地归档类型
     ARCH_DEST               = /dm7/data/EP02/DAMENG/arch    #本地归档文件存放路径
     ARCH_FILE_SIZE          = 128    #单位 Mb，本地单个归档文件最大值
     ARCH_SPACE_LIMIT        = 0      #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.4.4.4 配置 dmmpp.ctl

同 7.4.3.4 配置 dmmpp.ctl，拷贝 dmmpp.ctl 到/dm7/data/EP02/DAMENG 目录即可。

7.4.4.5 启动主库

以 mount 方式启动主库

```
./dmserver /dm7/data/EP02/DAMENG/dm.ini mount
```


注意：**一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。**

7.4.4.6 设置 OGUID

启动命令行工具 DIsql，使用 MPP 类型为 LOCAL 方式，登录主库设置 OGUID 值。

```
SQL>sp_set_oguid(45331);
```


注意：**系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。**

7.4.4.7 修改数据库模式

启动命令行工具 DIsql，使用 MPP 类型为 LOCAL 方式，登录主库修改数据库为 Primary 模式。

```
SQL>alter database primary;
```

7.4.5 配置备库 GRP1_MPP_EP11

7.4.5.1 配置 dm.ini

在 B 机器上配置主库的实例名为 GRP1_MPP_EP11，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP1_MPP_EP11
PORT_NUM                = 5237  #数据库实例监听端口
DW_PORT                 = 5244  #守护环境下，监听守护进程连接端口
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
```

```
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.4.5.2 配置 dmmal.ini

直接将 A 机器上实例 GRP1_MPP_EP01 配置的 dmmal.ini 拷贝到 B 机器 /dm7/data/EP01/DAMENG 目录中。

7.4.5.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和 MPP 归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP1_MPP_EP11 是备库，守护系统配置完成后，可能在各种故障处理中，GRP1_MPP_EP11 切换为新的主库，正常情况下，GRP1_MPP_EP01 会切换为新的备库，需要向 GRP1_MPP_EP01 同步数据，因此 MPP 归档的 ARCH_DEST 配置为 GRP1_MPP_EP01。

```
[ARCHIVE_MARCH1]
     ARCH_TYPE              = MARCH          #MPP 归档类型
     ARCH_DEST              = GRP1_MPP_EP01 #MPP 归档目标实例名
[ARCHIVE_LOCAL1]
     ARCH_TYPE              = LOCAL #本地归档类型
     ARCH_DEST              = /dm7/data/EP01/DAMENG/arch    #本地归档文件存放路径
     ARCH_FILE_SIZE         = 128   #单位 Mb，本地单个归档文件最大值
     ARCH_SPACE_LIMIT       = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.4.5.4 启动备库

以 mount 方式启动备库

```
./dmserver /dm7/data/EP01/DAMENG/dm.ini mount
```


注意：**一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。**

7.4.5.5 设置 OGUID

启动命令行工具 DIsql，登录备库设置 OGUID 值。

```
SQL>sp_set_oguid(45330);
```


注意：**系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。**

7.4.5.6 修改数据库模式

启动命令行工具 DIsql，登录实例修改数据库为 Standby 模式

```
SQL>alter database standby;
```

7.4.6 配置备库 GRP2_MPP_EP22

7.4.6.1 配置 dm.ini

在 A 机器上配置主库的实例名为 GRP2_MPP_EP22，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME          = GRP2_MPP_EP22
PORT_NUM               = 5237  #数据库实例监听端口
DW_PORT                = 5244  #守护环境下，监听守护进程连接端口
DW_ERROR_TIME          = 60    #接收守护进程消息超时时间
```

```
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.4.6.2 配置 dmmal.ini

直接将 A 机器上实例 GRP1_MPP_EP01 配置的 dmmal.ini 拷贝到 /dm7/data/EP02/DAMENG 目录中。

7.4.6.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和 MPP 归档。

除了本地归档外，其他归档配置项中的 ARCH_DEST 表示实例是 Primary 模式时，需要同步归档数据的目标实例名。

当前实例 GRP2_MPP_EP22 是备库，守护系统配置完成后，可能在各种故障处理中，GRP2_MPP_EP22 切换为新的主库，正常情况下，GRP2_MPP_EP02 会切换为新的备库，需要向 GRP2_MPP_EP02 同步数据，因此 MPP 归档的 ARCH_DEST 配置为 GRP2_MPP_EP02。

```
[ARCHIVE_MARCH1]
    ARCH_TYPE               = MARCH         #MPP 归档类型
    ARCH_DEST               = GRP2_MPP_EP02 #MPP 归档目标实例名
[ARCHIVE_LOCAL1]
    ARCH_TYPE               = LOCAL #本地归档类型
    ARCH_DEST               = /dm7/data/EP02/DAMENG/arch    #本地归档文件存放路径
    ARCH_FILE_SIZE          = 128   #单位 Mb，本地单个归档文件最大值
    ARCH_SPACE_LIMIT        = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.4.6.4 启动实例

以 mount 方式启动实例

```
./dmserver /dm7/data/EP02/DAMENG/dm.ini mount
```



注意：一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。

7.4.6.5 设置 OGUID

启动命令行工具 DIsql，登录实例设置 OGUID 值。

```
SQL>sp_set_oguid(45331);
```



注意：系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。

7.4.6.6 修改数据库模式

启动命令行工具 DIsql，登录 Normal 数据库，修改数据库为 Standby 模式

```
SQL>alter database standby;
```

7.4.7 配置 dmwatcher.ini

一般来说，每个单独的实例都是使用一个单独的守护进程守护。比如前面的实时守护类型和读写分离守护类型的配置。

本例中，由于同一台机器上有不同组的两个实例，我们可以只配置一个守护进程，同时守护两个实例。

A 机器上配置 dmwatcher.ini，配置为 MPP 守护类型，使用自动切换模式。

```
[GRP1]
```

```
    DW_TYPE             = MARCH  #MPP 守护类型
    DW_MODE             = AUTO   #自动切换模式
    DW_ERROR_TIME       = 10     #远程守护进程故障认定时间
    INST_RECOVER_TIME   = 60        #主库守护进程启动恢复的间隔时间
    INST_ERROR_TIME     = 10     #本地实例故障认定时间
    INST_OGUID          = 45330  #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/EP01/DAMENG/dm.ini #dm.ini 配置文件路径
    INST_AUTO_RESTART   = 1      #打开实例的自动启动功能
    INST_STARTUP_CMD    = /dm7/bin/dmserver  #命令行方式启动
[GRP2]
    DW_TYPE             = MARCH  #MPP 守护类型
    DW_MODE             = AUTO   #自动切换模式
    DW_ERROR_TIME       = 10     #远程守护进程故障认定时间
    INST_RECOVER_TIME   = 60     #主库守护进程启动恢复的间隔时间
    INST_ERROR_TIME     = 10     #本地实例故障认定时间
    INST_OGUID          = 45331  #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/EP02/DAMENG/dm.ini #dm.ini 配置文件路径
    INST_AUTO_RESTART   = 1      #打开实例的自动启动功能
    INST_STARTUP_CMD    = /dm7/bin/dmserver #命令行方式启动
```

B 机器上配置 dmwatcher.ini，配置为 MPP 守护类型，使用自动切换模式。

```
[GRP1]
    DW_TYPE             = MARCH  #MPP 守护类型
    DW_MODE             = AUTO   #自动切换模式
    DW_ERROR_TIME       = 10     #远程守护进程故障认定时间
    INST_RECOVER_TIME   = 60        #主库守护进程启动恢复的间隔时间
    INST_ERROR_TIME     = 10     #本地实例故障认定时间
    INST_OGUID          = 45330  #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/EP01/DAMENG/dm.ini #dm.ini 配置文件路径
    INST_AUTO_RESTART   = 1      #打开实例的自动启动功能
    INST_STARTUP_CMD    = /dm7/bin/dmserver  #命令行方式启动
```

```
[GRP2]
    DW_TYPE             = MARCH  #MPP 守护类型
    DW_MODE             = AUTO   #自动切换模式
    DW_ERROR_TIME       = 10     #远程守护进程故障认定时间
    INST_RECOVER_TIME  = 60        #主库守护进程启动恢复的间隔时间
    INST_ERROR_TIME     = 10     #本地实例故障认定时间
    INST_OGUID          = 45331  #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/EP02/DAMENG/dm.ini #dm.ini 配置文件路径
    INST_AUTO_RESTART  = 1       #打开实例的自动启动功能
    INST_STARTUP_CMD   = /dm7/bin/dmserver #命令行方式启动
```

7.4.8 生成 dmwatcher.ctl

同一个守护进程组，必须使用同一份 dmwatcher.ctl。因此，只需要使用 dmctlcvt 工具生成一份 dmwatcher.ctl 文件，然后分别拷贝到各个数据库目录下即可。

在配置完成 dmwatcher.ini 后，使用 dmctlcvt 工具生成 dmwatcher.ctl：

```
./dmctlcvt t2dwctl /dm7/data/EP01/DAMENG/dmwatcher.ini /dm7/data
```

在/dm7/data 目录生成两个按照组名命名的子目录，每个子目录中生成了该组的 dmwatcher.ctl 文件。将该文件拷贝到对应的组实例库目录下。

将/dm7/data/GRP1 中 ctl 文件分别拷贝到 A/B 机器的/dm7/data/EP01/DAMENG，

将/dm7/data/GRP2 中 ctl 文件分别拷贝到 A/B 机器的/dm7/data/EP02/DAMENG。

7.4.9 配置监视器

由于主库和实时备库的守护进程配置为自动切换模式，因此这里选择配置确认监视器。和普通监视器相比，确认监视器除了相同的命令支持外，在主库发生故障时，能够自动通知实时备库接管为新的主库，具有自动故障处理的功能。

确认监视器最多只能配置一个。

修改 dmmonitor.ini 配置确认监视器，其中 MON_DW_IP 中的 IP 和 PORT 和 dmmal.ini 中的 MAL_HOST 和 MAL_DW_PORT 配置项保持一致。

```
MON_DW_CONFIRM          = 1     #确认监视器模式
```

```
MON_LOG_PATH            = /dm7/data/log     #监视器日志文件存放路径
MON_LOG_INTERVAL        = 60    #每隔 60s 定时记录系统信息到日志文件
MON_LOG_FILE_SIZE       = 32    #每个日志文件最大 32M
MON_LOG_SPACE_LIMIT     = 0     #不限定日志文件总占用空间
[GRP1]
    MON_INST_OGUID          = 45330   #组 GRP1 的唯一 OGUID 值
    #以下配置为监视器到组 GRP1 的守护进程的连接信息，以“IP:PORT”的形式配置
    #IP 对应 dmmal.ini 中的 MAL_HOST，PORT 对应 dmmal.ini 中的 MAL_DW_PORT
    MON_DW_IP               = 192.168.0.141:5253
    MON_DW_IP               = 192.168.0.142:5254
[GRP2]
    MON_INST_OGUID          = 45331     #组 GRP2 的唯一 OGUID 值
    #以下配置为监视器到组 GRP2 的守护进程的连接信息，以“IP:PORT”的形式配置
    #IP 对应 dmmal.ini 中的 MAL_HOST，PORT 对应 dmmal.ini 中的 MAL_DW_PORT
    MON_DW_IP               = 192.168.0.142:5253
    MON_DW_IP               = 192.168.0.141:5254
```

7.4.10 启动守护进程

分别启动 A、B 机器上的守护进程，例如：

```
./dmwatcher /dm7/data/EP01/DAMENG/dmwatcher.ini
```

守护进程启动后，进入 Startup 状态，此时实例都处于 Mount 状态。守护进程开始广播自身和其监控实例的状态信息，结合自身信息和远程守护进程的广播信息，守护进程将本地实例 Open，并切换为 Open 状态。

7.4.11 启动监视器

启动监视器：

```
./dmmonitor /dm7/data/dmmonitor.ini
```

监视器提供一系列命令，支持当前守护系统状态查看以及故障处理，可输入 help 命令，查看各种命令说明使用，结合实际情况选择使用。

至此实时守护类型的数据守护系统搭建完毕，在搭建步骤和各项配置都正确的情况下，在监视器上执行 show 命令，可以监控到所有实例都处于 Open 状态，所有守护进程也都处于 Open 状态，即为正常运行状态。

7.5 配置异步备库

在实际应用中，如果数据库规模很大，并且对数据的实时性要求不是很严格，则可以配置多个异步备库用于分担统计报表等任务。

这里以实时主备环境为例，仍然采用 7.2 配置实时主备 小节的配置环境，对异步备库的搭建步骤做举例说明，读写分离集群和 MPP 主备的异步备库搭建步骤是相同的，只需要根据实际情况调整配置项的具体值即可。

这里以一个异步备库为例，配置在主库 GRP1_RT_01 上，异步备库实例名为 GRP1_LOCAL_01。如果需要配置多个异步备库，对应的源实例配置可以参考主库 GRP1_RT_01 的配置步骤，异步备库自身的配置可以参考 GRP1_LOCAL_01 的配置步骤。



注意：源实例之间不能有相同的异步归档配置，避免同时向异步备库发送归档，导致异步备库数据乱掉的情况发生。

如果源实例自身出现故障，在故障恢复之前，都不会再发送归档到异步备库，所以如果有多个异步备库，可以考虑将异步备库分别配置到不同的源实例上，

7.5.1 环境说明

这里采用 7.2 配置实时主备 的实时主备环境，增加一台机器用来部署异步备库，其他环境不变。

新增的机器事先安装了 DM，安装路径为'/dm7'，执行程序保存在'/dm7/bin'目录中，数据存放路径为'/dm7/data'。

新增的异步备库实例名为“GRP1_LOCAL_01”，按照“组名_守护类型_序号”的建议方式命名，注意总长度不能超过 16。

表 7.10 配置环境说明

机器名	IP 地址	初始状态	操作系统	备注
DW_S2	192.168.1.133 192.168.0.143	异步备库 GRP1_LOCAL_01	Linux rh6-143.localdomain	192.168.1.133 外部服务 IP；

			2.6.32-220.el6.x86_64 #1 SMP Wed Nov 9 08:03:13 EST 2011 x86_64 x86_64 x86_64 GNU/Linux	192.168.0.143 内部通信 IP

7.5.2 数据准备

按照 7.1 数据准备 中的方法准备异步备库的数据。

如果实时主备守护环境已经处于运行状态，需要先正常退出主备库实例和守护进程。

7.5.3 配置主库 GRP1_RT_01

7.5.3.1 配置 dm.ini

在 7.2.3.1 配置 dm.ini的基础上，打开定时器配置，其他配置不变。

```
#配置有异步归档时，打开定时器，定时同步归档到异备库
TIMER_INI               = 1
```

7.5.3.2 配置 dmmal.ini

在 7.2.3.2 配置 dmmal.ini的基础上，增加异步备库 GRP1_LOCAL_01 的 mal 配置项。

```
[MAL_INST3]
    MAL_INST_NAME      = GRP1_LOCAL_01 #实例名，和 dm.ini 中的 INSTANCE_NAME 一致
    MAL_HOST               = 192.168.0.143 #MAL 系统监听 TCP 连接的 IP 地址
    MAL_PORT               = 61143         #MAL 系统监听 TCP 连接的端口
    MAL_INST_HOST          = 192.168.1.133 #实例的对外服务 IP 地址
    MAL_INST_PORT          = 32143 #实例的对外服务端口，和 dm.ini 中的 PORT_NUM 一致
    MAL_DW_PORT            = 52143 #实例对应的守护进程监听 TCP 连接的端口
```

7.5.3.3 配置 dmarch.ini

在 7.2.3.3 配置 dmarch.ini的基础上，增加异步归档的配置项。

```
[ARCHIVE_ASYNC]
    ARCH_TYPE               = ASYNC         #异步归档类型
    ARCH_DEST               = GRP1_LOCAL_01 #异步归档目标实例名
    ARCH_TIMER_NAME         = RT_TIMER      #定时器名称，和 dmtimer.ini 中的名称一致
```

7.5.3.4 配置 dmtimer.ini

配置 dmtimer.ini，用于定时触发实例发送归档日志到异步备实例。

下面示例中定时器配置为每天 00:00:00 触发主库发送归档日志到异步备库，可以根据实际情况再做调整。

```
[RT_TIMER] #和 dmarch.ini 中的 ARCH_TIMER_NAME 一致
     TYPE                        = 2
     FREQ_MONTH_WEEK_INTERVAL     = 1
     FREQ_SUB_INTERVAL    = 0
     FREQ_MINUTE_INTERVAL = 0
     START_TIME           = 00:00:00
     END_TIME               = 00:00:00.1047296
     DURING_START_DATE      = 2016-02-11 17:36:09
     DURING_END_DATE        = 9999-12-31 23:59:59
     NO_END_DATE_FLAG       = 1
     DESCRIBE               = RT TIMER
     IS_VALID               = 1
```

7.5.3.5 其他配置说明

如果配置异步备库时，实时主备环境已经配置完成，这里对主库不需要再做其他配置，由于在配置异步备库之前实时主备环境已经被正常关闭，此处可以使用 mount 方式将主库

重启启动，并启动主库的守护进程。

如果同时也在搭建主库和实时备库，则主库其他的配置步骤请再参考 7.2.3 配置主库 GRP1_RT_01的配置说明。

7.5.4 配置备库 GRP1_RT_02

该示例中备库上没有异步归档配置，但是由于 dmmal.ini 必须配置一致，因此需要修改备库上的 dmmal.ini 配置，其他已有的配置项不需要改动。

7.5.4.1 配置 dmmal.ini

dmmal.ini 中需要增加到异步备库 GRP1_LOCAL_01 的 mal 配置项，可以直接拷贝 7.5.3.2 配置 dmmal.ini修改后的 dmmal.ini 文件到本地/dm7/data/DAMENG/目录中。

7.5.4.2 其他配置说明

如果配置异步备库时，实时主备环境已经配置完成，这里对实时备库不需要再做其他配置，由于在配置异步备库之前实时主备环境已经被正常关闭，此处可以使用 mount 方式将实时备库重新启动，并启动实时备库的守护进程。

如果同时也在搭建主库和实时备库，则实时备库其他的配置步骤请再参考 7.2.4 配置备库 GRP1_RT_02的配置说明。

7.5.5 配置异步备库 GRP1_LOCAL_01

7.5.5.1 配置 dm.ini

在 DW_S2 机器上配置备库的实例名为 GRP1_LOCAL_01，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME            = GRP1_LOCAL_01
PORT_NUM                 = 32143 #数据库实例监听端口
DW_PORT                  = 33143 #守护环境下，监听守护进程连接端口
```

```
DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间
MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.5.5.2 配置 dmmal.ini

同一个守护进程组中，所有主备库必须使用相同的 dmmal.ini 配置文件，这里可以直接拷贝 7.5.3.2 配置 dmmal.ini的 dmmal.ini 文件到本地的/dm7/data/DAMENG/目录中。

7.5.5.3 配置 dmarch.ini

本地守护类型的备库只需要配置本地归档。

```
[ARCHIVE_LOCAL1]
    ARCH_TYPE               = LOCAL #本地归档类型
    ARCH_DEST               = /dm7/data/DAMENG/arch  #本地归档文件路径
    ARCH_FILE_SIZE          = 128   #单位 Mb，本地单个归档文件最大值
    ARCH_SPACE_LIMIT        = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.5.5.4 配置 dmwatcher.ini

修改 dmwatcher.ini 配置守护进程，配置为本地守护类型，异步备库不具备故障自动切换等功能，DW_MODE 配置并不起作用，此处配置为 MANUAL 即可，另外异步备库不需要配置 dmwatcher.ctl 控制文件。

```
[GRP1]
    DW_TYPE             = LOCAL     #本地守护类型
    DW_MODE             = MANUAL    #故障手动切换模式
    DW_ERROR_TIME       = 10        #远程守护进程故障认定时间
```

```
    INST_ERROR_TIME     = 10         #本地实例故障认定时间
    INST_OGUID          = 453331     #守护系统唯一 OGUID 值
    INST_INI            = /dm7/data/DAMENG/dm.ini   #dm.ini 配置文件路径
    INST_AUTO_RESTART   = 1          #打开实例的自动启动功能
    INST_STARTUP_CMD    = /dm7/bin/dmserver  #命令行方式启动
```

7.5.5.5 启动异步备库

以 mount 方式启动备库

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```

一定要以 mount 方式启动数据库实例，否则系统启动时会重构回滚表空间，生成 Redo 日志；并且，启动后应用可能连接到数据库实例进行操作，破坏主备库的数据一致性。数据守护配置结束后，守护进程会自动 Open 数据库。

7.5.5.6 设置 OGUID

启动命令行工具 DIsql，登录异步备库设置 OGUID 值。

```
SQL>sp_set_oguid(453331);
```

系统通过 OGUID 值确定一个守护进程组，由用户保证 OGUID 值的唯一性，并确保数据守护系统中，数据库、守护进程和监视器配置相同的 OGUID 值。

7.5.5.7 修改数据库模式

启动命令行工具 DIsql，登录异备库修改数据库为 Standby 模式

```
SQL>alter database standby;
```

7.5.6 配置监视器

在 7.2.5 配置监视器 确认监视器的配置基础上，需要增加到异步备库守护进程的“IP:PORT”信息，否则监视器无法收到异步备库消息，其他配置项不需要改动。

修改 dmmonitor.ini 配置确认监视器，增加到 GRP1_LOCAL_01 守护进程的

MON_DW_IP 配置项，需要找到 dmmal.ini 中实例 GRP1_LOCAL_01 的配置项信息，MON_DW_IP 中的 IP 信息和 dmmal.ini 配置项中的 MAL_HOST 一致，PORT 信息和 dmmal.ini 配置项中的 MAL_DW_PORT 配置项保持一致。

```
MON_DW_Confirm          = 1     #确认监视器模式
MON_LOG_PATH            = /dm7/data/log #监视器日志文件存放路径
MON_LOG_INTERVAL        = 60    #每隔 60s 定时记录系统信息到日志文件
MON_LOG_FILE_SIZE       = 32    #每个日志文件最大 32M
MON_LOG_SPACE_LIMIT     = 0     #不限定日志文件总占用空间
[GRP1]
    MON_INST_OGUID          = 453331 #组 GRP1 的唯一 OGUID 值
    #以下配置为监视器到组 GRP1 的守护进程的连接信息，以“IP:PORT”的形式配置
    #IP 对应 dmmal.ini 中的 MAL_HOST，PORT 对应 dmmal.ini 中的 MAL_DW_PORT
    MON_DW_IP               = 192.168.0.141:52141
    MON_DW_IP               = 192.168.0.142:52142
    #新增到异步备库 GRP1_LOCAL_01 守护进程的连接信息
    MON_DW_IP               = 192.168.0.143:52143
```

7.5.7 启动守护进程

启动异步备库的守护进程：

```
./dmwatcher /dm7/data/DAMENG/dmwatcher.ini
```

守护进程启动后，进入 Startup 状态，此时实例处于 Mount 状态，异步备库的守护进程会将本地实例自动 Open，并切换守护进程自身为 Open 状态。

7.5.8 启动监视器

修改监视器配置后，需要重新启动监视器：

```
./dmmonitor /dm7/data/dmmonitor.ini
```

监视器提供一系列命令，支持当前守护系统状态查看以及故障处理，可输入 help 命令，查看各种命令说明使用，结合实际情况选择使用。

至此实时主备环境下的异步备库搭建完成，可以根据实际情况考虑配置多个异步备库，

每个异步备库源实例的配置方式都可以参考主库 GRP1_RT_01 的配置，在搭建步骤和各项配置都正确的情况下，在监视器上执行 show 命令，可以监控到异步备库实例和异步备库守护进程都处于 Open 状态。

7.6 注册服务

如果想让 dmserver、dmwatcher 和 dmmonitor 服务开机自启动，需要手动注册服务，注意以下注册步骤必须用 root 用户进行，注册完成后重启机器时，就会自动启动 dmserver、dmwatcher 和 dmmonitor 服务。

7.6.1 注册 dmserver 服务

手动注册 dmserver 服务步骤如下：

1、复制服务文件 DmServeice

```
cp /opt/dmdbms/bin/service_template/DmService
/etc/rc.d/init.d/DmServiceDMSERVER
```

2、修改服务文件 DmServiceDMSERVER

```
修改 DmServiceDMSERVER
svc_prog=DmServiceDMSERVER
model=mount
path=INI 路径
```

3、修改启动服务的等级连接

```
ln -s /etc/rc.d/init.d/DmServiceDMSERVER /etc/rc.d/rc0.d/K02DmServiceDMSERVER
ln -s /etc/rc.d/init.d/DmServiceDMSERVER /etc/rc.d/rc2.d/S98DmServiceDMSERVER
ln -s /etc/rc.d/init.d/DmServiceDMSERVER /etc/rc.d/rc3.d/S98DmServiceDMSERVER
ln -s /etc/rc.d/init.d/DmServiceDMSERVER /etc/rc.d/rc5.d/S98DmServiceDMSERVER
ln -s /etc/rc.d/init.d/DmServiceDMSERVER /etc/rc.d/rc6.d/K02DmServiceDMSERVER
```

7.6.2 注册 dmwatcher 服务

手动注册 dmwatcher 服务步骤如下：

1、复制服务文件 DmWatcherService

```
cp /opt/dmdbms/bin/service_template/DmWatcherService
/etc/rc.d/init.d/DmWatcherService
```

2、修改服务文件 DmWatcherService

```
修改 DmWatcherService
svc_prog=DmWatcherService
path=INI 路径
```

3、修改启动服务的等级连接

```
ln -s /etc/rc.d/init.d/DmWatcherService /etc/rc.d/rc0.d/K02 DmWatcherService
ln -s /etc/rc.d/init.d/DmWatcherService /etc/rc.d/rc2.d/S98 DmWatcherService
ln -s /etc/rc.d/init.d/DmWatcherService /etc/rc.d/rc3.d/S98 DmWatcherService
ln -s /etc/rc.d/init.d/DmWatcherService /etc/rc.d/rc5.d/S98 DmWatcherService
ln -s /etc/rc.d/init.d/DmWatcherService /etc/rc.d/rc6.d/K02 DmWatcherService
```

7.6.3 注册 dmmonitor 服务

一般只有确认监视器需要注册服务，确保机器启动后，自动启动 dmmonitor。

手动注册 dmmonitor 服务步骤如下：

1. 复制服务文件 DmMonitorService

```
cp /opt/dmdbms/bin/service_template/DmMonitorService
/etc/rc.d/init.d/DmMonitorService
```

2. 修改服务文件 DmMonitorService

```
修改 DmMonitorService
svc_prog=DmMonitorService
path=INI 路径
```

3. 修改启动服务的等级连接

```
ln -s /etc/rc.d/init.d/DmMonitorService /etc/rc.d/rc0.d/K02 DmMonitorService
ln -s /etc/rc.d/init.d/DmMonitorService /etc/rc.d/rc2.d/S98 DmMonitorService
ln -s /etc/rc.d/init.d/DmMonitorService /etc/rc.d/rc3.d/S98 DmMonitorService
ln -s /etc/rc.d/init.d/DmMonitorService /etc/rc.d/rc5.d/S98 DmMonitorService
```

```
ln -s /etc/rc.d/init.d/DmMonitorService /etc/rc.d/rc6.d/K02 DmMonitorService
```

7.7 动态增加读写分离集群节点

当需要进行系统扩容，希望系统运行不中断，或者影响运行的时间尽可能短，可通过动态增加集群节点的方式进行。下面举例对读写分离集群进行动态增加节点。

本例是在 7.3 配置读写分离集群 的基础上，再增加一个备库，实例名为 GRP1_RWW_04。

表 7.11 配置环境说明

机器名	IP 地址	初始状态	操作系统	备注
DW_S3	192.168.1.134 192.168.0.144	主库 GRP1_RWW_04	Linux rh6 x86_64	192.168.1.134 外部服务 IP； 192.168.0.144 内部通信 IP

详细步骤如下：

7.7.1 数据准备

1.对主库进行联机备份操作：

```
SQL> BACKUP DATABASE BACKUPSET ' BACKUP_FILE_01';
```

2.初始化备机数据库

```
dminit.exe path=/dm7/data/
```

3.还原恢复新增备库

拷贝生成的备份集目录 BACKUP_FILE_01 到 144 上/dm7/data/目录，使用 DMRMAN 工具脱机还原。

```
./dmrman CTLSTMT="RESTORE DATABASE '/dm7/data/DAMENG/dm.ini' FROM BACKUPSET
'/dm7/data/BACKUP_FILE_01'"
./dmrman CTLSTMT="RECOVER DATABASE '/dm7/data/DAMENG/dm.ini' FROM BACKUPSET
'/dm7/data/BACKUP_FILE_01'"
```

7.7.2 配置新备库

7.7.2.1 配置 dm.ini

在 DW_S3 机器上配置备库的实例名为 GRP1_RWW_04，dm.ini 参数修改如下：

```
#实例名，建议使用“组名_守护类型_序号”的命名方式，总长度不能超过 16
INSTANCE_NAME           = GRP1_RWW_04


PORT_NUM                = 32144 #数据库实例监听端口
DW_PORT                 = 33144 #守护环境下，监听守护进程连接端口


DW_ERROR_TIME           = 60    #接收守护进程消息超时时间
ALTER_MODE_STATUS       = 0     #不允许手工方式修改实例模式/状态
ENABLE_OFFLINE_TS       = 2     #不允许备库 OFFLINE 表空间


MAL_INI                 = 1     #打开 MAL 系统
ARCH_INI                = 1     #打开归档配置
HA_INST_CHECK_FLAG      = 1     #检测是否多个实例进程同时启动
```

7.7.2.2 配置 dmmal.ini

拷贝一份原系统 dmmal.ini 文件，并加上自己一项，最终配置如下：

```
MAL_CHECK_INTERVAL      = 5     #MAL 链路检测时间间隔
MAL_CONN_FAIL_INTERVAL  = 5     #判定 MAL 链路断开的时间


[MAL_INST1]
    MAL_INST_NAME        = GRP1_RWW_01
    MAL_HOST              = 192.168.0.141
    MAL_PORT                = 61141
    MAL_INST_HOST           = 192.168.1.131
```

```
    MAL_INST_PORT           = 32141
    MAL_DW_PORT         = 52141

[MAL_INST2]
    MAL_INST_NAME           = GRP1_RWW_02
    MAL_HOST                = 192.168.0.142
    MAL_PORT                = 61142
    MAL_INST_HOST           = 192.168.1.132
    MAL_INST_PORT           = 32142
    MAL_DW_PORT         = 52142

[MAL_INST3]
    MAL_INST_NAME           = GRP1_RWW_03
    MAL_HOST                = 192.168.0.143
    MAL_PORT                = 61143
    MAL_INST_HOST           = 192.168.1.133
    MAL_INST_PORT           = 32143
    MAL_DW_PORT         = 52143

[MAL_INST4]
    MAL_INST_NAME           = GRP1_RWW_04
    MAL_HOST                = 192.168.0.144
    MAL_PORT                = 61144
    MAL_INST_HOST           = 192.168.1.134
    MAL_INST_PORT           = 32144
    MAL_DW_PORT         = 52144
```

7.7.2.3 配置 dmarch.ini

修改 dmarch.ini，配置本地归档和即时归档。

```
[ARCHIVE_TIMELY1]
    ARCH_TYPE               = TIMELY        #即时归档类型
    ARCH_DEST               = GRP1_RWW_01   #即时归档目标实例名

[ARCHIVE_TIMELY2]
    ARCH_TYPE               = TIMELY        #即时归档类型
    ARCH_DEST               = GRP1_RWW_02   #即时归档目标实例名

[ARCHIVE_TIMELY3]
    ARCH_TYPE               = TIMELY        #即时归档类型
    ARCH_DEST               = GRP1_RWW_03   #即时归档目标实例名

 [ARCHIVE_LOCAL1]
    ARCH_TYPE               = LOCAL         #本地归档类型
    ARCH_DEST               = /dm7/data/DAMENG/arch     #本地归档文件存放路径
    ARCH_FILE_SIZE          = 128   #单位 Mb，本地单个归档文件最大值
    ARCH_SPACE_LIMIT        = 0     #单位 Mb，0 表示无限制，范围 1024~4294967294M
```

7.7.2.4 配置 dmwatcher.ini

修改 dmwatcher.ini 配置守护进程，配置为读写分离守护类型，使用自动切换模式。

```
[GRP1]
     DW_TYPE            = TIMELY    #读写分离守护类型
     DW_MODE            = AUTO      #自动切换模式
     DW_ERROR_TIME      = 10        #远程守护进程故障认定时间
     INST_RECOVER_TIME  = 60        #主库守护进程启动恢复的间隔时间
     INST_ERROR_TIME    = 10        #本地实例故障认定时间
     INST_OGUID         = 453332    #守护系统唯一 OGUID 值
     INST_INI           = /dm7/data/DAMENG/dm.ini   #dm.ini 配置文件路径
     INST_AUTO_RESTART  = 1         #打开实例的自动拉起功能
```

```
    INST_STARTUP_CMD   = /dm7/bin/dmserver #命令行方式启动
```

7.7.2.5 配置 dmwatcher.ctl

同一个守护进程组，直接拷贝主库的 dmwatcher.ctl 文件到本地数据文件目录 /dm7/data/DAMENG。

7.7.2.6 启动备库

以 mount 方式启动备库

```
./dmserver /dm7/data/DAMENG/dm.ini mount
```

7.7.2.7 设置 OGUID

启动命令行工具 disql，登录备库设置 OGUID 值。

```
SQL>sp_set_oguid(453332);
```

7.7.2.8 修改数据库模式

启动命令行工具 disql，登录备库修改数据库为 Standby 模式

```
SQL>alter database standby;
```

7.7.3 动态添加 MAL 配置

本小节的步骤需要分别连接原系统中每个实例单独执行：

1.允许允许手工修改服务器的模式和状态

```
SP_SET_PARA_VALUE(1, 'ALTER_MODE_STATUS', 1);
```

2.停止或者退出原系统所有守护进程

3.修改数据库状态为 MOUNT

```
alter database mount;
```

4.动态增加 mal 中 GRP1_RWW_04 的相关配置信息

```
SF_MAL_CONFIG(1,0);
```

```
SF_MAL_INST_ADD('MAL_INST4','GRP1_RWW_04','192.168.0.144',61144,'192.168.1.1
34', 32144, 52144);
SF_MAL_CONFIG_APPLY();
SF_MAL_CONFIG(0,0);
```

7.7.4 动态添加归档配置

分别连接原系统中的所有实例（此时处于 MOUNT 状态），动态添加 dmarch.ini 中归档节点

```
SQL> alter database add archivelog 'DEST=GRP1_RWW_04, TYPE=TIMELY';
```

7.7.5 修改监视器 dmmonitor.ini

在 dmmonitor.ini 中添加新增的 GRP1_RWW_04 备机：

```
MON_DW_IP               = 192.168.0.144:52144
```

7.7.6 启动所有守护进程以及监视器

分别启动主库和备库（包括 GRP1_RWW_04）的所有守护进程，最后启动监视器。

8 自动化脚本部署工具

DM 数据守护系统包含诸多部件，需要在多个节点部署。部署过程包括数据准备、配置文件修改，数据一致性确认等多个步骤。纯手工方式搭建往往费时费力，各个节点配置参数的修改也容易出错。为了简化数据守护系统搭建过程，提高配置效率，我们提供了一套脚本部署工具 

，支持数据守护系统的自动化部署。其实现原理是，使用 VBS 脚本生成搭建数据守护系统所用到的所有配置文件，通过 Putty 工具与集群所在的 Linux 服务器进行交互，并使用 BAT 批处理文件实现数据守护系统的搭建。

DM_CTL_V1.0 工具中把不同类型的数据守护系统，统称为集群。根据归档类型的不同，又将集群类型分为：读写分离集群（TIMELY）、数据守护集群（REALTIME）和 MPP 主备集群（MARCH）。

 说明：	**更新发布在达梦公司内部交流工具：为知笔记 技术简报\达梦集群部署工具**

8.1 配置文件

DataWatch\data_watch.conf 该文件为默认的配置文件，可参照该进行创建新的配置文件。

编辑：data_watch.conf

```
#请养成用 dmdba 用户的习惯

#此部署工具需要在 Windows 系统上运行，确保该 Windows 机器可通过外网连接到集群所有机器。集群机器必须是 linux 机
器，支持 ssh。

#groupadd dinstall

#useradd -g dinstall dmdba

#passwd dmdba

#配置文件



[GLOBAL]

DM_PATH       = /home/dmdba/dm7          #达梦安装目录，若是/opt/dmdbms/bin，取 bin 所在的目录，即
```

```
/opt/dmdbms

DATA_PATH     = /home/dmdba/data          #集群数据目录，确保该目录只存储该集群数据

ARCH_PATH        = /home/dmdba/arch          #根据实际修改                    #实例的本地归档路径，为
/home/dmdba/arch/GDYH_RT 或/arch/GDYH_MPP_EPXX

BAK_PATH         = /home/dmdba/dbbak            #根据实际修改                 #实例的数据库备份的默认路
径，为/home/dmdba/dbbak/GDYH_RT 或/dbbak/GDYH_MPP_EPXX

DB_INIT_PARA  = page_size=16 extent_size=32   #数据库初始化参数

DB_NAME     = GDYH   #数据库名称,也是数据文件所在路径，data 所在目录为：/home/dmdba/data/GDYH(_MPP)

DB_INST_PORT  = 5236      #数据库实例端口号,MPP 的备库端口为(DB_INST_PORT+1)

OGUID         = 12000     #OGUID，代表集群的唯一标识



DW_TYPE       = MARCH       #搭建的集群类型：读写分离:TIMELY,数据守护：REALTIME,MPP 主备:MARCH

DW_MODE      = MANUAL       #故障切换模式：故障自动切换(AUTO)，故障手动切换(MANUAL)

AUTO_RESTART  = 1           #实例自动重启，1 表示自动重启，0 表示不自动重启

INST_RECOVER_TIME    = 60   #主库守护进程启动恢复的间隔时间

INST_ERROR_TIME  = 10        #本地数据库故障认定时间

DW_ERROR_TIME   = 10         #远程主（备）机守护进程故障认定时间

ARCH_WAIT_APPLY  = 1         #配置为即时归档方式时，是否需要等待备库做完日志，1 表示需要，0 表示不需要。



[MAIL]#一般不用改参数

MAL_SYS_BUF_SIZE=1000

MAL_BUF_SIZE=1000

MAL_VPOOL_SIZE=1000

MAL_CHECK_INTERVAL=5

MAL_CONN_FAIL_INTERVAL=5

MAL_COMPRESS_LEVEL=10



#各节点配置

#第 1 个 NODE 代表主库，后面的 NODE 代表多台备库,IP 地址不能重复，1 个 IP 地址只能对应 1 个 NODE，且部署时只需要第
1 个 NODE 机器上的 DM_PATH 路径下存在 bin 目录即可，其它机器不需要手工安装达梦数据库
```

```
#对于读写分离集群，第 2 个以后的 IP 地址均为即时备库。

#对于数据守护集群，第 2 个 IP 地址是实时备库，第 3 个以后的 IP 地址为异步备库,异步备库的序号为奇数时其源为主库，异步备库的序号为偶数时其源为实时备库。

#对于 MPP 主备集群，每个 IP 地址代表一个 MPP 的 EP 节点,交叉互备，IP 地址不允许重复

#实例名命名规则：

#MPP 集群: 主库 GRP(i)_MPP_EP0(i),备库 GRP(i)_MPP_EP1(i),i 代表第 i 个节点。

#数据守护: 主库 GRP1_RT_01, 备库 GRP1_RT_02  异步备库 GRP1_LOCAL_0(i)

#读写分离: 主库 GRP1_RWW_01, 备库 GRP1_RWW_0(i)



[NODE1]

MAL_HOST = 192.168.5.42    #内网 IP

INST_HOST = 223.254.20.42   #外网 IP

INST_UID  = dmdba

INST_PWD  = hust4400



[NODE2]

MAL_HOST = 192.168.5.43

INST_HOST = 223.254.20.43

INST_UID  = dmdba

INST_PWD  = hust4400



[NODE3]

MAL_HOST = 192.168.5.44

INST_HOST = 223.254.20.44

INST_UID  = dmdba

INST_PWD  = hust4400



[MONITOR]

MON_MAL_HOST = 192.168.5.45    #内网 IP
```

```
MON_INST_HOST = 223.254.20.45  #外网 IP

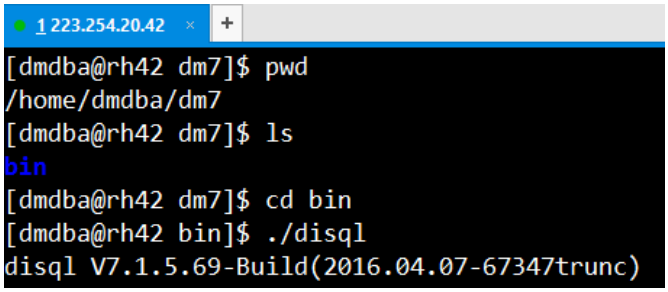
MON_INST_UID  = dmdba

MON_INST_PWD  = hust4400
```

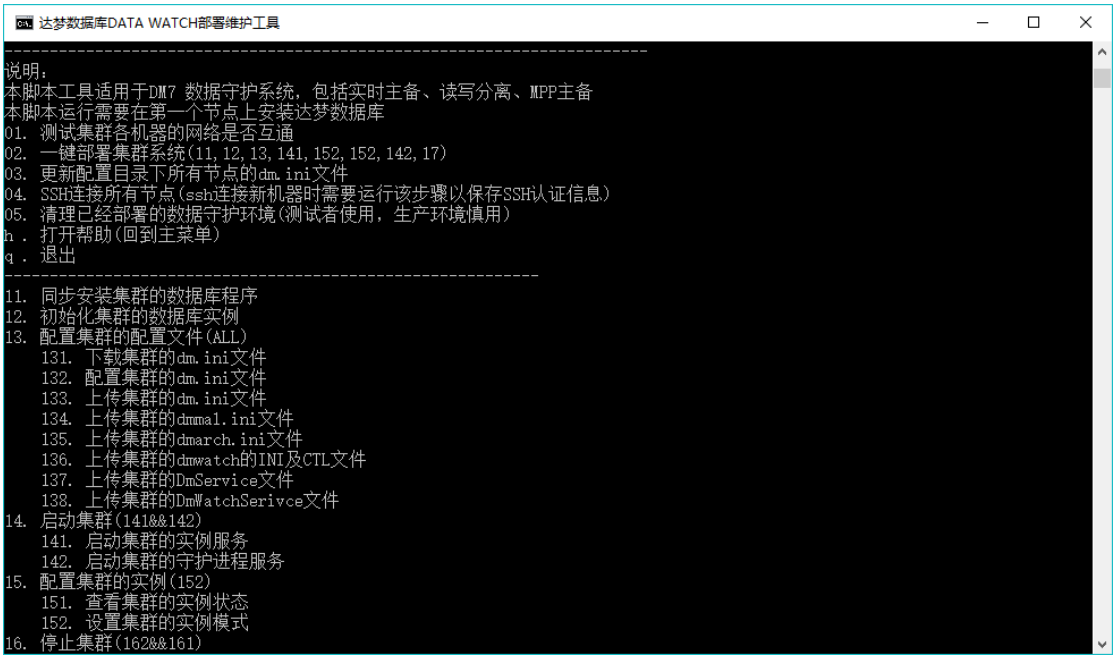
1. 根据集群服务器的资源，修改 NODE(1..x)的 IP 地址及用户名密码，推荐用非 root 用户。
2. 手工配置 NODE1 机器的 DM_PATH，确保该路径下存在达梦数据库的 bin 目录。
3. 根据需要，配置相关的参数：
   1) DW_TYPE 表示将要搭建的集群类型，读写分离:TIMELY, 数据守护:REALTIME, MPP 主备:MARCH
   2) DW_MODE 表示集群的故障切换模式：故障自动切换模式(AUTO)，故障手动切换模式(MANUAL)。
   3) DM_PATH 表示达梦安装目录，即 bin 所在的目录。
   4) DATA_PATH 表示集群数据目录，即 data 所在的目录。
   5) DB_INIT_PARA 表示集群实例初始化时的参数，如页大小、大小写敏感等。
   6) DB_NAME 表示数据库的名称，初始化实例时用到的 PATH，即为(DATA_PATH+DB_NAME)：/home/dmdba/data/GDYH(_MPP) 。
   7) DB_INST_PORT 表示数据库实例的端口号。对于 dmmal.ini 中的 MAL_PORT、MAL_DW_PORT 依次为 DB_INST_PORT+10 、 DB_INST_PORT+20 。即 DB_INST_PORT=5236，则 MAL_PORT=5246 、 MAL_DW_PORT=5256，而 DW_PORT= DB_INST_PORT+30,即 DW_PORT=5266 而对于 MPP+主备来说，备库的相应端口号为主库相应实例的端口号+1。
   8) DW_PORT 表示本机守护进程与数据库实例的 TCP 通信端口。
   9) OGUID 表示集群的唯一的标识，请确保多套集群间的该值不会重复。
   10) AUTO_RESTART 表示本机的数据库实例故障后，本机的守护进程是否自动将实例拉起来。
   11) INST_ERROR_TIME 表示本机的数据库故障认定时间。
   12) DW_ERROR_TIME 表示远程主（备）机守护进程故障认定时间
   13) ARCH_WAIT_APPLY 表示 配置为即时归档方式时，是否需要等待备库做完日志，1 表示需要，0 表示不需要

8.2 搭建步骤

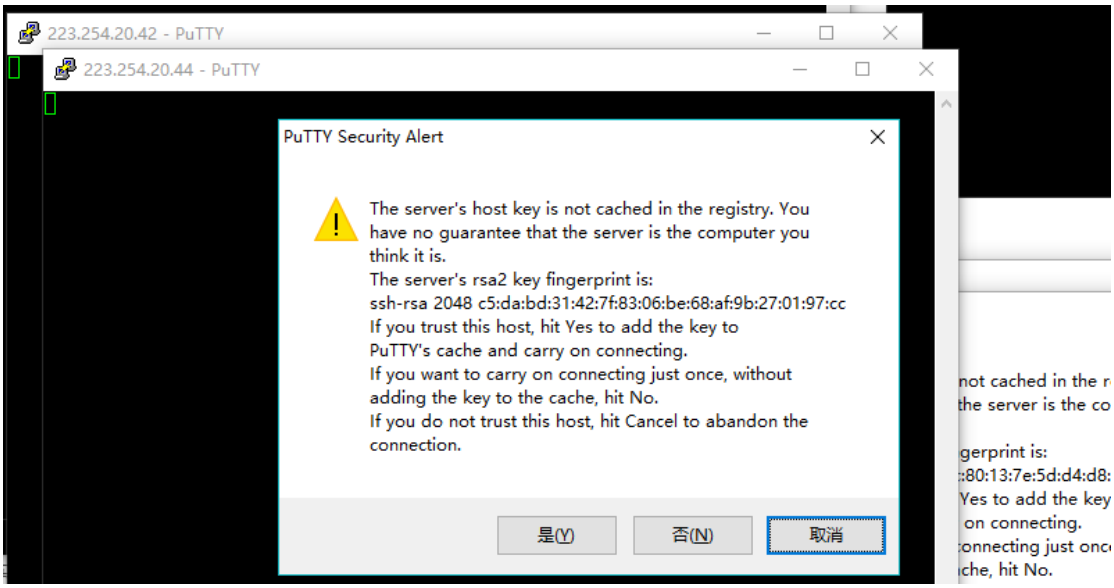
1. 配置好 data_watch.conf 文件后，检查相关的集群机器环境，并确保在第 1 个 node 的 dm_path 路径下存在达梦数据库的 bin 目录。其它机器上不存在该目录。



2. 运行 data_watch.bat，并回车：

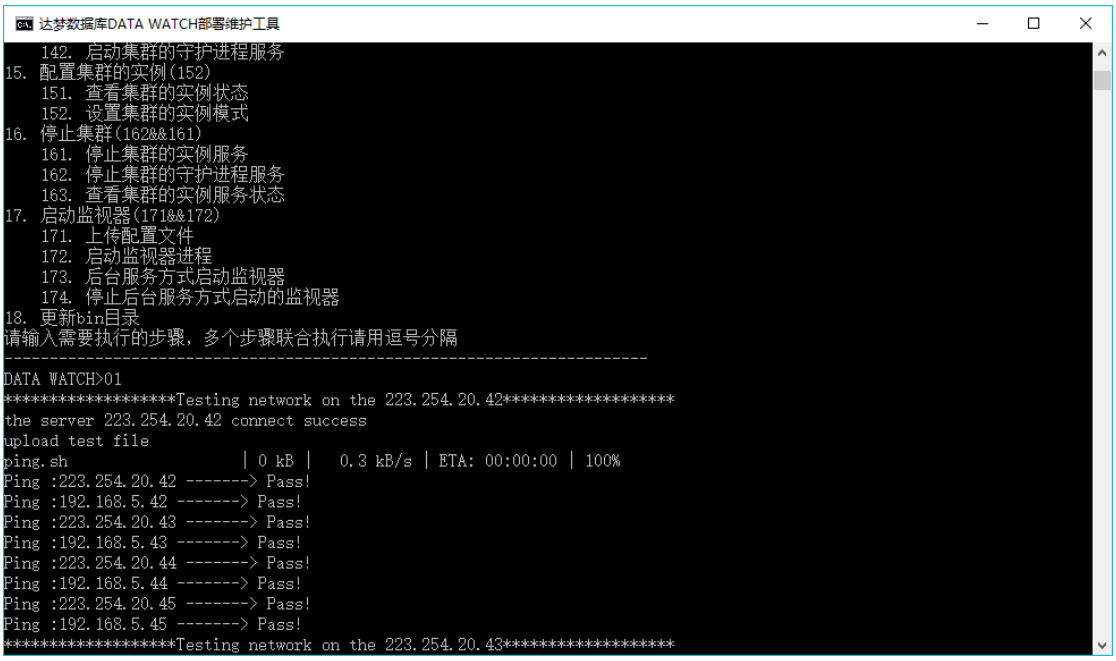


3. 运行 04，保存 SSH 的认证信息：



依次点击“是”。

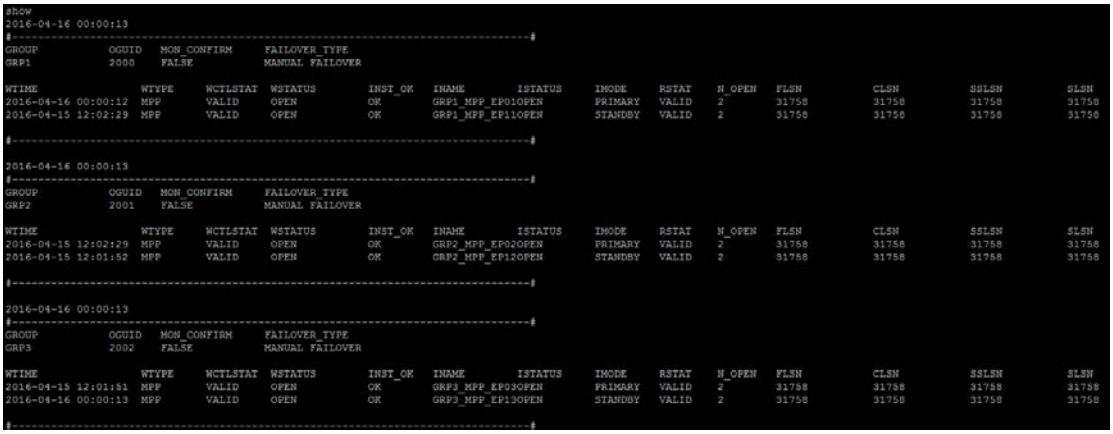
4． 运行 01，测试集群机器间的网络情况：



5． 运行 02，一键部署集群：

02 其实依次运行了：11,12,13,141,152,152,142,17。当然也可以手工依次运行 11,12,13,141,152,152,142,17 这些步骤，等同于 02。

6． 运行 172，启动监视器。查看集群的状态，此时集群的状态正常：



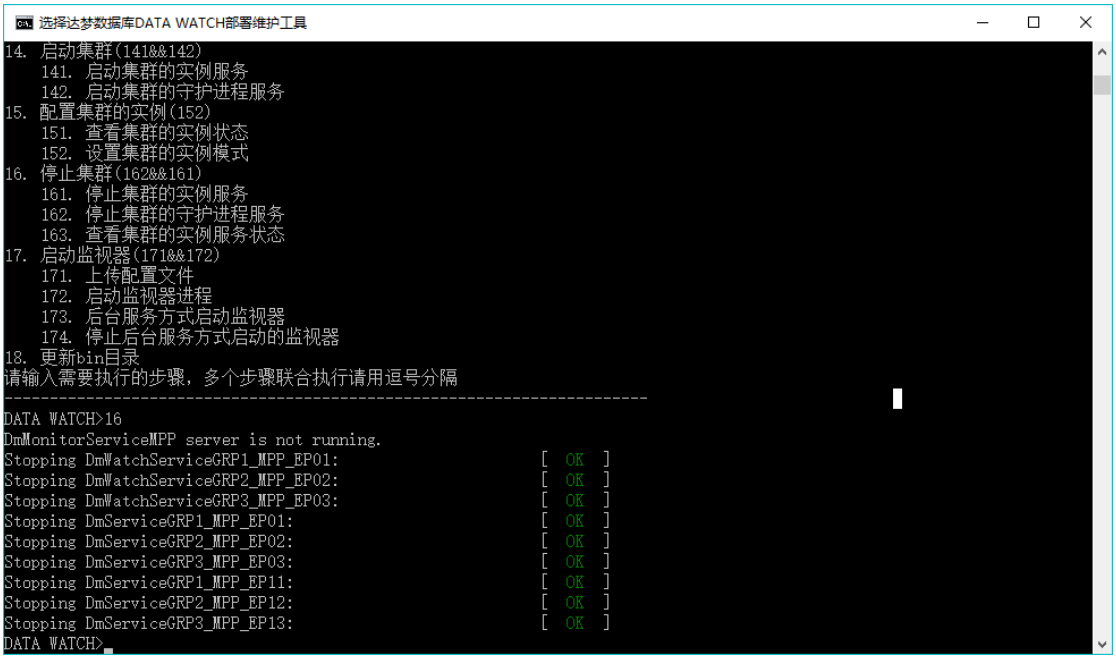
7. 至此集群搭建完成。上述以搭建 MPP 主备集群为例，搭建实时主备集群、读写分离集群同样的步骤，只需要将 DW_TYPE 改为 REALTIME、TIMELY 即可。

8.3 集群维护

8.3.1 重启集群

1. 停止集群

在集群部署工具中，运行 16，停止集群。该步骤等同于 162,161，即先停止守护进程服务，再停止数据库实例服务。另停止集群前，会先停止监视器的服务。



2. 启动集群

在集群部署工具中，运行 14，启动集群。该步骤等同于 141,142，即先启动数据库实例服务，再启动守护进程服务。



8.3.2 批量更新 dm.ini 参数

1. 配置 DataWatch\dm.ini (注：最后一行不能为空行)

```
MAX_OS_MEMORY                = 100

MEMORY_POOL                  = 8000

N_MEM_POOLS                  = 4

BUFFER                       = 2000

BUFFER_POOLS                 = 10

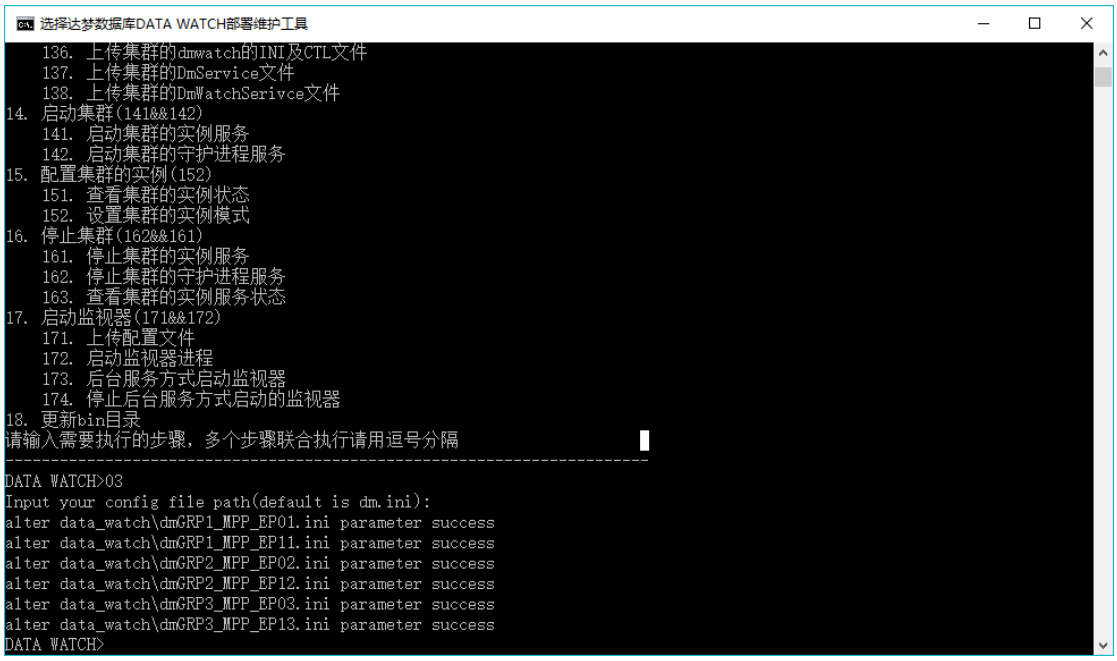
MAX_BUFFER                   = 2000

HUGE_BUFFER                  = 8000

RECYCLE                      = 8

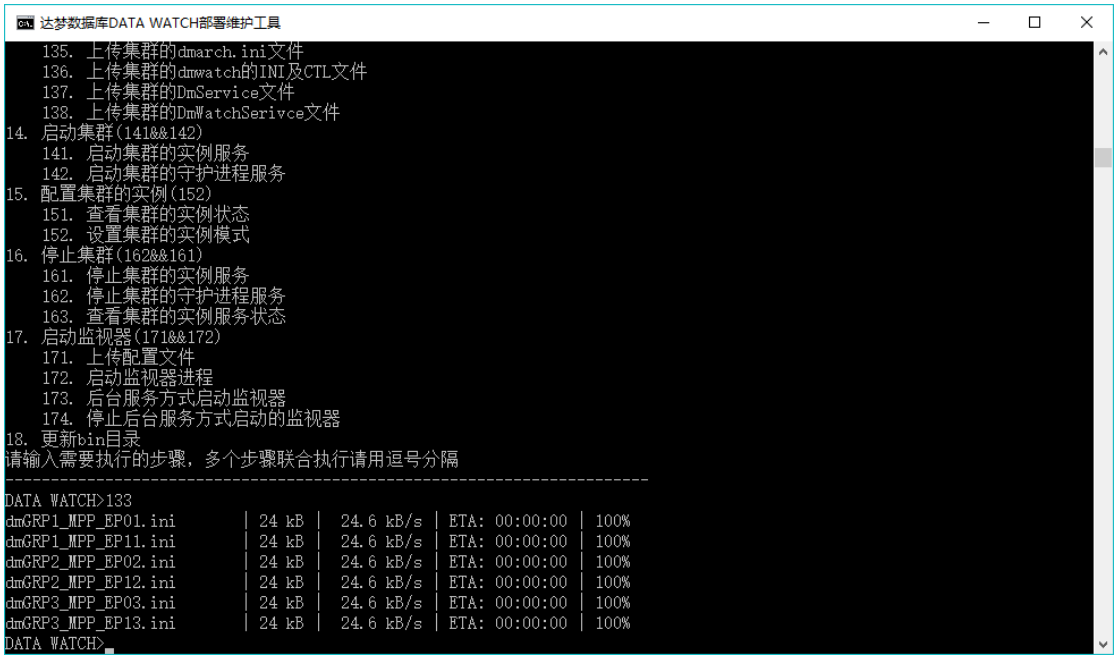
SORT_BUF_SIZE                = 2
```

2. 在集群部署工具中，运行 03，输入 dm.ini (若配件文件名称为 dm.ini，可不输入，直接回车)



则在 DataWatch\data_watch 目录下，将相应的 dm.ini 中的参数更新完成

3. 运行 133，将更新后的 dm.ini 配置文件，上传到集群服务器上。



4. 重启集群，使相应的参数生效。

8.3.3 配置 dm_svc.conf

在 DataWatch\data_watch 目录下会自动生成集群的 dm_svc.conf 配置文件，请自行复制它到客户端机器的相关目录下：dm_svc.conf 配置文件，Windows 平台下位于%SystemRoot%\system32 目录，Linux 平台下位于/etc 目录。

```
TIME_ZONE=(480)

LANGUAGE=(en)

DM_GDYH_MPP=(223.254.20.42:1236,223.254.20.42:1237,223.254.20.43:1236,223.25

4.20.43:1237,223.254.20.44:1236,223.254.20.44:1237)

LOGIN_PRIMARY=(1)

SWITCH_TIME=6000

SWITCH_INTERVAL=500
```

8.3.4 达梦集群部署工具的维护

达梦集群部署工具，是通过 DataWatch\bin\CONFIG.VBS 这个 VBS 脚本来生成达梦集群的配置文件、部署步骤的 BAT，并通过 putty 工具与集群服务器进行 ssh 交互，包括执行命令、上传下载配置文件、上传下载数据库文件。

对于达梦集群部署工具的维护，主要从下述 2 方面进行：

1．定位出错的步骤，并查看相关的 BAT 文件。

在集群步骤过程，若运行 133 步骤出错，则在 DataWatch\data_watch 文件夹下，编辑打开 133.BAT，检查其内容是否生成正确

```
pscp -batch -pw hust4400 DM_CONF\dmGRP1_MPP_EP01.ini dmdba@223.254.20.42:/home/dmdba/data/GDYH_MPP_EP01/DAMENG/dm.ini

pscp -batch -pw hust4400 DM_CONF\dmGRP1_MPP_EP11.ini dmdba@223.254.20.43:/home/dmdba/data/GDYH_MPP_EP01/DAMENG/dm.ini

pscp -batch -pw hust4400 DM_CONF\dmGRP2_MPP_EP02.ini dmdba@223.254.20.43:/home/dmdba/data/GDYH_MPP_EP02/DAMENG/dm.ini

pscp -batch -pw hust4400 DM_CONF\dmGRP2_MPP_EP12.ini dmdba@223.254.20.44:/home/dmdba/data/GDYH_MPP_EP02/DAMENG/dm.ini

pscp -batch -pw hust4400 DM_CONF\dmGRP3_MPP_EP03.ini dmdba@223.254.20.44:/home/dmdba/data/GDYH_MPP_EP03/DAMENG/dm.ini

pscp -batch -pw hust4400 DM_CONF\dmGRP3_MPP_EP13.ini dmdba@223.254.20.42:/home/dmdba/data/GDYH_MPP_EP03/DAMENG/dm.ini
```

若发现生成的 133.BAT 内容出错，则打开编辑 DataWatch\bin\CONFIG.VBS

2．修改 CONFIG.VBS

编辑 CONFIG.VBS，搜索 133.BAT，定位到生成 133.BAT 的方法，查看具体出错的地方并修正。

3．再次运行 data_watch.bat，查看生成的 133.BAT 的内容是否正确。

9 利用 DM 部署工具搭建数据守护

DM 部署工具 deploy.exe 提供数据守护的图形化搭建与管理功能。该工具位于安装目录的 tool 目录下，例如：/dmdbms/tool/deploy.exe。双击就可以打开。搭建与管理的守护类型有三种：MPP 主备、实时主备和读写分离主备。每一种类型都分为两大步骤：部署和监控。图形界面如下：

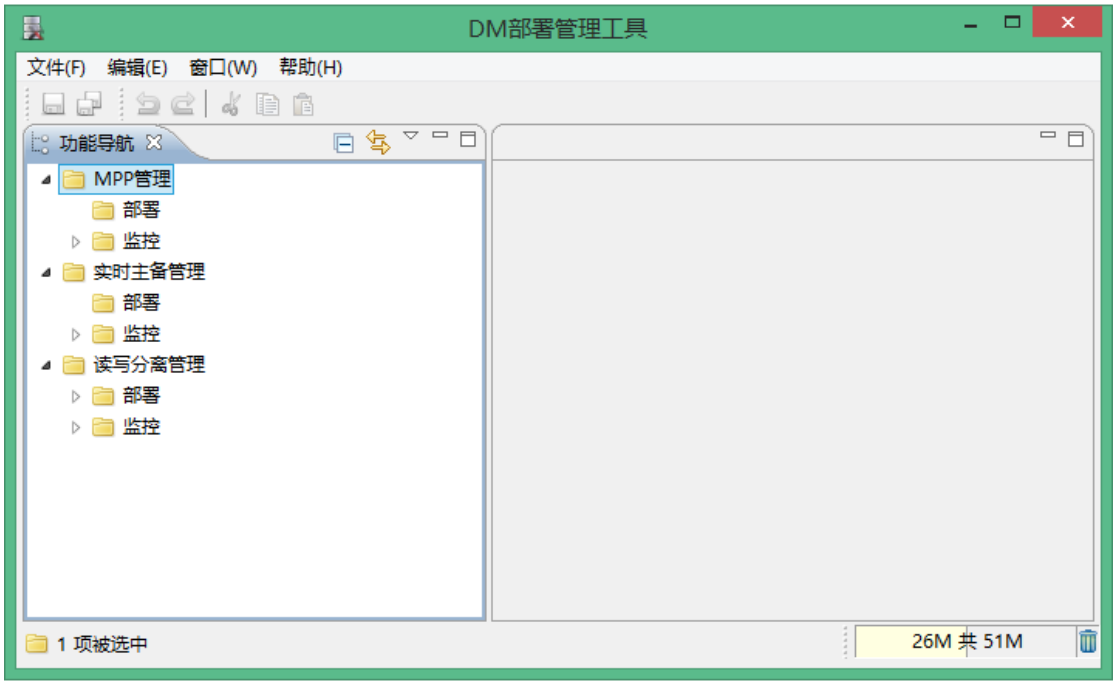


图 9.1 deploy 工具主页面

本章以实时主备管理为例，来介绍数据守护的搭建过程。MPP 管理、读写分离管理等更多详情，请参考 deploy 工具帮助文档。

9.1 实时主备管理

下面将介绍实时主备管理是如何搭建的。实时主备管理分为两大步骤：部署和监控。部署和监控的自动化搭建操作一气呵成，不需要各自单独进行配置。

9.1.1 部署

部署主界面。

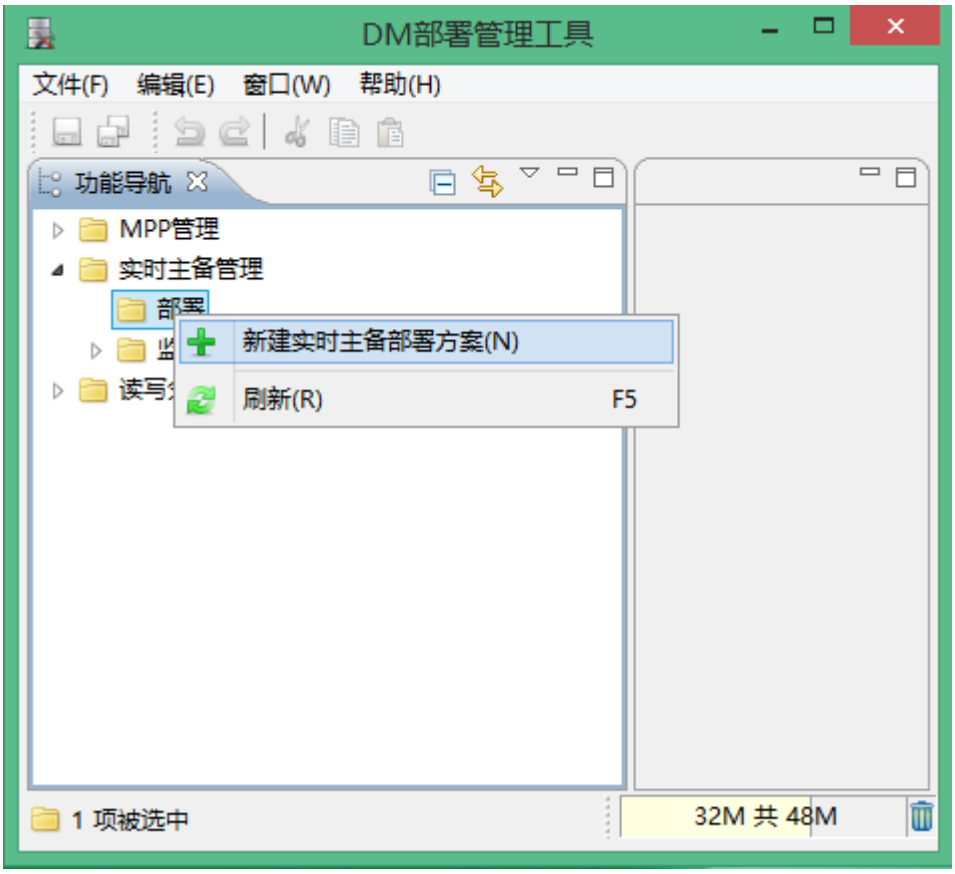


图 9.2 部署界面-新建

右键，选择新建实时主备部署方案：

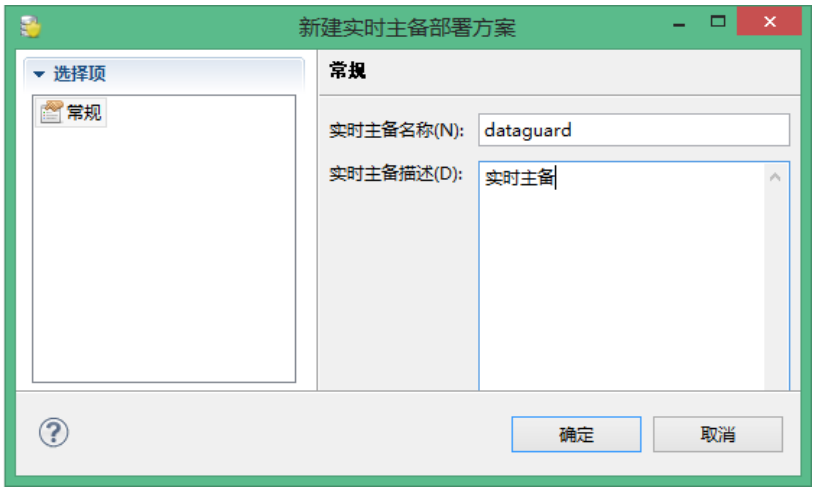


图 9.3 部署界面-常规

点击“确定”以后，进入选择“数据库代理”阶段：

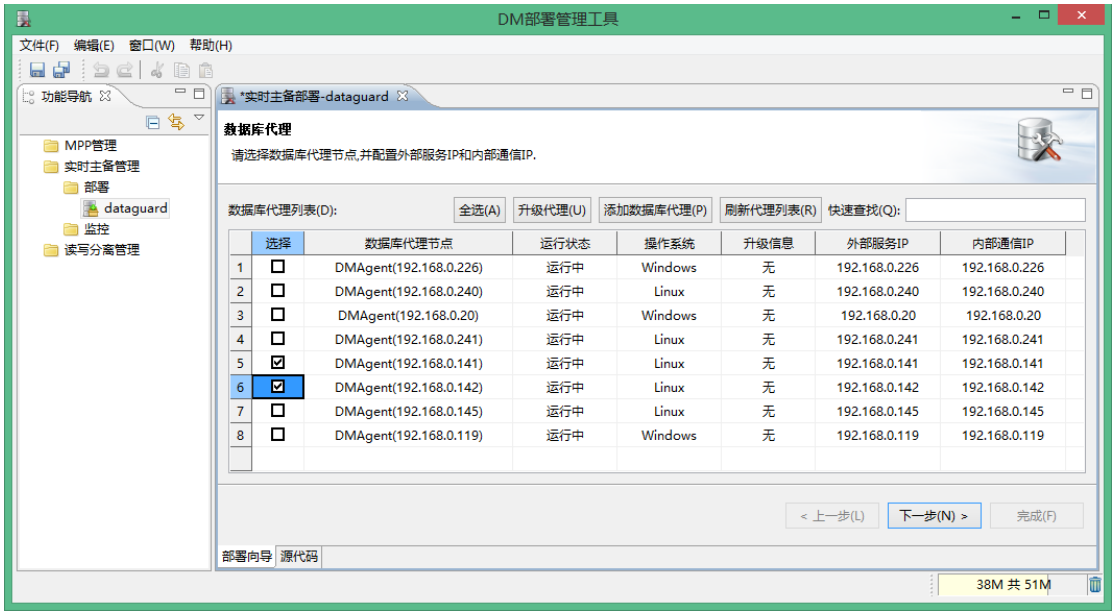


图 9.4 数据库代理界面

参数详解：

全选：全选所有数据库代理节点。

升级代理：在线升级代理版本。

添加数据库代理：添加新的数据库代理节点。

刷新代理列表：刷新数据库代理节点列表。

快速查找：输入 IP 地址，快速查找指定代理节点。

点击“下一步”，进入“数据准备”界面：

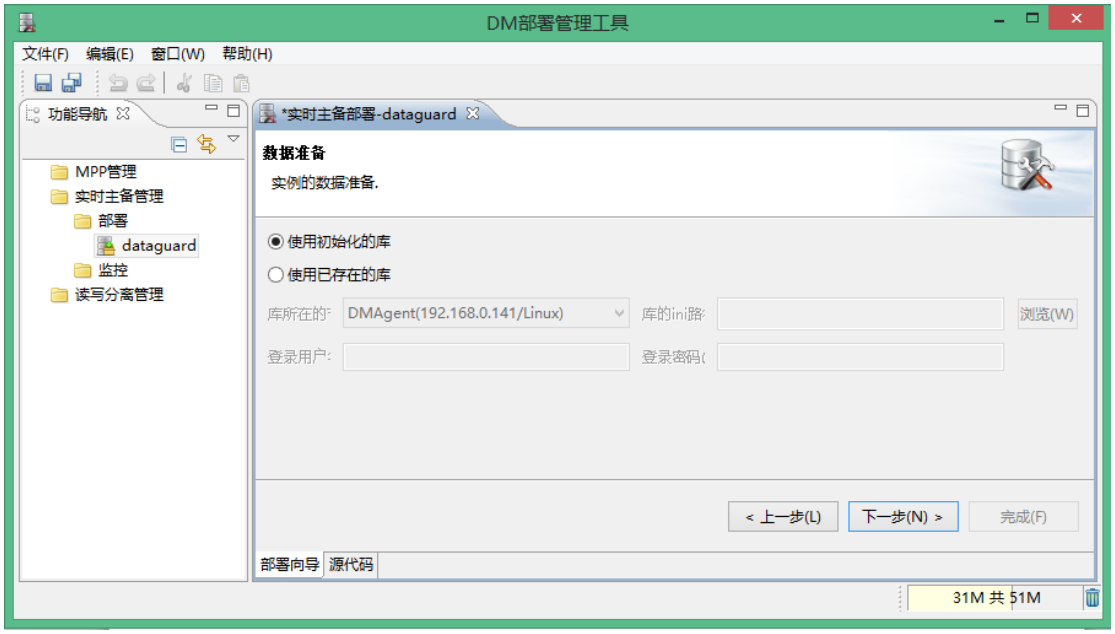


图 9.5 数据库准备界面

参数详解：

使用初始化的库：初始化一个新库，作为守护系统的实例。

使用已存在的库：指定已经存在的库作为守护系统的实例。指定库所在的 IP、INI 路径、库的登录用户名和密码。

点击“下一步”，进入“数据库初始化参数”设置界面：

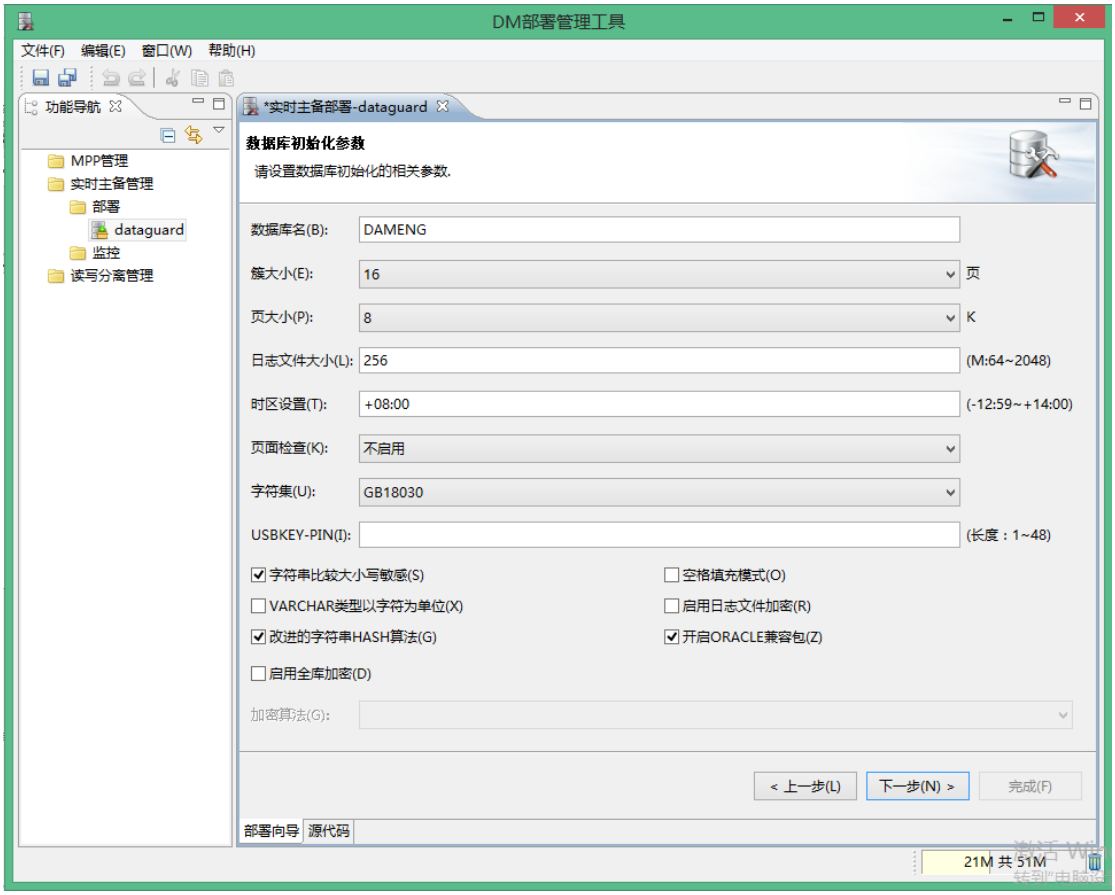


图 9.6 数据库初始化参数界面

点击“下一步”，进入“数据库密码参数”设置界面：

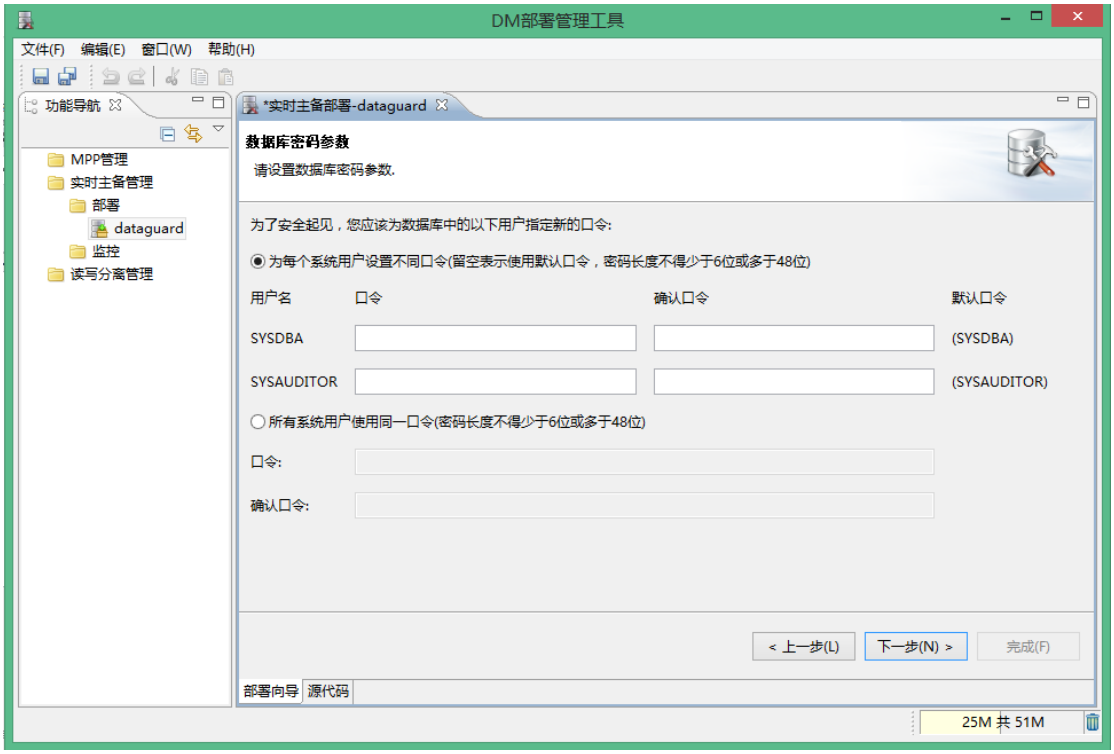


图 9.7 数据库密码参数界面

点击“下一步”，进入“实例规则”设置界面，配置实例的机器分布和规划端口。

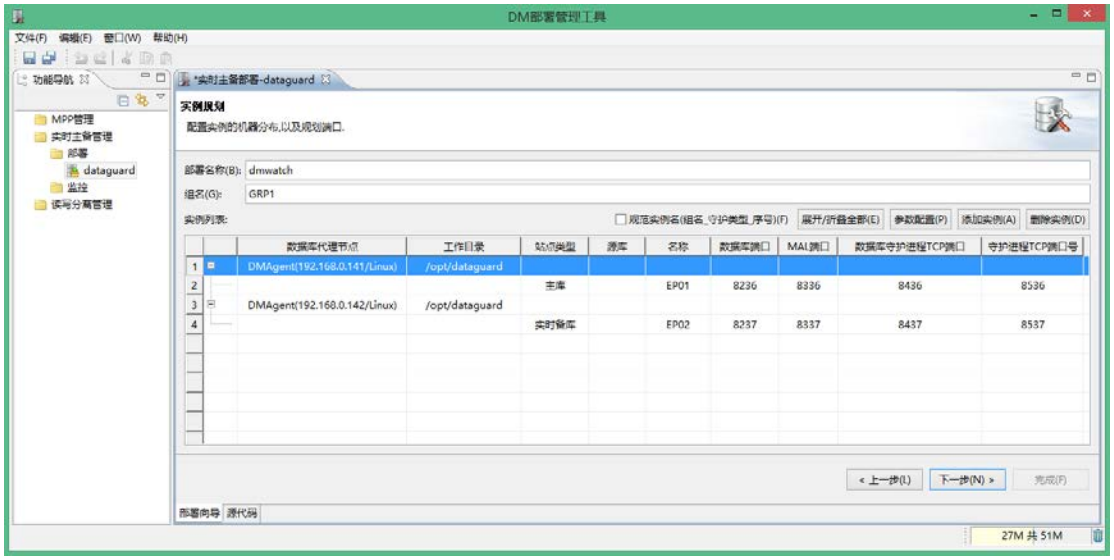


图 9.8 实例规则界面

参数详解：

规范实例名（组名_守护类型_序号）：勾选此项后，实例名由系统自动生成，用户不能修改。

参数配置：设置实例列表里的参数。

添加实例：添加异步备库，添加之后会出现在实例列表的源库中。

删除实例：删除选中的实例。

点击“下一步”，进入“配置 dm.ini”界面，配置 dm.ini 相关属性。

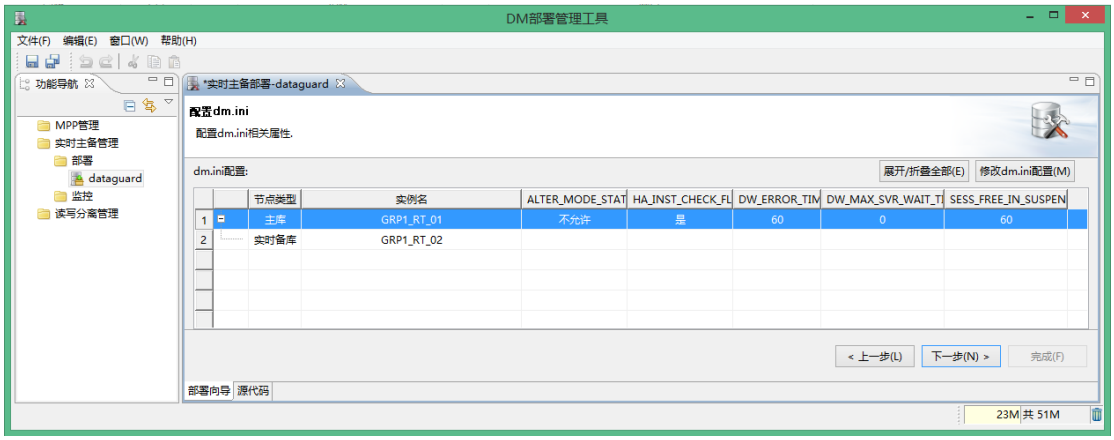


图 9.9 配置 dm.ini 界面

参数说明：

修改 dm.ini 配置：修改列表中的 dm.ini 参数。

点击“下一步”，进入“配置 dmmal.ini”界面，配置 dmmal.ini 相关属性。

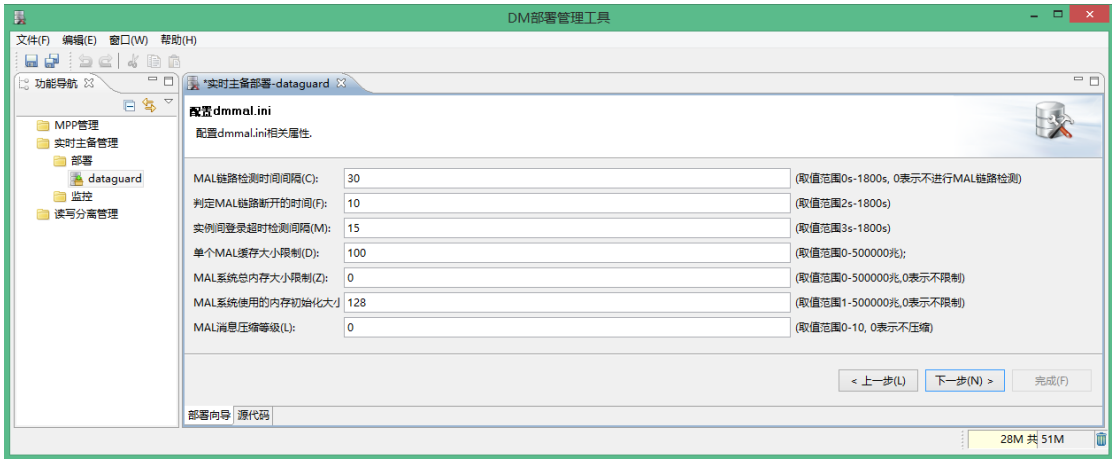


图 9.10 配置 dmmal.ini 界面

点击“下一步”，进入“配置 dmarch.ini”界面，配置 dmarch.ini 的属性，异步备库需要配置定时器信息。

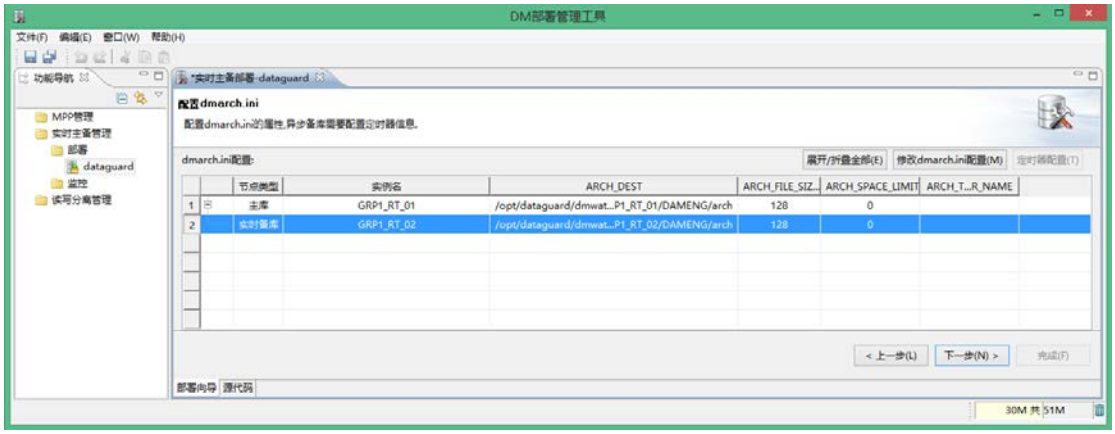


图 9.11 配置 dmarch.ini 界面

点击“下一步”，进入“配置 dmwatcher.ini”界面，配置 dmwatcher.ini 的属性。

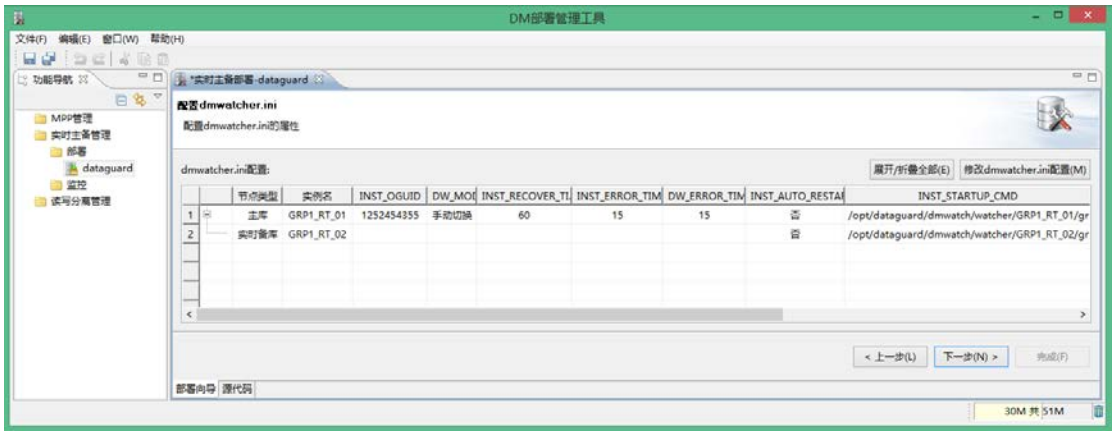


图 9.10 配置 dmwatcher.ini 界面

点击“下一步”，进入“配置监视器”界面，配置监视器相关属性。

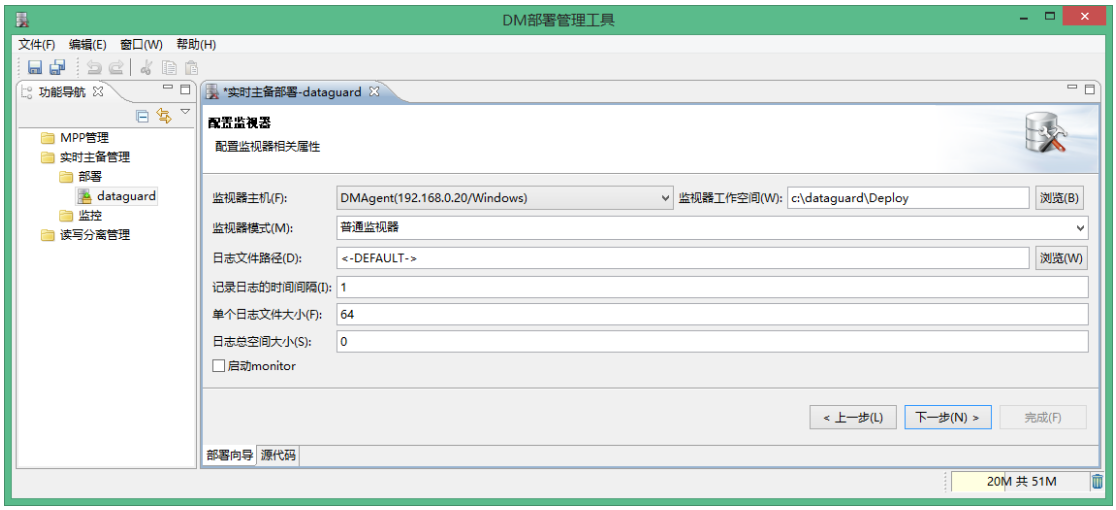


图 9.11 配置监视器界面

点击“下一步”，进入“服务器配置文件”界面，请选择上传的服务器文件。

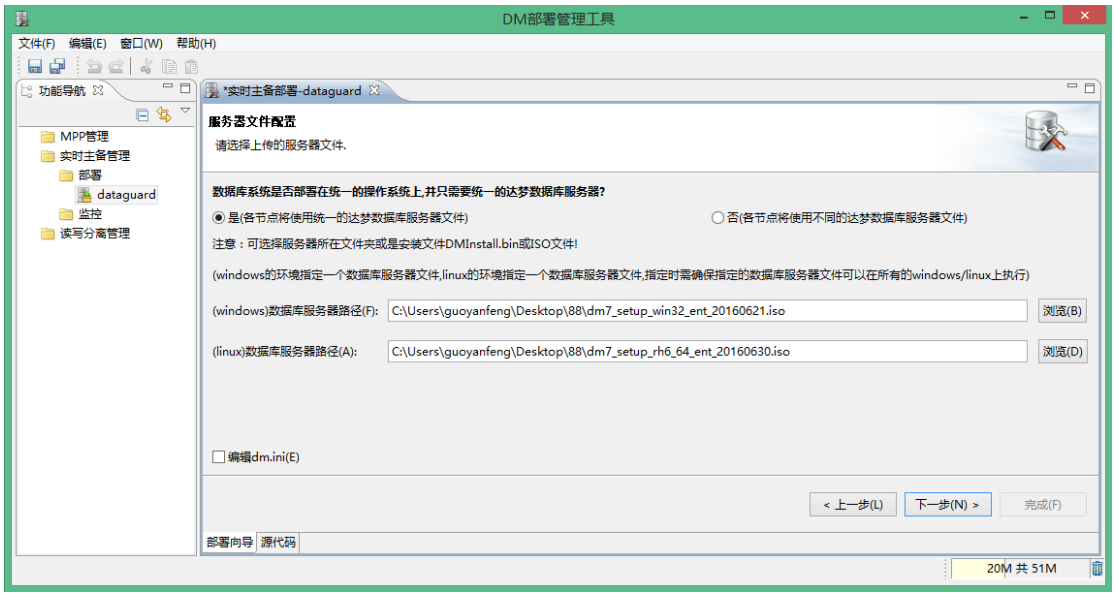


图 9.12 服务器文件配置界面

点击“下一步”，开始解析数据库服务器文件。解析的结果如下：

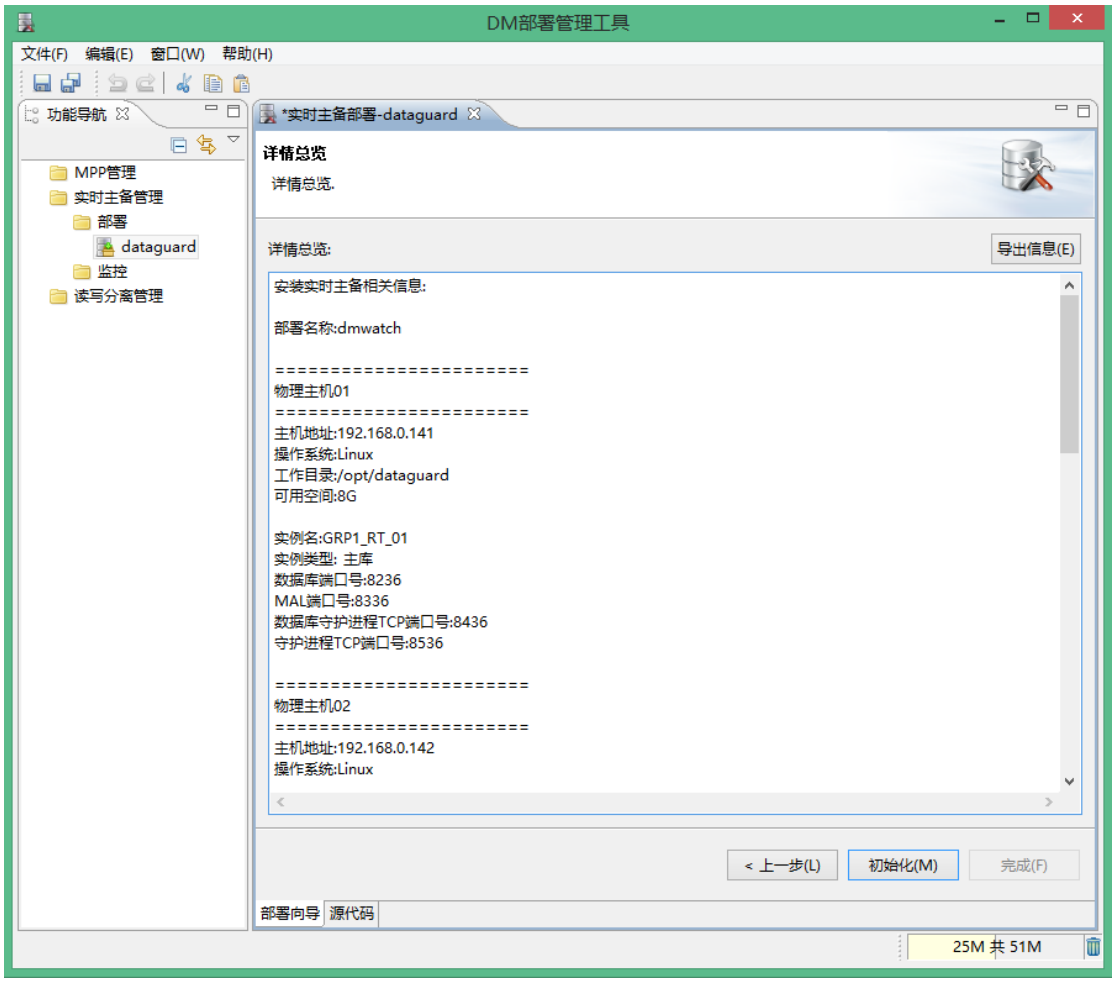


图 9.13 详情总览

点击“初始化”，开始执行初始化任务。结果如下：

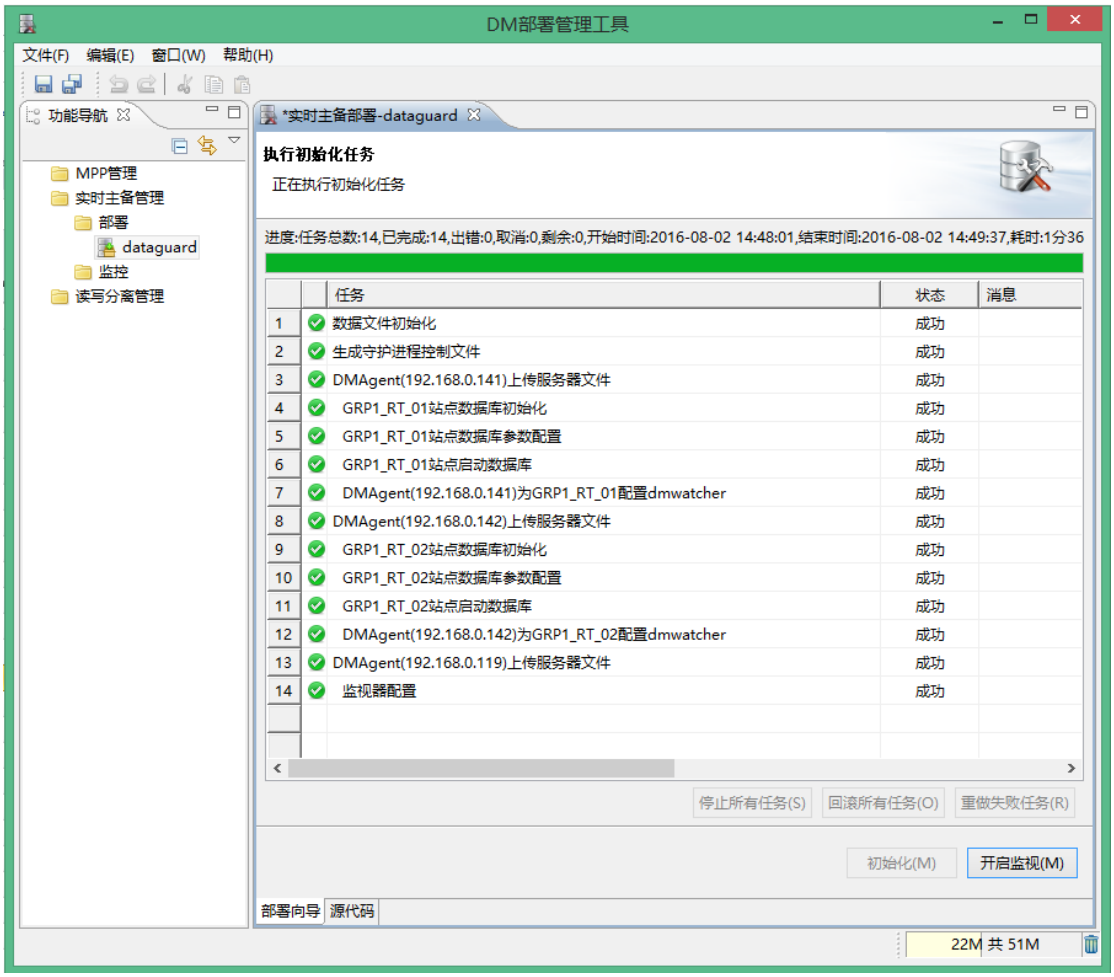


图 9.14 执行初始化任务界面

点击“开始监控”。

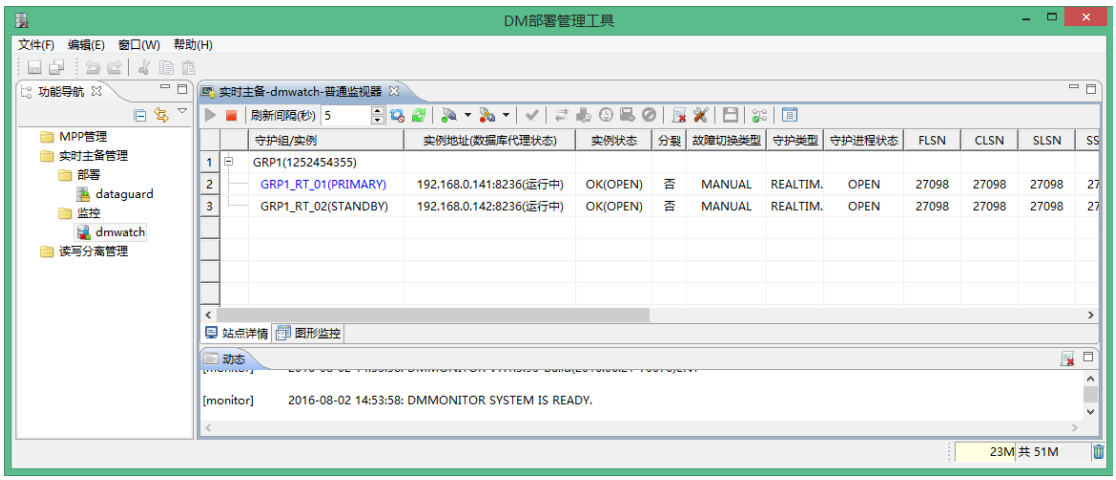


图 9.15 配置完成

至此，数据守护搭建完成。

9.1.2 监控

如果用户想监控现有的数据守护系统，可以使用“导入实时监控方案”，或者直接“扫描实时主备监控方案”。选择监控即可。监控界面如下：

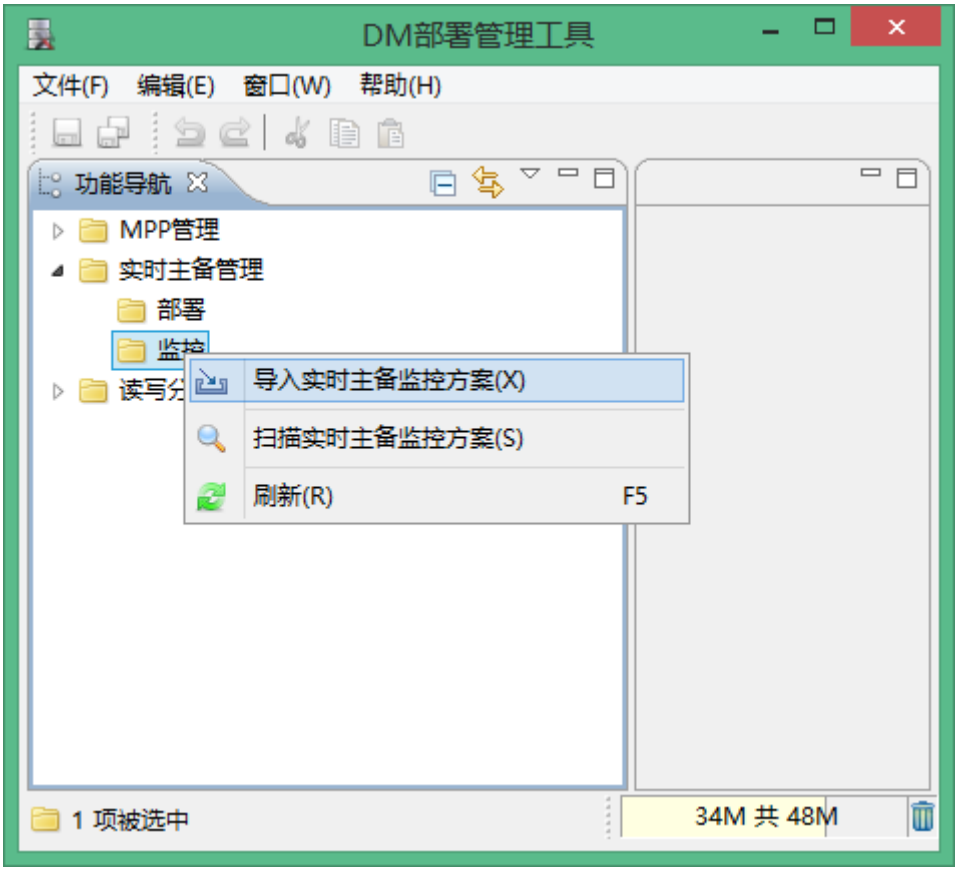


图 9.16 监控页面

本节以扫描实时主备监控方案为例。使用 9.1.2 节中配置的 dmmonitor.ini，保存在 C:\Users\guoyanfeng\Desktop\88 文件夹中。

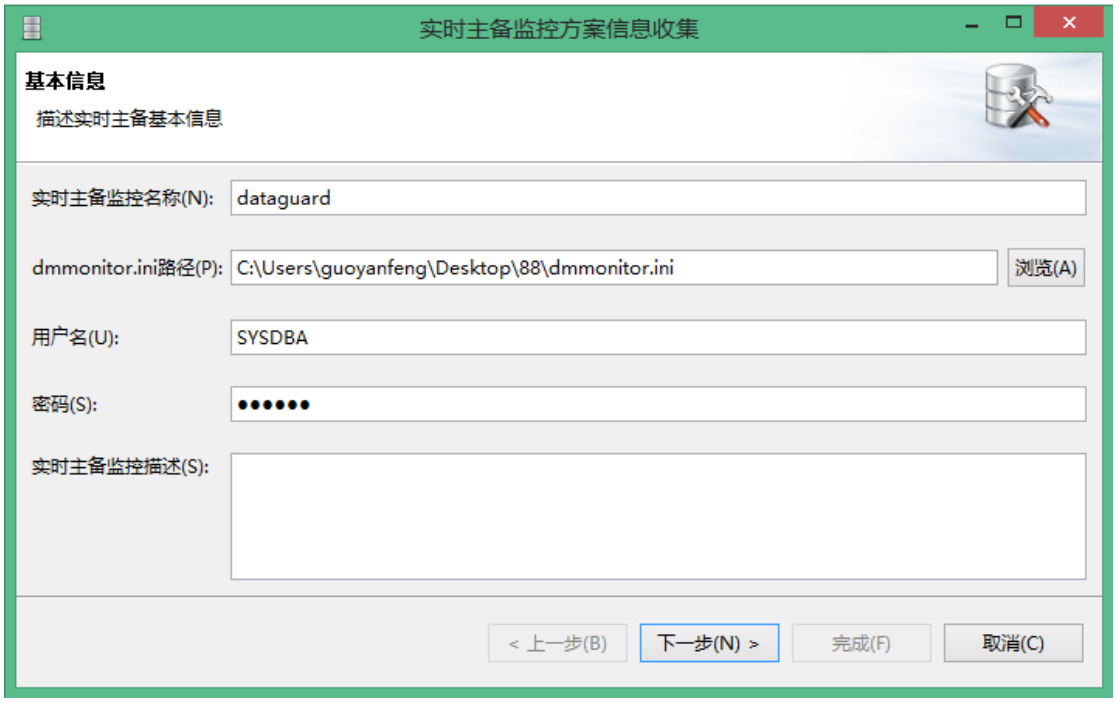


图 9.17 监控页面-基本信息

点击“下一步”，进入“连接节点”界面。

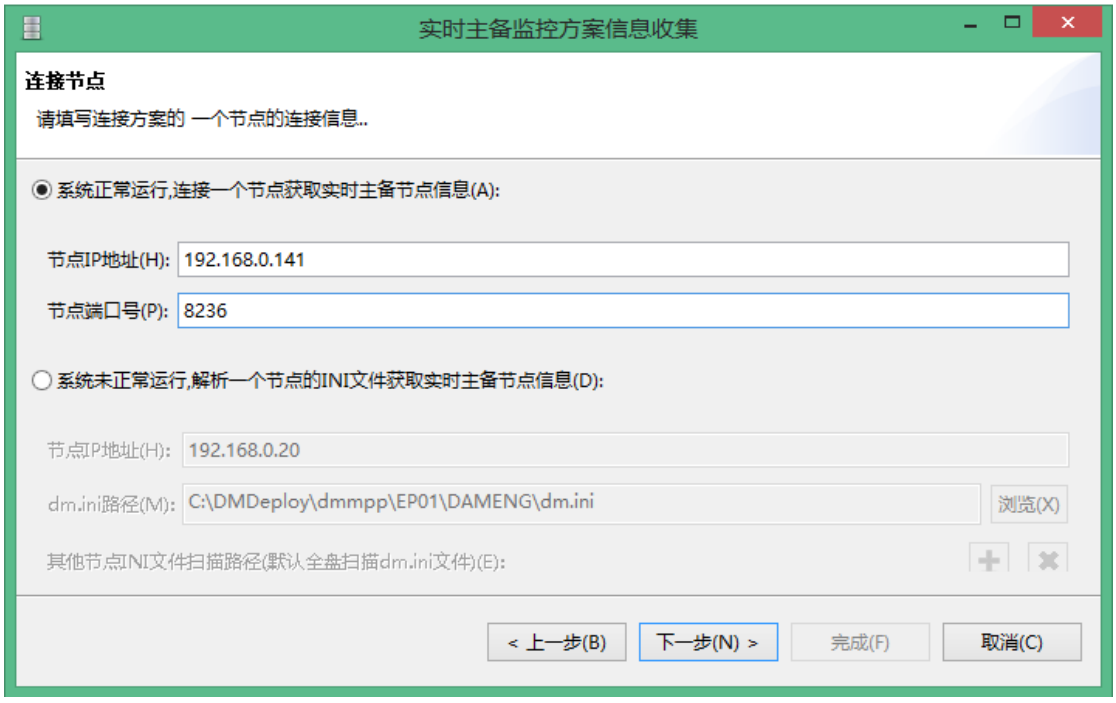


图 9.18 监控页面-连接节点

点击“下一步”，进入实时主备节点信息确认。



图 9.19 监控页面-实时主备节点信息确认

点击“完成”，就启动了一个已存在的实时主备监控方案。如下图的 dataguard。如果想查看更多的监控信息，右击 dataguard，查看属性。

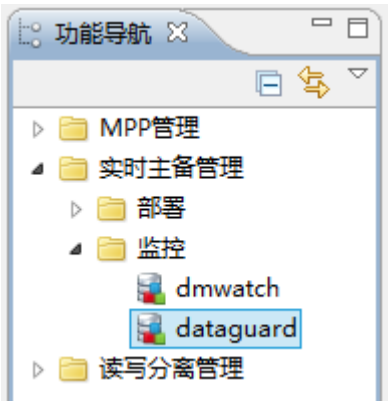


图 9.18 监控页面-启动完成

10 附录

10.1 系统视图

用于数据守护环境下的一些系统信息查询。

1. V$UTSK_INFO

查询守护进程向服务器发送请求的执行情况。

注意在 RUNNING 字段值为 TRUE 时，DSEQ、CODE 和 SEND_LSN 的值才有意义，另外 SEND_LSN 和 Recover_BREAK 在主库上查询才有意义，并且需要主库的守护进程处于 Recovery 状态。

序号	列	数据类型	说明
1	RUNNING	VARCHAR(64)	当前系统是否正在处理守护进程请求，TRUE 表示正在处理，FALSE 表示没有。
2	DSEQ	INTEGER	守护进程发送的报文序号
3	CODE	INTEGER	DSEQ 对应的执行结果
4	SEND_LSN	BIGINT	主库发送给备库的最大归档 LSN
5	RECOVER_BREAK	VARCHAR(64)	用于显示主库的 Recover 流程是否被中断，TRUE 表示被中断，FALSE 表示没有
6	AUTO_SWITCH	VARCHAR(64)	守护系统配置的切换模式，TRUE 表示故障自动切换，FALSE 表示故障手动切换。

2. V$RECOVER_STATUS

该视图需要在主库上查询，用于查询备库的恢复进度，如果已恢复完成，查询结果为空。

注意这里显示的是主库向备库发送日志的进度，由于备库重做日志也需要时间，在最后一批日志发送完成后，KBYTES_TO_Recover 为 0，Recover_PERCENT 为 100%，表示主库已经完成所有日志发送，需要等待备库将最后一批日志重做完成，此时主库的守护进程可能仍然处于 Recovery 状态，待备库重做完成后，主库的守护进程会自动切换 Open 状态。

```
RECOVER_PERCENT = (KBYTES_TOTAL - KBYTES_TO_Recover)/ KBYTES_TOTAL
```

序号	列	数据类型	说明
1	PRIMARY_NAME	VARCHAR(256)	主库实例名
2	STANDBY_NAME	VARCHAR(256)	当前正在恢复的备库实例名
3	PRIM_LSN	BIGINT	主库当前的 FLSN
4	SEND_LSN	BIGINT	主库已发给备库的最大 LSN
5	KBYTES_TOTAL	BIGINT	主库本地的日志总量，单位为 KB
6	KBYTES_TO_RECOVER	BIGINT	主库需要发送给备库的日志量，单位为 KB
7	FILES_TO_RECOVER	INTEGER	主库需要发给备库重做的日志文件个数
8	RECOVER_PERCENT	VARCHAR(64)	主库向备库已发送日志的比例

3． V$KEEP_BUF

该视图需要在备库上查询，用于查询备库上的 KEEP_BUF 信息。

读写分离集群下备库没有 KEEP_BUF 机制，该视图查询结果为空。

序号	列	数据类型	说明
1	TYPE	VARCHAR(64)	KEEP_BUF 中保存的日志类型。
2	MIN_LSN	BIGINT	KEEP_BUF 中保存的最小 LSN
3	MAX_LSN	BIGINT	KEEP_BUF 中保存的最大 LSN
4	PTX_OFF	SMALLINT	KEEP_BUF 中的 PTX 偏移
5	LOG_SIZE	INTEGER	KEEP_BUF 中的日志长度，单位为字节

4． V$RAPPLY_SYS

该视图需要在备库上查询，用于查询备库重做日志时的一些系统信息。

序号	列	数据类型	说明
1	APPLYING	VARCHAR(64)	备库的重做线程是否正在处理任务，查询结果为 TRUE 或 FALSE
2	SEC_MAX_LSN	BIGINT	备库收到的第二大 LSN 值
3	MAX_LSN	BIGINT	备库收到的最大 LSN 值
4	TASK_NUM	INTEGER	备库上的重做任务数
5	Primary_NAME	VARCHAR(128)	备库对应的主库实例名
6	HAS_KEEP_BUF	VARCHAR(64)	备库上是否有 KEEP_BUF，查询结果为 TRUE 或 FALSE

5. V$RAPPLY_LOG_TASK

该视图需要在备库上查询，用于查询备库所有重做任务的日志信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(64)	日志类型
2	MIN_LSN	BIGINT	日志的最小 LSN 值
3	MAX_LSN	BIGINT	日志的最大 LSN 值
4	PTX_OFF	SMALLINT	重做日志的 PTX 偏移
5	LOG_SIZE	INTEGER	日志长度，单位为字节

6. V$ARCH_FILE

查询本地归档日志信息。

序号	列	数据类型	说明
1	DB_MAGIC	INTEGER	数据库的 MAGIC 值
2	STATUS	VARCHAR(64)	归档日志状态
3	LEN	BIGINT	日志长度，单位为字节
4	FREE	BIGINT	写日志偏移位置
5	ARCH_LSN	BIGINT	归档文件起始 LSN，仅对归档文件有意义
6	CLSN	BIGINT	检查点的 LSN
7	ARCH_SEQ	INTEGER	归档文件起始 SEQ，仅对归档文件有意义
8	NEXT_SEQ	INTEGER	下一个 SEQ 值
9	CREAT_TIME	TIMESTAMP	归档文件创建时间
10	CLOSE_TIME	TIMESTAMP	归档文件关闭时间
11	PATH	VARCHAR(256)	归档文件所在路径，包括归档文件名

7. V$ARCH_STATUS

查询归档状态信息。

序号	列	数据类型	说明
1	ARCH_TYPE	VARCHAR(256)	归档类型
2	ARCH_DEST	VARCHAR(256)	归档目标，本地归档为归档路径，其他类型为归档目标实例名。
3	ARCH_STATUS	VARCHAR(256)	归档状态，Valid 为有效状态，Invalid

			为无效状态。

8. V$MAL_LINK_STATUS

查询本地实例到远程实例的 MAL 链路连接状态。

序号	列	数据类型	说明
1	SRC_SITE	VARCHAR(256)	源站点实例名
2	DEST_SITE	VARCHAR(256)	目的站点实例名。
3	CTL_LINK_STATUS	VARCHAR(256)	控制链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开。
4	DATA_LINK_STATUS	VARCHAR(256)	数据链路连接状态，CONNECTED 表示连接已建立，DISCONNECT 表示连接断开。

9. V$DM_ARCH_INI

归档参数信息。

序号	列	数据类型	说明
1	ARCH_NAME	VARCHAR (128)	归档名称
2	ARCH_TYPE	VARCHAR (128)	归档类型
3	ARCH_DEST	VARCHAR (256)	归档实例名
4	ARCH_FILE_SIZE	INTEGER	归档文件大小
5	ARCH_SPACE_LIMIT	INTEGER	归档文件的磁盘空间限制，单位是 MB
6	ARCH_TIMER_NAME	VARCHAR (128)	定时器名称
7	ARCH_IS_VALID	CHAR(1)	归档状态，是否有效
8	ARCH_WAIT_APPLY	INTEGER	性能模式，是否等待重演完成，取值 0：高性能模式， 1：数据一致模式。本地归档取值 NULL

10. V$DM_MAL_INI

MAL 参数信息。

序号	列	数据类型	说明
1	MAL_TYPE	VARCHAR (128)	MAL 名
2	INST_NAME	VARCHAR (256)	站点名
3	IP	VARCHAR (30)	IP 地址
4	MAL_PORT	INTEGER	端口号

5	INST_IP	VARCHAR (256)	对应实例 IP 地址
6	INST_PORT	INTEGER	对应实例的端口号

11. V$DM_TIMER_INI

定时器参数信息。

序号	列	数据类型	说明
1	TIMER_NAME	VARCHAR(128)	定时器名称
2	TYPE	TINYINT	类型
3	FREQ_MONTH_WEEK_INTERVAL	INTEGER	月或周的间隔
4	FREQ_SUB_INTERVAL	INTEGER	间隔天数
5	FREQ_MINUTE_INTERVAL	INTEGER	分钟间隔
6	START_TIME	TIME	开始时间
7	END_TIME	TIME	结束时间
8	DURING_START_DATE	DATETIME	有效时间段的开始日期时间
9	DURING_END_DATE	DATETIME	有效时间段的结束日期时间
10	NO_END_DATE_FLAG	CHAR(1)	无结束日期标识
11	DESCRIBE	VARCHAR(256)	描述
12	IS_VALID	CHAR(1)	是否有效

12. V$DMWATCHER

守护本实例的守护进程信息。

序号	列	数据类型	说明
1	N_GROUP	INTEGER	守护进程守护的实例个数
2	GROUP_NAME	VARCHAR(128)	本实例所在的组名
3	DW_TYPE	VARCHAR(32)	本组守护类型
4	DW_MODE	VARCHAR(32)	本组守护模式
5	AUTO_RESTART	INTEGER	本组是否自动重启
6	DW_STATUS	VARCHAR(64)	本组守护进程状态
7	DW_SUB_STATUS	VARCHAR(64)	本组守护进程子状态

8	LAST_MSG_TIME	DATETIME	最近收到守护进程消息的时间

13. V$MAL_SYS

MAL 系统信息视图。

序号	列	数据类型	说明
1	SYS_STATUS	INTEGER	mal 系统状态：0:OPEN,1:PRE_SHUTDOWN,2:SHUTDOWN
2	STMT_ID	INTEGER	mal 系统当前 stmtid
3	NEXT_MAL_ID	BIGINT	下一个 MAL_ID
4	MAL_PORT	INTEGER	mal 监听端口
5	N_SITE	INTEGER	mal 配置的站点数目
6	MAL_NUM	INTEGER	mal 系统邮箱数目
7	MAL_SEQ_NO	INTEGER	站点本身的 mal 序号
8	EMPTY_LET_NUM	INTEGER	空邮件数目
9	MAL_CHECK_INTERVAL	INTEGER	链路检测间隔
10	MAL_CONN_FAIL_INTERVAL	INTEGER	认定链路断开的时间间隔
11	MAL_COMPRESS_LEVEL	INTEGER	邮件压缩级别

14. V$MAL_SITE_INFO

MAL站点信息视图，MPP模式下，自动收集MPP各个站点的信息。

序号	列	数据类型	说明
1	SRC_SITE_SEQ	INTEGER	发送邮件的源站点
2	DEST_SITE_SEQ	INTEGER	目标站点序号
3	MAL_PORT_NUM	INTEGER	目标站点到本站点的 mal 链路数
4	BUILD_TIME	BIGINT	下一个邮件发送序号
5	LBTAPFDS	BIGINT	收到的邮件中已处理的最后一个邮件的序号
6	CUR_LETTER_NUM	INTEGER	当前保存不连续邮件的个数
7	MAX_LETTER_NUM	INTEGER	目前为止不连续邮件的最大个数
8	TOTAL_LINK_NUM	INTEGER	已创建到目标站点的 mal_link 数
9	FREE_LINK_NUM	INTEGER	当前空闲的 mal_link 数
10	SEND_LETTER_NUM	INTEGER	当前发送的邮件数

10.2 监视器接口

10.2.1 DLL 依赖库

监视器 DLL 依赖了其他的一些动态链接库，编程时需要将达梦 bin 目录的以下库拷贝到和 dwmon.dll 或 libdwmon.so 相同目录下，也可以在加载动态链接库时直接指定 bin 目录，就不需要再拷贝库文件：

序号	WINDOWS	非 WINDOWS（LINUX 等）
1	dmcalc.dll	libdmcalc.so
2	dmcfg.dll	libdmcfg.so
3	dmcomm.dll	libdmcomm.so
4	dmcvt.dll	libdmcvt.so
5	dmcyt.dll	libdmcyt.so
6	dmelog.dll	libdmelog.so
7	dmmem.dll	libdmmem.so
8	dmmout.dll	libdmmout.so
9	dmmsg.dll	libdmmsg.so
10	dmos.dll	libdmos.so
11	dmstrt.dll	libdmstrt.so
12	dmsys.dll	libdmsys.so
13	dmutl.dll	libdmutl.so
14	dwctl.dll	libdwctl.so

10.2.2 返回值说明

监视器接口总体的返回值策略为：小于0表示执行失败，其他值表示执行成功。对于启动/关闭守护进程监控和实例的相关接口，返回值为100表示执行前状态已经和预期状态一致，接口实际并未执行。

相关的错误码说明请参考10.2.7 错误码汇编，也可借助dwmon_get_error_msg_by_code接口获取错误码对应的错误描述信息。

10.2.3 C 接口说明

1. dwmon_alloc_handle

函数原型：

```
DWMON_RETURN
dwmon_alloc_handle(
   mhandle*       mhdle
);
```

功能说明：

分配监视器操作句柄，监视器的接口都通过此句柄调用执行。

参数说明：

mhdle：输出参数，分配到的监视器句柄。

返回值：

0：执行成功。

<0：执行失败。

2. dwmon_get_error_msg_by_code

函数原型：

```
DWMON_RETURN
dwmon_get_error_msg_by_code(
   mhandle        mhdle,
   mint4          code,
   mschar*        buf_msg,
   muint4         buf_len,
   muint4*        msg_len_out
```

```
)
```

功能说明：

获取输入 code 对应的错误描述信息，code 必须是小于 0 的值。

参数说明：

mhdle：输入参数，分配到的监视器句柄，注意如果是分配句柄或初始化环境时失败，允许 mhdle 是无效的，否则要求 mhdle 必须是有效句柄。

code：输入参数，需要获取错误信息的错误码 code，注意 code 必须是小于 0 的值。

buf_msg：输出参数，输出错误描述信息，注意 buf_msg 缓存长度建议大于 4096，避免输出信息被截断。

buf_len:输入参数，指定 buf_msg 可写入的最大长度。

msg_len_out：输出参数，buf_msg 缓存实际写入长度。

返回值：

0：执行成功。

<0：执行失败，-17 表示没有找到 code 对应的错误信息。

3. dwmon_init

函数原型：

```
DWMON_RETURN
dwmon_init(
   mhandle        mhdle,
   mschar*      ini_path,
   mschar*        log_path
);
```

功能说明：

初始化监视器环境。

参数说明：

mhdle：输入参数，监视器操作句柄。

ini_path： 输入参数，指定 dmmonitor.ini 的绝对路径。

log_path：输入参数，指定日志文件的存放路径，如果为 NULL 或空串，则将

dmmonitor.ini 中配置的 MON_LOG_PATH 作为日志文件路径，如果 dmmonitor.ini 中没有配置 MON_LOG_PATH，则将 dmmonitor.ini 的同级目录作为日志文件路径。

返回值：

0：执行成功。

<0：执行失败，-15 表示监视器尝试建立到所有守护进程的连接失败，可能守护进程都未启动，允许忽略此错误继续执行其他操作，如果不忽略此错误，认定初始化失败，则需要先正常销毁监视器环境（调用 dwmon_deinit 接口），再释放操作句柄（调用 dwmon_free_handle 接口）。

4. dwmon_msg_event_wait

函数原型：

```
void
dwmon_msg_event_wait(
   mhandle        mhdle
);
```

功能说明：

等待消息事件。监视器检测到守护进程或实例状态发生变化，或者接口命令执行过程中，产生中间输出消息时会将消息写入到缓存并通知消息事件，调用者只需要调用此接口等待即可，等待事件发生后，可通过接口dwmon_get_exec_msg去读取输出消息。

这种方式需要单独起一个线程来处理消息，以免消息被阻塞。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

无

5. dwmon_get_exec_msg

函数原型：

```
DWMON_RETURN
```

```
dwmon_get_exec_msg(
   mhandle        mhdle,
   mschar*        buf_msg,
   muint4         buf_len,
   muint4*        msg_len_out,
   muint4*        get_flag
);
```

功能说明：

获取输出信息，和 dwmon_msg_event_wait 配合使用。

参数说明：

mhdle:

输入参数，监视器操作句柄。

buf_msg：

输出参数，保存获取到的输出消息，注意 buf_msg 缓存长度需要大于 4096，以免长度不够导致消息被截断。

buf_len:

输入参数，指定 buf_msg 可写入的最大长度，建议 buf_msg 缓存长度及 buf_len 输入值大于 4096，避免消息被截断。

msg_len_out:

输出参数，写消息成功后，输出实际写入的消息长度。

get_flag:

输出参数，是否继续读取消息，TRUE 表示还有消息可读，FALSE 表示已全部读完。

返回值：

0：执行成功。

<0：执行失败。

6. dwmon_get_msg_exit_flag

函数原型：

```
mbool
```

```
dwmon_get_msg_exit_flag(
   mhandle        mhdle
);
```

功能说明：

如果上层应用程序中有单独的消息线程，可通过调用此接口获取退出标记，在标记为 TRUE 时可正常退出线程。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

1：允许线程退出。

0：不允许线程退出，需要继续等待获取输出消息。

7. dwmon_set_msg_exit_event

函数原型：

```
void
dwmon_set_msg_exit_event(
   mhandle        mhdle
);
```

功能说明：

设置消息退出事件。如果上层应用程序中有单独的消息线程，在 dwmon_get_msg_exit_flag 接口获取到退出标记为 TRUE 时，需要调用此接口设置退出事件。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

无

8. dwmon_msg_event_deinit

函数原型：

```
void
dwmon_msg_event_deinit(
   mhandle        mhdle
);
```

功能说明：

通知并等待消息退出。使用等待事件方式获取监视器消息时，需要单独起一个线程，在程序结束，退出线程时，需要调用此接口通知并等待消息线程退出。

消息线程相关的接口有 dwmon_msg_event_wait、dwmon_get_exec_msg、dwmon_get_msg_exit_flag、dwmon_set_msg_exit_event 和 dwmon_msg_event_deinit，这几个接口需要配合使用。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

无

9. dwmon_get_n_group

函数原型：

```
DWMON_RETURN
dwmon_get_n_group(
   mhandle        mhdle,
   muint4*        n_group,
   mschar**       group_name_arr
);
```

功能说明：

获取当前监控的组个数和组名称。

只有 MPP 守护类型下允许配置多组。

参数说明：

mhdle：输入参数，监视器操作句柄。

n_group：输入输出参数，监视器配置的监控组个数。输入参数：指定可获取的最大组个数，输出参数：实际获取的组个数，建议输入值不小于 16，避免取不到完整信息。

group_name_arr：输出参数，输出监视器配置的监控组名，参数是字符串指针数组类型，数组长度是 n_group 的输入值，每个指针元素指向的缓存长度不能小于 65，避免长度溢出。

返回值：

0：执行成功。

<0：执行失败。

10. dwmon_get_group_info

函数原型：

```
DWMON_RETURN
dwmon_get_group_info(
   mhandle        mhdle,
   mschar*        group_name,
   muint4*        oguid,
   muint4*        n_dw,
   mschar**       dw_ip_arr,
   msint2*        port_arr,
   mschar*        dw_type,
   mschar*        dw_mode,
   mschar*        primary_name,
   muint4*        n_split,
   mschar**       split_iname_arr,
   muint4*        n_inst,
   mschar**       inst_name_arr,
   muint4*        n_local,
```

```
    mschar**        local_name_arr
);
```

功能说明：

获取指定组的守护进程组信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入输出参数，指定要获取信息的组名，只有一组时如果没有指定组名，则作为输出参数，输出守护组名。

oguid：输出参数，输出指定守护进程组的 OGUID 值。

n_dw:输入输出参数，指定组配置的守护进程个数。输入参数：可获取的最大守护进程个数，输出参数：实际获取的守护进程个数，建议输入值不小于 25，避免取不到完整信息。

dw_ip_arr：输出参数,输出指定组配置的守护进程 IP,参数是字符串指针数组类型，数组长度是 n_dw 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免长度溢出。

port_arr：输出参数，输出指定组配置的守护进程端口，参数是 msint2 数组类型，数组长度是 n_dw 的输入值。

dw_type：输出参数，输出守护进程类型，注意缓存长度不能小于 65，避免长度溢出。

dw_mode：输出参数，输出守护进程切换模式，注意缓存长度不能小于 65，避免溢出。

primary_name：输出参数,输出当前的主库实例名,如果组中有多个 Primary 实例，则返回处于 Open 状态的 Primary 实例，如果都是 Mount 状态，则返回 LSN 最大的 Primary 实例，如果有多个 Open 状态的 Primary 实例，则接口会报错返回。注意缓存长度不能小于 65，避免溢出。

n_split：输入输出参数，当前的子组个数，如果组中有发生分裂，每个分裂实例是一个子组，剩余没有分裂的正常实例也是一个子组，如果组中没有分裂情况，则子组个数就是整个的组个数，输出值为 1。输入参数：可获取的最大子组个数，输出参数：实际获取的子组个数，建议输入值不小于 9，避免取不到完整信息。

split_iname_arr：输出参数，输出子组中其他实例可加入的实例名称，参数是字符串指针数组类型，数组长度是 n_split 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免溢出。

n_inst：输入输出参数，配置为非 local 守护类型的实例个数。输入参数：可获取的最大实例个数，输出参数：实际获取的实例个数，建议输入值不小于 9，避免取不到完整信息。

inst_name_arr：输出参数，输出配置为非 local 守护类型的实例名，参数是字符串指针数组类型，数组长度是 n_inst 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免溢出。

n_local：输入输出参数，配置为 local 守护类型的实例个数。输入参数：可获取的最大实例个数，输出参数：实际获取的实例个数，建议输入值不小于 16，避免取不到完整信息。

local_name_arr：输出参数，输出配置为 local 守护类型的实例名，参数是字符串指针数组类型，数组长度是 n_local 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

11. dwmon_get_split_sub_group_inst

函数原型：

```
DWMON_RETURN
dwmon_get_split_sub_group_inst(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name,
   muint4*        n_inst,
   mschar**       inst_name_arr
);
```

功能说明：

获取可加入指定实例的实例信息，可以和 dwmon_get_group_info 接口配合使用，其中 inst_name 是 dwmon_get_group_info 中 split_iname_arr 数组取出的某个实

例名。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要获取可加入信息的实例名，此参数必须指定。

n_inst：输入输出参数，可加入指定实例的实例个数。输入参数：指定可获取的最大实例个数，输出参数：实际获取的实例个数，建议输入值不小于 8，避免取不到完整信息。

inst_name_arr：输出参数，输出可加入指定实例的实例名，参数是字符串指针数组类型，数组长度是 n_inst 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

12. dwmon_get_nth_inst_info

函数原型：

```
DWMON_RETURN
dwmon_get_nth_inst_info(
   mhandle        mhdle,
   mschar*        group_name,
   muint4         nth,
   mschar*        dw_time,
   mschar*        dw_ip,
   msint2*        dw_port,
   mschar*        dw_type,
   mschar*        dw_mode,
   muint4*        dw_err_time,
   mschar*        dw_sta,
   mschar*        dw_sub_sta,
```

```
    mschar*        dw_ctl_sta,
    mschar*       inst_name,
    mschar*        inst_ok_str,
    muint4*        open_cnt,
    mschar*        svr_mode,
    mschar*        svr_status,
    mschar*        prim_inst,
mschar*        prim_mal_sta,
mschar*        prim_arch_type,
    mschar*        prim_arch_sta,
    muint8*        fLSN,
    muint8*        cLSN,
    muint8*        ssLSN,
    muint8*        sLSN,
    mschar*        arch_type,
    muint4*        n_arch,
    muint4*        n_async
);
```

功能说明：

获取指定组中第 nth 个实例信息，这里的 nth 是根据 dmmonitor.ini 指定组中所配置的守护进程先后顺序来排序的，每个组的 nth 都是从 0 开始，依次递增。

参数说明：



对字符串类型的输出参数，指向的缓存长度不能小于 65，避免溢出。

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定的守护进程组名，只有一组时可以不指定。

nth：输入参数，输入要获取的守护实例下标。

dw_time：输出参数，输出守护进程本地的当前时间。

dw_ip：输出参数，输出守护进程配置的 IP 地址。

dw_port: 输出参数，输出守护进程配置的端口。

dw_type: 输出参数，输出配置的守护类型。

dw_mode：输出参数，输出配置的切换模式。

dw_err_time: 输出参数，输出配置的守护进程故障认定时间。

dw_sta: 输出参数，输出守护进程状态。

dw_sub_sta: 输出参数，输出守护进程子状态。

dw_ctl_sta: 输出参数，输出守护进程控制文件状态。

inst_name: 输出参数，输出守护实例名。

inst_ok_str: 输出参数，输出守护进程认定的实例状态(OK/Error)。

open_cnt: 输出参数，输出实例的 Open 次数。

svr_mode: 输出参数，输出实例模式(NORMAL/Primary/Standby)。

svr_status: 输出参数，输出实例状态(Startup/AFTER Redo/Mount/Open/SUSPEND/Shutdown)。

prim_inst: 输出参数，当前实例是 Standby 模式时该字段才有意义，输出对应的 Primary 实例名，如果是 Primary 模式，则输出为空。

prim_mal_sta: 输出参数，当前实例是 Standby 模式时该字段才有意义，输出 prim_inst 到当前实例的 mal 链路状态。

prim_arch_type: 输出参数，当前实例是 Standby 模式时该字段才有意义，输出 prim_inst 到当前实例的归档类型，包括 REALTIME/TIMELY/MPP/ASYNC 这四种类型。

prim_arch_sta: 输出参数，当前实例是 Standby 模式时该字段才有意义，输出 prim_inst 到当前实例的归档状态，local 守护类型的除外。

fLSN: 输出参数，输出实例的 fLSN 值。

cLSN: 输出参数，输出实例的 cLSN 值。

ssLSN: 输出参数，输出实例的 ssLSN 值。

sLSN: 输出参数，输出实例的 sLSN 值。

arch_type: 输出参数，输出 inst_name 实例自身配置的归档类型，只统计 REALTIME/TIMELY/MARCH 这三种归档类型，不考虑本地 LOCAL 归档类型。

n_arch: 输出参数，输出 inst_name 实例自身配置的归档个数，只包含 REALTIME/TIMELY/MARCH 这三种归档类型下配置的归档个数。

n_async: 输出参数，输出 inst_name 实例自身配置的异步归档个数。

返回值：

0：执行成功。

<0：执行失败。

13. dwmon_get_inst_info_by_name

函数原型：

```
DWMON_RETURN

dwmon_get_inst_info_by_name(

   mhandle        mhdle,

   mschar*        group_name,

   mschar*      inst_name,

   mschar*        dw_time,

   mschar*        dw_ip,

   msint2*        dw_port,

   mschar*        dw_type,

   mschar*        dw_mode,

   muint4*        dw_err_time,

   mschar*        dw_sta,

   mschar*        dw_sub_sta,

   mschar*        dw_ctl_sta,

   mschar*        inst_ok_str,

   muint4*        open_cnt,

   mschar*        svr_mode,

   mschar*        svr_status,

   mschar*        prim_inst,

mschar*       prim_mal_sta,

mschar*       prim_arch_type,

   mschar*        prim_arch_sta,

   muint8*        fLSN,
```

```
    muint8*         cLSN,
    muint8*         ssLSN,
    muint8*         sLSN,
    mschar*         arch_type,
    muint4*         n_arch,
    muint4*         n_async
);
```

功能说明：

根据指定的实例名获取实例信息。

参数说明：

对字符串类型的输出参数，指向的缓存长度不能小于 65，避免溢出。

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定的守护进程组名，只有一组时可以不指定。

inst_name：输入参数，指定要获取信息的实例名。

dw_time：输出参数，输出守护进程本地的当前时间。

dw_ip：输出参数，输出守护进程配置的 IP 地址。

dw_port：输出参数，输出守护进程配置的端口。

dw_type：输出参数，输出配置的守护类型。

dw_mode：输出参数，输出配置的切换模式。

dw_err_time：输出参数，输出配置的守护进程故障认定时间。

dw_sta：输出参数，输出守护进程状态。

dw_sub_sta：输出参数，输出守护进程子状态。

dw_ctl_sta：输出参数，输出守护进程控制文件状态。

inst_ok_str：输出参数，输出守护进程认定的实例状态(OK/Error)。

open_cnt：输出参数，输出实例的 Open 次数。

svr_mode：输出参数，输出实例模式(NORMAL/Primary/Standby)。

svr_status：输出参数，输出实例状态(Startup/AFTER Redo/Mount/Open/SUSPEND/Shutdown)。

prim_inst：输出参数，当前实例是 Standby 模式时该字段才有意义，输出对应的 Primary 实例名，如果是 Primary 模式，则输出为空。

prim_mal_sta：输出参数，当前实例是 Standby 模式时该字段才有意义，输出 prim_inst 到当前实例的 mal 链路状态。

prim_arch_type：输出参数，当前实例是 Standby 模式时该字段才有意义，输出 prim_inst 到当前实例的归档类型，包括 REALTIME/TIMELY/MPP/ASYNC 这四种类型。

prim_arch_sta：输出参数，当前实例是 Standby 模式时该字段才有意义，输出 prim_inst 到当前实例的归档状态，local 守护类型的除外。

fLSN：输出参数，输出实例的 fLSN 值。

cLSN：输出参数，输出实例的 cLSN 值。

ssLSN：输出参数，输出实例的 ssLSN 值。

sLSN：输出参数，输出实例的 sLSN 值。

arch_type：输出参数，输出 inst_name 实例自身配置的归档类型，只统计 REALTIME/TIMELY/MARCH 这三种归档类型，不考虑本地 LOCAL 归档类型。

n_arch：输出参数，输出 inst_name 实例自身配置的归档个数，只包含 REALTIME/TIMELY/MARCH 这三种归档类型下配置的归档个数。

n_async：输出参数，输出 inst_name 实例自身配置的异步归档个数。

返回值：

0：执行成功。

<0：执行失败。

14. dwmon_get_inst_arch_info

函数原型：

```
DWMON_RETURN
dwmon_get_inst_arch_info(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*       inst_name,
   mschar*        arch_type,
```

```
    muint4*         n_arch,
    mschar**        arch_dest,
    mschar**        mal_sta,
    mschar**        arch_sta,
    muint4*         n_async,
mschar**        async_dest,
    mschar**          async_mal_sta
);
```

功能说明：

获取指定实例配置的归档信息，包括归档目标实例名，到归档目标的 mal 链路状态和归档状态，以及异步归档信息。

参数说明：

arch_dest、mal_sta、arch_sta、async_dest 和 async_mal_sta 都是字符串指针数组类型，其中 arch_dest、mal_sta 和 arch_sta 的数组长度是 n_arch 的输入值，async_dest 和 async_mal_sta 的数组长度是 n_async 的输入值，建议 n_arch 和 n_async 的输入值不小于 8，避免取不到完整信息，每个数组中的指针元素指向的缓存长度不能小于 65，避免溢出。

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取归档信息的实例名。

arch_type：输出参数，输出指定实例配置的归档类型，只检查 REALTIME、TIMELY、MARCH 这三种归档类型，注意缓存长度不能小于 65，避免溢出。

n_arch：输入输出参数，指定实例配置的归档个数，只检查 REALTIME、TIMELY、MARCH 这三种归档类型。输入参数：指定可获取的最大归档个数，输出参数：实际获取的归档个数，建议输入值不小于 8，避免取不到完整信息。

arch_dest：输出参数，输出指定实例配置的归档目标名称，只检查 REALTIME、TIMELY、MARCH 这三种归档类型。

mal_sta：输出参数，输出指定实例到归档目标的 mal 链路状态，只检查 REALTIME、

TIMELY、MARCH 这三种归档类型。

arch_sta： 输出参数，输出指定实例到归档目标的归档状态数组，指定实例是 Primary 模式时，该参数输出的是有效值，只检查 REALTIME、TIMELY、MARCH 这三种归档类型。

n_async: 输入输出参数，指定实例配置的异步归档个数。输入参数：指定可获取的最大归档个数，输出参数：实际获取的归档个数，建议输入值不小于 8，避免取不到完整信息。

async_dest: 输出参数，输出指定实例配置的异步归档目标名称。

async_mal_sta：输出参数，输出指定实例到异步归档目标的 mal 链路状态。

返回值：

0：执行成功。

<0：执行失败。

15. dwmon_get_inst_arch_send_info

函数原型：

```
DWMON_RETURN
dwmon_get_inst_arch_send_info(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name,
   mschar*        source_name,
   muint4*        time_in_ini,
   muint4*        time_in_mem,
   mschar*        last_recover_time,
   mint4*         last_recover_code,
   mschar*        arch_type,
   mschar*        mal_sta,
   mschar*        arch_sta,
   mschar*        send_type,
```

```
    mschar*         start_time,
    mschar*         end_time,
    ulint64*        start_lsn,
    ulint64*        end_lsn,
    mint4*          code,
    mschar*         desc_info
);
```

功能说明：

获取源库到指定备库的归档发送信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，输入指定备库实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取归档发送信息的备库实例名。

source_name：输出参数，输出备库 inst_name 对应的源库实例名，如果是异步备库，源库也可能是 Standby 模式，注意缓存长度不能小于 65，避免溢出。

time_in_ini：输出参数，输出源库守护进程 dmwatcher.ini 中配置 INST_RECOVER_TIME 值。

time_in_mem：输出参数，输出源库守护进程内存中的 INST_RECOVER_TIME 值。

last_recover_time：输出参数，输出源库上次对指定备库执行恢复操作的时间，对 local 守护类型的备库无效，NONE 表示未执行过恢复操作，注意缓存长度不能小于 65，避免溢出。

last_recover_code：输出参数，输出源库上次对指定备库执行恢复操作的执行结果 code，对 local 守护类型无效，0 表示执行成功，100 表示实际未执行，小于 0 表示执行失败。

arch_type：输出参数，输出源库 source_name 到备库 inst_name 的归档类型，包括 REALTIME/TIMELY/MPP/ASYNC 这四种类型，注意缓存长度不能小于 65，避免溢出。

mal_sta：输出参数，输出源库 source_name 到备库 inst_name 的 mal 链路状态，注意缓存长度不能小于 65，避免溢出。

arch_sta：输出参数，输出源库 source_name 到备库 inst_name 的归档状态，注意缓存长度不能小于 65，避免溢出。

send_type：输出参数，输出源库 source_name 到备库 inst_name 的归档发送类型，NORMAL SEND 表示正常的归档发送，FOR RECOVER SEND 表示守护进程发起的归档发送，FOR ASYNC SEND 表示定时器触发的异步归档发送，注意缓存长度不能小于 65，避免溢出。

start_time：输出参数，输出源库 source_name 到备库 inst_name 的归档发送起始时间，NONE 表示未执行过发送操作，注意缓存长度不能小于 65，避免溢出。

end_time：输出参数，输出源库 source_name 到备库 inst_name 的归档发送结束时间，end_time 为 NONE 时，如果 start_time 也为 NONE，则表示未执行过发送操作，如果 start_time 为具体时间，则表示发送尚未结束，注意缓存长度不能小于 65，避免溢出。

start_lsn：输出参数，输出源库 source_name 到备库 inst_name 的归档发送起始 lsn。

end_lsn：输出参数，输出源库 source_name 到备库 inst_name 的归档发送结束 lsn。

code：输出参数，输出源库 source_name 到备库 inst_name 的归档发送 code，0 表示执行成功，100 表示实际未执行，小于 0 表示执行失败。

desc_info：输出参数，输出源库 source_name 到备库 inst_name 的归档发送描述信息，注意缓存长度不能小于 513，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

16. dwmon_get_watch_ctl_info

函数原型：

```
DWMON_RETURN
dwmon_get_watch_ctl_info(
   mhandle        mhdle,
```

```
    mschar*         group_name,
    mschar*         inst_name,
    muint4*         n_item,
    mschar*         ctl_status,
    mschar*         cur_tguid
);
```

功能说明：

获取指定实例的守护进程控制文件信息，包括控制文件中的记录项个数，控制文件状态和当前 TGUID 值这些全局信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取控制文件信息的守护进程对应的实例名。

n_item：输出参数，输出守护进程控制文件中的记录项个数。

ctl_status：输出参数，输出守护进程控制文件状态，注意缓存长度不能小于 65，避免溢出。

cur_tguid：输出参数，输出守护进程控制文件的当前 TGUID 值，注意缓存长度不能小于 65，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

17. dwmon_get_nth_watch_ctl_item

函数原型：

```
DWMON_RETURN
dwmon_get_nth_watch_ctl_item(
    mhandle         mhdle,
    mschar*         group_name,
```

```
    mschar*         inst_name,
    muint4          nth,
    mschar*         tguid_org,
    mschar*         tguid_new,
    muint8*         LSN,
    mschar*         time,
    muint8*         mid,
    mschar*         mon_ip,
    mschar*         desc
);
```

功能说明：

获取指定实例的守护进程控制文件中第 nth 个记录项信息，记录项总数 n_item 可通过接口 dwmon_get_watch_ctl_info 获取，nth 取值范围为 0 到 n_item - 1。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取控制文件信息的守护进程对应的实例名。

nth：输入参数，指定获取第 nth 个记录项内容。

tguid_org：输出参数，输出记录项原始的 TGUID 值，注意缓存长度不能小于 65，避免溢出。

tguid_new：输出参数，输出记录项新的 TGUID 值，注意缓存长度不能小于 65，避免溢出。

LSN：输出参数，输出执行操作当时实例的 sLSN 值。

time：输出参数，输出记录产生时间，注意缓存长度不能小于 65，避免溢出。

mid：输出参数，输出执行操作的监视器 ID。

mon_ip：输出操作，输出执行操作的监视器 IP，注意缓存长度不能小于 65，避免溢出。

desc：输出操作，输出操作描述信息，注意缓存长度不能小于 257，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

18. dwmon_get_watch_config_info

函数原型：

```
DWMON_RETURN
dwmon_get_watch_config_info(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name,
   mschar*        dw_type,
   mschar*        dw_mode,
   muint4*        dw_error_time,
   muint4*        inst_oguid,
   muint4*        inst_error_time,
   muint4*        inst_recover_time,
   mschar*        inst_ini,
   muint4*        inst_auto_restart,
mschar*        inst_startup_cmd,
muint4*        inst_host_check
);
```

功能说明：

获取指定实例的守护进程配置信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入输出参数，当前有多个组时，作为输入参数，指定实例所在的守护进程组名，只有一组时允许不指定，如果没有指定组名，则作为输出参数，输出实例所在的守护进程组名。

inst_name：输入参数，指定要获取配置信息的守护进程对应的实例名。

dw_type：输出参数，输出 dmwatcher.ini 的 dw_type 配置值，注意缓存长度不能小于 65，避免溢出。

dw_mode：输出参数，输出 dmwatcher.ini 的 dw_mode 配置值，注意缓存长度不能小于 65，避免溢出。

dw_error_time：输出参数，输出 dmwatcher.ini 的 dw_error_time 配置值。

inst_oguid：输出参数，输出 dmwatcher.ini 的 inst_oguid 配置值。

inst_error_time：输出参数，输出 dmwatcher.ini 的 inst_error_time 配置值。

inst_recover_time：输出参数，输出 dmwatcher.ini 的 inst_recover_time 配置值。

inst_ini：输出参数，输出 dmwatcher.ini 的 inst_ini 配置值，注意缓存长度不能小于 257，避免溢出。

inst_auto_restart：输出参数，输出 dmwatcher.ini 的 inst_auto_restart 配置值。

inst_startup_cmd：输出参数，输出 dmwatcher.ini 的 inst_startup_cmd 配置值，注意缓存长度不能小于 257，避免溢出。

inst_host_check：输出参数，输出守护进程是否检查实例对外服务 IP 的网卡故障，0 表示不检查，1 表示检查。

返回值：

0：执行成功。

<0：执行失败。

19. dwmon_get_monitor_info

函数原型：

```
DWMON_RETURN
dwmon_get_monitor_info(
   mhandle        mhdle,
   mschar*        group_name,
```

```
    mschar*         inst_name,
    muint4*         n_mon,
    mschar**        conn_time_arr,
    muint4*         confirm_arr,
    muint8*         mid_arr,
    mschar**        mon_ip_arr
)
```

功能说明：

获取指定守护进程的监视器连接信息。同一个守护系统中允许最多 10 个监视器同时启动连接到守护进程，可通过调用此接口获取指定守护进程上的监视器连接信息，包括连接时间、确认监视器配置、监视器 ID 和 IP 信息。

其中 inst_name 是输入输出参数，表示指定守护进程对应的实例名，如果此参数为空，则根据 dmmonitor.ini 对应组中的配置顺序，选择第一个活动的守护进程获取监视器连接信息，并输出选中的实例名称；如果指定有合法值，则使用指定实例的守护进程获取连接信息。

另外输出的数组参数中第一个数组元素存放的是本地监视器的连接信息。

参数说明：

conn_time_arr 和 mon_ip_arr 是字符串指针数组类型，数组长度不能小于 10，每个数组中的指针元素指向的缓存长度不能小于 65，避免溢出。

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入输出参数，指定要获取信息的守护进程对应的实例名。

n_mon：输入输出参数，指定守护进程上连接的监视器总个数。输入参数：可获取的最大个数，输出参数：实际获取的个数，建议输入值不小于 10，避免取不到完整信息。

conn_time_arr：输出参数，输出守护进程上建立连接的时间，数组长度是 n_mon 的输入值。

confirm_arr：输出参数，输出监视器的确认配置信息，参数为 muint4 数组类型，数组长度是 n_mon 的输入值。

mid_arr：输出参数，输出监视器的唯一 ID，参数为 muint8 数组类型，数组长度是 n_mon 的输入值。

mon_ip_arr：输出参数，输出监视器的 IP 地址，数组长度是 n_mon 的输入值。

返回值：

0：执行成功。

<0：执行失败。

20. dwmon_set_cmd_execing

函数原型：

```
DWMON_RETURN
dwmon_set_cmd_execing(
   mhandle       mhdle
);
```

功能说明：

设置命令执行标记，在确认监视器模式下，调用命令接口之前，需要先调用此接口，设置执行标记，避免和确认监视器自动接管故障主库的操作发生冲突。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

21. dwmon_reset_cmd_execing

函数原型：

```
DWMON_RETURN
dwmon_reset_cmd_execing(
   mhandle       mhdle
);
```

功能说明：

重置命令执行标记，在调用命令接口执行完成后，需要调用此接口，重置执行标记，避免确认监视器无法执行自动接管故障主库的操作。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

22. dwmon_get_dw_confirm

函数原型：

```
DWMON_RETURN
dwmon_get_dw_confirm(
   mhandle        mhdle,
   mbool*         dw_confirm
)
```

功能说明：

获取确认监视器标记，用于判断当前的监视器是否为确认监视器。

参数说明：

mhdle：输入参数，监视器操作句柄

dw_confirm：输出参数，输出确认监视器标记，1：是确认监视器， 0：不是确认监视器。

返回值：

0：执行成功。

<0：执行失败。

23. dwmon_set_show_interval

函数原型：

```
DWMON_RETURN
dwmon_set_show_interval(
   mhandle        mhdle,
   muint4         show_interval
)
```

功能说明：

设置自动显示间隔。监视器根据指定的 show_interval 值，每隔 show_interval 个时间间隔，自动输出各实例的状态信息给用户，需要使用 dwmon_get_exec_msg 接口获取到输出消息，show_interval 单位为秒，默认值为 0。

参数说明：

mhdle：输入参数，监视器操作句柄

show_interval：输入参数，设置定时输出状态信息的时间间隔，单位为秒，取值为 0，或 5~3600，取值为 0 表示取消自动输出，默认值为 0。

返回值：

0：执行成功。

<0：执行失败。

24. dwmon_get_show_interval

函数原型：

```
DWMON_RETURN
dwmon_get_show_interval(
   mhandle        mhdle,
   muint4*        show_interval
);
```

功能说明：

获取当前设置的自动显示时间间隔，单位为秒。

参数说明：

mhdle：输入参数，监视器操作句柄。

show_interval：输出参数，输出当前设置的自动显示时间间隔。

返回值：

0：获取成功。

<0：获取失败。

25. dwmon_set_log_interval

函数原型：

```
DWMON_RETURN
dwmon_set_log_interval(
   mhandle        mhdle,
   muint4         log_interval
);
```

功能说明：

设置自动写入日志文件间隔。监视器按照设置的 log_interval 值，每隔 log_interval 个时间间隔，自动将各实例的状态信息写入到日志文件，log_interval 单位为秒，默认值为 1。

参数说明：

mhdle：输入参数，监视器操作句柄。

log_interval：输入参数，设置定时写入状态信息到日志文件的时间间隔，单位为秒，取值为 0，1 或 5~3600。

取值为 0 表示不写入任何信息到日志文件。

取值为 1 表示只写入普通的状态变化日志，例如收到新的信息或者检测到状态变化等。

取值为 5~3600 范围内的数值时，除了生成普通日志信息外，会按照设置的间隔定期写入站点状态信息到日志文件中。

返回值：

0：执行成功。

<0：执行失败。

26. dwmon_get_log_interval

函数原型：

```
DWMON_RETURN
dwmon_get_log_interval(
   mhandle        mhdle,
   muint4*        log_interval
)
```

功能说明：

获取当前设置的自动写入日志的时间间隔，单位为秒。

参数说明：

mhdle：输入参数，监视器操作句柄。

log_interval：输出参数，输出当前设置的时间间隔。

返回值：

0：执行成功。

<0：执行失败。

27. dwmon_set_log_file_size

函数原型：

```
DWMON_RETURN
dwmon_set_log_file_size(
   mhandle        mhdle,
   muint4         log_file_size
)
```

功能说明：

设置监视器单个的日志文件大小，达到最大值后，会自动生成并切换到新的日志文件中，如果达到最大的日志总空间限制，会自动删除创建时间最早的日志文件，日志文件命名方式为“dmmonitor_年月日时分秒.log”，例如“dmmonitor _20160201155820.log”。

参数说明：

mhdle：输入参数，监视器操作句柄。

log_file_size：输入参数，设置单个日志文件大小，单位为 M，取值范围为 16~2048，默认值为 64。

返回值：

0：执行成功。

<0：执行失败。

28. dwmon_get_log_file_size

函数原型：

```
DWMON_RETURN
dwmon_get_log_file_size(
   mhandle        mhdle,
   muint4*        log_file_size
);
```

功能说明：

获取当前设置的单个日志文件大小，单位为 M。

参数说明：

mhdle：输入参数，监视器操作句柄。

log_file_size：输出参数，输出当前设置的单个日志文件大小。

返回值：

0：执行成功。

<0：执行失败。

29. dwmon_get_log_path

函数原型：

```
DWMON_RETURN
dwmon_get_log_path(
   mhandle        mhdle,
```

```
    mschar*         log_path
);
```

功能说明：

获取当前的日志文件路径。

参数说明：

mhdle：输入参数，监视器操作句柄。

log_path：输出参数，输出当前的日志文件路径，log_path 的缓存长度不能小于 257，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

30. dwmon_set_log_space_limit

函数原型：

```
DWMON_RETURN
dwmon_set_log_space_limit(
    mhandle         mhdle,
    muint4          log_space_limit
);
```

功能说明：

调用此接口，可以设置监视器的日志总空间大小，达到设置的总空间大小后，会自动删除创建时间最早的日志文件。

参数说明：

mhdle：输入参数，监视器操作句柄。

log_space_limit：输入参数，设置日志总空间大小，单位为 M，取值为 0 或 256~4096，默认值为 0，表示没有空间限制。

返回值：

0：执行成功。

<0：执行失败。

31. dwmon_get_log_space_limit

函数原型：

```
DWMON_RETURN
dwmon_get_log_space_limit(
   mhandle        mhdle,
   muint4*        log_space_limit
);
```

功能说明：

获取当前设置的日志总空间大小，单位为 M。

参数说明：

mhdle：输入参数，监视器操作句柄。

log_space_limit：输出参数，输出当前设置的日志总空间大小。

返回值：

0：执行成功。

<0：执行失败。

32. dwmon_login

函数原型：

```
DWMON_RETURN
dwmon_login(
   mhandle        mhdle,
   mschar*        username,
   mschar*        password
);
```

功能说明：

登录监视器，这里的登录口令和服务器的登录口令是一致的，且必须有 DBA 权限。

校验登录信息时由守护进程转发给服务器校验，并返回校验结果给监视器，因此需要保证系统中至少有一个活动实例，否则无法校验成功。

参数说明：

mhdle：输入参数，监视器操作句柄。

username：输入参数，登录用户名。

password：输入参数，登录密码。

返回值：

0：执行成功。

<0：执行失败。

33. dwmon_logout

函数原型：

```
DWMON_RETURN
dwmon_logout(
   mhandle        mhdle
);
```

功能说明：

退出登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

34. dwmon_set_para_value

函数原型：

```
DWMON_RETURN
dwmon_set_para_value(
   mhandle        mhdle,
   mschar*        group_name,
```

```
    mschar*         para_name,
    mschar*         para_value
);
```

功能说明：

修改指定组的所有守护进程的指定参数值，并同步修改配置文件 dmwatcher.ini 中的参数值，如果组中某个守护进程故障，则跳过不修改，如果要重启故障的守护进程，需要在重启前修改守护进程 dmwatcher.ini 中的 DW_ERROR_TIME 和远程一致，否则 dmmonitor 执行命令时会报错处理。

目前可修改的参数包括：DW_ERROR_TIME、INST_RECOVER_TIME、INST_ERROR_TIME、INST_AUTO_RESTART。

执行此操作需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要修改的守护进程组名，只有一组时可以不指定。

para_name：输入参数，指定要修改的参数名称，目前只允许是 DW_ERROR_TIME、INST_ERROR_TIME、INST_AUTO_RESTART 中的一种。

para_value：输入参数，指定要修改的参数值，和 para_name 对应。

返回值：

0：执行成功。

<0：执行失败。

35. dwmon_set_inst_recover_time

函数原型：

```
DWMON_RETURN
dwmon_set_inst_recover_time(
mhandle         mhdle,
    mschar*         group_name,
    mschar*         inst_name,
    muint4          recover_time
```

```
);
```

功能说明：

设置指定备库的恢复间隔时间，设置成功后，recover_time 值保存在对应主库的守护进程内存中，并且只对指定的备库起作用。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定执行操作的守护进程组名，只有一组时可以不指定组名。

inst_name：输入参数，指定要设置恢复间隔时间的备库实例名。

recover_time：输入参数，指定要设置的恢复间隔时间，取值 0~86400s，设置为 0 时，表示指定备库不需要有时间间隔限制，满足恢复条件后，主库的守护进程就会启动恢复流程，详细说明可参考 3.1.8 小节。

返回值：

0：执行成功。

<0：执行失败。

36. dwmon_set_inst_arch_invalid

函数原型：

```
DWMON_RETURN
dwmon_set_inst_arch_invalid(
mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name
);
```

功能说明：

设置主库到指定备库的归档状态无效。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定执行操作的守护进程组名，只有一组时可以不指定组

名。

inst_name：输入参数，指定要修改归档状态无效的备库实例名。

返回值：

0：执行成功。

<0：执行失败。

37. dwmon_startup_watch

函数原型：

```
DWMON_RETURN
dwmon_startup_watch(
   mhandle        mhdle,
   mschar*        group_name
);
```

功能说明：

打开指定组的守护进程监控功能，守护进程启动后，默认监控功能处于打开状态。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要打开监控的守护进程组名，只有一组时可以不指定组名。

返回值：

>=0：执行成功，执行前已经有守护进程的监控处于打开状态时，返回值为 100。

<0：执行失败。

38. dwmon_startup_watch_all

函数原型：

```
DWMON_RETURN
dwmon_startup_watch_all(
   mhandle        mhdle
```

```
);
```

功能说明：

打开所有组的守护进程监控功能，守护进程启动后，默认监控功能处于打开状态。

只有 MPP 守护类型允许调用此接口。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

>=0：执行成功，执行前已经有守护进程的监控处于打开状态时，返回值为 100。

<0：执行失败。

39. dwmon_stop_watch

函数原型：

```
DWMON_RETURN
dwmon_stop_watch(
   mhandle        mhdle,
   mschar*        group_name
);
```

功能说明：

关闭指定组的所有守护进程的监控功能，监控功能被关闭后，守护进程还可以正常的接收和发送消息，但无法根据当前的实例状态来自动处理故障以及实例的自动拉起等，关闭后守护进程处于 Shutdown 状态。

如果需要正常的退出实例或者保持实例为 Mount 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

执行此操作需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要关闭监控的守护进程组名，只有一组时可以不指定组名。

返回值：

>=0：所有守护进程的监控关闭成功。执行前已经有有守护进程的监控处于关闭状态时，返回值为 100。

<0：执行失败

40. dwmon_stop_watch_all

函数原型：

```
DWMON_RETURN
dwmon_stop_watch_all(
   mhandle        mhdle
);
```

功能说明：

关闭所有组的守护进程监控功能，监控功能被关闭后，守护进程还可以正常的接收和发送消息，但无法根据当前的实例状态来自动处理故障以及实例的自动拉起等，关闭后守护进程处于 Shutdown 状态。

如果需要正常的退出实例或者保持实例为 Mount 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

执行此操作需要先登录监视器，只有 MPP 守护类型允许调用此接口。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

>=0：所有守护进程的监控关闭成功，执行前已经有守护进程的监控处于关闭状态时，返回值为 100。

<0：执行失败。

41. dwmon_startup_instance

函数原型：

```
DWMON_RETURN
dwmon_startup_instance(
```

```
    mhandle        mhdle,
    mschar*        group_name
);
```

功能说明：

启动指定组的所有实例，在 dmwatcher.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要启动实例的守护进程组名，只有一组时可以不指定。

返回值：

>=0：所有实例都启动成功，执行前已经有实例处于活动状态时，返回值为 100。

<0：执行失败。

42. dwmon_startup_instance_all

函数原型：

```
DWMON_RETURN
dwmon_startup_instance_all(
    mhandle        mhdle
);
```

功能说明：

启动所有组的实例，在 dmwatcher.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

只有 MPP 守护类型允许调用此接口。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

>=0：所有实例都启动成功，执行前已经有实例处于活动状态时，返回值为 100。

<0：执行失败。

43. dwmon_stop_instance

函数原型：

```
DWMON_RETURN
dwmon_stop_instance(
   mhandle        mhdle,
   mschar*        group_name
);
```

功能说明：

关闭指定组的所有实例，关闭实例之前，会先关闭所有守护进程的监控功能，避免守护进程将实例再次拉起。

执行此操作需要先登录监视器，另外关闭守护进程只是将守护进程切换为 Shutdown 状态，并没有退出。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要关闭实例的守护进程组名，只有一组时可以不指定。

返回值：

>=0：所有实例都退出成功，执行前实例已经全部退出时返回值为 100。

<0：执行失败。

44. dwmon_stop_instance_all

函数原型：

```
DWMON_RETURN
dwmon_stop_instance_all(
   mhandle        mhdle
);
```

功能说明：

关闭所有组的实例，关闭实例之前，会先关闭所有守护进程的监控功能，避免守护进程将实例再次拉起。

只有 MPP 守护类型允许调用此接口，执行此操作需要先登录监视器，另外关闭守护进程只是将守护进程切换为 Shutdown 状态，并没有退出。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

>=0：所有实例都退出成功，执行前有某组实例已经全部退出时返回值为 100。

<0：执行失败。

45. dwmon_open_instance

函数原型：

```
DWMON_RETURN
dwmon_open_instance(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name
);
```

功能说明：

在系统中存在其他故障实例时，守护进程无法自动 Open 主库，或者系统中没有活动主库时，守护进程无法自动 Open 备库，这种情况下，可以调用此接口强制 Open 实例。

执行此操作需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要强制 Open 实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要强制 Open 的实例名，此参数必须指定。

返回值：

>=0：执行成功，执行前实例已经处于 Open 状态时返回值为 100。

<0：执行失败。

46. dwmon_remove_instance

函数原型：

```
DWMON_RETURN
dwmon_remove_instance(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name
);
```

功能说明：

在守护系统正常运行的情况下，如果想主动退出维护备库，并且不想触发主库的 failover 故障处理流程，可通过此命令将备库正常移出守护系统，注意，这里并不会修改相关配置文件，命令会依次执行下列操作：

1. 设置指定备库的恢复间隔为 86400 s（相当于执行 set recover time 命令）
2. 修改备库的归档状态无效（相当于执行 set arch invalid 命令）
3. 切换备库的守护进程为 shutdown 状态（防止守护进程将备库自动拉起）
4. 通知备库实例正常退出

在备库维护结束重启后，如果要尽快启动主库的 recover 恢复流程，可再通过 set recover time 命令缩短恢复间隔（当前为 86400s）。

另外对于异步备库也支持此接口操作，会跳过 1,2 步骤，直接执行 3,4 步骤，正常退出异步备库。

执行此操作需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要执行操作的守护进程组名，只有一组时可以不指定。

inst_name：输入参数，指定要正常移出的备库实例名，此参数必须指定。

返回值：

0：执行成功。

<0：执行失败。

47. dwmon_get_switchover_standby

函数原型：

```
DWMON_RETURN
dwmon_get_switchover_standby(
   mhandle        mhdle,
   mschar*        group_name,
   muint4*        n_inst,
   mschar**       inst_name_arr
);
```

功能说明：

获取可以切换为主库的备库实例信息，这里的切换是指主备库都正常，且处于 Open 状态时，交换主备库模式，将备库切换为新的主库，主库切换为备库的操作。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要获取切换信息的组名，只有一组时可以不指定。

n_inst：输入输出参数，可以执行切换的备库个数。输入参数：指定可获取的最大备库个数，输出参数：实际获取的备库个数，建议输入值不小于 8，避免取不到完整信息。

inst_name_arr：输出参数，输出可以执行切换的备库实例名，参数是字符串指针数组类型，数组长度是 n_inst 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

48. dwmon_switchover

函数原型：

```
DWMON_RETURN
dwmon_switchover(
```

```
    mhandle        mhdle,
    mschar*        group_name,
    mschar*        inst_name
);
```

功能说明：

使用指定的备库和当前的主库执行切换操作，这里的切换是指主备库都正常，且处于 Open 状态时，交换主备库模式，将备库切换为新的主库，主库切换为备库的操作。

此接口只有在指定实例满足切换条件时才能够执行成功，可以先调用 dwmon_get_switchover_standby 获取到满足条件的备库实例。

执行此操作需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要执行切换的实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要执行切换的实例名，只有一个备库时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

49. dwmon_get_takeover_standby

函数原型：

```
DWMON_RETURN
dwmon_get_takeover_standby(
    mhandle        mhdle,
    mschar*        group_name,
    mbool          force_flag,
    muint4*        n_inst,
    mschar**       inst_name_arr
);
```

功能说明：

获取可以接管为主库的备库信息，在主库发生故障时，可以通过调用此接口查找可以接管为新的主库的备库实例，其中包括正常接管和强制接管，通过输入参数 force_flag 可以指定接管方式。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要获取接管信息的组名，只有一组时可以不指定。

force_flag：输入参数，指定接管方式，TRUE：强制接管，FALSE：正常接管。

n_inst：输入输出参数，可以执行接管操作的备库个数。输入参数：可获取的最大备库个数，输出参数：实际获取的备库个数，建议输入值不小于 8，避免取不到完整信息。

inst_name_arr：输出参数，输出可以执行接管操作的备库实例名，参数是字符串指针数组类型，数组长度是 n_inst 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

50. dwmon_takeover

函数原型：

```
DWMON_RETURN
dwmon_takeover(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name
);
```

功能说明：

主库故障后，可以通过调用此接口执行正常的接管操作，注意，此接口只有在指定实例满足正常接管条件时才能够执行成功，可以先调用 dwmon_get_takeover_standby 获取到符合正常接管条件的备库实例，force_flag 指定为 FALSE。

执行此操作需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要执行接管的实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要执行接管的实例名，只有一个备库时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

51. dwmon_takeover_force

函数原型：

```
DWMON_RETURN
dwmon_takeover_force(
   mhandle        mhdle,
   mschar*        group_name,
   mschar*        inst_name
);
```

功能说明：

主库故障后，可以通过调用此接口执行强制接管操作，注意，此接口只有在指定实例满足强制接管条件时才能够执行成功，可以先调用 dwmon_get_takeover_standby 获取到符合强制接管条件的备库实例，force_flag 指定为 TRUE。

执行此操作需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要执行强制接管的实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要执行强制接管的实例名，只有一个备库时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

52. dwmon_clear_watch_ctl

函数原型：

```
DWMON_RETURN
dwmon_clear_watch_ctl(
   mhandle        mhdle,
   mschar*        group_name
);
```

功能说明：

在守护进程控制文件过大时，可以调用此接口清理指定组的控制文件，此操作要求主备库运行正常，都处于 Open 状态，并且不存在分裂情况。

清理操作是重新初始化一个新的控制文件来完成的，控制文件的 TGUID 标识和清理前的不同，如果清理执行到一半，某个实例清理失败，会导致控制文件处于不一致状态，需要手动拷贝恢复到一致状态，并且重启对应的守护进程，避免误判出现分裂。

另外此操作要求先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要执行清理的守护进程组名，只有一组时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

53. dwmon_clear_watch_ctl_all

函数原型：

```
DWMON_RETURN
dwmon_clear_watch_ctl_all(
```

```
   mhandle        mhdle

);
```

功能说明：

在守护进程控制文件过大时，可以调用此接口清理所有组的控制文件，此操作要求主备实例运行正常，都处于 Open 状态，并且不存在分裂情况。

只有 MPP 守护类型允许调用此接口，并且要求先登录监视器。

清理操作是重新初始化一个新的控制文件来完成的，控制文件的 TGUID 标识和清理前的不同，如果清理执行到一半，某个实例清理失败，会导致控制文件处于不一致状态，需要手动拷贝恢复到一致状态，并且重启对应的守护进程，避免误判出现分裂。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

54. dwmon_check_mppctl

函数原型：

```
DWMON_RETURN

dwmon_check_mppctl(

   mhandle        mhdle

);
```

功能说明：

配置为 MPP 守护类型时，可通过调用此接口检查当前各个活动节点的 dmmpp.ctl 是否处于一致状态，只有 MPP 守护类型允许调用此接口。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：处于一致状态。

<0：检查失败，或者控制文件不一致(返回值为-12)。

55. dwmon_recover_mppctl

函数原型：

```
DWMON_RETURN
dwmon_recover_mppctl(
   mhandle        mhdle
);
```

功能说明：

配置为 MPP 守护类型时，如果某个活动节点的 dmmpp.ctl 不一致，可通过调用此接口恢复到一致状态。

只有 MPP 守护类型允许调用此接口，并且需要先登录监视器。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

56. dwmon_get_mpp_site_info

函数原型：

```
DWMON_RETURN
dwmon_get_mpp_site_info(
   mhandle        mhdle,
   muint4*        n_site,
   muint4*        ep_seqno,
   mschar**       inst_name
);
```

功能说明：

配置为 MPP 守护类型时，可通过调用此接口获取 MPP 节点信息，注意只有 MPP 守护类型允许调用此接口。

参数说明：

mhdle：输入参数，监视器操作句柄。

n_site：输入输出参数，MPP 节点数。输入参数：可获取的最大节点数，输出参数：实际获取的节点数，建议输入值不小于 1024，避免取不到完整信息。

ep_seqno：输出参数，输出 MPP 节点号，参数为 muint4 数组类型，数组长度是 n_site 的输入值。

inst_name：输出参数，输出 MPP 节点实例名，参数是字符串指针数组类型，数组长度是 n_site 的输入值，数组中每个指针元素指向的缓存长度不能小于 65，避免溢出。

返回值：

0：执行成功。

<0：执行失败。

57. dwmon_tip

函数原型：

```
DWMON_RETURN
dwmon_tip(
   mhandle        mhdle
);
```

功能说明：

可调用此接口查看系统当前运行状态，输出的系统状态信息可通过消息相关接口获取到。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功，并且系统运行正常。

<0：执行失败，或者系统运行异常（返回值为-11）。

58. dwmon_deinit

函数原型：

```
DWMON_RETURN
dwmon_deinit(
   mhandle        mhdle
);
```

功能说明：

销毁监视器执行环境。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

59. dwmon_free_handle

函数原型：

```
DWMON_RETURN
dwmon_free_handle(
   mhandle        mhdle
);
```

功能说明：

释放分配到的监视器句柄。

参数说明：

mhdle：输入参数，分配到的句柄。

返回值：

0：执行成功。

<0：执行失败。

10.2.4 JNI 接口说明

1. dwmon_init_handle

函数原型：

```
boolean

dwmon_init_handle();
```

功能说明：

分配监视器操作句柄。

参数说明：

无

返回值：

true：分配成功。

false:分配失败。

2. dwmon_get_error_msg_by_code

函数原型：

```
DwMonMsg

dwmon_get_error_msg_by_code(

    int         code

);
```

功能说明：

获取输入 code 对应的错误描述信息，code 必须是小于 0 的值。

参数说明：

code：输入参数，需要获取错误信息的错误码 code，注意 code 必须是小于 0 的值。

返回值：

返回 DwMonMsg 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonMsg 的其他对象成员获取输出消息。

<0：执行失败，-17 表示没有找到 code 对应的错误信息。

3. dwmon_init_env

函数原型：

```
int
dwmon_init_env(
    String          ini_path,
    String          log_path
);
```

功能说明：

初始化监视器环境。

参数说明：

ini_path: 输入参数，指定 dmmonitor.ini 的绝对路径。

log_path: 输入参数，指定日志文件的存放路径，如果为 NULL 或空串，则将 dmmonitor.ini 中配置的 MON_LOG_PATH 作为日志文件路径，如果 dmmonitor.ini 中没有配置 MON_LOG_PATH，则将 dmmonitor.ini 的同级目录作为日志文件路径。

返回值：

0：执行成功。

<0：执行失败，-15 表示监视器尝试建立到所有守护进程的连接失败，可能守护进程都未启动，允许忽略此错误继续执行其他操作，如果不忽略此错误，认定初始化失败，则需要先正常销毁监视器环境（调用 dwmon_deinit_env 接口），再释放操作句柄（调用 dwmon_free_handle 接口）。

4. dwmon_msg_event_wait

函数原型：

```
void
dwmon_msg_event_wait();
```

功能说明：

监视器检测到守护进程或实例状态发生变化，或者接口命令执行过程中，产生中间输出消息时会将消息写入到缓存并通知消息事件，调用者只需要调用此接口等待即可，等待事件

发生后，可通过接口dwmon_get_exec_msg去读取输出消息。

这种方式需要单独起一个线程来处理消息，以免消息被阻塞。

参数说明：

无

返回值：

无

5. dwmon_get_exec_msg

函数原型：

```
DwMonMsg

dwmon_get_exec_msg();
```

功能说明：

获取输出信息，和 dwmon_msg_event_wait 配合使用。

参数说明：

无

返回值：

返回 DwMonMsg 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonMsg 的其他对象成员获取输出消息。

<0：执行失败。

6. dwmon_get_msg_exit_flag

函数原型：

```
boolean

dwmon_get_msg_exit_flag();
```

功能说明：

如果上层应用程序中有单独的消息线程，可通过调用此接口获取退出标记，在标记为 TRUE 时可正常退出线程。

参数说明：

无

返回值：

true：允许线程退出。

false：不允许线程退出，需要继续等待获取输出消息。

7. dwmon_set_msg_exit_event

函数原型：

```
void

dwmon_set_msg_exit_event();
```

功能说明：

如果上层应用程序中有单独的消息线程，在 dwmon_get_msg_exit_flag 接口获取到退出标记为 true 时，需要调用此接口设置退出事件。

参数说明：

无

返回值：

无

8. dwmon_msg_event_deinit

函数原型：

```
void

dwmon_msg_event_deinit();
```

功能说明：

使用等待事件方式获取监视器消息时，需要单独起一个线程，在程序结束，退出线程时，需要调用此接口通知并等待消息线程退出。

消息线程相关的接口有 dwmon_msg_event_wait、dwmon_get_exec_msg、dwmon_get_msg_exit_flag、dwmon_set_msg_exit_event 和 dwmon_msg_event_deinit，这几个接口需要配合使用。

参数说明：

无

返回值：

无

9. dwmon_get_n_group

函数原型：

```
DwMonGrpNameArray

dwmon_get_n_group();
```

功能说明：

获取当前监控的组个数和组名称。只有 mpp 守护类型下允许配置多组。

参数说明：

无

返回值：

返回 DwMonGrpNameArray 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonGrpNameArray 的其他对象成员获取监控组信息。

<0：执行失败。

10. dwmon_get_group_info

函数原型：

```
DwMonGroup

dwmon_get_group_info(

    String          group_name

);
```

功能说明：

获取指定组的守护进程组信息。

参数说明：

group_name：输入参数，指定要获取信息的组名，只有一组时可以不指定。

返回值：

返回 DwMonGroup 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonGroup 的其他对象成员获取监控组信息。

<0：执行失败。

11. dwmon_get_split_sub_group_inst

函数原型：

```
DwMonInstNameArray
dwmon_get_split_sub_group_inst(
    String          group_name,
    String          inst_name
);
```

功能说明：

获取可加入指定实例的实例信息。

参数说明：

group_name：输入参数，指定实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要获取可加入信息的实例名，此参数必须指定。

返回值：

返回 DwMonInstNameArray 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonInstNameArray 的其他对象成员获取实例信息。

<0：执行失败。

12. dwmon_get_nth_inst_info

函数原型：

```
DwMonInst
dwmon_get_nth_inst_info(
    String          group_name,
    int             nth
);
```

功能说明：

获取指定组中第 nth 个实例信息，这里的 nth 是根据 dmmonitor.ini 指定组中所配置的守护进程先后顺序来排序的，每个组的 nth 都是从 0 开始，依次递增。

参数说明：

group_name：输入参数，指定的守护进程组名，只有一组时可以不指定。

nth：输入参数，输入要获取的守护实例下标。

返回值：

返回 DwMonInst 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonInst 的其他对象成员获取实例信息。

<0：执行失败。

13. dwmon_get_inst_info_by_name

函数原型：

```
DwMonInst
dwmon_get_inst_info_by_name(
    String          group_name,
    String          inst_name
);
```

功能说明：

根据指定的实例名获取实例信息。

参数说明：

group_name：输入参数，指定的守护进程组名，只有一组时可以不指定。

inst_name：输入参数，指定要获取信息的实例名。

返回值：

返回 DwMonInst 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonInst 的其他对象成员获取实例信息。

<0：执行失败。

14. dwmon_get_inst_arch_info

函数原型：

```
DwMonArch
dwmon_get_inst_arch_info(
    String          group_name,
    String          inst_name
);
```

功能说明：

获取指定实例配置的归档信息，包括归档目标实例名，到归档目标的 mal 链路状态和归档状态，以及异步归档信息。

参数说明：

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取归档信息的实例名。

返回值：

返回 DwMonArch 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonArch 的其他对象成员获取相关归档信息。

<0：执行失败。

15. dwmon_get_inst_arch_send_info

函数原型：

```
ArchSendInfo
dwmon_get_inst_arch_send_info(
    String      group_name,
    String      inst_name
);
```

功能说明：

获取源库到指定备库实例的归档发送信息。

参数说明：

group_name：输入参数，输入指定备库实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取归档发送信息的备库实例名。

返回值：

返回 ArchSendInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 ArchSendInfo 的其他对象成员获取相关归档发送信息。

<0：执行失败。

16. dwmon_get_watch_ctl_info

函数原型：

```
DwCtlInfo
dwmon_get_watch_ctl_info(
    String          group_name,
    String          inst_name
);
```

功能说明：

获取指定实例的守护进程控制文件信息，包括控制文件中的记录项个数，控制文件状态和当前 TGUID 值这些全局信息。

参数说明：

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取控制文件信息的守护进程对应的实例名。

返回值：

返回 DwCtlInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwCtlInfo 的其他对象成员获取相关控制文件信息。

<0：执行失败。

17. dwmon_get_nth_watch_ctl_item

函数原型：

```
DwCtlItem
dwmon_get_nth_watch_ctl_item(
    String          group_name,
    String          inst_name,
    int             nth
);
```

功能说明：

获取指定实例的守护进程控制文件中第 nth 个记录项信息，记录项总数 n_item 可通过接口 dwmon_get_watch_ctl_info 获取，nth 取值范围为 0 到 n_item - 1。

参数说明：

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取控制文件信息的守护进程对应的实例名。

nth：输入参数，指定获取第 nth 个记录项内容。

返回值：

返回 DwCtlItem 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwCtlItem 的其他对象成员获取记录项信息。

<0：执行失败。

18. dwmon_get_watch_config_info

函数原型：

```
DwConfigInfo
dwmon_get_watch_config_info(
    String      group_name,
    String      inst_name
);
```

功能说明：

获取指定实例的守护进程配置信息。

参数说明：

group_name：输入参数，只有一组时允许不指定。

inst_name：输入参数，指定要获取配置信息的守护进程对应的实例名。

返回值：

返回 DwConfigInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwConfigInfo 的其他对象成员获取配置信息。

<0：执行失败。

19. dwmon_get_monitor_info

函数原型：

```
DwMonInfo
dwmon_get_monitor_info(
    String          group_name,
    String          inst_name
);
```

功能说明：

同一个守护系统中允许最多 10 个监视器同时启动连接到守护进程，可通过调用此接口获取指定守护进程上的监视器连接信息，包括连接时间、确认监视器配置、监视器 ID 和 IP 信息。

其中 inst_name 是输入参数，表示指定守护进程对应的实例名，如果此参数为空，则根据 dmmonitor.ini 对应组中的配置顺序，自动选择第一个活动的守护进程获取监视器连接信息，如果指定有合法值，则使用指定实例的守护进程获取监视器连接信息。

参数说明：

group_name：输入参数，输入指定实例所在的守护进程组名，如果只有一组可以不指定。

inst_name：输入参数，指定要获取信息的守护进程对应的实例名。

返回值：

返回 DwMonInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonInfo 的其他对象成员获取记录项信息，取出的第一个数组元素是当前 monitor 的连接信息。

<0：执行失败。

20. dwmon_set_cmd_execing

函数原型：

```
int
dwmon_set_cmd_execing();
```

功能说明：

设置命令执行标记，在确认监视器模式下，调用命令接口之前，需要先调用此接口，设置执行标记，避免和确认监视器自动接管故障主库的操作发生冲突。

参数说明：

无

返回值：

0：执行成功。

<0：执行失败。

21. dwmon_reset_cmd_execing

函数原型：

```
int
dwmon_reset_cmd_execing();
```

功能说明：

重置命令执行标记，在调用命令接口执行完成后，需要调用此接口，重置执行标记，避免确认监视器无法执行自动接管故障主库的操作。

参数说明：

无

返回值：

0：执行成功。

<0：执行失败。

22. dwmon_get_dw_confirm

函数原型：

```
DwMonData
dwmon_get_dw_confirm();
```

功能说明：

获取确认监视器标记，用于判断当前的监视器是否为确认监视器。

参数说明：

无

返回值：

返回 DwMonData 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过对象成员 dataValue 获取确认标记，1：是确认监视器， 0：不是确认监视器。

<0：执行失败。

23. dwmon_set_show_interval

函数原型：

```
int
dwmon_set_show_interval(
    int             show_interval
);
```

功能说明：

监视器根据指定的 show_interval 值，每隔 show_interval 个时间间隔，自动输出各实例的状态信息给用户，需要使用 dwmon_get_exec_msg 接口获取到输出消息，show_interval 单位为秒，默认值为 0。

参数说明：

show_interval：输入参数，设置定时输出状态信息的时间间隔，单位为秒，取值为 0，或 5~3600，取值为 0 表示取消自动输出，默认值为 0。

返回值：

0：执行成功。

<0：执行失败。

24. dwmon_get_show_interval

函数原型：

```
DwMonData
dwmon_get_show_interval();
```

功能说明：

获取当前设置的自动显示时间间隔，单位为秒。

参数说明：

无

返回值：

返回 DwMonData 对象，可通过对象成员 returnCode 获取执行结果。

0：获取成功，可通过对象成员 dataValue 获取自动显示间隔。

<0：获取失败。

25. dwmon_set_log_interval

函数原型：

```
int
dwmon_set_log_interval(
    int             log_interval
);
```

功能说明：

监视器按照设置的 log_interval 值，每隔 log_interval 个时间间隔，自动将各实例的状态信息写入到日志文件，log_interval 单位为秒，默认值为 1。

参数说明：

log_interval：输入参数，设置定时写入状态信息到日志文件的时间间隔，单位为秒，取值为 0，1 或 5~3600。

取值为 0 表示不写入任何信息到日志文件。

取值为 1 表示只写入普通的状态变化日志，例如收到新的信息或者检测到状态变化等。

取值为 5~3600 范围内的数值时，除了生成普通日志信息外，会按照设置的间隔定期写入站点状态信息到日志文件中。

返回值：

0：执行成功。

<0：执行失败。

26. dwmon_get_log_interval

函数原型：

```
DwMonData
dwmon_get_log_interval();
```

功能说明：

获取当前设置的自动写入日志的时间间隔，单位为秒。

参数说明：

无

返回值：

返回 DwMonData 对象，可通过对象成员 returnCode 获取执行结果。

0：获取成功，可通过对象成员 dataValue 获取自动写入日志文件间隔。

<0：获取失败。

27. dwmon_set_log_file_size

函数原型：

```
int
dwmon_set_log_file_size(
```

```
    int             log_file_size
);
```

功能说明：

调用此接口，可以设置监视器单个的日志文件大小，达到最大值后，会自动生成并切换到新的日志文件中，如果达到最大的日志总空间限制，会自动删除创建时间最早的日志文件，日志文件命名方式为“dmmonitor _年月日时分秒.log”，例如“dmmonitor _20160201155820.log”。

参数说明：

log_file_size：输入参数，设置单个日志文件大小，单位为 M，取值范围为 16~2048，默认值为 64。

返回值：

0：执行成功。

<0：执行失败。

28. dwmon_get_log_file_size

函数原型：

```
DwMonData
dwmon_get_log_file_size();
```

功能说明：

获取当前设置的单个日志文件大小，单位为 M。

参数说明：

无**返回值：**

返回 DwMonData 对象，可通过对象成员 returnCode 获取执行结果。

0：获取成功，可通过对象成员 dataValue 获取单个日志文件大小。

<0：获取失败。

29. dwmon_get_log_path

函数原型：

```
DwMonData

dwmon_get_log_path();
```

功能说明：

获取当前的日志文件路径。

参数说明：

无

返回值：

返回 DwMonData 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过对象成员 stringValue 获取日志文件路径。

<0：执行失败。

30. dwmon_set_log_space_limit

函数原型：

```
int

dwmon_set_log_space_limit(

    int             log_space_limit

);
```

功能说明：

调用此接口，可以设置监视器的日志总空间大小，达到设置的总空间大小后，会自动删除创建时间最早的日志文件。

参数说明：

log_space_limit：输入参数，设置日志总空间大小，单位为 M，取值为 0 或 256~4096，默认值为 0，表示没有空间限制。

返回值：

0：执行成功。

<0：执行失败。

31. dwmon_get_log_space_limit

函数原型：

```
DwMonData
dwmon_get_log_space_limit();
```

功能说明：

获取当前设置的日志总空间大小，单位为 M。

参数说明：

无

无**返回值：**

返回 DwMonData 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过对象成员 dataValue 获取当前设置的日志总空间大小。

<0：执行失败。

32. dwmon_login

函数原型：

```
int
dwmon_login(
    String          username,
    String          password
);
```

功能说明：

登录监视器，这里的登录口令和服务器的登录口令是一致的，且必须有 DBA 权限。

校验登录信息时由守护进程转发给服务器校验，并返回校验结果给监视器，因此需要保证系统中至少有一个活动实例，否则无法校验成功。

参数说明：

username: 输入参数，登录用户名。

password: 输入参数，登录密码。

返回值：

0：执行成功。

<0：执行失败。

33. dwmon_logout

函数原型：

```
int
dwmon_logout();
```

功能说明：

退出登录监视器。

参数说明：

无

返回值：

0：执行成功。

<0：执行失败。

34. dwmon_set_para_value

函数原型：

```
int
dwmon_set_para_value(
    String      group_name,
    String      para_name,
    String      para_value
);
```

功能说明：

修改指定组的所有守护进程的指定参数值，并同步修改配置文件 dmwatcher.ini 中的参数值，如果组中某个守护进程故障，则跳过不修改，如果要重启故障的守护进程，需要在重启前修改守护进程 dmwatcher.ini 中的 DW_ERROR_TIME 和远程一致，否则 dmmonitor 执行命令时会报错处理。

目前可修改的参数包括：DW_ERROR_TIME、INST_RECOVER_TIME、INST_ERROR_TIME、INST_AUTO_RESTART。

注意：执行此操作需要先登录监视器。

参数说明：

group_name：输入参数，指定要修改的守护进程组名，只有一组时可以不指定。

para_name：输入参数，指定要修改的参数名称，目前只允许是 DW_ERROR_TIME、INST_ERROR_TIME、INST_AUTO_RESTART 中的一种。

para_value：输入参数，指定要修改的参数值，和 para_name 对应。

返回值：

0：执行成功。

<0：执行失败。

35. dwmon_set_inst_recover_time

函数原型：

```
int
dwmon_set_inst_recover_time(
    String      group_name,
    String      inst_name,
    int         recover_time
);
```

功能说明：

设置指定备库的恢复间隔时间，设置成功后，recover_time 值保存在对应主库的守护进程内存中，并且只对指定的备库起作用。

参数说明：

group_name：输入参数，指定执行操作的守护进程组名，只有一组时可以不指定组名。

inst_name：输入参数，指定要设置恢复间隔时间的备库实例名。

recover_time：输入参数，指定要设置的恢复间隔时间，取值 0~86400 s，设置为 0 时，表示指定备库不需要有时间间隔限制，满足恢复条件后，主库的守护进程就会启

动恢复流程，详细说明可参考 3.1.8 故障恢复处理 。

返回值：

0：执行成功。

<0：执行失败。

36. dwmon_set_inst_arch_invalid

函数原型：

```
int
dwmon_set_inst_arch_invalid(
    String      group_name,
    String      inst_name
);
```

功能说明：

设置主库到指定备库的归档状态无效。

参数说明：

group_name：输入参数，指定执行操作的守护进程组名，只有一组时可以不指定组名。

inst_name：输入参数，指定要修改归档状态无效的备库实例名。

返回值：

0：执行成功。

<0：执行失败。

37. dwmon_startup_watch

函数原型：

```
int
dwmon_startup_watch(
    String          group_name
);
```

功能说明：

打开指定组的守护进程监控功能，守护进程启动后，默认监控功能处于打开状态。

参数说明：

group_name：输入参数，指定要打开监控的守护进程组名，只有一组时可以不指定组名。

返回值：

>=0：执行成功，执行前已经有守护进程的监控处于打开状态时，返回值为 100。

<0：执行失败。

38. dwmon_startup_watch_all

函数原型：

```
int
dwmon_startup_watch_all();
```

功能说明：

打开所有组的守护进程监控功能，守护进程启动后，默认监控功能处于打开状态。

只有 MPP 守护类型允许调用此接口。

参数说明：

无**返回值：**

>=0：执行成功，执行前已经有守护进程的监控处于打开状态时，返回值为 100。

<0：执行失败。

39. dwmon_stop_watch

函数原型：

```
int
dwmon_stop_watch(
    String          group_name
);
```

功能说明：

关闭指定组的所有守护进程的监控功能，监控功能被关闭后，守护进程还可以正常的接收和发送消息，但无法根据当前的实例状态来自动处理故障以及实例的自动拉起等，关闭后守护进程处于 Shutdown 状态。

如果需要正常的退出实例或者保持实例为 Mount 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

执行此操作需要先登录监视器。

参数说明：

group_name：输入参数，指定要关闭监控的守护进程组名，只有一组时可以不指定组名。

返回值：

>=0：所有守护进程的监控关闭成功。执行前已经有有守护进程的监控处于关闭状态时，返回值为 100。

<0：执行失败

40. dwmon_stop_watch_all

函数原型：

```
int
dwmon_stop_watch_all();
```

功能说明：

关闭所有组的守护进程监控功能，监控功能被关闭后，守护进程还可以正常的接收和发送消息，但无法根据当前的实例状态来自动处理故障以及实例的自动拉起等，关闭后守护进程处于 Shutdown 状态。

如果需要正常的退出实例或者保持实例为 Mount 状态，则可以关闭监控功能，否则建议保持监控功能为打开状态，以保证能够及时处理系统的各种情况。

执行此操作需要先登录监视器，只有 MPP 守护类型允许调用此接口。

参数说明：

无

返回值：

>=0：所有守护进程的监控关闭成功，执行前已经有守护进程的监控处于关闭状态时，

返回值为 100。

<0：执行失败。

41. dwmon_startup_instance

函数原型：

```
int
dwmon_startup_instance(
    String          group_name
);
```

功能说明：

启动指定组的所有实例，在 dmwatcher.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

参数说明：

group_name：输入参数，指定要启动实例的守护进程组名，只有一组时可以不指定。

返回值：

>=0：所有实例都启动成功，执行前已经有实例处于活动状态时，返回值为 100。

<0：执行失败。

42. dwmon_startup_instance_all

函数原型：

```
int
dwmon_startup_instance_all();
```

功能说明：

启动所有组的实例，在 dmwatcher.ini 中 INST_AUTO_RESTART 配置为 0 时该接口可以成功执行，配置为 1 时在守护进程的监控功能打开的情况下，守护进程可以自动完成启动实例的功能，不需要再调用接口执行。

只有 MPP 守护类型允许调用此接口。

参数说明：

无**返回值：**

>=0：所有实例都启动成功，执行前已经有实例处于活动状态时，返回值为 100。

<0：执行失败。

43. dwmon_stop_instance

函数原型：

```
int
dwmon_stop_instance(
String          group_name
);
```

功能说明：

关闭指定组的所有实例，关闭实例之前，会先关闭所有守护进程的监控功能，避免守护进程将实例再次拉起。

执行此操作需要先登录监视器，另外关闭守护进程只是将守护进程切换为 Shutdown 状态，并没有退出。

参数说明：

group_name：输入参数，指定要关闭实例的守护进程组名，只有一组时可以不指定。

返回值：

>=0：所有实例都退出成功，执行前实例已经全部退出时返回值为 100。

<0：执行失败。

44. dwmon_stop_instance_all

函数原型：

```
int
dwmon_stop_instance_all();
```

功能说明：

关闭所有组的实例，关闭实例之前，会先关闭所有守护进程的监控功能，避免守护进程

将实例再次拉起。

只有 MPP 守护类型允许调用此接口，执行此操作需要先登录监视器，另外关闭守护进程只是将守护进程切换为 Shutdown 状态，并没有退出。

参数说明：

无

返回值：

>=0：所有实例都退出成功，执行前有某组实例已经全部退出时返回值为 100。

<0：执行失败。

45. dwmon_open_instance

函数原型：

```
int
dwmon_open_instance(
String          group_name,
String          inst_name
);
```

功能说明：

在系统中存在其他故障实例时，守护进程无法自动 Open 主库，或者系统中没有活动主库时，守护进程无法自动 Open 备库，这种情况下，可以调用此接口强制 Open 实例。

执行此操作需要先登录监视器。

参数说明：

group_name：输入参数，指定要强制 Open 实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要强制 Open 的实例名，此参数必须指定。

返回值：

>=0：执行成功，执行前实例已经处于 Open 状态时返回值为 100。

<0：执行失败。

46. dwmon_remove_instance

函数原型：

```
int
dwmon_remove_instance(
String      group_name,
String      inst_name
);
```

功能说明：

在守护系统正常运行的情况下，如果想主动退出维护备库，并且不想触发主库的 failover 故障处理流程，可通过此命令将备库正常移出守护系统，注意，这里并不会修改相关配置文件，命令会依次执行下列操作：

1. 设置指定备库的恢复间隔为 86400 s（相当于执行 set recover time 命令）
2. 修改备库的归档状态无效（相当于执行 set arch invalid 命令）
3. 切换备库的守护进程为 shutdown 状态（防止守护进程将备库自动拉起）
4. 通知备库实例正常退出

在备库维护结束重启后，如果要尽快启动主库的 recover 恢复流程，可再通过 set recover time 命令缩短恢复间隔（当前为 86400 s）。

另外对于异步备库也支持此接口操作，会跳过 1,2 步骤，直接执行 3,4 步骤，正常退出异步备库。

执行此操作需要先登录监视器。

参数说明：

group_name：输入参数，指定要执行操作的守护进程组名，只有一组时可以不指定。

inst_name：输入参数，指定要正常移出的备库实例名，此参数必须指定。

返回值：

0：执行成功。

<0：执行失败。

47. dwmon_get_switchover_standby

函数原型：

```
DwMonInstNameArray
dwmon_get_switchover_standby(
    String          group_name
);
```

功能说明：

获取可以切换为主库的备库实例信息，这里的切换是指主备库都正常，且处于 Open 状态时，交换主备库模式，将备库切换为新的主库，主库切换为备库的操作。

参数说明：

group_name：输入参数，指定要获取切换信息的组名，只有一组时可以不指定。

返回值：

返回 DwMonInstNameArray 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonInstNameArray 的其他对象成员获取备实例实例信息。

<0：执行失败。

48. dwmon_switchover

函数原型：

```
int
dwmon_switchover(
    String          group_name,
    String          inst_name
);
```

功能说明：

使用指定的备库和当前的主库执行切换操作，这里的切换是指主备库都正常，且处于 Open 状态时，交换主备库模式，将备库切换为新的主库，主库切换为备库的操作。

此接口只有在指定实例满足切换条件时才能够执行成功，可以先调用 dwmon_get_switchover_standby 获取到满足条件的备库实例。

执行此操作需要先登录监视器。

参数说明：

group_name：输入参数，指定要执行切换的实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要执行切换的实例名，只有一个备库时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

49. dwmon_get_takeover_standby

函数原型：

```
DwMonInstNameArray
dwmon_get_takeover_standby(
    String          group_name,
    boolean             force_flag
);
```

功能说明：

获取可以接管为主库的备库信息，在主库发生故障时，可以通过调用此接口查找可以接管为新的主库的备库实例，其中包括正常接管和强制接管，通过输入参数 force_flag 可以指定接管方式。

参数说明：

group_name：输入参数，指定要获取接管信息的组名，只有一组时可以不指定。

force_flag：输入参数，指定接管方式，true：强制接管，false：正常接管。

返回值：

返回 DwMonInstNameArray 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonInstNameArray 的其他对象成员获取备实例实例信息。

<0：执行失败。

50. dwmon_takeover

函数原型：

```
int
dwmon_takeover(
    String          group_name,
    String          inst_name
);
```

功能说明：

主库故障后，可以通过调用此接口执行正常的接管操作，注意，此接口只有在指定实例满足正常接管条件时才能够执行成功，可以先调用 dwmon_get_takeover_standby 获取到符合正常接管条件的备库实例，force_flag 指定为 false。

执行此操作需要先登录监视器。

参数说明：

group_name：输入参数，指定要执行接管的实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要执行接管的实例名，只有一个备库时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

51. dwmon_takeover_force

函数原型：

```
int
dwmon_takeover_force(
    String          group_name,
    String          inst_name
);
```

功能说明：

主库故障后，可以通过调用此接口执行强制接管操作。此接口只有在指定实例满足强制接管条件时才能够执行成功，可以先调用 dwmon_get_takeover_standby 获取到符合强制接管条件的备库实例，force_flag 指定为 TRUE。

执行此操作需要先登录监视器。

参数说明：

group_name：输入参数，指定要执行强制接管的实例所在的组名，只有一组时可以不指定。

inst_name：输入参数，指定要执行强制接管的实例名，只有一个备库时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

52. dwmon_clear_watch_ctl

函数原型：

```
int
dwmon_clear_watch_ctl(
    String          group_name
);
```

功能说明：

在守护进程控制文件过大时，可以调用此接口清理指定组的控制文件，此操作要求主备库运行正常，都处于 Open 状态，并且不存在分裂情况。

清理操作是重新初始化一个新的控制文件来完成的，控制文件的 TGUID 标识和清理前的不同，如果清理执行到一半，某个实例清理失败，会导致控制文件处于不一致状态，需要手动拷贝恢复到一致状态，并且重启对应的守护进程，避免误判出现分裂。

另外此操作要求先登录监视器。

参数说明：

group_name：输入参数，指定要执行清理的守护进程组名，只有一组时可以不指定。

返回值：

0：执行成功。

<0：执行失败。

53. dwmon_clear_watch_ctl_all

函数原型：

```
int
dwmon_clear_watch_ctl_all();
```

功能说明：

在守护进程控制文件过大时，可以调用此接口清理所有组的控制文件，此操作要求主备实例运行正常，都处于 Open 状态，并且不存在分裂情况。

只有 MPP 守护类型允许调用此接口，并且要求先登录监视器。

清理操作是重新初始化一个新的控制文件来完成的，控制文件的 TGUID 标识和清理前的不同，如果清理执行到一半，某个实例清理失败，会导致控制文件处于不一致状态，需要手动拷贝恢复到一致状态，并且重启对应的守护进程，避免误判出现分裂。

参数说明：

无

返回值：

0：执行成功。

<0：执行失败。

54. dwmon_check_mppctl

函数原型：

```
int
dwmon_check_mppctl();
```

功能说明：

配置为 mpp 守护类型时，可通过调用此接口检查当前各个活动节点的 dmmpp.ctl 是否处于一致状态，注意只有 MPP 守护类型允许调用此接口。

参数说明：

无

返回值：

0：处于一致状态。

<0：检查失败，或者控制文件不一致(返回值为-12)。

55. dwmon_recover_mppctl

函数原型：

```
int
dwmon_recover_mppctl();
```

功能说明：

配置为 mpp 守护类型时，如果某个活动节点的 dmmpp.ctl 不一致，可通过调用此接口恢复到一致状态。

只有 MPP 守护类型允许调用此接口，并且需要先登录监视器。

参数说明：

无

返回值：

0：执行成功。

<0：执行失败。

56. dwmon_get_mpp_site_info

函数原型：

```
DwMonMpp
dwmon_get_mpp_site_info();
```

功能说明：

配置为 mpp 守护类型时，可通过调用此接口获取 mpp 节点信息，注意只有 MPP 守护类型允许调用此接口。

参数说明：

无

返回值：

返回 DwMonMpp 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 DwMonMpp 其他对象成员获取 MPP 节点信息。

<0：执行失败。

57. dwmon_tip

函数原型：

```
int
dwmon_tip();
```

功能说明：

可调用此接口查看系统当前运行状态，输出的系统状态信息可通过消息相关接口获取到。

参数说明：

无

返回值：

0：执行成功，并且系统运行正常。

<0：执行失败，或者系统运行异常（返回值为-11）。

58. dwmon_deinit_env

函数原型：

```
int
dwmon_deinit_env();
```

功能说明：

销毁监视器执行环境。

参数说明：

无

返回值：

0：执行成功。

<0：执行失败。

59. dwmon_free_handle

函数原型：

```
int
dwmon_free_handle();
```

功能说明：

释放分配到的监视器句柄。

参数说明：

无

返回值：

0：执行成功。

<0：执行失败。

10.2.5 C 编程示例

这里给出 C 程序的编程示例，运行环境为 VS2010，通过配置项目属性，使用 dwmon.lib 隐式加载 dll 的方式， 注意此处仅为使用示例，实际使用时可以根据自己的需求再做调整。

```
#include "dwmon_dll.h"
#include "stdio.h"
#include "stdlib.h"
#include "windows.h"
#include "process.h"

/* 消息线程 */
DWORD
WINAPI
dwmon_get_msg_thread(
   mhandle    handle
)
```

```
{
   mschar        buf_msg[4097];
   muint4        msg_len_out;
   muint4        get_flag;
   DWMON_RETURN   code;
   while (TRUE)
   {
      /* 等待消息事件 */
      dwmon_msg_event_wait(handle);
      /* 获取退出标记 */
      if (dwmon_get_msg_exit_flag(handle) == TRUE)
      {
         break;
      }
      do
      {
         /* 获取并打印消息 */
         code  =  dwmon_get_exec_msg(handle,  buf_msg,  4097,  &msg_len_out,
&get_flag);
         if (code >=0 && msg_len_out > 0)
         {
            fprintf(stdout, "%s", buf_msg);
         }
      } while (get_flag); /* 判断是否还有未读消息 */
   }
   /* 设置退出事件 */
   dwmon_set_msg_exit_event(handle);
   return 0;
}
/* 清理环境 */
```

```
void
dwmon_clear_env(
   mhandle    handle
)
{
   /* 通知并等待 dmmon_get_msg_thread 消息线程退出 */
   dwmon_msg_event_deinit(handle);
   /* 销毁监视器环境 */
   dwmon_deinit(handle);
   /* 释放句柄 */
   dwmon_free_handle(handle);
}
int
main()
{
   DWMON_RETURN    ret;
   mhandle        handle;
    HANDLE         thread_handle;
    char           msg[4097];
    int            msg_len;
   /* 分配操作句柄 */
   ret = dwmon_alloc_handle(&handle);
   if (ret < 0)
   {
      fprintf(stdout, "dwmon alloc handle failed!\n");
    /* 获取 ret 对应的错误描述信息 */
       dwmon_get_error_msg_by_code(handle, ret, msg, 4097, &msg_len);
      if (msg_len > 0)
      {
         fprintf(stdout, "code:%d, error msg:%s", ret, msg);
```

```
      }
      return ret;
   }
   /* 创建消息线程 */
   thread_handle = (HANDLE)_beginthreadex(NULL, 0, dwmon_get_msg_thread, handle,
0, NULL);
   if (thread_handle == NULL)
   {
      ret = GetLastError();
      fprintf(stderr, "_beginthreadex error! desc:%s, code:%d\n", strerror(ret),
ret);
      return ret;
   }
    /* 初始化监视器环境，可以选择忽略-15 错误（建立到所有守护进程连接失败，可能当前守护进程
都还未启动） */
   ret = dwmon_init(handle, "D:\\dwmon\\dmmonitor.ini", "D:\\dwmon\\log");
   if (ret < 0 && ret != -15)
    {
       fprintf(stdout, "dwmon init failed!\n");
      /* 获取 ret 对应的错误描述信息 */
        dwmon_get_error_msg_by_code(handle, ret, msg, 4097, &msg_len);
      if (msg_len > 0)
      {
         fprintf(stdout, "code:%d, error msg:%s", ret, msg);
      }
      dwmon_clear_env(handle);
      return ret;
   }
   /* 登录监视器 */
   ret = dwmon_login(handle, "SYSDBA", "SYSDBA");
```

```
   if (ret < 0)
   {
      fprintf(stdout, "dwmon login failed!\n");
       /* 获取 ret 对应的错误描述信息 */
       dwmon_get_error_msg_by_code(handle, ret, msg, 4097, &msg_len);
      if (msg_len > 0)
      {
         fprintf(stdout, "code:%d, error msg:%s", ret, msg);
      }
   }
   /* 执行其他命令 */
   /*...*/
   /* 退出登录 */
   dwmon_logout(handle);
   /* 清理环境 */
   dwmon_clear_env(handle);
   system("pause");
   return 0;
}
```

10.2.6 Java 编程示例

这里给出 Java 程序的编程示例，运行环境为 Eclipse，使用加载监视器 jar 包的方式，jar 包名称为 com.dameng.jni_7.0.0.jar， 注意此处仅为使用示例，实际使用时可以根据自己的需求再做调整。

```
package com.dameng.test;
import com.dameng.console.dwmon.*;
public class DwMonTest {
    public DwMonDLL monDll = null;
    public DwMonMsg monMsg = null;
```

```
public Thread msgThread = null;
public String ini_path = "D:\\dwmon\\dmmonitor.ini";
public String log_path = "D:\\dwmon\\log";
public String username = "SYSDBA";
public String password = "SYSDBA";
//接收消息线程
public void startThread()
{
    msgThread = new Thread(new Runnable()
   {
      @Override
      public void run()
      {
        while (true)
          {
            //等待消息事件
            monDll.dwmon_msg_event_wait();

            //获取退出标记
            if (monDll.dwmon_get_msg_exit_flag() == true)
                break;
            do
              {
                //获取并打印消息
                monMsg = monDll.dwmon_get_exec_msg();
                if(monMsg.getReturnCode() < 0)
                    break;
                if(monMsg.getMsg().length() > 0)
                {
                    System.out.print(monMsg.getMsg());
```

```
                }


            } while (monMsg.getMsgFlag() == 1); //是否还有未读消息
          }
        //设置退出标记
        monDll.dwmon_set_msg_exit_event();
      }
   });
    //启动线程
    msgThread.start();
}
//退出线程
public void endThread() throws InterruptedException
{
    if (msgThread != null && msgThread.isAlive())
    {
        //通知并等待消息线程退出
        monDll.dwmon_msg_event_deinit();
    }
}
public void test() throws InterruptedException
{
    boolean    ret1 = false;
    int     ret2 = 0;
   DwMonMsg retMsg = null;


    monDll = new DwMonDLL();
    //分配句柄
    ret1 = monDll.dwmon_init_handle();
    if (ret1 == false)
```

```
        {
            System.out.println("dwmon alloc handle failed!");
            return;
        }
        else
        {
            System.out.println("dwmon alloc handle success!");
        }
        //启动消息接收线程
        startThread();
        //初始化监视器，可以选择忽略-15 错误（建立到所有守护进程连接失败，可能当前守护进程
都还未启动）
        ret2 = monDll.dwmon_init_env(ini_path, log_path);
        if (ret2 < 0 && ret2 != -15)
        {
            System.out.println("dwmon init failed!");
            /* 获取 ret2 对应的错误描述信息 */
            retMsg = monDll.dwmon_get_error_msg_by_code(ret2);
            if (retMsg.getReturnCode() >= 0)
            {
                System.out.println("code:" + ret2 + "," + retMsg.getMsg());
            }
            //初始化失败，退出线程，销毁句柄
            endThread();
            monDll.dwmon_free_handle();
            return;
        }
       else
        {
            System.out.println("dwmon init success!");
```

```
        }
        //登录监视器
        ret2 = monDll.dwmon_login(username, password);
        if (ret2 < 0)
        {
            System.out.println("login failed!");
            /* 获取 ret2 对应的错误描述信息 */
            retMsg = monDll.dwmon_get_error_msg_by_code(ret2);
            if (retMsg.getReturnCode() >= 0)
            {
                System.out.println("code:" + ret2 + "," + retMsg.getMsg());
            }
        }
        //退出登录
        monDll.dwmon_logout();
        //退出消息线程
        endThread();
        //销毁监视器环境
        monDll.dwmon_deinit_env();
        //销毁句柄
        monDll.dwmon_free_handle();
    }
    /**
     * @param args
     * @throws InterruptedException
    **/
    public static void main(String[] args) throws InterruptedException {
        DwMonTest test = new DwMonTest();
        test.test();
    }
```

```
}
```

10.2.7 错误码汇编

错误码	解释
-1	普通错误
-2	无效的句柄
-3	初始化日志信息失败
-4	初始化系统信息失败
-5	未登录监视器或服务器公钥发生变化，请重新登录
-6	读取 dmmonitor.ini 文件失败（路径错误或配置错误或没有权限）
-7	无效的日志文件路径
-8	设置命令执行标记失败，此标记已被设置，在标记被重置后才允许再次设置
-9	重置命令执行标记失败，已经被重置过
-10	实例故障或者实例对应的守护进程故障
-11	系统运行异常
-12	MPP 控制文件不一致
-13	守护进程控制文件没有记录项
-14	同一个守护系统中有多个确认监视器
-15	建立到所有守护进程的连接失败，可能守护进程都未启动，由用户决定是否忽略此错误码继续往下执行，如果不忽略此错误，认定初始化失败，则需要先销毁监视器环境，再释放操作句柄。
-16	非法的参数，参数为空或长度超长或值非法
-17	没有找到 code 对应的错误信息
-18	内存不足
-19	监视器已达到最大个数（同一个守护进程组中最多允许同时启动 10 个监视器）
-8014	用户自定义异常或者未知错误

-8145	消息校验失败
-8013	接收守护进程消息超时
-8206	用户名或密码错误，或用户没有 DBA 权限
-8191	用户名加密失败
-8192	密码加密失败
-8197	用户名或密码为空或长度超长
-8204	当前没有活动实例或守护进程状态不匹配，登录信息校验失败
-8243	监视器还未收到操作组的任何一个守护进程消息(不包括 local 守护类型)
-8342	监视器未收到操作组的所有守护进程消息(不包括 local 守护类型)
-8345	尚未收到守护进程的控制文件消息
-8380	守护进程的控制文件不是有效状态
-8346	主备库的守护进程控制文件不一致
-8347	不符合 clear dmwatcher ctl 执行条件
-8021	实例故障
-8037	实例不是 MOUNT 状态
-8038	有消息正在发送中,您的命令当前不能执行
-8042	实例故障, 操作终止
-8043	接收守护进程消息超时，操作终止
-8044	实例执行命令失败
-8045	实例名无效或尚未收到实例消息
-8496	组中当前没有可执行的备实例
-8497	实例的守护进程是 local 守护类型
-8456	未指定组名
-8455	未指定实例名
-8146	无效的组名
-8046	守护进程不是 Startup 或 OPEN 状态
-8371	守护进程不是 Startup 状态
-8062	实例的模式不匹配，预期为 Standby 模式

-8055	STOP 守护进程组失败
-8058	Startup 守护进程组失败
-8106	SWITCHOVER 所用的实例是 Primary 模式
-8083	使用实例接管失败
-8078	实例状态不匹配
-8079	组中存在多个 Primary 实例，或不存在实例满足：1.实例是 Primary&OPEN 状态 2.守护进程是 OPEN 状态 3.实例状态(INST_OK)OK
-8344	实例不满足条件：1.实例是 Standby&OPEN 状态 2.守护进程是 OPEN 状态 3.实例归档状态为 VALID
-8082	实例切换失败
-8089	实例不是 MOUNT 或 SUSPEND 状态，不允许 OPEN
-8069	实例执行 SQL 语句失败
-8388	组中有活动 Primary 实例，不再查找可接管实例列表
-8099	修改所有实例归档为无效状态失败
-8100	组中存在多个 OPEN 状态的 Primary 实例
-8111	待切换实例归档不是有效状态
-8113	监视器操作冲突，终止操作
-8440	实例不是 OPEN 状态
-8439	守护进程不是 OPEN 状态
-8121	实例非 Primary 或 Standby 模式
-8123	守护进程状态不匹配，不是 Startup、OPEN、SHUTDOWN 或 RECOVERY 状态
-8441	守护进程状态为 ERROR
-8125	实例故障，实例不是 OK 状态
-8126	实例的归档状态无效
-8137	检测到故障实例已恢复，重新修改实例为 Standby&OPEN 状态
-8148	系统组内没有或有多个活动 Primary 实例
-8435	组中没有活动 Primary 实例

-8149	内存分配失败
-8155	实例的 MPPCTL 文件更新失败
-8157	MPPCTL 文件恢复失败
-8159	构造新的 MPPCTL 文件失败
-8163	通知实例更新 MPPCTL 文件失败
-8165	尚未收到组的 MPPCTL 消息或组中不存在活动的 Primary 实例
-8265	尚未收到任何组的 MPPCTL 消息
-8359	实例非 Primary 模式
-8178	实例 OPEN 失败
-8180	到守护进程的连接断开
-8182	根据实例名获取发送端口失败
-8183	MPP 控制文件不一致
-8199	监视器尚未收到公钥信息，请稍候重试
-8201	监视器生成密钥对失败，请重试登录
-8203	守护进程不是 Startup/OPEN/RECOVERY 状态，或实例非 OK 状态，校验登录信息失败
-8220	守护进程不是 Startup/OPEN/SHUTDOWN 状态，不能修改参数
-8234	STOP 所有守护进程组失败
-8236	Startup 所有守护进程组失败
-8219	守护类型未知或是 LOCAL 守护类型(可使用 show 命令查看系统运行状态)
-8222	通知启动实例失败（启动实例的守护进程失败）
-8223	通知启动实例失败，实例已处于运行状态，或 watch 已配置自动重启，或 watch 的启动参数配置有误
-8229	实例 SHUTDOWN 失败
-8231	退出组中所有活动实例失败
-8238	启动所有组的实例失败
-8240	退出所有组的活动实例失败
-8257	组中所有实例均已处于 SHUTDOWN 状态

-8431	组中守护进程故障或实例故障，或尚未收到消息
-8272	STOP 实例的守护进程失败
-8275	Startup 实例的守护进程失败
-8277	守护进程不是 Startup 或 SHUTDOWN 状态
-8280	启动组中的所有实例失败
-8299/-8427	切换守护进程状态失败
-8311	切换守护进程为 OPEN 状态失败
-8309	修改实例实时归档为无效状态失败
-8368	请等待守护进程自动 OPEN 实例，不允许手动 OPEN FORCE
-8333	不符合 OPEN FORCE 条件
-8424	守护进程配置不一致
-8340	通知守护进程增加 CTL 记录失败
-8351	清理守护进程 CTL 记录失败
-8355	清理组的守护进程 CTL 失败，需要手动拷贝恢复组中所有守护进程的控制文件到一致状态，并重启拷贝恢复过 CTL 的守护进程
-8432	清理所有组的守护进程控制文件失败
-8360	组中已经存在 Primary&OPEN 状态的实例
-8387	守护类型未知或不是 MPP 守护类型，不支持此命令
-8425	非 MPP 守护类型，不支持配置多组
-8389	存在分裂实例，不允许执行命令
-8390	守护进程不是 OPEN 状态
-8458	守护进程不是 OPEN 或 Startup 状态
-8392	实例不是 Standby&OPEN 状态
-8459	实例不是 Standby 模式
-8460	实例不是 OPEN 或 MOUNT 状态
-8393	存在 SLSN 更大的活动实例，不允许接管
-8394	存在 Primary & OPEN 状态的实例，不允许接管
-8395	没有找到实例对应的 Primary 实例，不允许接管
-8396	被接管实例故障前不是 Primary&OPEN，不允许接管

-8397	被接管实例的 flsn 大于接管用实例的 slsn，不允许接管
-8398	守护进程故障前不是 OPEN 或 RECOVERY 状态，不允许接管
-8399	被接管实例仍然处于活动状态，不允许接管
-8420	被接管实例仍然处于活动状态，模式/状态或 LSN 值不符合接管条件
-8400	被接管实例故障前到接管用实例的归档状态无效，不允许接管
-8401	守护进程的控制文件不是有效状态
-8402	守护进程的控制文件不一致
-8403	存在 SLSN 更大的实例，不允许接管
-8404	没有收到守护进程的控制文件消息，不允许接管
-8405	存在多个或没有 Primary&OPEN 状态的实例
-8419	未找到实例对应的 Primary 实例
-8406	备实例对应的 Primary 实例不是 Primary&OPEN 状态
-8407	实例不是 Primary&OPEN 状态
-8408	实例的归档状态无效或 MAL 链路异常
-8416	组中存在多个 Primary&OPEN 状态的实例
-8428	设置 DMWATCHER 的 MID 命令执行失败
-8430	实例 [Standby, DW_TYPE:LOCAL]配置错误，被同时配置在多个实例，只允许配置在一个源实例上
-8454	存在多个守护进程控制文件有效的实例，且不可互相加入
-8436	实例发生分裂，需要用户干预
-8442	实例可作为 Standby 模式重加入，请等待守护进程自动处理
-8444	实例无法自动重加入，需要等实例恢复正常，或者用户干预
-8446	实例运行异常
-8445	尚未收到组的 MPPCTL 消息，请检查组中有活动的 Primary 实例，并且 MPP_INI 已配置，且存在有 dmmpp.ctl 文件
-8457	组中没有活动的守护进程或尚未收到消息
-8461	实例属于其他组，不在指定的组中
-8498	无效的参数名或者参数值
-8500	修改配置参数失败

-8503	修改组中实例对应的守护进程配置参数失败
-8504	监视器不支持监控不同守护类型的组，且只有 MPP 守护类型支持多组
-8505	组中不存在 Primary&OPEN 状态的实例或者有多个 Primary 实例或者 Primary 实例故障
-8506	没有找到实例对应的 Primary 实例
-8507	查找实例的归档信息失败
-8508	实例对应的 Primary 实例的守护进程不是 STARTUP/OPEN/RECOVERY 状态，不允许设置恢复间隔时间
-8514	修改组中实例的恢复时间间隔失败
-8515	指定的恢复时间间隔非法，必须在 0~86400(s)范围内
-8520	修改实例的归档为无效状态失败
-8521	实例是 Primary 模式，不允许执行此命令
-8522	组中存在多个 Primary 实例
-8523	组中当前没有活动的 Primary 实例
-8524	实例的归档信息没有找到
-8525	没有找到实例的源实例或尚未收到消息
-8527	守护进程为 MPP 守护类型，但未收到对应实例的 mppctl 消息，不允许 OPEN（请确认是否打开 dm.ini 中的 MPP_INI 配置并且存在 dmmpp.ctl 文件）
-8528	不符合设置归档失效条件
-8529	实例对应的主库不唯一
-8530	不符合退出实例条件
-8532	将实例移出守护进程组失败

show arch info 命令以及 dwmon_get_inst_arch_send_info 接口中的输出 code 说明

守护进程通知恢复 code：

0	恢复执行成功
100	未执行恢复操作
-10050	恢复失败，远程实例的模式或状态不匹配

服务器发送归档 code：	
0	发送成功
100	未执行发送操作或正在等待归档目标响应
-6018	消息长度超出网络发送消息缓存区
-6010	MAL 连接丢失（源库到归档发送目标）
-6021	MAL 链路已断开（源库到归档发送目标）
-517	系统处于状态切换中
-703	服务器模式不匹配
-518	无效的系统状态
-533	数据库的 PERMANENT_MAGIC 不匹配
-701	日志复制 LSN 值错误（归档目标收到的日志和本地日志不连续）
-503	服务器内存不足
-9512	MPP 执行停止
-9513	MPP 清理执行
-9504	MPP 收到新的计划，终止当前执行
-715	归档日志不存在（源库）
-718	收集到的归档日志和起始 LSN 不连续（源库）
-721	守护进程发起的 recover 恢复操作被中断
-108	打开重做日志失败
-702	归档日志错，备库重做日志失败